Gerencia de Administración de Riesgo de Modelo
Gerencia de Validación Interna de Riesgo Financiero
Banco de Crédito

VIRF — Guía técnica para el agente corporativo

Qué hace cada parte, cómo se usa, qué archivo tocar si algo falla y cómo pedir el cambio con un límite de 20 archivos de entrada y 5 de salida.

Generada el 4 de noviembre de 2026 desde el código: 141 módulos, 100 endpoints y 41 archivos de test.

Índice

Parte 1 · Cómo usar esta guía y el paquete 10
Paquete para el agente corporativo — cómo usarlo 10
Parte 2 · Conocimiento VIRF 12
Conocimiento VIRF — base operativa para el agente corporativo 12
Parte 3 · Recorrido por la interfaz 18
Entrada (login) 18
Mis Modelos 19
Nuevo modelo 20
Expediente: portada y guía de pasos 21
Paso 01 · Configuración 22
Paso 02 · Estándares 23
Paso 03 · Evidencia 24
Paso 04 · Dominio y literatura 25
Paso 05 · Skills (capacidades) 26
Paso 06 · Cuestionamientos metodológicos 27
Paso 08 · Validar modelo 28
Paso 09 · Documentar 29
Consulta · Resultados 30
Consulta · Controles de datos 31
Consulta · Chat de validación 32
Guía de ayuda (botón «?») 33
Solo Informe · estándar, IA y manuales 34
Solo Informe · datos de la validación 35
Solo Informe · registrar observación 36
Móvil · Mis Modelos 37
Móvil · Solo Informe 38
Móvil · Expediente 39
Parte 4 · Flujo de una validación, extremo a extremo 40
FLUJO DEL AGENTE — VIRF Agent (extremo a extremo) 40

Parte 5 · Los informes Word por dentro 46
Solo Informe 46
Entregable del pipeline 49
«Generar informe» con capturas (corrida real) 52
Parte 6 · Skills de cambio 55
S01 — Diagnosticar un fallo 55
S02 — Informes Word 57
S03 — Regla de calificación y riesgos 60
S04 — Web: pantallas y endpoints 62
S05 — Proveedor de IA, credenciales y modelos 64
S06 — Estándar de validación (Excel) 66
S07 — RAG léxico y literatura por dominio 68
S08 — Calidad de datos, motores y notebook de controles 70
S09 — Evidencia: subida, lectura y capturas 72
S10 — Chat de sustentación y consolidación 74
S11 — Usar el grafo para acotar archivos 76
S12 — Regenerar la guía y el grafo 78
Parte 7 · Catálogo de módulos (generado del código) 79
Archivos de la raíz 80
agente.py 81
Carpeta config/ 82
config/dominio.py 83
config/ia.py 84
config/modelos.py 85
config/parametros.py 86
config/red.py 87
config/rutas.py 88
config/virf_plus.py 89
Carpeta motor/ 90
motor/Mx_00_CargaRutas.py 91

motor/Mx_01_LecturaEvidencias.py 92
motor/Mx_02_EstructuraPrompt.py 94
motor/Mx_03_LlamarIA.py 95
motor/Mx_04_MotorControlesData.py 96
Carpeta modulos/ 97
modulos/abstencion.py 100
modulos/aislamiento.py 101
modulos/analisis_estatico.py 102
modulos/automatizaciones.py 103
modulos/capturas.py 105
modulos/chunker.py 106
modulos/chunker_estructural.py 107
modulos/comparador.py 108
modulos/consola.py 109
modulos/consolidacion.py 110
modulos/control_evaluacion_data.py 111
modulos/controles_a_notebook.py 112
modulos/cuestionamientos_ext.py 113
modulos/custodia.py 114
modulos/document_cache.py 115
modulos/entailment.py 116
modulos/esquemas.py 117
modulos/excel_a_notebook.py 118
modulos/explicador_modelo.py 120
modulos/exportador_excel.py 121
modulos/feedback.py 122
modulos/feedback_mejorado.py 123
modulos/fusibles.py 124
modulos/generador_estadistica_feedback.py 125
modulos/generador_markdown.py 126

modulos/generador_word.py 127
modulos/huella_informe.py 130
modulos/huella_prompts.py 131
modulos/instrucciones_sesion.py 132
modulos/inyeccion.py 133
modulos/motor_ia.py 134
modulos/motor_ia_v2.py 136
modulos/motores_datos.py 137
modulos/motores_entidad.py 138
modulos/notebook_controles.py 139
modulos/override_humano.py 140
modulos/perfilador_datasets.py 141
modulos/pii.py 142
modulos/plantilla_bcp.py 143
modulos/prevuelo.py 146
modulos/registro_emisiones.py 147
modulos/reglas_informe.py 148
modulos/reporte_docx.py 149
modulos/repositorio_controles.py 150
modulos/revision_codigo.py 151
modulos/skills/base.py 152
modulos/skills/catalogo.py 153
modulos/validador_citas.py 154
modulos/validador_humano.py 155
modulos/ventana_contexto.py 156
modulos/versionado.py 157
modulos/word_estilo.py 158
Carpeta src/ 159
src/rag/ALM/indexer.py 160
src/rag/ALM/retriever.py 161

src/rag/base_indexer.py 162
src/rag/base_retriever.py 163
src/rag/buscador_lexico.py 164
src/rag/config_rag.py 165
src/rag/custom_retriever.py 166
src/rag/indexador_lexico.py 167
src/rag/LaR/indexer.py 168
src/rag/LaR/retriever.py 169
src/rag/shared/indexer.py 170
src/rag/shared/retriever.py 171
src/rag/tesauro.py 172
src/rag/VaR/indexer.py 173
src/rag/VaR/retriever.py 174
Carpeta web/ 175
web/app.py 177
web/config_web.py 178
web/routers/auth.py 179
web/routers/automatizaciones.py 180
web/routers/chat.py 181
web/routers/config.py 182
web/routers/controles.py 183
web/routers/cuestionamientos.py 184
web/routers/cuestionamientos_sesion.py 185
web/routers/debug_conexiones.py 186
web/routers/dominio_prevuelo.py 187
web/routers/entendimiento.py 188
web/routers/estandares.py 189
web/routers/guia.py 190
web/routers/informe.py 191
web/routers/modelos.py 192

web/routers/preview_prompt.py 193
web/routers/prompts.py 194
web/routers/rag.py 195
web/routers/resultados.py 196
web/routers/skills.py 197
web/routers/solo_informe.py 198
web/routers/upload.py 200
web/routers/validacion.py 201
web/services/analisis_codigo.py 202
web/services/chat_service.py 203
web/services/contexto_sesion.py 205
web/services/cuestionamientos_sesion.py 206
web/services/dominios_custom_store.py 207
web/services/env_service.py 208
web/services/estandares_service.py 209
web/services/informe_web.py 210
web/services/modelos_store.py 212
web/services/pipeline_runner.py 213
web/services/presupuesto_service.py 214
web/services/prompt_service.py 215
web/services/sesion_store.py 216
web/services/skills_store.py 217
web/services/solo_informe_ia.py 218
web/services/solo_informe_reglas.py 220
web/services/solo_informe_service.py 221
web/services/solo_informe_word.py 223
web/verificar_conexiones.py 224
Carpeta skills/ 225
skills/base_tipada.py 226
skills/calidad_datos.py 227

skills/cola_pendiente.py 228
skills/documentacion.py 229
skills/gobierno.py 230
skills/implementacion.py 231
skills/metodologia.py 232
skills/uso.py 233
Carpeta scripts/ 234
scripts/auditar_cobertura_rag.py 235
scripts/explicar_rag.py 236
scripts/generar_corpus_rag.py 237
scripts/generar_guia_agente.py 238
scripts/pipeline_calidad_datos.py 240
scripts/preguntas_oro_formula.py 241
scripts/preguntas_oro_rag.py 242
scripts/rebuild_rag_databricks.py 243
scripts/reconstruir_estandar_desde_informes.py 244
scripts/verificar_corpus_rag.py 245
Parte 8 · Catálogo de endpoints de la web 246
web/app.py 246
web/routers/auth.py 246
web/routers/automatizaciones.py 246
web/routers/chat.py 246
web/routers/config.py 247
web/routers/controles.py 247
web/routers/cuestionamientos.py 247
web/routers/cuestionamientos_sesion.py 248
web/routers/debug_conexiones.py 248
web/routers/dominio_prevuelo.py 248
web/routers/entendimiento.py 248
web/routers/estandares.py 249

web/routers/guia.py 249
web/routers/informe.py 249
web/routers/modelos.py 249
web/routers/preview_prompt.py 250
web/routers/prompts.py 250
web/routers/rag.py 250
web/routers/resultados.py 251
web/routers/skills.py 251
web/routers/solo_informe.py 251
web/routers/upload.py 252
web/routers/validacion.py 253
Parte 9 · Catálogo de tests 254
Parte 10 · Variables de entorno 265
Parte 11 · El grafo del código 268
Nodos centrales (cambiarlos afecta a mucho) 268
Comunidades del grafo 270
Parte 12 · Matriz de fallos frecuentes 285

Parte 1 · Cómo usar esta guía y el paquete

Paquete para el agente corporativo — cómo usarlo

Este paquete permite que un agente de IA con **acceso limitado** (máximo **20 archivos de entrada** y **5 archivos devueltos** por pedido) revise VIRF, diagnostique fallos y proponga cambios seguros.

Qué contiene

Archivo	Para qué	¿Subirlo siempre?
GUIA_VIRF_AGENTE_CORPORATIVO.pdf	Guía completa (+200 páginas): qué hace cada parte, capturas, catálogo de módulos, endpoints, tests, variables y grafo.	Solo para estudio o dudas generales: ocupa 1 archivo pero es largo.
CONOCIMIENTO_VIRF.md	Resumen operativo: arquitectura, reglas, mapa síntoma → skill.	Sí (1 archivo).
skills/Sxx_*.md	Una «skill» por tipo de cambio: qué archivos subir, qué puede devolver, pasos, tests.	Sí, solo la que aplique (1 archivo).
docs/agente_corporativo/capturas/	Pantallas de la web (también están dentro del PDF).	No.

Cómo armar cada pedido (regla 20 / 5)

1. Sube **CONOCIMIENTO_VIRF.md + la skill** del tipo de cambio → 2 archivos.
2. Sube los archivos de código que la skill indica en «Qué subir» → **máximo 18**. Si el problema no está claro, empieza por **S01 — Diagnosticar un fallo**: te dice qué skill usar y qué archivos pedir.
3. El agente devuelve **como máximo 5 archivos modificados**. Cada skill indica cuáles son los que normalmente cambian («Qué puede devolver»). Si un cambio necesita tocar más de 5, la skill explica cómo **partirlo en varios pedidos** que dejen el sistema funcionando en cada paso.
4. Después de aplicar los archivos devueltos, ejecuta los tests que indica la skill (siempre con el entorno `.venv`, sin llamadas reales a la IA).

Plantilla de pedido

```
Contexto: te adjunto CONOCIMIENTO_VIRF.md y la skill <Sxx>.
Problema: <qué falla o qué quieres cambiar; pega el error o la captura si la hay>.
Archivos adjuntos: <lista>.
Pide: diagnóstico + archivos modificados completos (máx. 5) + tests que debo correr.
Respeta las «Reglas que no se pueden romper» de la skill.
```

Regenerar este paquete

El PDF se genera desde el código (no se escribe a mano):

```
.venv\Scripts\python.exe scripts\generar_guia_agente.py --pdf
```

Ver la skill **S12 — Regenerar la guía y el grafo**.

Parte 2 · Conocimiento VIRF

Conocimiento VIRF — base operativa para el agente corporativo

> *Documento corto y autosuficiente. Súbelo SIEMPRE junto con la skill del cambio. El detalle completo (catálogo de módulos, endpoints, tests, variables, grafo y capturas) está en GUIA_VIRF_AGENTE_CORPORATIVO.pdf.*

1. Qué es VIRF

VIRF es un **agente de validación de modelos de riesgo financiero** (VaR, LaR, ALM…) usado por la Gerencia de Validación Interna de Riesgo Financiero (segunda línea de defensa). Recibe la evidencia de un modelo (manuales, código, datos, actas), la contrasta contra un **estándar de validación** (Excel de ámbitos, controles y actividades) y produce un **informe Word** con la calificación del modelo.

Tiene dos formas de uso:

- **Web** (FastAPI + JavaScript sin framework): el validador sigue 9 pasos guiados, conversa con el agente en un chat de sustentación y descarga el informe. Arranque: `.venv\Scripts\python.exe -m uvicorn web.app:app --port 8000`.
- **Línea de comandos**: `agente.py --prdc …` ejecuta el pipeline completo y emite el entregable regulatorio. La web lo lanza como subproceso.

Y un tercer modo, **Solo Informe**: genera el informe solo con el estándar y observaciones registradas a mano, sin ejecutar el pipeline.

2. Arquitectura en capas

```
Navegador ── web/static/js/app.js · web/templates/index.html · web/static/css/styles.css
   │  HTTP /api/... y WebSocket /ws/...
   ▼
web/routers/*.py      ← endpoints (validan entrada, devuelven JSON o archivos)
   ▼
web/services/*.py     ← lógica de la web (sesión, estándares, chat, informes web, Solo Informe)
   ▼
modulos/*.py          ← núcleo de negocio (motor IA, informes Word, reglas, PII, custodia…)
config/*.py           ← parámetros, rutas, proveedor de IA (ÚNICO lugar que elige proveedor)
motor/Mx_0x_*.py      ← pipeline de lectura de evidencias, prompt, llamada a IA, controles de datos
src/rag/*.py          ← RAG léxico (BM25 + TF-IDF, local, sin embeddings)
agente.py             ← orquestador del pipeline (CLI)
tests/*.py            ← ~1200 tests, todos sin coste (IA simulada)
```

Regla de dependencias: `web/` puede importar `modulos/`, `config/`, `src/`; **modulos/ no importa web/** (el pipeline CLI no tiene web). Por eso la plantilla de informes y la regla de calificación viven en `modulos/`.

3. Datos y rutas

- Sesiones web: `web/data/sesiones/{codigo_validador}/{SLUG_MODELO}/inputs|outputs`.
  - `inputs/documentacion/modelamiento|seguimiento|controles_datos`, `inputs/implementacion` (código del modelo, se descomprime del ZIP), `inputs/calidad_datos/…`, `inputs/automatizaciones` (Excels 00–03), `<sesión>/inputs/estandares/`.
  - `outputs/AAAA-MM-DD/resultados_*.json` (resultado del pipeline), `outputs/VIRF_Informe_*_Vn_*.docx` (informe web versionado), `<sesión>/outputs/capturas/` (pantallazos de evidencia).
- Solo Informe: `web/data/sesiones/{codigo}/_solo_informe/` (estándar, borrador, imágenes, manuales, salida).
- Globales: `inputs/Estandares_VIRF.xlsx`, `inputs/rag/` (literatura por dominio), `inputs/logoempresa/` (logo BCP), `inputs/repositorio_controles/<dominio>/` (controles en Python).
- Credenciales: `.env` en la raíz (**nunca** se edita desde el código ni se sube al agente); plantilla documentada en `.env.example`.

4. Funciones principales y dónde viven

Función	Pantalla	Archivos clave
Login y modelos	Login, Mis Modelos	web/routers/auth.py, modelos.py, web/services/sesion_store.py, modelos_store.py
Paso 01 Configuración (proveedor IA, tier)	Configuración	web/routers/config.py, web/services/env_service.py, config/ia.py, config/parametros.py
Paso 02 Estándares	Estándares	web/routers/estandares.py, web/services/estandares_service.py, motor/Mx_01_LecturaEvidencias.py (leer_estandares_virf)
Paso 03 Evidencia	Evidencia	web/routers/upload.py, web/services/contexto_sesion.py, analisis_codigo.py
Paso 04 Dominio & RAG	Dominio y literatura	web/routers/rag.py, src/rag/*, web/services/dominios_custom_store.py
Pasos 05–06 Skills y cuestionamientos	Skills, Cuestionamientos	web/routers/skills.py, cuestionamientos*.py, modulos/skills/, modulos/cuestionamientos_ext.py
Paso 07 Pre-vuelo	Pre-vuelo	web/routers/dominio_prevuelo.py, modulos/prevuelo.py

Función	Pantalla	Archivos clave
Paso 08 Validar	Validar modelo	web/routers/validacion.py, web/services/pipeline_runner.py, agente.py, modulos/motor_ia.py, motor/*
Paso 09 Documentar (informe web)	Documentar	web/routers/informe.py, web/services/informe_web.py, modulos/plantilla_bcp.py, modulos/capturas.py
Resultados	Resultados	web/routers/resultados.py
Controles de datos / notebook	Controles de datos	web/routers/controles.py, modulos/motores_datos.py, notebook_controles.py, repositorio_controles.py
Chat de sustentación	Chat de validación	web/routers/chat.py, web/services/chat_service.py, modulos/consolidacion.py
Solo Informe (+ IA opcional)	Solo Informe	web/routers/solo_informe.py, web/services/solo_informe_*.py
Informe del pipeline (CLI)	—	modulos/generador_word.py → modulos/plantilla_bcp.py
Calificación (los 3 informes)	—	modulos/reglas_informe.py

5. Los tres informes Word (misma plantilla)

`modulos/plantilla_bcp.py` define la estructura común: portada BCP, encabezado «Página | N» con logo, pie «VALIDACIÓN INTERNA DE RIESGO FINANCIERO · Informe Final», índice con hipervínculos, **I. Objetivos · II. Alcance · III. Cumplimiento de Estándar · IV. Información Relevante (ficha del documento, contexto, limitaciones, fechas, conclusión) · V. Cuadro Resumen de Observaciones · VI. Glosario · VII. Barrido de Controles** y anexos A, B, C…

- Solo Informe → `web/services/solo_informe_word.py` (adaptador fino).
- «Generar informe» de la web → `web/services/informe_web.py` (añade puntaje, evidencia, contradicciones manual↔código, capturas, diálogo del chat; anexos: tabla final, entendimiento, automatizaciones, trazabilidad).
- Entregable del pipeline → `modulos/generador_word.py::generar_word_informe` (anexos: firma, custodia, escala, correcciones, análisis transversales).

6. Regla de calificación (única para los tres)

`modulos/reglas_informe.py::concluir(observaciones)`:

- **Deficiente**: más de 2 críticas.
- **Regular**: 1 o 2 críticas; o más de 1 moderada (umbral 3 o más si hay al menos una moderada de Documentación y una de Gobierno).
- **Aceptable**: 1 moderada (o 2 con la excepción Doc+Gob); o 5 o más bajas/potenciales. Nunca lleva críticas.
- **Satisfactorio**: hasta 4 bajas/potenciales.
- Las Oportunidades de Mejora no cuentan y no dejan el control como incumplido.

Puntaje del agente → riesgo (`riesgo_desde_puntaje`): 0–2 Crítico (§6), 3 Moderado, 4 Oportunidad de Mejora, 5 sin observación, sin puntaje = no evaluada (fallo técnico, no se califica). La falta de evidencia (`sin_evidencia`) es **falta crítica**. En la web, un APROBADO del chat cuenta como subsanado.

7. IA

- Proveedor: `IA_PROVEEDOR` en `.env` (por defecto `azure_openai`; alternativas `anthropic`, `openai`). La selección vive SOLO en `config/ia.py` (`get_cliente_ia`, `cliente_dedicado`). Sin plan B automático entre proveedores (lo vigilan tests).
- VIRF PLUS: GPT + Claude con árbitro (`config/virf_plus.py`).
- Salvaguardas antes de enviar texto al modelo: `modulos/pii.py` (enmascarado de datos personales) y `modulos/inyeccion.py` (neutraliza instrucciones incrustadas).
- Solo Informe: IA opcional (pulir, gráficos, manuales) en `web/services/solo_informe_ia.py`; overrides `SOLO_INFORME_IA_PROVEEDOR`, `SOLO_INFORME_DEPLOYMENT`, `SOLO_INFORME_MODELO`.
- RAG: léxico local (`src/rag/`), **sin HuggingFace, sin embeddings, sin vector store**.

8. Reglas que ningún cambio puede romper

5. Usar `.venv` (el Python del sistema tiene paquetes prohibidos).
6. No editar `.env`; documentar variables nuevas en `.env.example`.
7. Tests sin coste: IA simulada (mocks/`monkeypatch`), nada de llamadas reales.
8. La selección de proveedor de IA solo en `config/`; la palabra «fallback» no puede aparecer en código vivo de `config/ia.py`.
9. `modulos/` no importa `web/`.
10. Sin detección automática de modos: flags explícitos.
11. Sin HuggingFace / ChromaDB / embeddings.
12. La calificación solo en `modulos/reglas_informe.py`.
13. No lanzar POST a ciegas contra la web en marcha (escriben en rutas globales).
14. Toda afirmación del informe debe poder rastrearse a la evidencia (citas verificables).

9. Cómo probar

```
.venv\Scripts\python.exe -m pytest -q                # suite completa (~5 min)
.venv\Scripts\python.exe -m pytest tests/test_X.py -q # un área
```

Tests por área: Solo Informe `test_solo_informe*.py`; plantilla `test_plantilla_bcp.py`; capturas `test_capturas.py`; informes y reglas regulatorias `test_regresion.py`, `test_pendientes_auditoria.py`; proveedor de IA `test_estructura_config.py`, `test_azure_mocks.py`, `test_virf_plus.py`; RAG `test_rag_lexico.py`; estándares `test_estandares_sesion.py`; chat `test_consolidacion.py`, `test_acta_cierre.py`.

10. El grafo del código (graphify)

`graphify-out/` tiene un mapa del código: `GRAPH_REPORT.md` (más de 300 comunidades con nombre en español, nodos centrales), `graph.json` (consultable) y `graph.html` (visual). Úsalo para elegir los archivos que subir:

```
graphify query "cómo se genera el informe word"     # recorre el grafo desde la pregunta
graphify explain "concluir()"                        # vecinos de un nodo y su
archivo:línea
graphify path "ia_pulir()" "cliente_dedicado()"      # camino entre dos funciones
graphify affected "riesgo_desde_puntaje()"           # qué se ve afectado si lo cambias
graphify update .                                    # actualizar tras cambiar código (sin
IA)
```

Limitación: el grafo no une JavaScript con Python (hablan por HTTP). La pantalla llama a `/api/...`; busca ese prefijo en `web/routers/`.

11. Mapa síntoma → skill

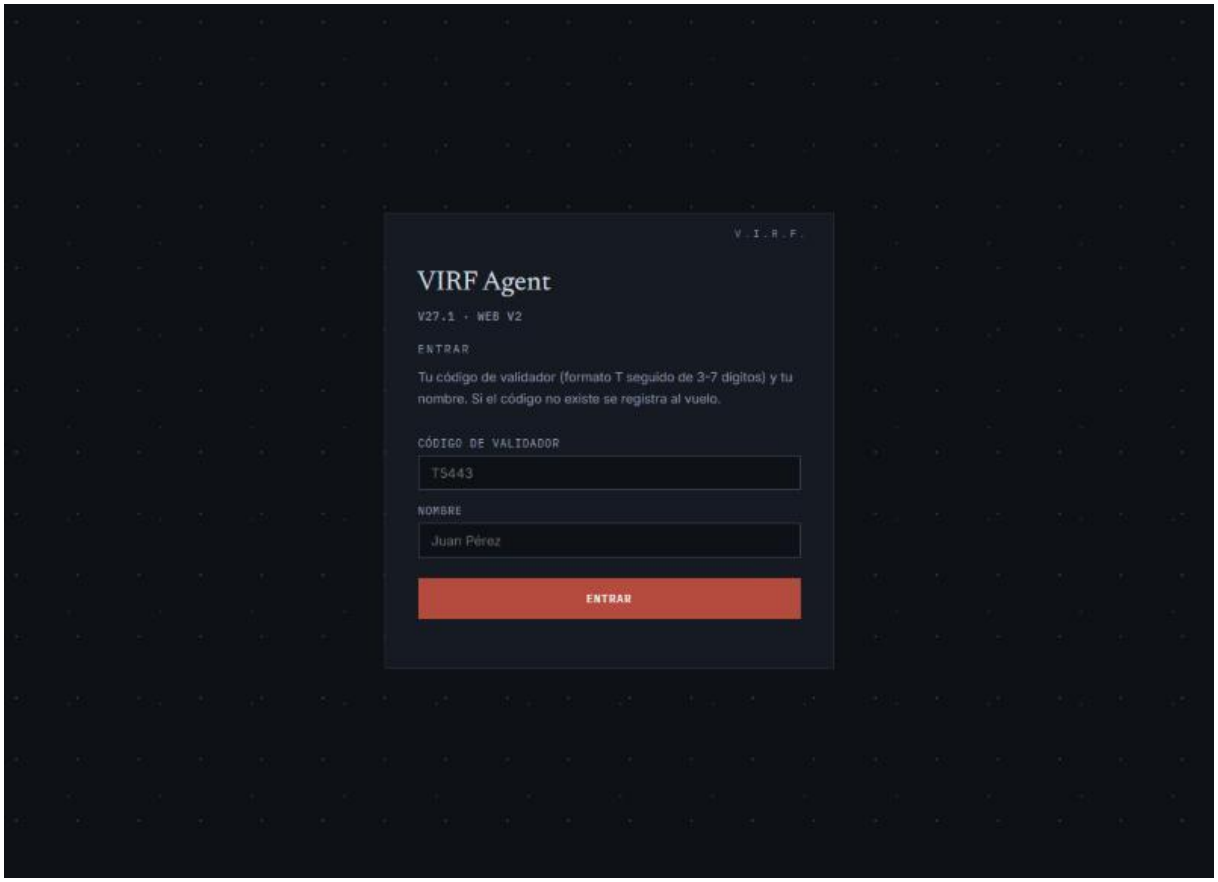
Síntoma	Skill
No sé dónde está el problema / error genérico	S01 Diagnosticar un fallo
El Word sale mal (estructura, estética, contenido, imágenes)	S02 Informes Word
La calificación o el riesgo no es el esperado	S03 Regla de calificación
Una pantalla o botón falla, o quiero una pantalla/endpoint nuevo	S04 Web
Error 401/503, cambiar de modelo o proveedor de IA	S05 Proveedor de IA
El Excel de estándar no se lee o faltan actividades	S06 Estándar de validación
La literatura/RAG no aparece o cita mal	S07 RAG léxico

Síntoma	**Skill**
Controles de calidad de datos / notebook	S08 Calidad de datos
La evidencia no se sube, no se lee o las capturas fallan	S09 Evidencia y capturas
El chat no decide bien o la consolidación es incorrecta	S10 Chat de sustentación
Necesito acotar qué archivos subir	S11 Usar el grafo
Actualizar esta guía o el grafo	S12 Regenerar guía y grafo

Parte 3 · Recorrido por la interfaz

Cada pantalla de la web con su propósito, los endpoints que usa, los archivos que la implementan y la skill que aplica si falla. Las capturas son de la versión actual (escritorio a 1183 px y móvil a 375 px).

Entrada (login)



Captura: Entrada (login)

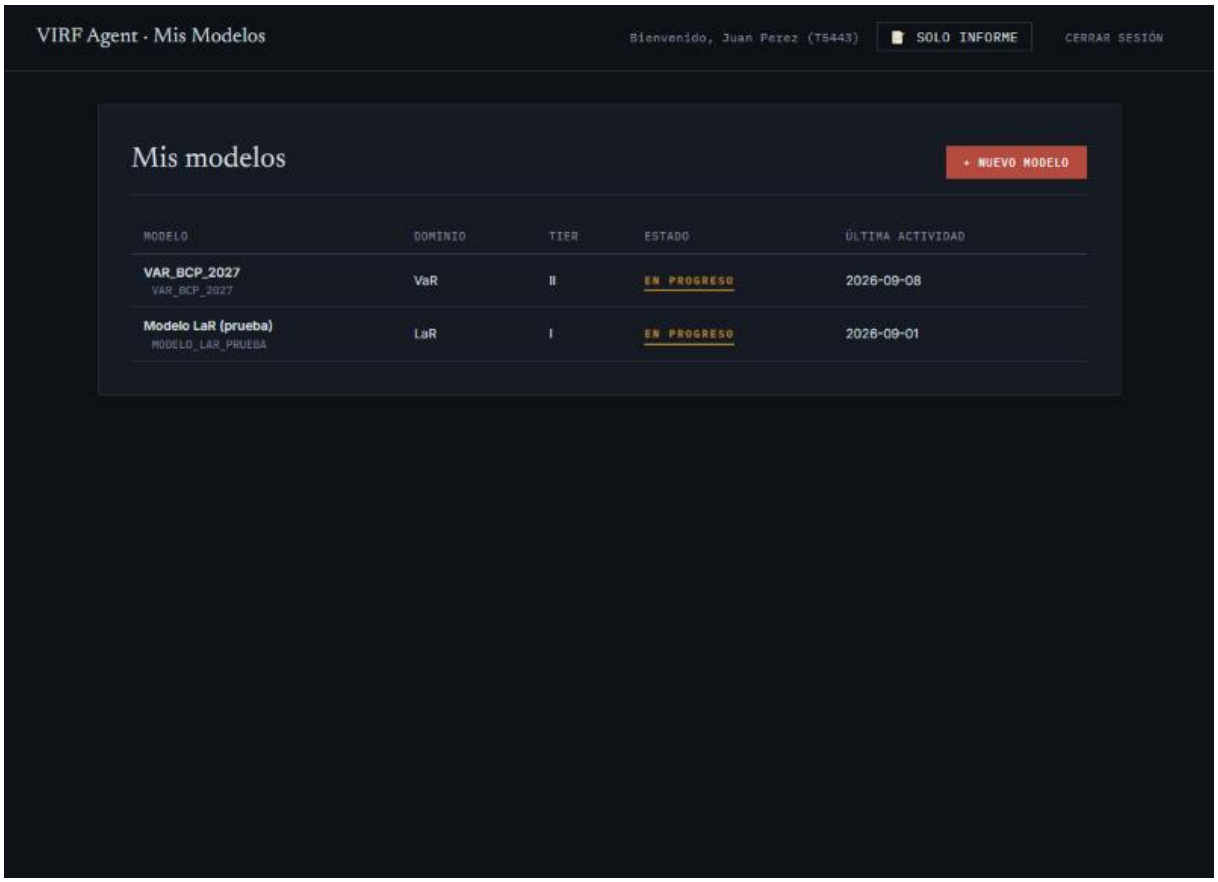
Para qué sirve: El validador entra con su código (T + 3–7 dígitos) y su nombre. Si el código no existe se registra al vuelo. La sesión vive en memoria del servidor: al reiniciarlo hay que volver a entrar.

Endpoints: POST /api/auth/login · GET /api/auth/actual · POST /api/auth/logout

Archivos: web/routers/auth.py · web/services/sesion_store.py · web/static/js/app.js (submitLogin)

Si falla: skill S04 (ver Parte 6)

Mis Modelos



Captura: Mis Modelos

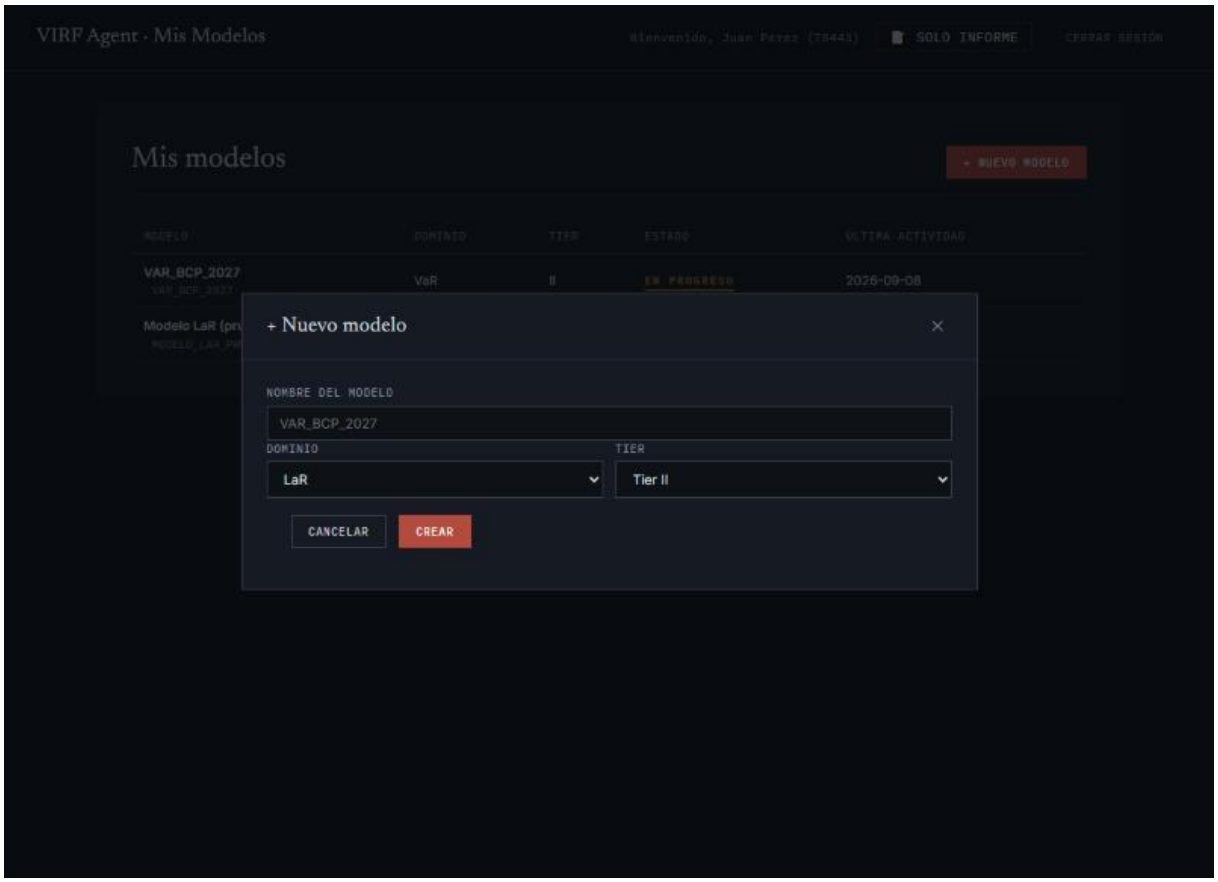
Para qué sirve: Lista los modelos del validador con dominio, tier, estado y última actividad. Una fila abre el expediente del modelo (también con Enter). Desde aquí se entra a «Solo Informe».

Endpoints: GET /api/modelos · POST /api/modelos/{slug}/activar

Archivos: web/routers/modelos.py · web/services/modelos_store.py · app.js (pintarListaModelos)

Si falla: skill S04 (ver Parte 6)

Nuevo modelo



Captura: Nuevo modelo

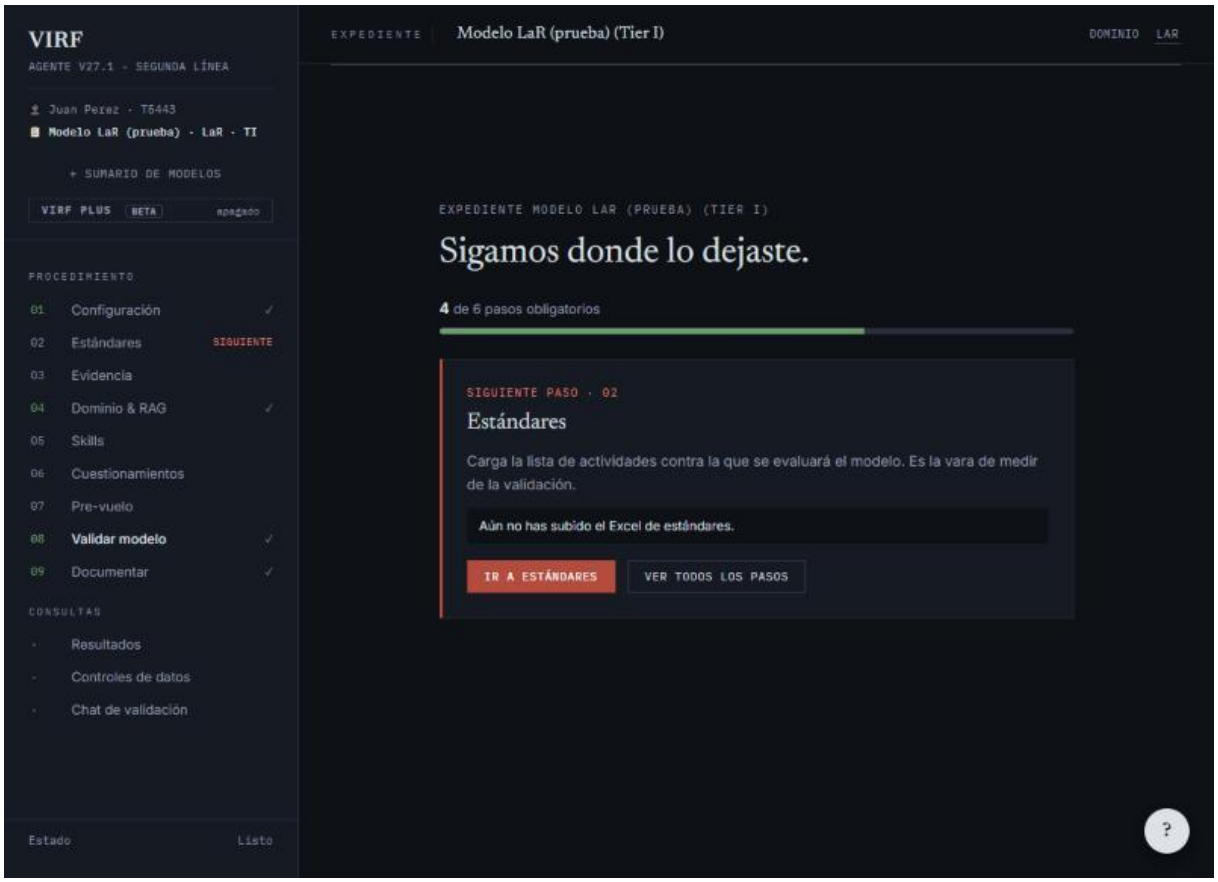
Para qué sirve: Crea un expediente: nombre, dominio (VaR, LaR, ALM o un dominio personalizado) y tier. El dominio decide qué literatura (RAG) se usa y no se cambia después.

Endpoints: POST /api/modelos · GET /api/rag/dominios

Archivos: web/routers/modelos.py · web/services/modelos_store.py · web/services/dominios_custom_store.py

Si falla: skill S04 (ver Parte 6)

Expediente: portada y guía de pasos



Captura: Expediente: portada y guía de pasos

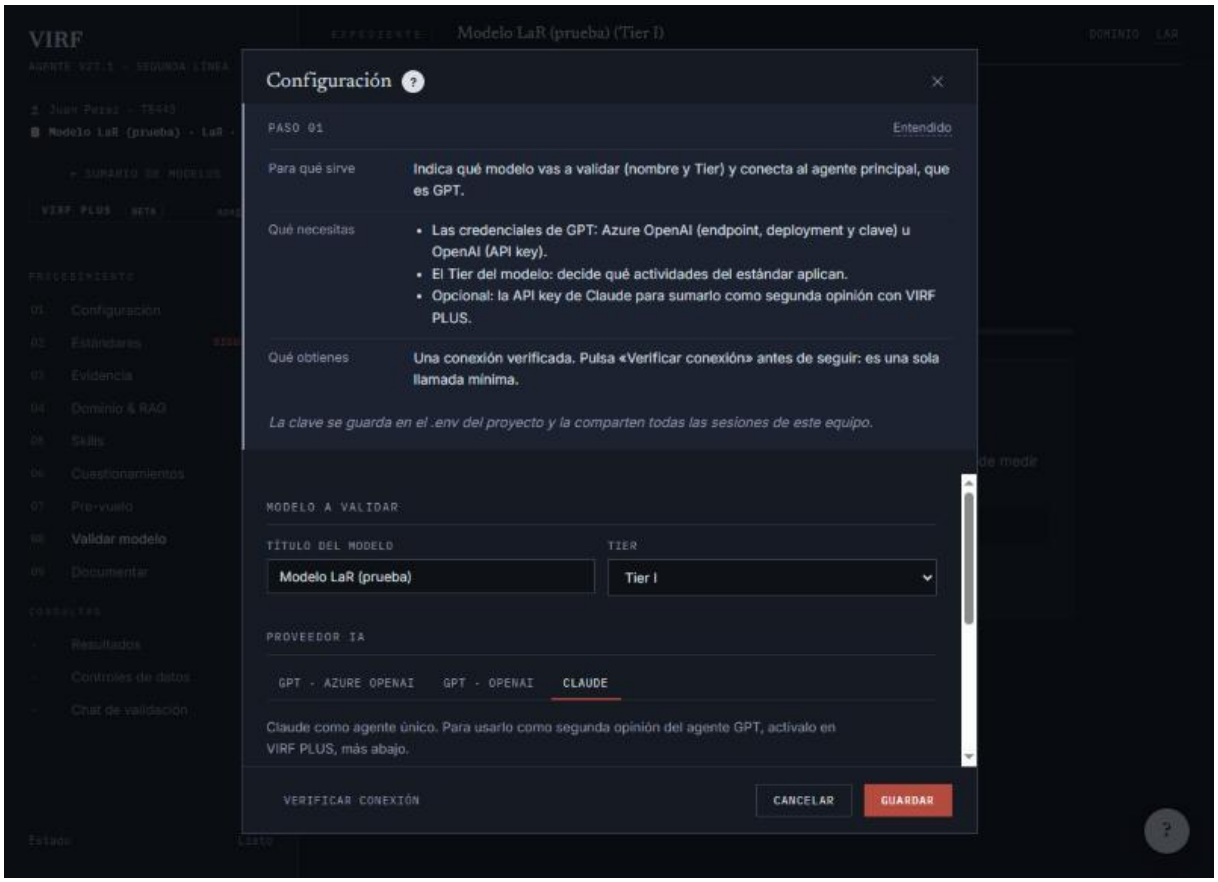
Para qué sirve: Muestra en qué paso está la validación (9 pasos, 6 obligatorios), el siguiente paso y una barra de progreso. El menú lateral lleva a cada paso y a las consultas.

Endpoints: GET /api/guia/progreso

Archivos: web/routers/guia.py · web/static/js/guia.js

Si falla: skill S04 (ver Parte 6)

Paso 01 · Configuración



Captura: Paso 01 · Configuración

Para qué sirve: Título y tier del modelo, proveedor de IA (Azure OpenAI, OpenAI o Claude) y VIRF PLUS. «Verificar conexión» hace una llamada mínima. Las claves se guardan en el .env del proyecto.

Endpoints: GET/POST /api/config · POST /api/config/verificar

Archivos: web/routers/config.py · web/services/env_service.py · config/ia.py · config/parametros.py

Si falla: skill S05 (ver Parte 6)

Paso 02 · Estándares



Captura: Paso 02 · Estándares

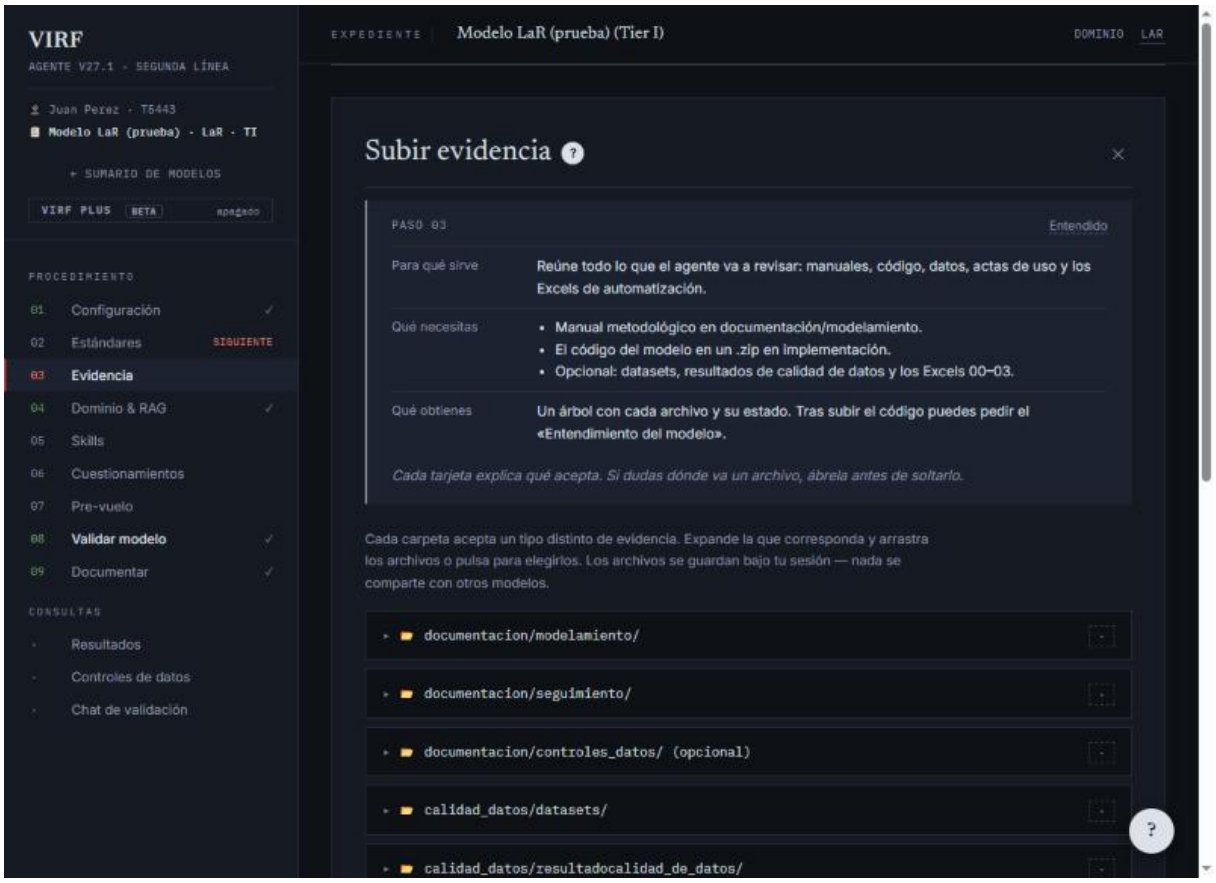
Para qué sirve: Se sube el Estandares_VIRF.xlsx del modelo; se ve una vista previa por ámbito y se confirma. El estándar confirmado se pasa al pipeline con --estandares.

Endpoints: POST /api/estandares/upload · POST /api/estandares/confirmar · GET /api/estandares/estado

Archivos: web/routers/estandares.py · web/services/estandares_service.py

Si falla: skill S06 (ver Parte 6)

Paso 03 · Evidencia



Captura: Paso 03 · Evidencia

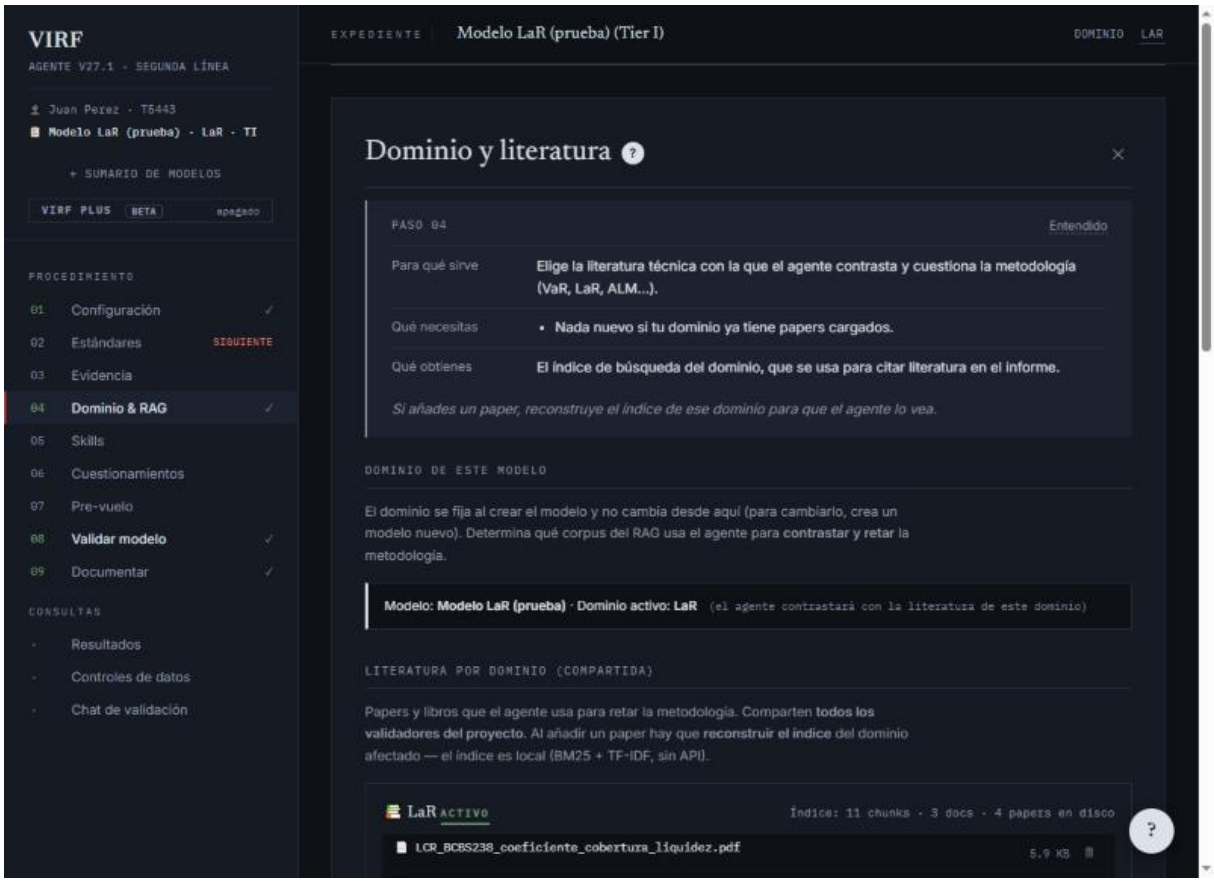
Para qué sirve: Carpetas por tipo de evidencia: documentación (modelamiento, seguimiento, controles de datos), datasets, resultados de calidad de datos, implementación (ZIP del código), usos, Excels de automatización. Tras subir el código se puede pedir el «Entendimiento del modelo».

Endpoints: GET /api/inputs/arbol · POST /api/inputs/upload · POST /api/inputs/upload-zip · /api/entendimiento/*

Archivos: web/routers/upload.py · web/services/contexto_sesion.py · web/services/analisis_codigo.py

Si falla: skill S09 (ver Parte 6)

Paso 04 · Dominio y literatura



Captura: Paso 04 · Dominio y literatura

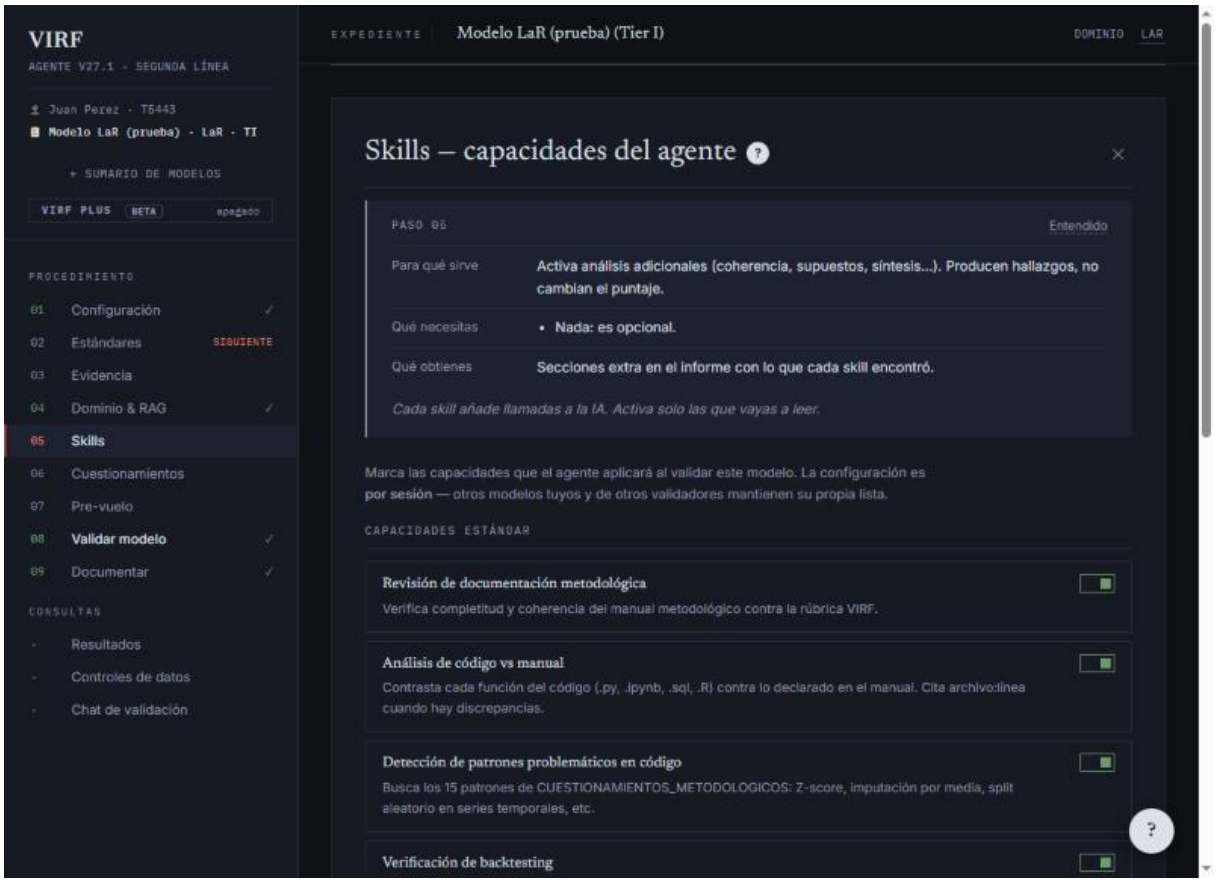
Para qué sirve: Muestra el dominio del modelo y la literatura técnica por dominio (compartida por todos los validadores). Se pueden subir papers y reconstruir el índice léxico (BM25 + TF-IDF, local).

Endpoints: GET /api/rag/dominios · POST /api/rag/upload · POST /api/rag/rebuild

Archivos: web/routers/rag.py · src/rag/* · web/services/dominios_custom_store.py

Si falla: skill S07 (ver Parte 6)

Paso 05 · Skills (capacidades)



Captura: Paso 05 · Skills (capacidades)

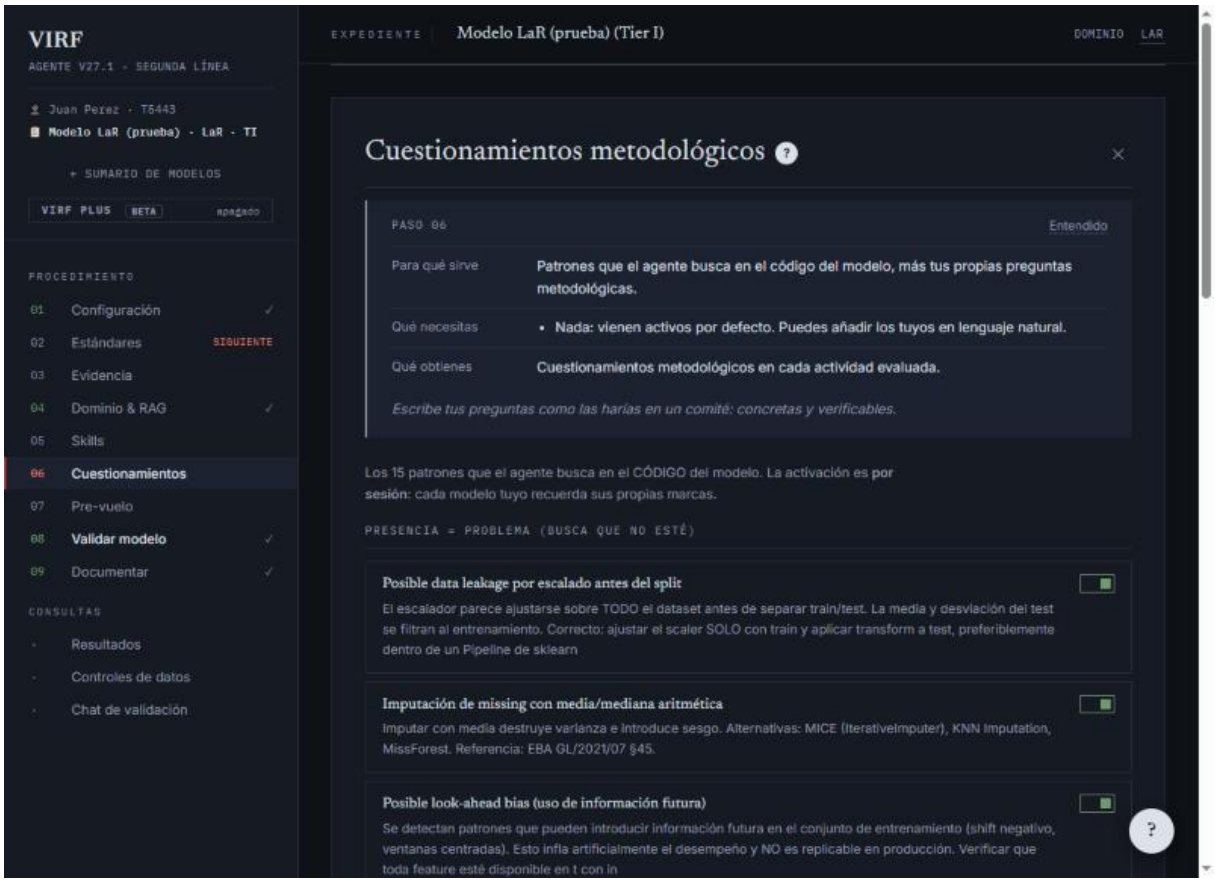
Para qué sirve: Análisis adicionales (coherencia documental, código vs manual, patrones problemáticos, backtesting, síntesis…). Producen hallazgos, no cambian el puntaje. Configuración por sesión.

Endpoints: GET/POST /api/skills

Archivos: web/routers/skills.py · web/services/skills_store.py · modulos/skills/

Si falla: skill S04 (ver Parte 6)

Paso 06 · Cuestionamientos metodológicos



Captura: Paso 06 · Cuestionamientos metodológicos

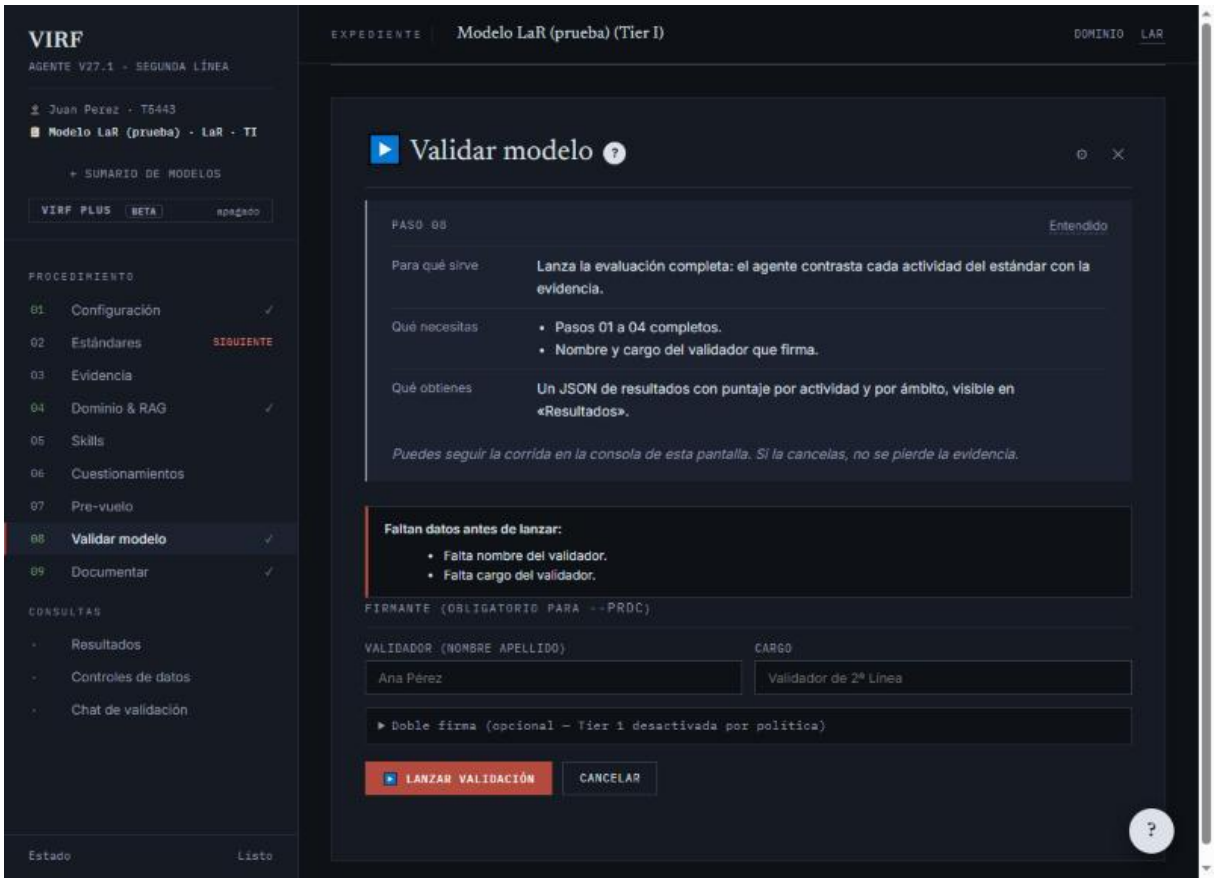
Para qué sirve: 15 patrones que el agente busca en el código del modelo (data leakage, imputación por media, look-ahead bias…) más preguntas propias del validador en lenguaje natural.

Endpoints: GET/POST /api/cuestionamientos*

Archivos: web/routers/cuestionamientos.py · cuestionamientos_sesion.py · modulos/cuestionamientos_ext.py

Si falla: skill S04 (ver Parte 6)

Paso 08 · Validar modelo



Captura: Paso 08 · Validar modelo

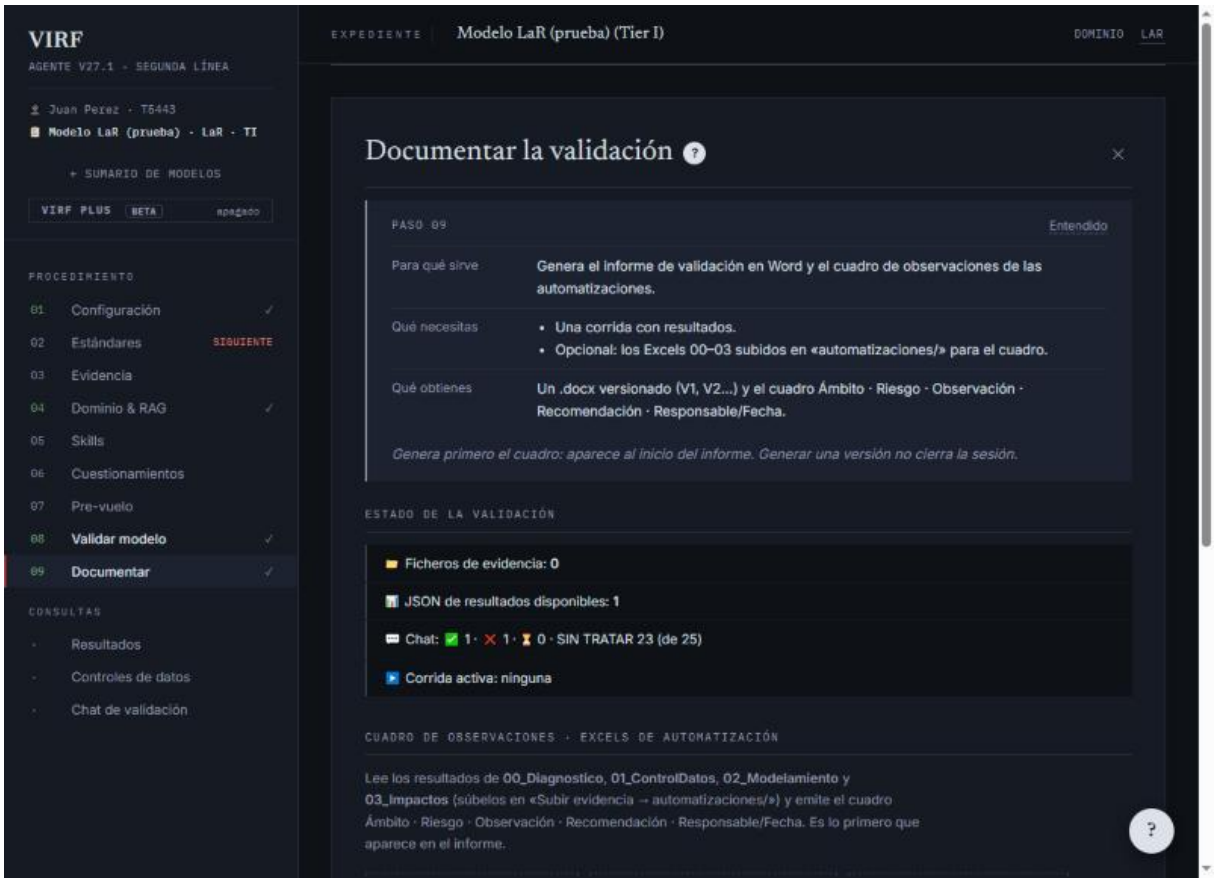
Para qué sirve: Lanza el pipeline (agente.py --prdc) como subproceso y muestra su consola en vivo. Exige nombre y cargo del validador firmante (doble firma opcional). Al terminar deja un JSON de resultados.

Endpoints: POST /api/validar/lanzar · GET /api/validar/estado · WS /ws/validar/{run_id}

Archivos: web/routers/validacion.py · web/services/pipeline_runner.py · agente.py · modulos/motor_ia.py

Si falla: skill S01 (ver Parte 6)

Paso 09 · Documentar



Captura: Paso 09 · Documentar

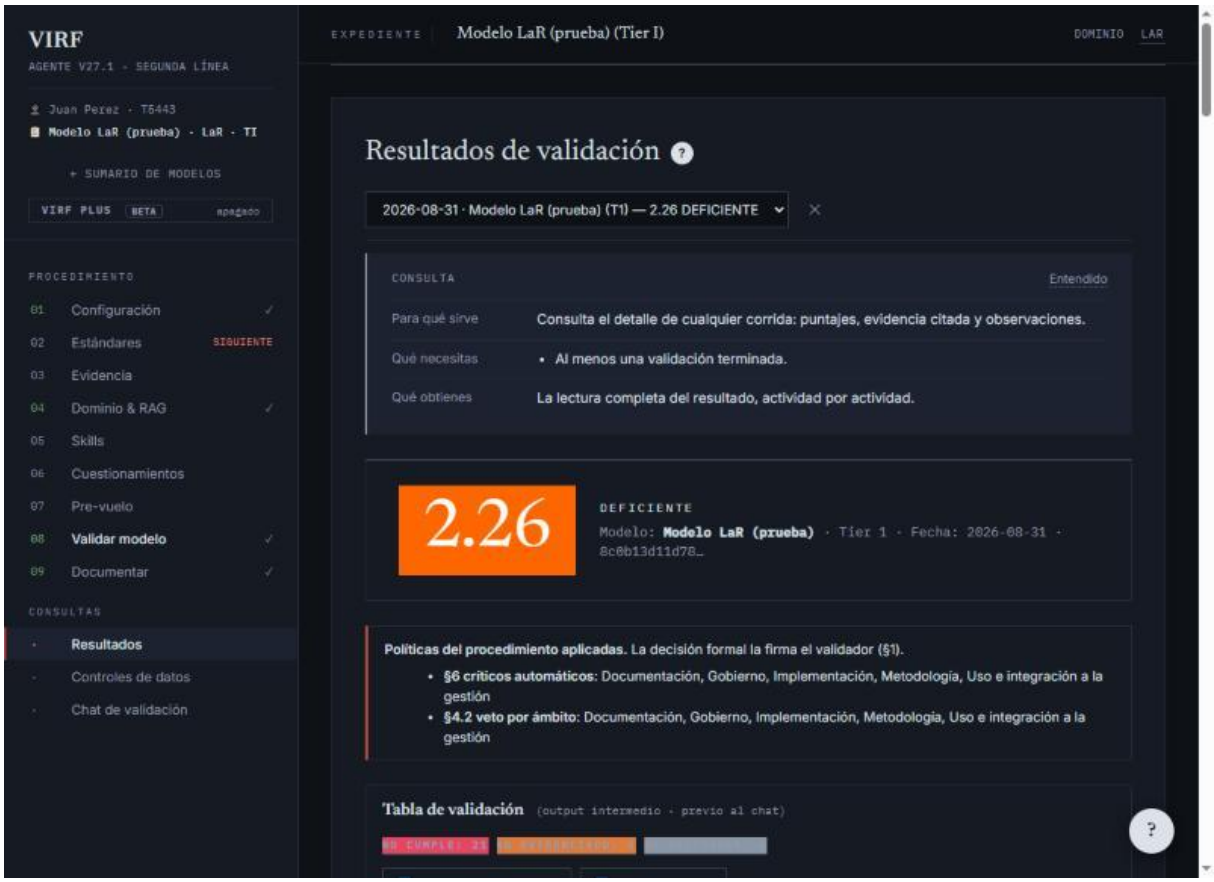
Para qué sirve: Estado de la validación (evidencia, JSON, chat, corrida), cuadro de observaciones de los Excels de automatización y botón para generar el informe Word versionado (V1, V2…).

Endpoints: GET /api/informe/estado · POST /api/informe/consolidado · POST /api/automatizaciones/cuadro

Archivos: web/routers/informe.py · web/services/informe_web.py · modulos/plantilla_bcp.py · modulos/capturas.py

Si falla: skill S02 (ver Parte 6)

Consulta · Resultados



Captura: Consulta · Resultados

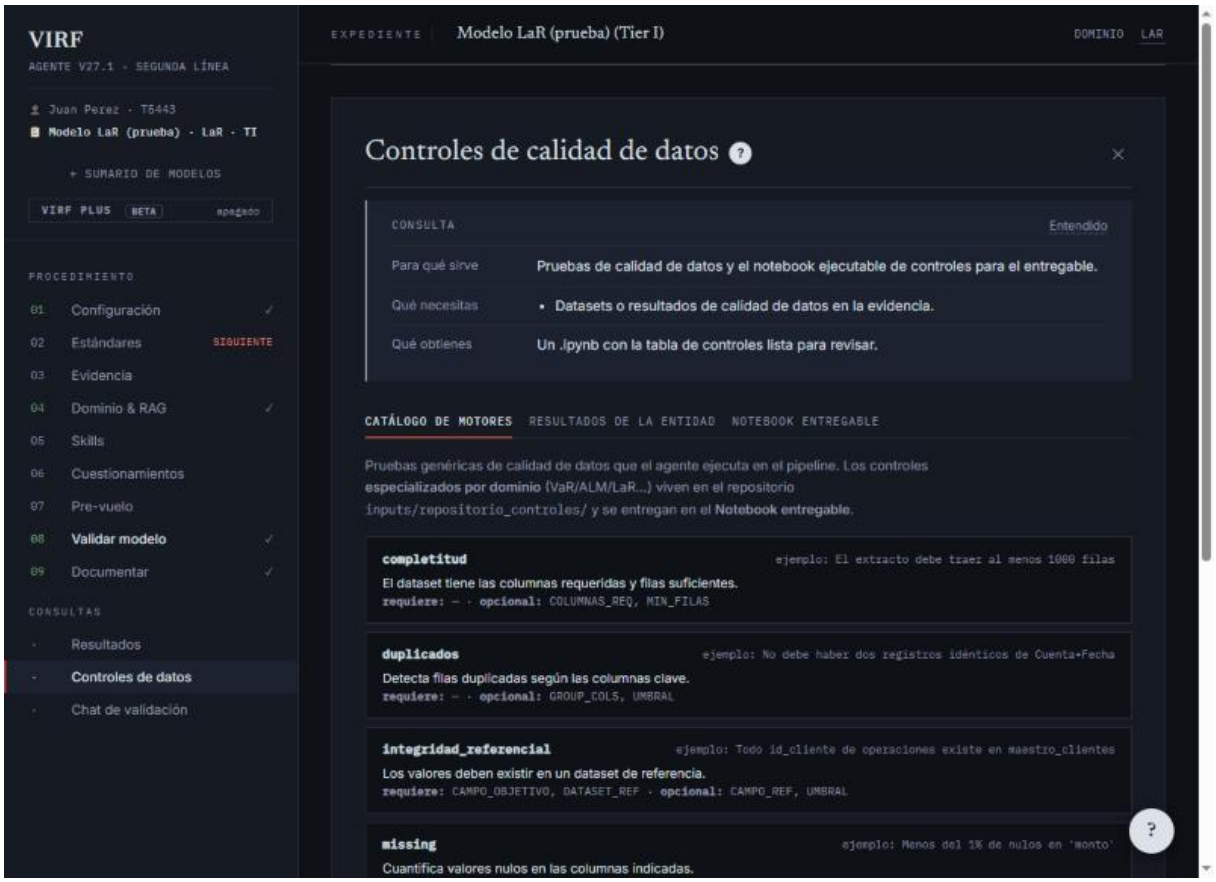
Para qué sirve: Lectura de una corrida: puntaje global y semáforo, políticas aplicadas (§6 críticos, §4.2 veto), tabla de validación intermedia y el detalle por actividad con la evidencia citada.

Endpoints: GET /api/resultados/actual · GET /api/resultados/lista

Archivos: web/routers/resultados.py · modulos/tabla_validacion (datos del JSON)

Si falla: skill S01 (ver Parte 6)

Consulta · Controles de datos



Captura: Consulta · Controles de datos

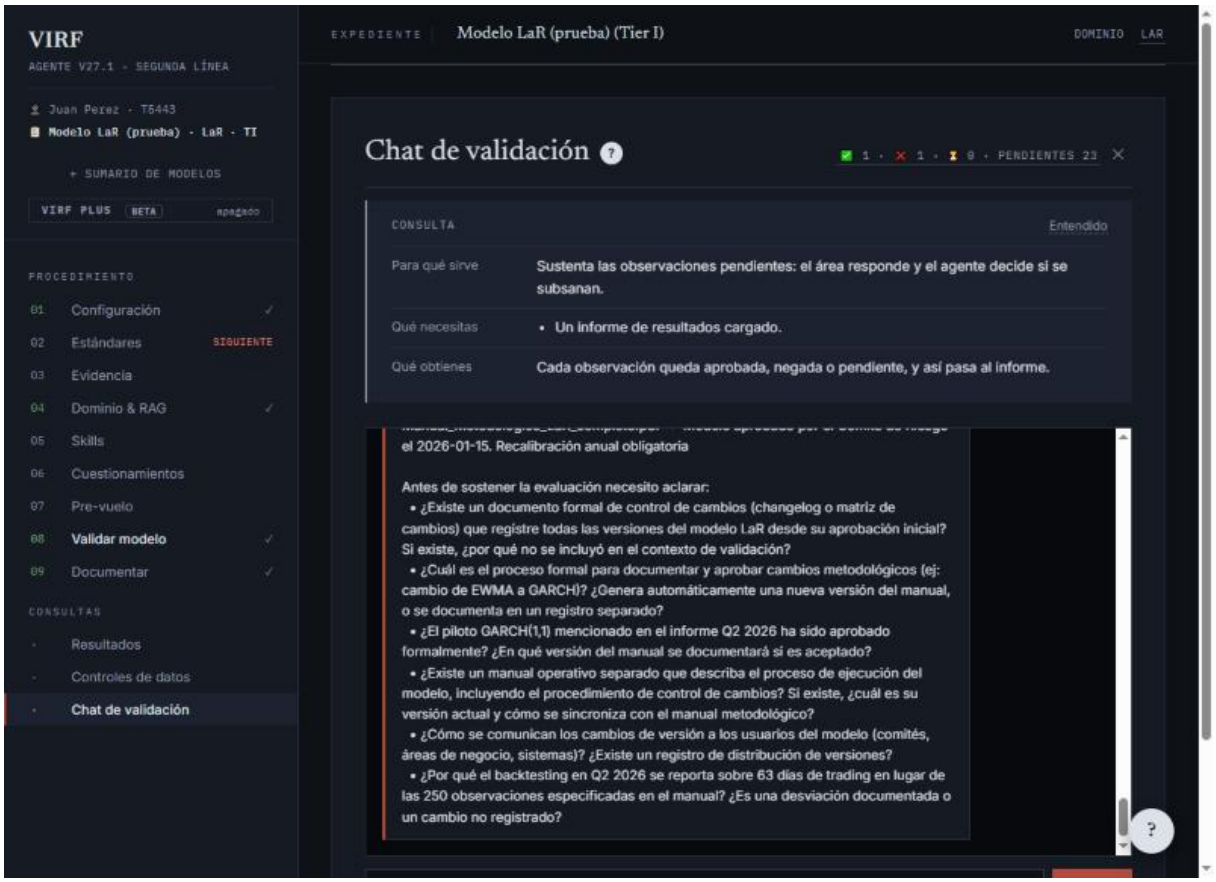
Para qué sirve: Catálogo de motores de calidad de datos, resultados de la entidad y el notebook ejecutable de controles para el entregable.

Endpoints: GET /api/motores/* · GET /api/motores/notebook

Archivos: web/routers/controles.py · modulos/motores_datos.py · modulos/notebook_controles.py

Si falla: skill S08 (ver Parte 6)

Consulta · Chat de validación



Captura: Consulta · Chat de validación

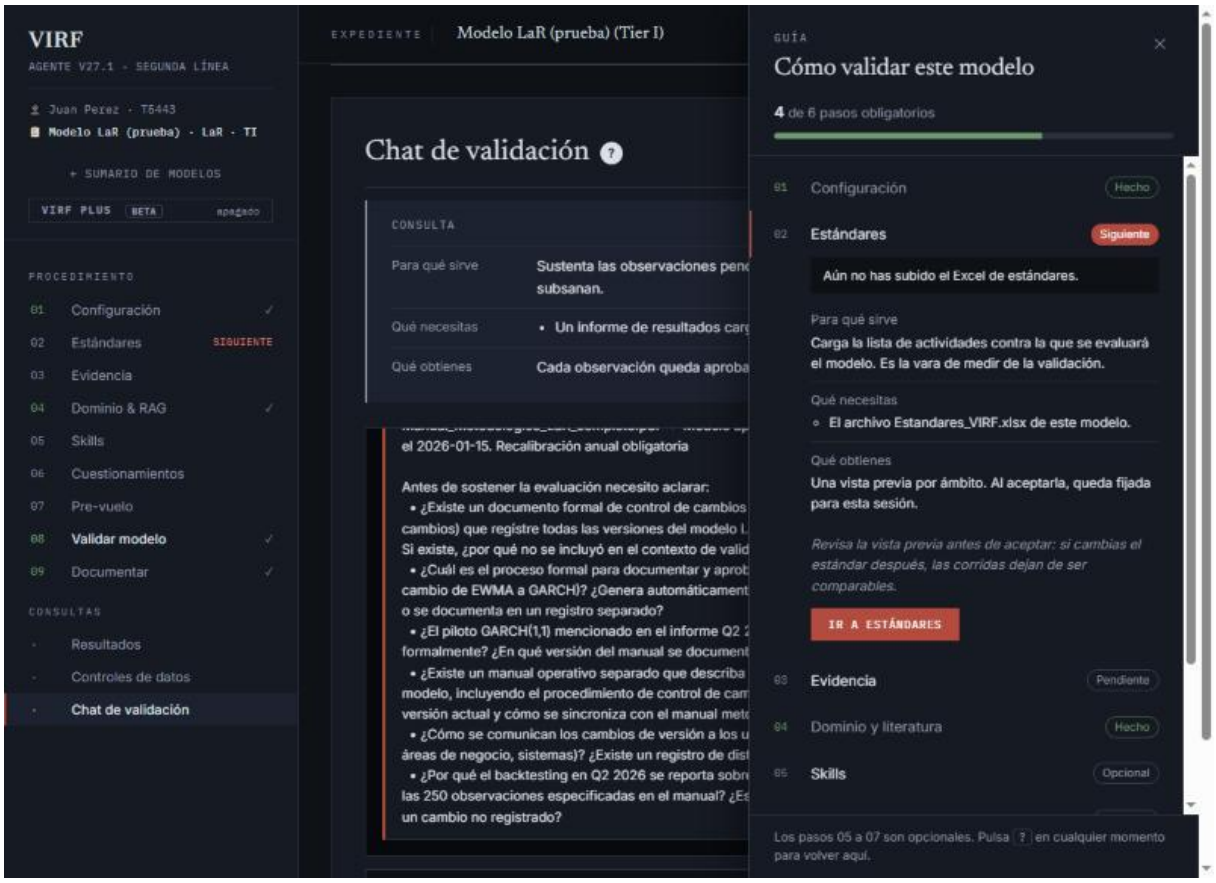
Para qué sirve: El área sustenta las observaciones; el agente propone y el validador cierra cada una como APROBADO, NEGADO o PENDIENTE. Se puede adjuntar evidencia nueva. Al consolidar se recalcula el puntaje final y se descarga el acta de cierre.

Endpoints: POST /api/chat/iniciar · WS /ws/chat · POST /api/chat/consolidar · GET /api/chat/acta.xlsx

Archivos: web/routers/chat.py · web/services/chat_service.py · modulos/consolidacion.py

Si falla: skill S10 (ver Parte 6)

Guía de ayuda (botón «?»)



Captura: Guía de ayuda (botón «?»)

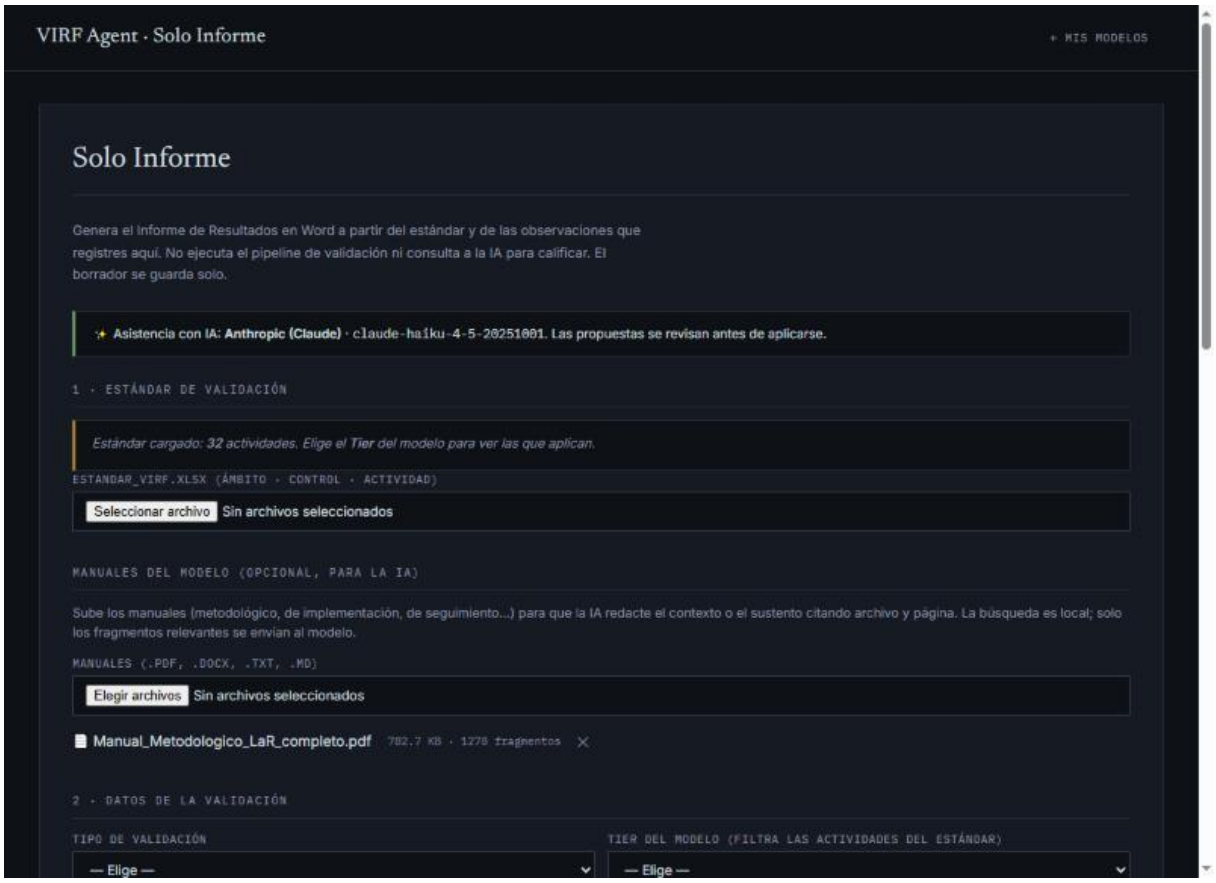
Para qué sirve: Panel lateral con los pasos, su estado (hecho, siguiente, pendiente, opcional) y qué hace falta en cada uno.

Endpoints: GET /api/guia/progreso

Archivos: web/static/js/guia.js · web/routers/guia.py

Si falla: skill S04 (ver Parte 6)

Solo Informe · estándar, IA y manuales



Captura: Solo Informe · estándar, IA y manuales

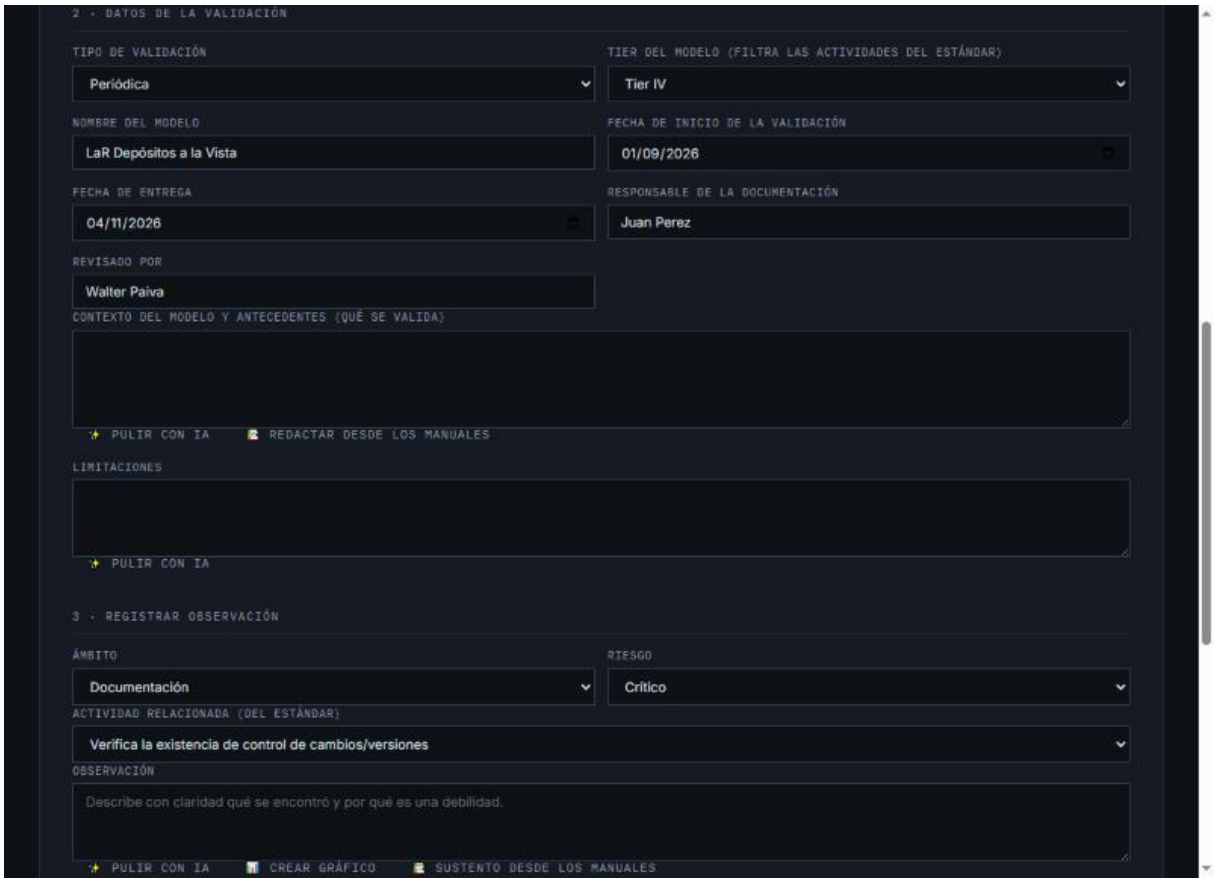
Para qué sirve: Genera el Informe de Resultados SIN ejecutar el pipeline. Muestra el proveedor de IA activo, el estándar cargado (filtrable por Tier) y los manuales del modelo para que la IA cite archivo y página.

Endpoints: GET /api/solo-informe/estado · POST /api/solo-informe/estandar · /api/solo-informe/manuales

Archivos: web/routers/solo_informe.py · web/services/solo_informe_service.py · solo_informe_ia.py

Si falla: skill S02 (ver Parte 6)

Solo Informe · datos de la validación



Captura: Solo Informe · datos de la validación

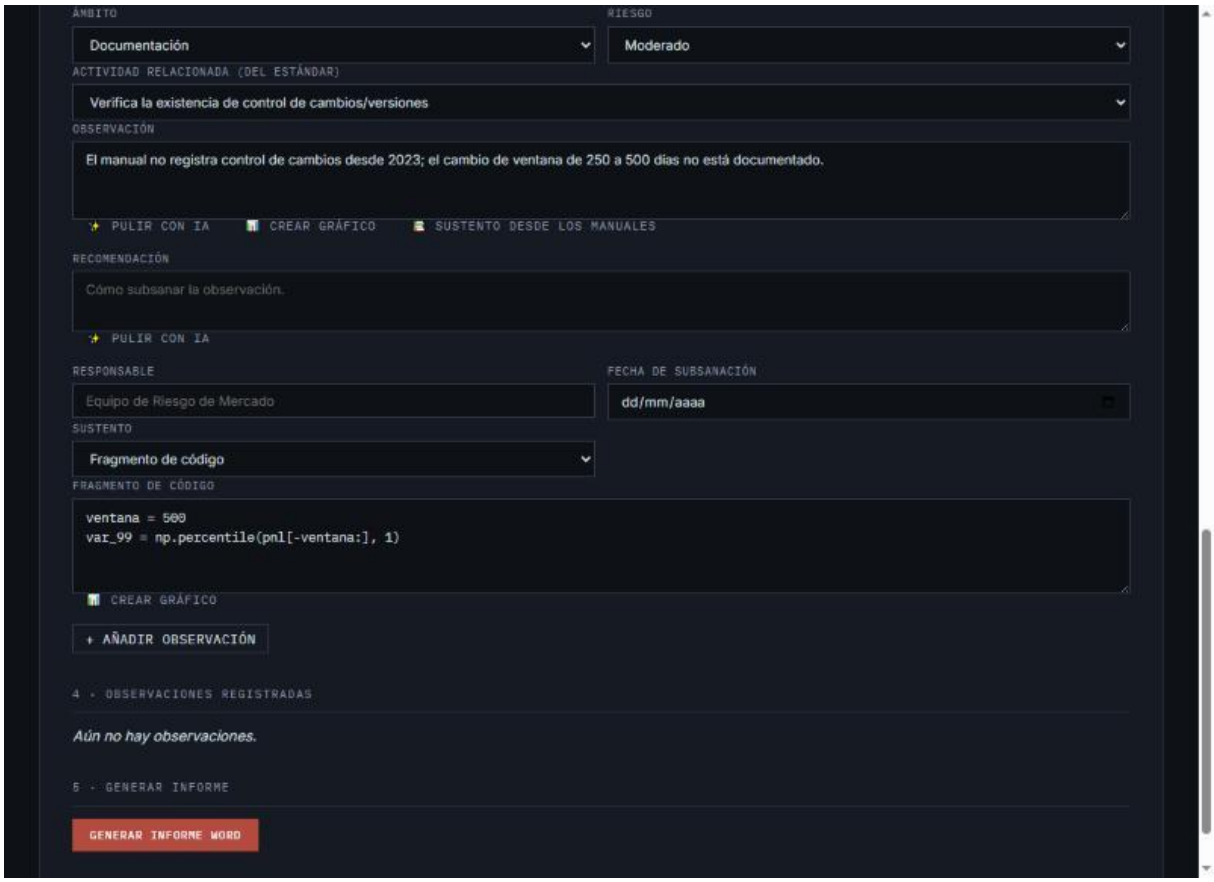
Para qué sirve: Tipo (periódica o primera vez), tier, nombre del modelo, fechas, responsable de la documentación, revisor, contexto y limitaciones; cada texto con «Pulir con IA» y «Redactar desde los manuales».

Endpoints: PUT /api/solo-informe/borrador · POST /api/solo-informe/ia/pulir · /ia/manuales

Archivos: web/static/js/app.js (bloque Solo Informe) · web/services/solo_informe_ia.py

Si falla: skill S02 (ver Parte 6)

Solo Informe · registrar observación



Captura: Solo Informe · registrar observación

Para qué sirve: Ámbito, riesgo, actividad del estándar, observación, recomendación, responsable/fecha (no para Oportunidad de Mejora) y sustento (imagen, código o texto). La IA puede crear un gráfico de textos largos. «Generar informe Word» aplica la regla de calificación.

Endpoints: POST /api/solo-informe/imagen · /ia/grafico · POST /api/solo-informe/generar

Archivos: web/services/solo_informe_word.py · modulos/reglas_informe.py · modulos/plantilla_bcp.py

Si falla: skill S03 (ver Parte 6)

Móvil · Mis Modelos



Captura: Móvil · Mis Modelos

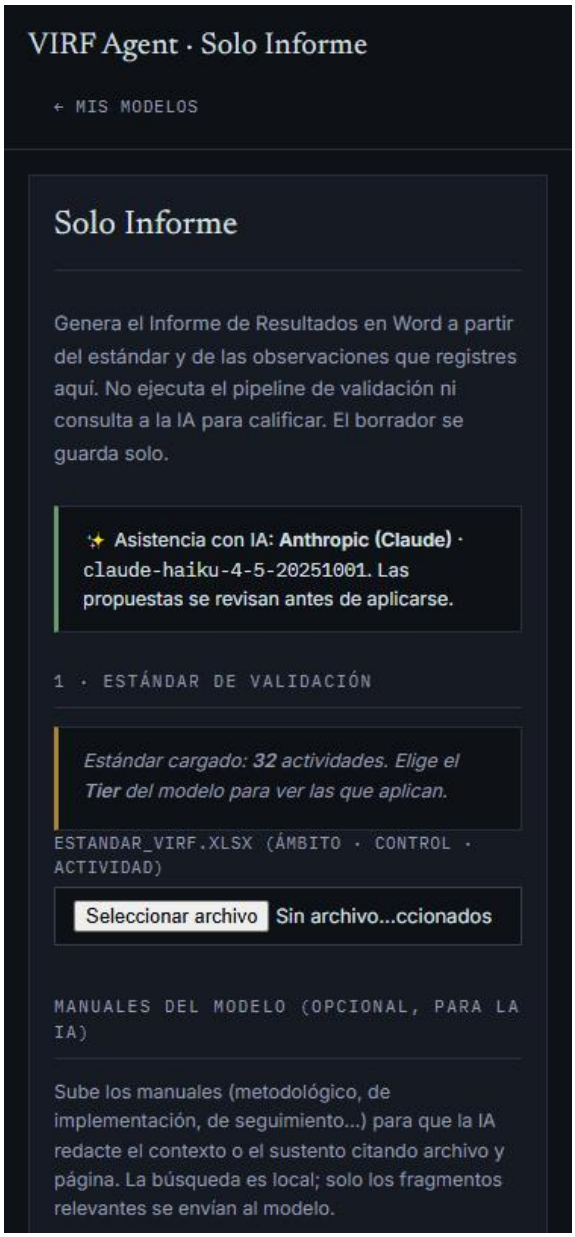
Para qué sirve: A 375 px la cabecera se parte en líneas, la tabla muestra modelo, dominio y estado, y el foco de teclado se ve como un anillo mostaza.

Endpoints: —

Archivos: web/static/css/styles.css (@media max-width: 760px)

Si falla: skill S04 (ver Parte 6)

Móvil · Solo Informe



Captura: Móvil · Solo Informe

Para qué sirve: Formulario en una sola columna; las tablas anchas se desplazan dentro de su recuadro.

Endpoints: —

Archivos: web/static/css/styles.css

Si falla: skill S04 (ver Parte 6)

Móvil · Expediente



Captura: Móvil · Expediente

Para qué sirve: El menú lateral pasa a barra horizontal desplazable; el contenido usa todo el ancho.

Endpoints: —

Archivos: web/static/css/styles.css

Si falla: skill S04 (ver Parte 6)

Parte 4 · Flujo de una validación, extremo a extremo

FLUJO DEL AGENTE — VIRF Agent (extremo a extremo)

> *Qué es este documento. Traza el recorrido completo de una validación, de la entrada hasta la entrega del .docx sellado, en las 4 fases que pidió el validador: (a) cuándo se dialoga con el agente, (b) cuándo replantea/autocorrige, (c) cuándo resuelve —concluye o se abstiene—, (d) cuándo entrega el producto final. Regla de honestidad. El agente está a medio construir. Se distingue SIEMPRE entre: - ACTUAL — lo que ya corre hoy, con evidencia archivo:línea leída en el código. - OBJETIVO — lo que quedará cuando se encienda el día de API (coordinador ReAct + skills tipadas + NLI + abstención cableada). Se marca como pendiente; no se dibuja como si existiera. Baseline git: caf3beb (hito *Skills-1*). API-free: este documento no ejecutó ninguna corrida ni gastó crédito; toda afirmación ACTUAL está anclada a archivo:línea. Documento de solo lectura: no se modificó código de producción.*

1. Resumen en prosa — las 4 fases

(a) Cuándo se dialoga con el agente — ACTUAL. El diálogo es de **arranque**, no conversacional-agéntico. Una validación entra por dos bocas: la **web** (el validador completa los pasos 1–8 del asistente y pulsa «Validar»; el navegador abre un WebSocket `/ws/validar/{runId}` — app.js:1669 (web/static/js/app.js) — y el backend lanza el CLI por subprocess — pipeline_runner.py:255 (web/services/pipeline_runner.py)), o el **CLI directo** `python agente.py --prdc …` — agente.py:305 (agente.py). En ambos casos el "diálogo" es la fijación de parámetros ANTES de gastar: título, tier, **dominio RAG declarado** (`--dominio-rag`), validador(es) y estándar. Hay además un **chat post-informe** (`/ws/chat`, app.js:1227 (web/static/js/app.js)) para preguntar sobre el resultado ya emitido. Lo que **no** existe hoy es un bucle donde el agente pida aclaraciones a mitad de razonamiento: eso es OBJETIVO (coordinador ReAct, día de API).

(b) Cuándo replantea / autocorrige — HOY casi nada; es sobre todo OBJETIVO. Hoy la única "autocorrección" es de **contrato**: si el modelo devuelve un JSON inválido, se reintenta el parseo un número acotado de veces (`INTENTOS_CONTRATO`, parametros.py:254 (config/parametros.py)), y las citas se **verifican literalmente** pero solo se **declaran** dudosas, no se re-piensa la respuesta (motor_ia.py:513-515 (modulos/motor_ia.py)). El "cambia de opinión" del enunciado —un verificador que invalida una afirmación y fuerza un nuevo intento, con cap de iteraciones y fusibles— está **construido pero NO cableado**: `fusibles.py` (`MAX_ITER=3`, `FUSIBLE_SEG=10`, `FUSIBLE_TOKENS=20000`) existe, pero parametros.py:264 (config/parametros.py) admite que activarlo "no altera nada" hoy porque **no hay bucle agéntico** que lo consuma. Ese bucle es OBJETIVO (SK-05).

(c) Cuándo resuelve — concluye (ACTUAL) o se abstiene (OBJETIVO parcial). El agente **concluye** hoy convirtiendo el score de cada ámbito en un **semáforo** de 5 niveles (SATISFACTORIO → CRÍTICO, dominio.py:104 (config/dominio.py)) y un score global, más un flag `dictamen_confiable` que baja si hubo fallos técnicos (agente.py:1840-1843 (agente.py)). La **abstención activa formal** —responder «EVIDENCIA INSUFICIENTE» cuando el retrieval es débil, con umbral β=0.19— está **implementada y probada** en abstencion.py (modulos/abstencion.py), pero **NO la invoca el pipeline --prdc**: hoy la falta de evidencia se refleja como `sin_evidencia`/score 0.0 por actividad, no como un veredicto de abstención. Cablear β es OBJETIVO.

(d) Cuándo entrega el producto final — ACTUAL. Al terminar los ámbitos (y, si se pidieron, las skills), el agente consolida un `payload` que se valida contra el contrato Pydantic `ValidationReport` (esquemas.py (modulos/esquemas.py)), genera el informe **.docx** con `generar_word_informe` (modulos/generador_word.py) y **sella la cadena de custodia** por hash. Las skills corren **antes** del sellado a propósito, para que sus hallazgos queden cubiertos por el hash (agente.py:1857-1863 (agente.py)). El `.docx` usa hoy la escala de semáforo; la tríada formal **APROBADO/PENDIENTE/NO APROBADO** por ámbito es OBJETIVO.

2. Flujo ACTUAL (lo que corre hoy), paso a paso

Cada paso lleva su evidencia `archivo:línea`.

15. **Entrada y diálogo de arranque.**
    - Web: el asistente reúne sesión/modelo, estándares (paso 02, `estandares_service.parsear()`), upload/entendimiento y prompts; al validar, abre WS `/ws/validar/{runId}` (app.js:1669 (web/static/js/app.js)); el servidor lo atiende en validacion.py:74 (web/routers/validacion.py) y delega en el runner.
    - El runner **construye los args** y lanza `agente.py --prdc` por subprocess (pipeline_runner.py:151-177, :255 (web/services/pipeline_runner.py)), inyectando en el entorno el presupuesto de contexto del deployment y el override del rol si el validador lo editó (pipeline_runner.py:69-110 (web/services/pipeline_runner.py)).
    - CLI directo: `main()` con argparse (agente.py:305 (agente.py)). El **dominio RAG se declara** con `--dominio-rag`; sin el flag se **infiere por heurística del título** (que puede enrutar al corpus equivocado) — motor_ia.py:180-189 (modulos/motor_ia.py). El informe registra si el dominio fue *declarado* o *inferido*.
16. **Orquestación — secuencial, sin coordinador.** `agente.py` recorre los ámbitos, los evalúa y opcionalmente ejecuta skills, todo en orden fijo. Conviven dos pipelines: **Pipeline 1** por control de datos (`--control`, `motor/Mx_0x`) y **Pipeline 2** por ámbito (motor_ia.py (modulos/motor_ia.py)), que es el que la web usa por defecto. `resultados_ambitos[ambito]` se va poblando por ámbito (agente.py:1315 (agente.py)).
17. **Recuperación de evidencia — RAG léxico por dominio.** Por actividad, `_enriquecer_con_rag` (motor_ia.py:168 (modulos/motor_ia.py)) consulta el índice del dominio (BM25 + TF-IDF, buscador_lexico.py (src/rag/buscador_lexico.py)). Dos controles **nuevos y ya cableados** actúan dentro de `buscar`:
    - **Versionado**: `versionado.chunk_activo` descarta chunks marcados `is_active=False` (normativa derogada) — buscador_lexico.py:451-463 (src/rag/buscador_lexico.py).
    - **Aislamiento multi-tenant**: hard pre-filter **opcional** por `(tenant, model_id)`, adicional al de dominio, con aserción anti-contaminación — buscador_lexico.py:556-565 (src/rag/buscador_lexico.py). Es opt-in: si el llamante no pasa `model_id`, el comportamiento es el histórico.
18. **Evaluación por ámbito.** Se arma el prompt (evidencia + RAG enmarcado), se llama al modelo y se parsea el JSON con rescate acotado. El score de cada ámbito sale de `calcular_score_ambito` (agente.py:1315, :1412 (agente.py)).
19. **Verificación de citas — determinista y literal.** `_verificar_citas` comprueba que cada cita entrecomillada de >40 caracteres **exista textualmente** en el contexto (motor_ia.py:504 (modulos/motor_ia.py)). Clave:

declara, no invalida — una cita dudosa se registra en el JSON de auditoría para el revisor humano, pero **no tumba** la evaluación (motor_ia.py:513-515 (modulos/motor_ia.py)).

20. **Skills — análisis transversales prompt-based (opcionales).** Si se pasó `--skills`, `ejecutar_skills` corre las skills del catálogo actual (`modulos/skills/`, salida `ResultadoSkill(dict)`) sobre el contexto deduplicado y la literatura RAG regulatoria (agente.py:1952 (agente.py)). **No alteran el score** a propósito, para que dos corridas con distinta selección de skills sigan siendo comparables (agente.py:1862-1864 (agente.py)).
21. **Resolución — semáforo + dictamen de confianza.** `score_global = calcular_score_global(...)` y `semaforo = get_semaforo(...)` (agente.py:1682-1683 (agente.py)); las bandas son SATISFACTORIO/ACEPTABLE/CON OBSERVACIONES/DEFICIENTE/CRÍTICO (dominio.py:104 (config/dominio.py)). Se computa `dictamen_confiable` (falso si hubo fallos técnicos o ninguna actividad válida) y el conteo de actividades `sin_evidencia` (agente.py:1817-1843 (agente.py)).
22. **Producto final — contrato, .docx y sellado.** Se arma el `_payload` (metadatos de huella, integridad, scores, resultados, skills…) (agente.py:1965-1995 (agente.py)); se valida contra `ValidationReport` (esquemas.py (modulos/esquemas.py)); se genera el `.docx` con `generar_word_informe` (modulos/generador_word.py) (importado en agente.py:54 (agente.py)) y, como **último paso**, se sella la cadena de custodia por hash sobre todo lo anterior —por eso las skills corrieron antes (agente.py:1857-1863, :1997-1999 (agente.py)). El markdown paralelo sale de `generar_informe_md` (agente.py:2053 (agente.py)).

> *Nota de artefacto. El pipeline usa generador_word.py; no existe ningún generador_word_mejorado.py en el árbol activo. El .docx de hoy rotula el veredicto con la escala de semáforo, no con la tríada APROBADO/PENDIENTE/NO APROBADO.*

3. Flujo OBJETIVO (lo que quedará tras el día de API)

Piezas **construidas y probadas como módulos aislados**, pero **aún NO cableadas** al pipeline de veredicto. Se encienden con el coordinador ReAct.

Pieza OBJETIVO	Estado real hoy	Evidencia	Se habilita en
Abstención activa β («EVIDENCIA INSUFICIENTE», β=0.19)	Módulo listo y testeado; el pipeline --prdc no lo llama (solo lo referencia skills/base_tipada.py:158 y los tests)	[abstencion.py](modulos/abstencion.py); β en [parametros.py:289](config/parametros.py)	Cableado al veredicto (WP-01 encendido)
Verificación NLI / entailment de citas	verificar_entailment devuelve NA-diferido; hoy solo hay verificación literal	[validador_citas.py:143](modulos/validador_citas.py)	Día de API (WP-04)
Linaje [doc:pág:chunk] reconstruible	Validador determinista de ids existe pero no lo importa agente.py/motor_ia.py; la cita hoy es fichero+texto	[validador_citas.py](modulos/validador_citas.py)	Día de API (WP-03/04)
Fusibles + cap de iteraciones en un bucle real	Config y clase listas, pero sin bucle agéntico que las consuma («no altera nada» hoy)	[fusibles.py](modulos/fusibles.py); [parametros.py:264-272](config/parametros.py)	Con el coordinador (SK-05)

Pieza OBJETIVO	Estado real hoy	Evidencia	Se habilita en
Coordinador ReAct + 6 skills tipadas (tool_choice=auto, max_iterations=5)	Paquete skills/ declarado (SK-00..04); solo Calidad de Datos operativa (python -m skills.calidad_datos); las otras 5 son stubs	skills/; [plan_skills.md](docs/evidencia/plan_skills.md)	SK-05 (coordinador) + SK-06..10 (día de API)
Veredicto formal APROBADO/PENDIENTE/NO APROBADO por ámbito + pie paginado + referencias consolidadas	El .docx usa el semáforo; sin footer de paginación	[dominio.py:104](config/dominio.py); [generador_word.py](modulos/generador_word.py)	WP-08 (encendido pleno)
Cola HITL PENDIENTE (acciones de impacto se encolan)	Módulo aislado, no cableado a la web	skills/cola_pendiente.py	Día de API

4. Tabla ACTUAL vs OBJETIVO por fase

Fase (validador)	ACTUAL (corre hoy)	OBJETIVO (día de API)
(a) Diálogo	Diálogo de arranque: web (WS /ws/validar) o CLI fijan parámetros antes de gastar; chat post-informe (/ws/chat) sobre el resultado ya emitido	Diálogo durante el razonamiento: el coordinador ReAct pide aclaraciones / re-consulta herramientas en bucle
(b) Replanteo / autocorrección	Solo reintento de contrato JSON (INTENTOS_CONTRATO); citas se declaran dudosas, no se re-piensa	Verificador NLI que invalida afirmaciones y fuerza reintento; fusibles (cap≤3, tiempo, tokens) que cortan y escalan a HITL
(c) Resolución	Concluye vía semáforo (5 bandas) + dictamen_confiable; falta de evidencia = sin_evidencia/0.0	Se abstiene formalmente con «EVIDENCIA INSUFICIENTE» (umbral β) cuando el retrieval es débil
(d) Entrega	ValidationReport (Pydantic) + .docx con semáforo + sellado de custodia por hash	.docx con veredicto APROBADO/PENDIENTE/NO APROBADO por ámbito, pie paginado, referencias con linaje [doc:pág:chunk]

5. Diagrama del flujo (Mermaid)

Los nodos **OBJETIVO** llevan la clase `:::pendiente` (borde punteado); el resto es ACTUAL.

```
flowchart TD
    A["Entrada: Web /ws/validar o CLI --prdc<br/>fija titulo, tier, dominio-rag, validador"] --> B["Runner lanza agente.py por subprocess<br/>pipeline_runner.py:255"]
    B --> C["Orquestacion SECUENCIAL por ambito<br/>agente.py (sin coordinador)"]
```

```
    C --> D["RAG lexico por dominio<br/>buscador_lexico.py"]
    D --> D1["Filtro versionado is_active<br/>buscador_lexico.py:451"]
    D --> D2["Aislamiento model_id (opt-in)<br/>buscador_lexico.py:556"]
    D1 --> E["Evaluacion por ambito -> score<br/>motor_ia.py"]
    D2 --> E
    E --> F["Verificacion LITERAL de citas<br/>declara, no invalida<br/>motor_ia.py:504"]
    F --> G["Skills prompt-based (opcional)<br/>no alteran score<br/>agente.py:1952"]
    G --> H["Resolucion: semaforo + dictamen_confiable<br/>agente.py:1682 /
dominio.py:104"]
    H --> I["ValidationReport (Pydantic)<br/>esquemas.py"]
    I --> J["Genera .docx<br/>generar_word_informe"]
    J --> K["Sella cadena de custodia (hash)<br/>agente.py:1997"]

    %% ---- Nodos OBJETIVO (dia de API) ----
    RA["Coordinador ReAct<br/>tool_choice=auto, max_iterations=5<br/>SK-05"]:::pendiente
    NLI["Verificador NLI / entailment<br/>validador_citas.py:143 (NA-
diferido)"]:::pendiente
    FUS["Fusibles: cap<=3, tiempo, tokens<br/>fusibles.py (no cableado)"]:::pendiente
    ABS["Abstencion activa: EVIDENCIA INSUFICIENTE<br/>abstencion.py (no
cableada)"]:::pendiente
    LIN["Linaje doc:pagina:chunk + veredicto<br/>APROBADO/PENDIENTE/NO
APROBADO"]:::pendiente
    HITL["Cola PENDIENTE (HITL)<br/>cola_pendiente.py"]:::pendiente

    C -.reemplaza a.-> RA
    F -.se suma.-> NLI
    G -.gobernado por.-> FUS
    H -.se suma.-> ABS
    J -.enriquece.-> LIN
    ABS -.escala a.-> HITL

    classDef pendiente stroke-dasharray: 5 5,stroke-width:2px;
```

6. Los 3 puntos donde el flujo actual más difiere del objetivo

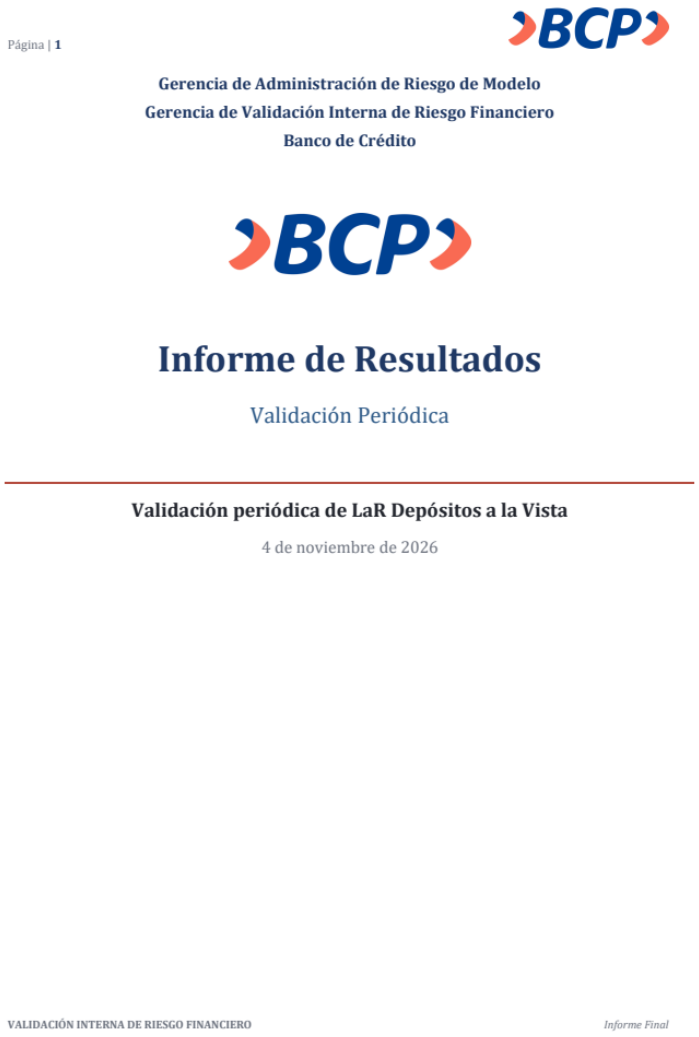
Lo que el **día de API** cierra:

23. **Abstención β sin cablear.** El módulo existe y está probado, pero el pipeline `--prdc` nunca emite «EVIDENCIA INSUFICIENTE»: la evidencia débil hoy sale como `sin_evidencia`/score 0.0, no como abstención formal.
24. **Sin verificación semántica (NLI).** Hoy la verificación de citas es solo **literal** y **declara, no invalida**; `verificar_entailment` está diferido (NA). Una cita decorativa que exista textualmente pasa el filtro actual.
25. **Sin coordinador ReAct.** La orquestación es **secuencial y prompt-based**: **1 de 6** skills tipadas está operativa (Calidad de Datos), no hay bucle Actor/Evaluador, ni fusibles activos, ni cola HITL cableada. Ese coordinador es el corazón del "replanteo/autocorrección" que aún no ocurre.

Parte 5 · Los informes Word por dentro

Los tres informes (Solo Informe, «Generar informe» de la web y el entregable del pipeline) usan la plantilla común `modulos/plantilla_bcp.py`. Estas son páginas reales de los ejemplos de `ejemplos/` (generados sin IA).

Solo Informe



Página | 1

Gerencia de Administración de Riesgo de Modelo
Gerencia de Validación Interna de Riesgo Financiero
Banco de Crédito

Informe de Resultados

Validación Periódica

Validación periódica de LaR Depósitos a la Vista

4 de noviembre de 2026

VALIDACIÓN INTERNA DE RIESGO FINANCIERO

Informe Final

Ejemplo_Informe_Resultados_LaR.pdf · página 1 («Informe de Resultados»)

III. Cumplimiento de Estándar

Ámbito	Total de Controles	Controles Satisfactorios (%)	Controles no cumplidos (%)
Documentación	6	5 (83.3%)	1 (16.7%)
Calidad de datos	6	5 (83.3%)	1 (16.7%)
Metodología	6	6 (100.0%)	0 (0.0%)
Gobierno	2	1 (50.0%)	1 (50.0%)
Implementación	4	4 (100.0%)	0 (0.0%)
Uso e integración a la gestión	5	5 (100.0%)	0 (0.0%)
Total	**29**	**26 (89.7%)**	**3 (10.3%)**

Cumplimiento del estándar por ámbito. Un control se considera no cumplido cuando tiene al menos una observación de riesgo.

IV. Información Relevante

Campo	Detalle
Título	Validación Periódica LaR Depósitos a la Vista
Descripción del documento	El presente documento resume el análisis de la validación periódica realizada entre septiembre y noviembre de 2026.
Responsable de la documentación	Juan Pérez
Revisado por	Walter Paiva
Fecha de aprobación	4 de noviembre de 2026

Ficha del documento.

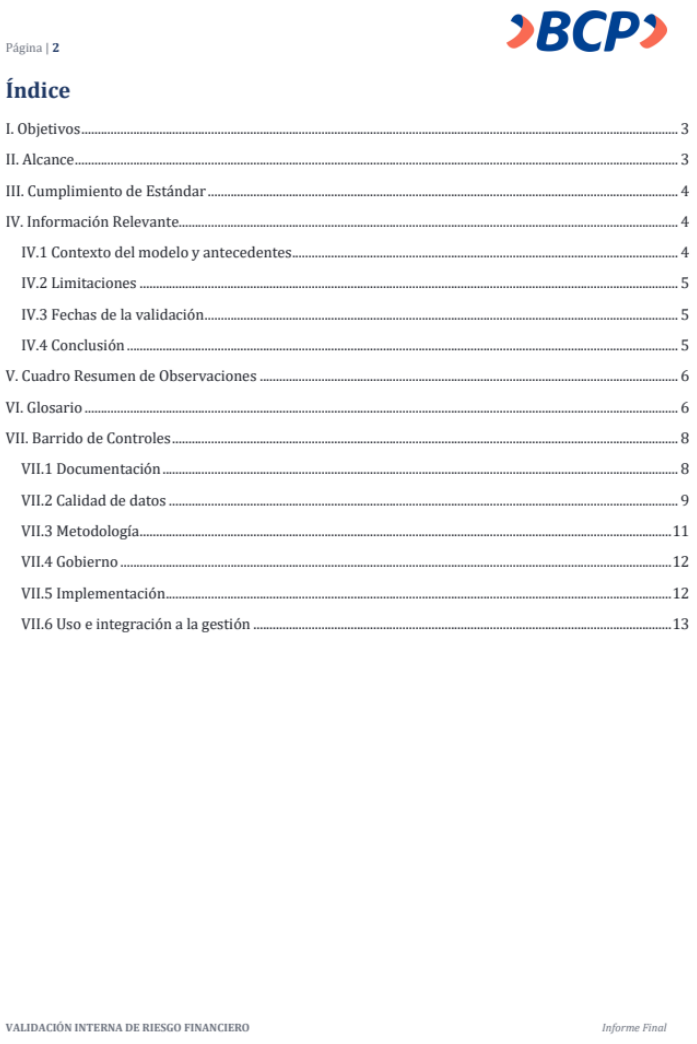
IV.1 Contexto del modelo y antecedentes

El modelo LaR (Liquidity at Risk) estima la máxima pérdida potencial de liquidez de los depósitos a la vista en un horizonte de 30 días con un nivel de confianza del 99%, usando volatilidad EWMA (lambda 0.94). Se emplea en la gestión del riesgo de liquidez y en la fijación de límites internos reportados al comité ALCO.

La última validación se realizó en noviembre de 2025 con resultado Aceptable y dos observaciones moderadas, ambas reportadas como subsanadas por el área usuaria.

VALIDACIÓN INTERNA DE RIESGO FINANCIERO *Informe Final*

Ejemplo_Informe_Resultados_LaR.pdf · página 4 («Ficha del documento»)



Página | 2

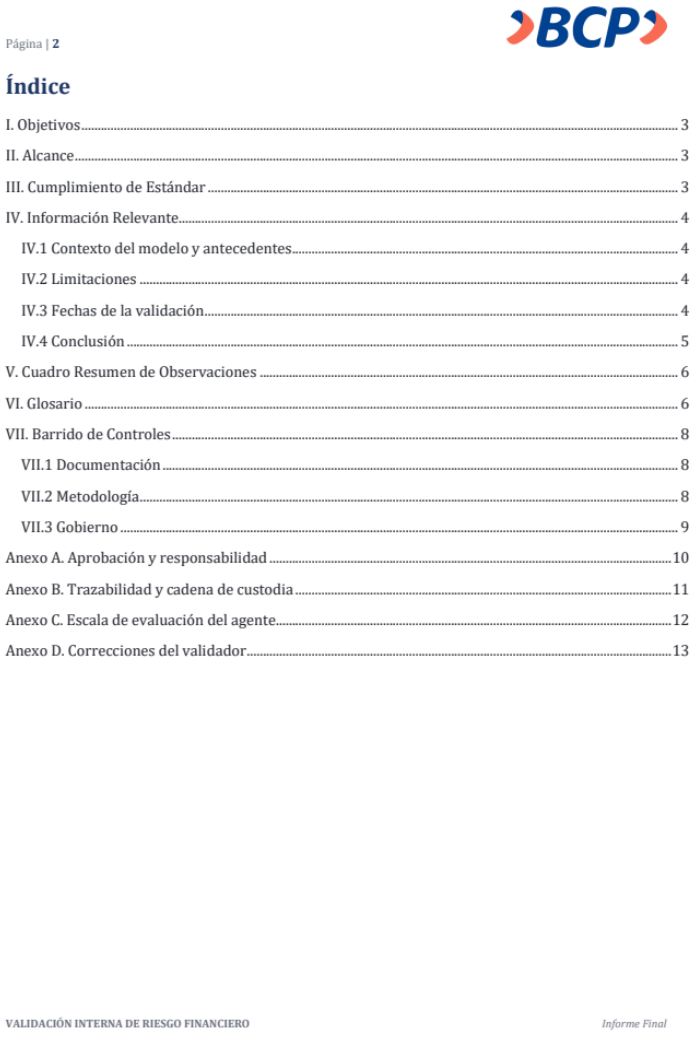
Índice

I. Objetivos 3
II. Alcance 3
III. Cumplimiento de Estándar 4
IV. Información Relevante 4
IV.1 Contexto del modelo y antecedentes 4
IV.2 Limitaciones 5
IV.3 Fechas de la validación 5
IV.4 Conclusión 5
V. Cuadro Resumen de Observaciones 6
VI. Glosario 6
VII. Barrido de Controles 8
VII.1 Documentación 8
VII.2 Calidad de datos 9
VII.3 Metodología 11
VII.4 Gobierno 12
VII.5 Implementación 12
VII.6 Uso e integración a la gestión 13

VALIDACIÓN INTERNA DE RIESGO FINANCIERO *Informe Final*

Ejemplo_Informe_Resultados_LaR.pdf · página 2 («VII. Barrido de Controles»)

Entregable del pipeline



Página | 2

Índice

I. Objetivos ... 3
II. Alcance ... 3
III. Cumplimiento de Estándar ... 3
IV. Información Relevante ... 4
IV.1 Contexto del modelo y antecedentes ... 4
IV.2 Limitaciones ... 4
IV.3 Fechas de la validación ... 4
IV.4 Conclusión ... 5
V. Cuadro Resumen de Observaciones ... 6
VI. Glosario ... 6
VII. Barrido de Controles ... 8
VII.1 Documentación ... 8
VII.2 Metodología ... 8
VII.3 Gobierno ... 9
Anexo A. Aprobación y responsabilidad ... 10
Anexo B. Trazabilidad y cadena de custodia ... 11
Anexo C. Escala de evaluación del agente ... 12
Anexo D. Correcciones del validador ... 13

VALIDACIÓN INTERNA DE RIESGO FINANCIERO *Informe Final*

Ejemplo_Informe_Pipeline_LaR.pdf · página 2 («IV.4 Conclusión»)



Página | **10**

Anexo A. Aprobación y responsabilidad

Modelo validado	LaR Depósitos a la Vista
Tier	Tier IV
Validador responsable	Juan Pérez
Cargo / rol	Validador 2LoD
Fecha de aprobación (UTC)	

El validador identificado asume la responsabilidad del contenido de este informe. El agente VIRF actúa como herramienta de apoyo: sus resultados constituyen una propuesta sujeta a revisión humana, no un dictamen automático.

Firma: _____

VALIDACIÓN INTERNA DE RIESGO FINANCIERO *Informe Final*

Ejemplo_Informe_Pipeline_LaR.pdf · página 10 («Anexo A.»)





Anexo C. Escala de evaluación del agente

Puntaje	Estado	Criterio	Riesgo en el informe
1	No cumple	Evidencia ausente o completamente insuficiente	Crítico
2	Cumple parcialmente - obs. críticas	Requiere remediación inmediata — observaciones críticas	Crítico
3	Cumple parcialmente	Brechas importantes pero no bloquean el proceso	Moderado
4	Cumple con observaciones menores	Cumplimiento sustancial con mejoras recomendadas	Oportunidad de Mejora
5	Cumple plenamente	Evidencia sólida, completa y bien documentada	Sin observación

Escala de puntaje del agente y su traducción a riesgo para la calificación del informe. 0 = sin evidencia (falta crítica).

Ejemplo_Informe_Pipeline_LaR.pdf · página 12 («Anexo C.»)

«Generar informe» con capturas (corrida real)



SECCION-001 — Componente metodológico 1

MARCADOR-UNICO-001-INICIO

El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones. El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones.

1.1 Supuestos y validación

El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones. El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones.

Tabla 1.1 — Resultados del backtesting

SECCION-002 — Componente metodológico 2

MARCADOR-UNICO-002-INICIO

El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones. El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante Jarque-Bera y se reporta la curtosis observada. El backtesting aplica las pruebas de Kupiec (POF) y Christoffersen (independencia) sobre 250 observaciones. El modelo LaR estima la máxima pérdida potencial de liquidez en un horizonte de 30 días con nivel de confianza del 99%. Se emplea volatilidad EWMA con lambda 0.94 conforme a RiskMetrics. El cuantil crítico z(99%) es 2.326. Los supuestos de normalidad se contrastan mediante

VALIDACIÓN INTERNA DE RIESGO FINANCIERO — *Informe Final*

Portada del documento revisado — Manual_Metodologico_LaR_completo.pdf

> **Evidencia en el manual / documento:** *«Manual_Metodologico_LaR_completo.pdf → "Versión: Manual Metodológico LaR v4.2" y "Propietario: BANCO — Gerencia de Riesgo de Liquidez"*
>
> *Manual_Metodologico_LaR_completo.pdf → "Modelo aprobado por el Comité de Riesgo el 2026-01-15. Recalibración anual obligatoria."*
>
> *informe_seguimiento_Q2_2026.md → "Recalibración adelantada al 2026-08-15 (originalmente Q4). Piloto GARCH(1,1) vs EWMA — dueño: Equipo Cuantitativo."*
>
> *No se encontró en el contexto disponible: (1) un registro de cambios (changelog) que documente transiciones entre versiones; (2) acta de aprobación formal de los cambios propuestos en Q2 2026; (3) documentación de la versión recalibrada posterior a 2026-08-15; (4) justificación documentada de por qué se mantiene λ=0.94 o por qué se descartaron alternativas.»*

Ejemplo_Informe_Web_LaR_real_con_capturas.pdf · página 32 («Portada del documento revisado»)

informe_seguimiento_Q2_2026.md → "Durante Q2 2026 el modelo LaR produjo 3 excedencias en 63 días de trading, por encima del valor esperado (0.63). Se activó revisión metodológica."

*Manual_Metodologico_LaR_completo.pdf (Bloque 1/3) → "**Propietario**: BANCO — Gerencia de Riesgo de Liquidez"»*



Evidencia — manual_metodologico_LaR.md · líneas 39–40

Ambigüedad de fuentes (severidad media):

manual_metodologico_LaR.md: *«## 5. Governance\n\nModelo aprobado por el Comité de Riesgo el 2026-01-15. Recalibración anual obligatoria. Owner: Gerencia de Riesgo de Liquidez.»*

informe_seguimiento_Q2_2026.md: *«Durante Q2 2026 el modelo LaR produjo 3 excedencias en 63 días de trading, por encima del valor esperado (0.63). Se activó revisión metodológica.»*

Por qué importa: El manual designa como 'Owner' a la Gerencia de Riesgo de Liquidez, pero el informe de seguimiento no identifica QUIÉN activó la revisión metodológica ni CÓMO se conecta con la gobernanza declarada. No hay evidencia de que la Gerencia de Riesgo de Liquidez haya ejecutado una acción documentada en respuesta a las excedencias. Esto sugiere que la cadena de responsabilidad y escalamiento no está clara o no se ejecutó conforme a lo documentado.



Sustento A — manual_metodologico_LaR.md · líneas 39–40



Sustento B — informe_seguimiento_Q2_2026.md · líneas 5–6

Ambigüedad de fuentes (severidad media):

manual_metodologico_LaR.md: *«## 1. Objetivo del modelo\n\nEstimar la máxima pérdida potencial de liquidez que la entidad puede sufrir en un horizonte de 30 días con un nivel de confianza del 99%, considerando salidas netas de depósitos, uso de líneas comprometidas y estrés de mercado.»*

VALIDACIÓN INTERNA DE RIESGO FINANCIERO — *Informe Final*

Ejemplo_Informe_Web_LaR_real_con_capturas.pdf · página 36 («Sustento A»)

se gestionaría la transición. Esto crea incertidumbre sobre cuál es la metodología actual en producción y viola el principio de trazabilidad: un cambio metodológico propuesto debe estar documentado, aprobado y reflejado en el manual antes de su uso.

informe_seguimiento_Q2_2026.md · líneas 20–27

```
| MAPE 30 días      | 8.2%  | ≤ 10%  | OK |

## Hallazgos

1. Concentración de excedencias en período de estrés de tasas (mayo 2026).
2. Volatilidad EWMA subestimada — considerar cambio a GARCH(1,1).
3. Datos de flujos de una cuenta corporativa top-10 llegaron con 2 días
   de rezago durante 5 fechas.

## Acciones correctivas

- Recalibración adelantada al 2026-08-15 (originalmente Q4).
- Piloto GARCH(1,1) vs EWMA — dueño: Equipo Cuantitativo.
- Automatización de ingesta desde el core bancario — dueño: TI Riesgo.
```

Sustento B — informe_seguimiento_Q2_2026.md · líneas 20–27

Ambigüedad de fuentes (severidad media):

Manual_Metodologico_LaR_completo.pdf: *«Recalibración anual obligatoria.»*

informe_seguimiento_Q2_2026.md: *«Recalibración adelantada al 2026-08-15 (originalmente Q4).»*

Por qué importa: El manual establece recalibración anual (implícitamente en Q4), pero el informe Q2 2026 adelanta la recalibración a agosto. No hay evidencia de que esta desviación del calendario documentado haya sido formalmente aprobada ni de que se haya actualizado el manual para reflejar el nuevo calendario. Esto sugiere que el manual no se mantiene sincronizado con las decisiones operativas reales.

informe_seguimiento_Q2_2026.md · líneas 26–26

```
3. Datos de flujos de una cuenta corporativa top-10 llegaron con 2 días
   de rezago durante 5 fechas.

## Acciones correctivas

- Recalibración adelantada al 2026-08-15 (originalmente Q4).
- Piloto GARCH(1,1) vs EWMA — dueño: Equipo Cuantitativo.
- Automatización de ingesta desde el core bancario — dueño: TI Riesgo.
```

Sustento B — informe_seguimiento_Q2_2026.md · líneas 26–26

Citas sin verificar (revisar contra la fuente): Propietario: BANCO — Gerencia de Riesgo de Liquidez · Recalibración adelantada al 2026-08-15 (originalmente Q4). Piloto GARCH(1,1) vs · Volatilidad EWMA subestimada — considerar cambio a GARCH(1,1). Piloto GARCH(1,1)

Preguntas pendientes del agente: ¿Existe un registro formal de cambios (changelog) que documente todas las versiones del Manual Metodológico LaR, incluyendo fechas, cambios específicos, justificación y aprobador? Si existe, ¿dónde se almacena y cómo se mantiene sincronizado con el manual en producción? · ¿Fue formalmente aprobado por el Comité de Riesgo el cambio de recalibración de Q4 a 2026-08-15? ¿Existe acta de esa aprobación? · ¿Cuál es el estado actual del piloto GARCH(1,1) propuesto en Q2 2026? ¿Ha sido aprobado? ¿Cuándo se espera que entre en producción? ¿Se actualizará el manual cuando se implemente? · ¿Existe un procedimiento documentado que defina cuándo y cómo se deben proponer cambios metodológicos, quién debe aprobarlos, y cómo se comunican e implementan? · ¿La versión recalibrada del modelo (post-2026-08-15) está documentada en un manual actualizado (v4.3 o posterior)? Si es así, ¿cuáles fueron los cambios específicos respecto a v4.2? · ¿Existe control de versiones en el código de cálculo del modelo (scripts, stored procedures, etc.)? ¿Se etiquetan las versiones de código con las versiones del manual metodológico?

Conclusión: A partir de la revisión realizada sobre este control, este ha resultado con riesgo crítico.

VALIDACIÓN INTERNA DE RIESGO FINANCIERO *Informe Final*

Ejemplo_Informe_Web_LaR_real_con_capturas.pdf · página 34 («Sustento B»)

Parte 6 · Skills de cambio

Una skill por tipo de cambio. Cada una dice qué subir (máx. 20 archivos), qué puede devolver el agente (máx. 5), los pasos, las reglas que no se pueden romper, cómo verificar y los errores frecuentes.

Campo	Valor
name	S01-diagnosticar-fallo
description	Triaje de cualquier fallo de VIRF cuando no se sabe en qué parte está. Devuelve qué skill aplicar y qué archivos pedir. Usar primero si el error es genérico o no se sabe qué subir.

S01 — Diagnosticar un fallo

Cuándo usarla

- Aparece un error y no está claro si es de la web, del pipeline, de la IA o del informe.
- Un resultado «se ve raro» (calificación, informe, pantalla vacía).
- Antes de cualquier otra skill, si el pedido no encaja claramente en una.

Qué subir (máx. 20)

26. `CONOCIMIENTO_VIRF.md` y esta skill.
27. **La evidencia del fallo** (pégala en el mensaje, no cuenta como archivo): texto completo del error, la URL o pantalla, la captura y qué se pulsó.
28. Según dónde ocurrió (elige UNA fila, máx. 6 archivos):

Dónde ocurre	Archivos
Pantalla de la web (botón, panel vacío, error rojo)	web/templates/index.html, web/static/js/app.js, el router de esa pantalla (tabla del §4 de CONOCIMIENTO), su servicio
Validación (paso 08) no termina o da error	web/services/pipeline_runner.py, agente.py, modulos/motor_ia.py, el log de la corrida (outputs/…/*.log o consola)
Informe Word	modulos/plantilla_bcp.py + el generador de ese informe (ver S02)
Error 401/403/503 de IA	config/ia.py, config/parametros.py, .env.example (nunca el .env)
Lectura de documentos o Excel	motor/Mx_01_LecturaEvidencias.py, web/services/estandares_service.py

29. El test del área (sección 9 de CONOCIMIENTO).

Qué puede devolver

Normalmente **ningún archivo**: un diagnóstico con (a) causa probable, (b) skill a aplicar, (c) lista exacta de archivos a subir en el siguiente pedido. Si el arreglo es de 1–2 líneas y evidente, puede devolver hasta 2 archivos corregidos.

Pasos

30. Leer el mensaje de error de abajo arriba: la última línea dice QUÉ falló; las líneas `File "…", line N` dicen DÓNDE. Ese archivo:línea es el primer sospechoso.
31. Clasificar el código HTTP si lo hay: 400 entrada inválida (router), 401 sesión caducada, 404 recurso inexistente, 502 la IA respondió mal, 503 faltan credenciales de IA, 500 error del servidor (traza en la consola de uvicorn).
32. Ubicar la función con el grafo (S11): `graphify explain "<función>"` da archivo y vecinos.
33. Reproducir con un test sin coste si es posible (los tests usan `tmp_path` y IA simulada).
34. Responder con la plantilla de diagnóstico.

Reglas que no se pueden romper

Las 10 reglas del §8 de CONOCIMIENTO. En diagnóstico, además: no proponer «probar con la API real» como primer paso; primero reproducir con mocks.

Cómo verificar

El diagnóstico se valida cuando el test del área reproduce el fallo (falla antes del arreglo y pasa después).

Plantilla de diagnóstico

```
Síntoma: …
Causa probable: archivo:línea — explicación
Skill a aplicar: Sxx
Archivos para el siguiente pedido (≤18): …
Test que lo reproduce / debe pasar: …
```

Errores frecuentes

Error	Causa habitual	Skill
sesión de login expirada	El servidor se reinició (las sesiones viven en memoria)	Volver a entrar; no es un fallo
no hay modelo activo	Se llamó a un endpoint de sesión sin elegir modelo	S04
rechazó la clave (401)	Clave mal copiada en .env (p. ej. sin la «s» inicial)	S05
faltan columnas obligatorias	Cabecera del Excel distinta	S06
Word sin imágenes de evidencia	El documento citado no está en la sesión o la cita no es literal	S09
ModuleNotFoundError (torch, chromadb…)	Se usó el Python del sistema en vez de .venv	Usar .venv

Campo	Valor
name	S02-informes-word
description	Cambiar o corregir los informes Word de VIRF (estructura, estética, contenido, anexos, imágenes). Cubre los tres informes, que comparten la plantilla modulos/plantilla_bcp.py.

S02 — Informes Word

Cuándo usarla

- Cambiar la portada, encabezado, pie, índice, secciones I–VII, glosario o anexos.
- Un informe sale con texto mal, tablas desbordadas, imágenes faltantes o Word dice «contenido ilegible».
- Añadir información nueva a un control del barrido o a un anexo.

Qué subir (máx. 20)

Siempre: `CONOCIMIENTO_VIRF.md`, esta skill, `modulos/plantilla_bcp.py`. Más el informe afectado:

Informe	Archivos
Solo Informe	web/services/solo_informe_word.py, web/services/solo_informe_service.py, tests/test_solo_informe.py
«Generar informe» (web)	web/services/informe_web.py, modulos/capturas.py, tests/test_plantilla_bcp.py, tests/test_capturas.py
Entregable del pipeline	modulos/generador_word.py, modulos/word_estilo.py, tests/test_plantilla_bcp.py (+ si toca firma/custodia: fragmentos de tests/test_regresion.py y tests/test_pendientes_auditoria.py que buscan esos textos)

Opcional: `modulos/reglas_informe.py` si el cambio toca la conclusión (ver S03); un `.docx` o PDF de ejemplo de `ejemplos/` para que el agente vea el resultado actual.

Qué puede devolver (máx. 5)

- Cambio de **estética o estructura común** → `modulos/plantilla_bcp.py` (+ su test).
- Cambio de **contenido de un informe** → el generador de ese informe (+ su test).
- Nunca más de un generador por pedido si además cambia la plantilla.

Pasos

35. Identificar si el cambio es común (plantilla) o de un informe (adaptador). Si los tres informes deben cambiar igual, va en la plantilla.
36. En la plantilla, las piezas son funciones: `_portada`, `_encabezado_pie`, `_indice`, `_objetivos`, `_alcance`, `_cumplimiento`, `_info_relevante`, `_ficha_documento`, `_cuadro_resumen`, `_glosario`, `_barrido`, y la API `generar(inf, ruta)`.

37. Los informes no pintan secciones: construyen el diccionario `inf` (ver docstring de `plantilla_bcp.generar`) y aportan ganchos: `detalle_control(doc, control, obs)`, `conclusion_extra(doc)`, `contexto`/`limitaciones` (texto o función) y `anexos` [(título, fn)].
38. Para destacar texto sin tablas usar `campo(doc, etiqueta, texto)` y `cita(doc, título, texto)`. Para fondos en párrafos usar `resaltar(parrafo, fill, acento)` (respeta el orden XML de Word).
39. Imágenes: `_imagen(doc, ruta, ancho_cm)` + `_caption(doc, pie)`.
40. Regenerar un ejemplo y abrirlo en Word para revisar visualmente.

Reglas que no se pueden romper

- La calificación NO se calcula en los informes: sale de `reglas_informe.concluir`.
- Textos regulatorios que los tests exigen en el entregable del pipeline: «APROBACIÓN Y RESPONSABILIDAD», «CADENA DE CUSTODIA», «CORRECCIONES DEL VALIDADOR», «ANÁLISIS TRANSVERSALES» (sin distinguir mayúsculas), «SIN FIRMA DE VALIDADOR» si falta firmante, el hash del informe, «--verificar-cadena», el aviso de procedencia del corpus. No eliminarlos.
- Nunca el literal «None» en el documento.
- El logo se busca con `word_estilo.localizar_logo` vía módulo (los tests lo sustituyen).
- Orden XML de `w:pPr`: no añadir `w:shd`/`w:pBdr` con `append` al final; usar `resaltar`.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_plantilla_bcp.py tests/test_solo_informe.py
tests/test_capturas.py tests/test_informe_tabla_final.py -q
.venv\Scripts\python.exe -m pytest tests/test_regresion.py
tests/test_pendientes_auditoria.py -q   # si tocó el entregable del pipeline
```

Visual: generar un ejemplo y abrirlo en Word (actualizar campos para el índice).

Plantilla de pedido

```
Skill S02. Informe afectado: <Solo Informe | web | pipeline>.
Cambio: <qué debe verse distinto; adjunta captura/PDF actual>.
Devuélveme los archivos modificados (≤5) y el test que lo comprueba.
```

Errores frecuentes

Síntoma	Causa	Arreglo
Word: «contenido ilegible»	Elemento XML fuera de orden (p. ej. w:shd tras w:jc)	Usar resaltar; no append directo en pPr

Síntoma	Causa	Arreglo
Hoja en blanco entre anexos	La función del anexo termina con salto de página	La plantilla ya los quita (_quitar_salto_final); no añadir otro
Índice sin números	Campos sin actualizar	Abrir en Word (se actualiza solo: updateFields)
Tabla cortada entre páginas	Falta no_partir(tabla)	Añadirlo tras crear la tabla
Imagen de sustento ausente	Ruta inexistente	Ver S09

Campo	Valor
name	S03-regla-calificacion
description	Cambiar o corregir la calificación del informe (Deficiente/Regular/Aceptable/Satisfactorio), la traducción puntaje→riesgo o qué cuenta como control incumplido. Un solo archivo de reglas para los tres informes.

S03 — Regla de calificación y riesgos

Cuándo usarla

- La conclusión de un informe no coincide con lo esperado.
- La gerencia cambia los umbrales (p. ej. «Deficiente con 2 críticas»).
- Cambia la traducción del puntaje del agente a riesgo, o el trato de las Oportunidades de Mejora.

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, `modulos/reglas_informe.py`, `tests/test_solo_informe.py` (clase `TestConcluir`, `TestControl`), `tests/test_plantilla_bcp.py` (`test_riesgo_desde_puntaje`). Opcional: `modulos/plantilla_bcp.py` (cómo se muestra), `web/services/informe_web.py` (cómo se aplica el chat).

Qué puede devolver (máx. 5)

`modulos/reglas_informe.py` + los tests actualizados (2). Nada más: la regla es pura.

Pasos

41. Leer `concluir()`: evalúa de lo más grave a lo menos grave; la primera condición gana. Los casos ambiguos están documentados como supuestos A1–A6 (se devuelven en `supuestos`).
42. Cambiar la condición y su texto `motivo`.
43. Actualizar/añadir casos en `TestConcluir` (parametrizados): un caso por umbral.
44. Si cambia `riesgo_desde_puntaje`, actualizar `test_riesgo_desde_puntaje` y la tabla de la «Escala de evaluación del agente» (anexo C del entregable, en `generador_word.py::escala`).

Reglas que no se pueden romper

- La regla es una función pura (sin E/S, sin IA). Ningún informe calcula la calificación por su cuenta.
- «Aceptable» nunca lleva críticas (decisión del usuario).
- La falta de evidencia es falta crítica (política de la entidad, protegida en `test_regresion.py::TestAuditoriaSinEvidenciaEsFaltaCriticaNoAusenciaDeDictamen`).
- Una actividad sin puntaje por fallo técnico no se califica (no se culpa al validado).

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_solo_informe.py tests/test_plantilla_bcp.py -
```

```
q
```

Plantilla de pedido

```
Skill S03. Regla actual: <cita>. Regla nueva: <texto exacto de la gerencia>.
Casos de ejemplo que deben dar cada nivel: <lista>.
```

Errores frecuentes

Síntoma	Causa	Arreglo
Sale Regular con 3 moderadas Doc+Gob	Supuesto A2 (hueco entre «más de 3» y «menos de 3»)	Confirmar con la gerencia y cambiar la condición
Una OM deja el control en rojo	OM_INCUMPLE = True	Debe ser False (decisión del usuario)

Campo	Valor
name	S04-web-pantallas-endpoints
description	Corregir o crear pantallas, botones y endpoints de la web de VIRF (FastAPI + JavaScript sin framework). Incluye estilos, responsive y accesibilidad.

S04 — Web: pantallas y endpoints

Cuándo usarla

- Un botón no hace nada, un panel queda vacío, aparece un error rojo en la web.
- Añadir un campo, una pantalla o un endpoint.
- Problemas de estilo, móvil o accesibilidad.

Cómo está armada la web

- `web/app.py` registra los routers y sirve `index.html` (con versión de caché de CSS/JS).
- `web/templates/index.html`: TODAS las pantallas (`.pantalla` login/modelos/Solo Informe, `.app` dashboard) y vistas (`#vista-*`) en un solo HTML; se muestran/ocultan con `hidden`.
- `web/static/js/app.js`: lógica por bloques (`cablear*` al arrancar, `abrir*` por vista, `pedirJSON(url, opciones)` para llamar a la API). `guia.js`: guía de pasos y ayuda.
- `web/static/css/styles.css`: tokens en `:root` (`--tinta`, `--capa`, `--texto`, `--texto-mu`, `--sello`, `--sello-txt`, `--ok`, `--warn`…). Responsive en `@media (max-width: 760px)`.
- Cada pantalla ↔ un router `web/routers/<x>.py` (prefijo `/api/<x>`) ↔ un servicio `web/services/<x>.py`. Sesión: `Depends(contexto_sesion)` (requiere login + modelo activo).

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, y:

- Pantalla: `web/templates/index.html`, `web/static/js/app.js` (o `guia.js`), `web/static/css/styles.css`.
- Backend de esa pantalla: `web/routers/<x>.py`, `web/services/<x>.py`, `web/app.py` (si es router nuevo).
- Test del área (`tests/test_<x>.py`).

Para localizar el router: busca en `app.js` la URL `/api/...` que llama el botón.

Qué puede devolver (máx. 5)

Típico: `index.html` + `app.js` + `styles.css` + router + servicio. Si además hace falta test, partir en dos pedidos: (1) backend + test, (2) frontend.

Pasos

45. Backend primero: endpoint en el router (modelo Pydantic para el cuerpo, `HTTPException` con mensaje claro: 400 entrada, 404 no existe, 503 falta configuración), lógica en el servicio.
46. Registrar el router en `web/app.py` si es nuevo.

47. Frontend: HTML de la vista, función `abrir<X>()` y `cablear<X>()` en `app.js`, llamada a `pedirJSON`. Mostrar errores en un `.verificacion.is-error`.
48. Estilos reutilizando clases existentes (`panel`, `config-section`, `field`, `btn`, `btn--primary`, `grid-2`, `tabla-modelos`). Recordar: `.field` y `.grid-2` fijan `display`, así que para ocultarlos con `hidden` hace falta la regla `[hidden]{display:none!important}` del contenedor.
49. Comprobar en escritorio y a 375 px de ancho; foco visible con Tab.

Reglas que no se pueden romper

- Endpoints de sesión con `Depends(contexto_sesion)`; los que escriben en rutas globales deben validar nombres (no aceptar `..`).
- No guardar estado importante en `localStorage`.
- Contraste de texto ≥ 4.5:1 (usar tokens, no colores sueltos).
- No llamar a la IA desde el frontend; siempre vía endpoint.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_<area>.py -q
.venv\Scripts\python.exe -m uvicorn web.app:app --port 8000   # y probar en el navegador
```

Los tests de endpoints usan `fastapi.testclient.TestClient` y `app.dependency_overrides` para sustituir la sesión (ver `tests/test_solo_informe.py::cliente`).

Errores frecuentes

Síntoma	Causa	Arreglo
Cambio de CSS/JS no se ve	Caché del navegador	index() añade ?v=<mtime>; recargar
Campo no se oculta con hidden	La clase fija display	Regla [hidden]{display:none!important} en el panel
401 tras reiniciar el servidor	Sesiones en memoria	Volver a entrar
400 «no hay modelo activo»	Endpoint de sesión sin modelo	Entrar a un modelo o usar dependencia de solo login

Campo	Valor
name	S05-proveedor-ia
description	Errores de conexión con la IA (401, 403, 503, 429, timeouts), cambio de proveedor (Azure OpenAI, OpenAI, Anthropic), de modelo o deployment, y VIRF PLUS.

S05 — Proveedor de IA, credenciales y modelos

Cuándo usarla

- «rechazó la clave (401)», «faltan en el .env», 403 de política de contenido, 429, timeouts.
- Pasar de Claude a Azure GPT (o al revés), cambiar de deployment/modelo, usar uno más barato.
- Activar o corregir VIRF PLUS (GPT + Claude con árbitro).

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, `config/ia.py`, `config/parametros.py`, `config/red.py`, `config/virf_plus.py`, `.env.example`, `web/routers/config.py`, `web/services/env_service.py`, `tests/test_estructura_config.py`, `tests/test_azure_mocks.py`, `tests/test_virf_plus.py`. Si el fallo es de Solo Informe: `web/services/solo_informe_ia.py`, `tests/test_solo_informe_ia.py`. **Nunca subir .env** (contiene claves). Describir qué variables hay (sin valores).

Qué puede devolver (máx. 5)

`config/ia.py`, `config/parametros.py`, `.env.example`, y como mucho un servicio y un test.

Configuración por proveedor (en `.env`, lo edita la persona, no el agente)

Proveedor	Variables
Azure OpenAI (principal)	IA_PROVEEDOR=azure_openai, AZURE_OPENAI_API_KEY, AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT, opcional AZURE_OPENAI_API_VERSION
OpenAI directo	IA_PROVEEDOR=openai, OPENAI_API_KEY, OPENAI_MODELO
Anthropic	IA_PROVEEDOR=anthropic, ANTHROPIC_API_KEY (empieza por sk-ant-api03-), ANTHROPIC_MODELO, opcional ANTHROPIC_WORKSPACE_ID
Solo Informe aparte	SOLO_INFORME_IA_PROVEEDOR, SOLO_INFORME_DEPLOYMENT, SOLO_INFORME_MODELO

Prueba de humo (una llamada mínima al proveedor activo; funciona con cualquier proveedor pese al nombre del flag): `.venv\Scripts\python.exe agente.py --verificar-conexion-azure`, o el botón «Verificar conexión» de Configuración (endpoint en `web/routers/config.py`, que llama a `config.ia.prueba_de_humo`).

Pasos

50. Leer el error: 401 = clave inválida; 503 en la web = falta una variable (el mensaje la nombra); 403 = filtro de contenido de Azure (`_diagnostico_filtro` explica la categoría); 429 = cuota.
51. Si es configuración: indicar a la persona qué variable corregir en `.env`; no tocar código.
52. Si es código: los clientes están en `config/ia.py` (`ClienteAzureOpenAIIA`, `ClienteAnthropicIA`, `ClienteOpenAIIA`), el único punto de instanciación es `_instanciar`, `get_cliente_ia()` usa la caché y `cliente_dedicado(proveedor, modelo)` crea uno aparte.
53. Reintentos/timeouts de red: `config/red.py` (`con_reintento`, `VIRF_HTTP_TIMEOUT`).

Reglas que no se pueden romper

- La selección de proveedor vive SOLO en `config/` (test `test_la_seleccion_de_proveedor_vive_solo_en_config`).
- Sin cambio automático a otro proveedor si uno falla; y la palabra «fallback» no puede aparecer en el código vivo de `config/ia.py` (test `test_no_existe_fallback_automatico_en_el_codigo`).
- `FABRICAS[proveedor]()` aparece una sola vez (en `_instanciar`).
- Tests sin llamadas reales (`monkeypatch` de `FABRICAS` o `_llm_de_prueba`).
- El default de `IA_PROVEEDOR` en el código es `azure_openai`.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_estructura_config.py
tests/test_azure_mocks.py tests/test_virf_plus.py tests/test_solo_informe_ia.py -q
```

Errores frecuentes

Síntoma	Causa	Arreglo
401 «invalid x-api-key»	Clave incompleta o con comillas/espacios	Corregir en .env y reiniciar la web
400 «workspace» con Anthropic	Clave «identity-linked»	Añadir ANTHROPIC_WORKSPACE_ID
Azure «deployment not found»	Nombre de deployment ≠ nombre del modelo	Usar el nombre del deployment del portal
403 content_filter	Política de contenido de Azure	Ver categoría en el mensaje; ajustar prompt
La web sigue usando la clave vieja	.env se lee al arrancar	Reiniciar uvicorn

Campo	Valor
name	S06-estandar-validacion
description	El Excel de estándar (Ámbito, Control, Actividad, Tier I–IV) no se lee, faltan actividades, el filtro por Tier falla o cambia el formato del Excel.

S06 — Estándar de validación (Excel)

Cuándo usarla

- «Excel inválido: faltan columnas», actividades que no aparecen, ámbitos mal agrupados.
- El filtro por Tier deja fuera o dentro actividades que no debería.
- La gerencia cambia el formato del estándar (columnas nuevas, otra hoja).

Hay tres lectores (por razones distintas)

Lector	Uso	Archivo
leer_estandares_virf(tier)	Pipeline CLI (ruta global RUTAS["estandares_virf"])	motor/Mx_01_LecturaEvidencias.py
estandares_service.parsear	Paso 02 de la web (por sesión, exige cabecera en fila 1 y TIER I–IV)	web/services/estandares_service.py
solo_informe_service.parsear_estandar	Solo Informe (tolerante: cabecera por palabra clave en las 30 primeras filas, Control General/Específico, TIER opcional)	web/services/solo_informe_service.py

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, el lector afectado, su test (`tests/test_estandares_sesion.py` o `tests/test_solo_informe.py`), `config/dominio.py` (`normalizar_tier`), `motor/Mx_00_CargaRutas.py` (`redirigir_estandar`, flag `--estandares`) y, si se puede, **el Excel** que falla (o una captura de su cabecera y 5 filas).

Qué puede devolver (máx. 5)

El lector + su test (+ `config/dominio.py` si cambian los tiers).

Pasos

54. Abrir el Excel: ¿en qué fila está la cabecera? ¿cómo se llaman las columnas? ¿hay celdas combinadas (ámbito/control)? openpyxl solo da valor a la primera celda combinada: el código propaga el último valor hacia abajo (forward-fill).
55. Comparar con el lector: palabras clave reconocidas (`ambito`/`dimension`, `actividad`/`tarea`, `control`, `especific`, `tier i..iv` o `tier 1..4`).

56. Ajustar el reconocimiento SIN adivinar el modo: si el formato nuevo es otro, añadir la palabra clave; si es incompatible, error explícito con el nombre de la columna que falta.
57. Añadir un test con un Excel construido en memoria (openpyxl) que reproduzca el formato.

Reglas que no se pueden romper

- Sin detección «mágica»: el estándar alternativo se pasa con `--estandares <ruta>`.
- Una actividad huérfana (sin ámbito) se descarta, no se inventa ámbito.
- Filas de totales/notas (`Total…`, `Nota:`) no son actividades.
- La marca de tier cuenta como «aplica» si no está vacía ni es `no/n/false/0`.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_estandares_sesion.py
tests/test_solo_informe.py -q
```

`test_estandar_real_del_repo` lee `inputs/Estandares_VIRF.xlsx` real.

Errores frecuentes

Síntoma	Causa	Arreglo
Paso 02 rechaza el Excel real	Cabecera en fila 2 (fila 1 vacía)	Usar el lector tolerante o mover la cabecera
Actividades con ámbito equivocado	Celda combinada no propagada	Revisar forward-fill de la columna
Tier IV muestra todas las actividades	Sin columnas TIER	El estándar no distingue tiers: es lo esperado

Campo	Valor
name	S07-rag-lexico
description	La literatura por dominio (VaR, LaR, ALM, dominios custom) no aparece, cita fragmentos irrelevantes, el índice está desactualizado o hay que añadir un dominio o un paper.

S07 — RAG léxico y literatura por dominio

Cómo funciona

- Corpus por dominio en `inputs/rag/<DOMINIO>/` (PDF/MD). Índice local por dominio.
- `src/rag/indexador_lexico.py` tokeniza (minúsculas, sin acentos, sin stopwords, stem) y guarda el índice; `src/rag/buscador_lexico.py` puntúa con **BM25 (0.7) + TF-IDF coseno (0.3)**; `src/rag/tesauro.py` expande sinónimos; parámetros en `src/rag/config_rag.py` (y `RAG_TOP_K`, `RAG_MIN_SCORE` en `config/parametros.py`).
- Cada dominio tiene `src/rag/<DOM>/indexer.py` y `retriever.py`; los custom usan `custom_retriever.py` / `base_indexer.py`.
- El pipeline consulta por actividad en `modulos/motor_ia.py::_enriquecer_con_rag`. El dominio se declara con `--dominio-rag` (si no, se infiere del título y el informe lo marca como inferido).
- Web: paso 04 «Dominio & RAG» (`web/routers/rag.py`): subir papers, reconstruir índice, dominios personalizados (`web/services/dominios_custom_store.py`).
- Controles extra dentro de `buscar`: versionado (`modulos/versionado.py`, descarta chunks derogados) y aislamiento opcional por modelo (`modulos/aislamiento.py`).

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, `src/rag/buscador_lexico.py`, `src/rag/indexador_lexico.py`, `src/rag/config_rag.py`, `src/rag/tesauro.py`, `src/rag/base_retriever.py`, `src/rag/<DOM>/retriever.py`, `web/routers/rag.py`, `tests/test_rag_lexico.py`, `tests/test_rag_desactualizado.py`. Para diagnóstico: salida de `.venv\Scripts\python.exe scripts\explicar_rag.py` (o del endpoint de diagnóstico) pegada en el mensaje.

Qué puede devolver (máx. 5)

Buscador/indexador/tesauro/config + su test.

Pasos

58. Reproducir la consulta: `BuscadorLexico("<DOM>").diagnostico_consulta("<texto>")` muestra tokens, expansión y puntajes.
59. Si no hay resultados: ¿existe el índice? ¿se reconstruyó tras añadir papers? (el panel avisa si está desactualizado).
60. Si los resultados son irrelevantes: revisar tesauro, pesos, `RAG_MIN_SCORE`.
61. Añadir un caso de prueba con un corpus pequeño en `tmp_path`.

Reglas que no se pueden romper

- **Sin HuggingFace, sin embeddings, sin ChromaDB** (`validate_project.py` y `tests/test_validador_entorno_hf.py` lo vigilan).
- Indexado y consulta usan la MISMA función `tokenizar` (si difieren, BM25 devuelve ceros).
- El corpus es material de apoyo, no fuente normativa: el informe lo declara.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_rag_lexico.py
tests/test_rag_desactualizado.py tests/test_validador_entorno_hf.py -q
```

Reconstruir índices: botón del paso 04 o `scripts/rebuild_rag_databricks.py`.

Errores frecuentes

Síntoma	Causa	Arreglo
Sin literatura en el informe	Índice no construido / dominio inferido mal	Reconstruir; declarar --dominio-rag
Resultados repetidos	Corpus con párrafos de plantilla	Deduplicar en el buscador
BM25 = 0 con corpus pequeño	IDF nulo en Okapi con pocos documentos	Usar BM25Plus o más documentos

Campo	Valor
name	S08-calidad-datos
description	Controles de calidad de datos (motores genéricos y repositorio de controles por dominio), su ejecución y el notebook entregable de controles.

S08 — Calidad de datos, motores y notebook de controles

Cómo funciona

- **Motores genéricos** (`modulos/motores_datos.py`): completitud, duplicados, integridad referencial, missing, rangos… Catálogo visible en «Controles de datos». Ejecución: `ejecutar_motor`, `motores_disponibles`.
- **Motores de entidad** (`modulos/motores_entidad.py`): resultados del área (no del agente).
- **Repositorio por dominio**: controles como funciones Python en `inputs/repositorio_controles/<dominio>/*.py` (p. ej. `inputs/repositorio_controles/var/christoffersen.py`, `inputs/repositorio_controles/alm/delta_eve.py`); `modulos/repositorio_controles.py` los carga.
- **Notebook entregable**: `modulos/notebook_controles.py` (8 columnas) y `modulos/controles_a_notebook.py` / `excel_a_notebook.py` (desde documento o Excel); endpoint `GET /api/motores/notebook` en `web/routers/controles.py`.
- Pipeline por control: `agente.py --control` + `motor/Mx_04_MotorControlesData.py`.
- Pipeline de calidad de datos por script: `scripts/pipeline_calidad_datos.py`.

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, y según el caso:

- Motor genérico: `modulos/motores_datos.py`, `tests/test_regresion.py` (clase `TestMotoresDatos`; puede pegarse solo esa clase).
- Control de un dominio: el `.py` del control, `modulos/repositorio_controles.py`, `modulos/notebook_controles.py`, `tests/test_notebook_controles.py`.
- Notebook: `modulos/notebook_controles.py`, `modulos/controles_a_notebook.py`, `web/routers/controles.py`, `tests/test_endpoint_notebook_controles.py`.
- Un dataset de muestra pequeño (CSV, sin datos personales reales).

Qué puede devolver (máx. 5)

El motor o control + el generador de notebook + su test.

Pasos

62. Un control nuevo: copiar la estructura de uno existente del mismo dominio (función `control(...)` con docstring de una línea que describe la prueba).
63. Registrar su metadato (nombre, requeridos, opcionales, ejemplo) igual que los demás.

64. Test con dataset sintético en `tmp_path`.

Reglas que no se pueden romper

- Los controles son deterministas (sin IA); la IA solo redacta, nunca calcula el resultado.
- Un dataset ausente no revienta: el control informa «no ejecutable».
- Sin datos personales reales en tests ni en ejemplos.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_notebook_controles.py
tests/test_endpoint_notebook_controles.py -q
.venv\Scripts\python.exe -m pytest tests/test_regresion.py -q -k "Motores"
```

Campo	Valor
name	S09-evidencia-y-capturas
description	Subida y lectura de evidencia (manuales, ZIP de código, datasets, Excels 00–03) y las capturas («pantallazos») de portada, página citada y código que se insertan en el informe web.

S09 — Evidencia: subida, lectura y capturas

Cómo funciona

- Subida: `web/routers/upload.py` (destinos definidos en `web/services/contexto_sesion.py::_DESTINOS_SESION`; el ZIP de implementación se descomprime con límites anti ZIP-bomb). Entendimiento del código: `web/services/analisis_codigo.py`.
- Lectura para el pipeline: `motor/Mx_01_LecturaEvidencias.py` (`_leer_pdf`, `_leer_docx`, …), caché `modulos/document_cache.py`, troceo `modulos/chunker.py` / `chunker_estructural.py`.
- Capturas (sin IA): `modulos/capturas.py`
  - `localizar(nombre, raices)` busca el archivo citado (sesión y, de respaldo, `inputs/`).
  - `captura_portada(pdf)` → primera página.
  - `captura_cita(doc, cita)` → página del PDF con la cita **literal** resaltada; en .md/.docx, el pasaje con líneas numeradas. Si la cita no es literal en el PDF, no hay captura (evita presentar una página equivocada como sustento).
  - `captura_codigo(fragmento, archivo)` → código estilo editor con números de línea reales.
- Uso en el informe web: `web/services/informe_web.py` (`_Capturas`, `_portadas`, `_captura_cita`, `_detalle_control`): portada en Documentación (una sola vez), evidencia, y en cada contradicción «Sustento A» (manual) y «Sustento B» (código u otro documento), máx. 3 contradicciones con captura por control.

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill y:

- Subida: `web/routers/upload.py`, `web/services/contexto_sesion.py`, `tests/test_upload_zip.py`.
- Lectura: `motor/Mx_01_LecturaEvidencias.py`, `modulos/document_cache.py`, `modulos/chunker.py`.
- Capturas: `modulos/capturas.py`, `web/services/informe_web.py`, `tests/test_capturas.py`, y si es posible el documento citado (o un extracto) y el fragmento del JSON de resultados con la cita.

Qué puede devolver (máx. 5)

`modulos/capturas.py`, `web/services/informe_web.py`, `tests/test_capturas.py` (+ lectores si aplica).

Pasos (capturas que no salen)

65. ¿El nombre del archivo citado coincide con uno subido? (`localizar` compara el nombre exacto, sin distinguir mayúsculas; tolera conectores como «y archivo.md»).

66. ¿La cita aparece literal en el PDF? Probar `captura_cita(ruta, cita, carpeta)`.
67. ¿Es un PDF escaneado sin texto? Entonces no hay texto que buscar (hace falta OCR, fuera de alcance).
68. Revisar `<sesión>/outputs/capturas/` (caché por contenido).

Reglas que no se pueden romper

- No capturar páginas «aproximadas» como sustento (`solo_exacta=True` por defecto).
- Las capturas no usan IA y nunca tumban el informe (devuelven None si fallan).
- No ejecutar nada de lo subido (el código del modelo se lee, no se ejecuta).

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_capturas.py tests/test_upload_zip.py -q
```

Campo	Valor
name	S10-chat-sustentacion
description	Chat de sustentación (el área responde las observaciones y el agente decide APROBADO/NEGADO/PENDIENTE), consolidación de decisiones, puntaje final y acta de cierre en Excel.

S10 — Chat de sustentación y consolidación

Cómo funciona

- Endpoints en `web/routers/chat.py`: `POST /api/chat/iniciar`, `GET /api/chat/estado`, `GET /api/chat/observaciones`, `POST /api/chat/adjuntar` (evidencia nueva → re-evaluar), `POST /api/chat/consolidar`, `GET /api/chat/acta.xlsx`, WebSocket `/ws/chat`.
- Lógica: `web/services/chat_service.py` (clases `Observacion`, `Mensaje`, `SesionChat`; veredictos `SIN_TRATAR | APROBADO | NEGADO | PENDIENTE`).
- Consolidación: `modulos/consolidacion.py::consolidar_decisiones` (puntaje final: APROBADO = `VIRF_PUNTAJE_CUMPLE`, por defecto 5) y `tabla_final` (estado por actividad).
- Acta: `modulos/exportador_excel.py::acta_cierre_bytes` (3 hojas).
- Aprendizaje: las correcciones del chat se guardan como feedback (`modulos/feedback.py`).
- En el informe web, APROBADO = observación subsanada (no cuenta para la calificación).

Qué subir (máx. 20)

`CONOCIMIENTO_VIRF.md`, esta skill, `web/routers/chat.py`, `web/services/chat_service.py`, `modulos/consolidacion.py`, `modulos/exportador_excel.py`, `tests/test_consolidacion.py`, `tests/test_acta_cierre.py`, `tests/test_chat_adjuntar.py`, `tests/test_aprendizaje_chat.py`. Si afecta al informe: `web/services/informe_web.py`.

Qué puede devolver (máx. 5)

Servicio de chat, consolidación, exportador y sus tests.

Reglas que no se pueden romper

- La decisión final la toma el validador; el agente propone.
- El puntaje inicial del agente queda sellado y comparable; el final es aparte.
- Tests sin IA real (el chat se prueba con sesiones construidas a mano).

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_consolidacion.py tests/test_acta_cierre.py
tests/test_chat_adjuntar.py tests/test_aprendizaje_chat.py -q
```

Campo	Valor
name	S11-usar-el-grafo
description	Usar el grafo de código (graphify) para encontrar qué archivos subir al agente, qué se rompe si se cambia algo y cómo se conectan las piezas, respetando el límite de 20 archivos.

S11 — Usar el grafo para acotar archivos

Qué es

`graphify-out/` contiene un mapa del código construido sin IA a partir del árbol sintáctico:

- `GRAPH_REPORT.md`: resumen legible — más de 300 comunidades con nombre en español («Informes · Plantilla común BCP», «Configuración · Clientes IA y proveedor»…), nodos centrales («god nodes»), conexiones sorprendentes y preguntas sugeridas.
- `graph.json`: el grafo (nodos = funciones, clases, módulos; aristas = llama, importa, contiene).
- `graph.html`: vista visual por comunidades (ábrelo en el navegador).

Comandos (en la raíz del proyecto, no necesitan IA)

Necesito…	Comando
Ver dónde vive una función y con qué se conecta	graphify explain "concluir()"
Saber qué se rompe si cambio algo	graphify affected "riesgo_desde_puntaje()"
Ver cómo llega un endpoint a una función	graphify path "ia_pulir()" "cliente_dedicado()"
Responder una pregunta en lenguaje natural	graphify query "cómo se capturan las páginas del manual"
Los nodos más conectados (cambiarlos es delicado)	graphify god-nodes --top 15
Actualizar el grafo tras cambiar código	graphify update .

Si `graphify` no está instalado en la máquina corporativa, usar `GRAPH_REPORT.md` (texto) y la sección «Comunidades del grafo» del PDF.

Receta: elegir los ≤18 archivos de código

69. Empezar por la función o endpoint del síntoma (`explain`).
70. `affected` sobre ella: los archivos que aparecen son los candidatos a romperse.
71. Quedarse con: el archivo de la función + los que la llaman directamente + su test.
72. Si salen más de 18, priorizar: (a) el archivo a modificar, (b) su test, (c) los que la llaman, (d) los que ella llama. Los demás se describen en el mensaje (nombre + qué hacen, según la ficha del PDF).

Limitaciones

- No une JavaScript con Python (HTTP). Buscar la URL `/api/...` de `app.js` en `web/routers/`.
- Se queda desactualizado si no se corre `graphify update .` tras los cambios.
- Las carpetas `archivo/` y `codigo_muerto_*` están en el grafo pero NO son código vivo.

Campo	Valor
name	S12-regenerar-guia-y-grafo
description	Regenerar la guía PDF del agente corporativo y el grafo de código después de cambiar VIRF, para que la documentación no mienta.

S12 — Regenerar la guía y el grafo

Cuándo usarla

Después de cualquier cambio que añada o quite módulos, endpoints, tests o pantallas.

Cómo

```
graphify update .                                                  # 1) grafo (sin IA)
.venv\Scripts\python.exe scripts\generar_guia_agente.py --pdf    # 2) guía DOCX + PDF
```

- Sin `--pdf` solo genera el `.docx` (por ejemplo en una máquina sin Word). El PDF se crea abriendo el `.docx` en Word y exportando, o con LibreOffice: `soffice --headless --convert-to pdf docs/agente_corporativo/GUIA_VIRF_AGENTE_CORPORATIVO.docx`.
- Las capturas de pantalla viven en `docs/agente_corporativo/capturas/` (`NN_nombre.jpg`); si cambia una pantalla, reemplazar la imagen con el mismo nombre y regenerar.
- El texto explicativo vive en `CONOCIMIENTO_VIRF.md`, `00_LEEME.md` y `skills/*.md`; el resto (fichas de módulos, endpoints, tests, variables, comunidades) se extrae del código al generar.

Qué subir (si se pide al agente que mejore la guía)

`scripts/generar_guia_agente.py`, `tests/test_guia_agente.py`, el `.md` a cambiar.

Cómo verificar

```
.venv\Scripts\python.exe -m pytest tests/test_guia_agente.py -q
```

El test comprueba, entre otras cosas, que todas las rutas de archivo citadas en las skills existen en el repositorio.

Parte 7 · Catálogo de módulos (generado del código)

Una ficha por archivo de código vivo: propósito (del docstring), tamaño, comunidades del grafo, quién lo importa, qué tests lo cubren y sus funciones y clases con la línea donde empiezan. Sirve para decidir qué archivos subir al agente (ver S11).

Archivos de la raíz

Archivo	Líneas	Propósito
agente.py	2325	PIPELINE 1: CALIDAD DE DATOS (nuevo - recomendado) python agente.py --titulo "Modelo LaR" --tier 1 --excel inputs/calidad_datos/ControlesDat

agente.py

Propósito: PIPELINE 1: CALIDAD DE DATOS (nuevo - recomendado) python agente.py --titulo "Modelo LaR" --tier 1 --excel inputs/calidad_datos/ControlesData.xlsx --control python agente.py --titulo "Modelo LaR" --tier 1 --excel inputs/calidad_datos/ControlesData.xlsx --control --prdc

Tamaño: 2325 líneas

Comunidades del grafo: Agente · Punto de entrada y cliente IA · Agente · Acotar contexto de skills · Configuración · Clientes IA y proveedor · Feedback · Estadísticas

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
modo_estadistica_feedback()	71	pública	
_setup_env_si_falta()	106	interna	Si no existe .env, lo copia desde .env.example con un aviso claro.
_aplicar_raiz_evidencia()	132	interna	Resuelve la raíz de evidencia y reapunta las carpetas de `inputs/`.
_acotar_contexto_skills()	197	interna	Reparte el presupuesto de contexto ENTRE los documentos y declara cuánto
_huella_corpus_rag()	284	interna	Huella del corpus de dominio que orientó el juicio. `None` si no aplica.
main()	306	pública	

Carpeta config/

Archivo	Líneas	Propósito
config/dominio.py	154	Parámetros de NEGOCIO del marco VIRF: tiers, pesos de ámbito, escalas de puntaje, semáforo y preguntas metodológicas.
config/ia.py	987	Clientes IA: interfaz agnóstica de proveedor + factory con caché de proceso.
config/modelos.py	283	CATÁLOGO DE MODELOS — parámetros óptimos por modelo, no por proveedor.
config/parametros.py	802	ÚNICA FUENTE DE VERDAD DE TODOS LOS PARÁMETROS AJUSTABLES DEL PROYECTO.
config/red.py	222	Capa de RED: proxy, inspección TLS y reintento ante cuota agotada.
config/rutas.py	113	Rutas del proyecto. Única fuente de verdad de dónde vive cada cosa.
config/virf_plus.py	318	VIRF PLUS — el agente principal (GPT) suma a Claude como segunda opinión.

config/dominio.py

Propósito: Parámetros de NEGOCIO del marco VIRF: tiers, pesos de ámbito, escalas de puntaje, semáforo y preguntas metodológicas.

Tamaño: 154 líneas

Comunidades del grafo: Configuración · Normalización de tier

Lo usan: motor/Mx_01_LecturaEvidencias.py, scripts/reconstruir_estandar_desde_informes.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
normalizar_tier()	42	pública	Devuelve el tier en notación romana, o None si no es un tier real.

config/ia.py

Propósito: Clientes IA: interfaz agnóstica de proveedor + factory con caché de proceso.

Tamaño: 987 líneas

Comunidades del grafo: Configuración · Clientes IA y proveedor · Tests · Catálogo y detección de modelos · Configuración · Cliente OpenAI · Tests · Filtro de contenido Azure

Lo usan: agente.py, config/virf_plus.py, modulos/automatizaciones.py, modulos/entailment.py, web/routers/config.py, web/services/chat_service.py, web/services/solo_informe_ia.py

Tests que lo importan: test_azure_mocks.py, test_estructura_config.py, test_pendientes_auditoria.py, test_regresion.py, test_solo_informe_ia.py, test_virf_plus.py

Nombre	Línea	Tipo	Qué hace
_detectar_key_en_example()	42	interna	[SEC-FIX] Devuelve la ruta de .env.example si la clave activa aparece
_extraer_categorias()	283	interna	Categorías marcadas en un bloque `content_filter_results`.
_diagnostico_filtro()	300	interna	Mensaje accionable cuando Azure bloquea por Responsible AI.
virf_plus_activo()	686	pública	¿Está encendido VIRF PLUS (Claude + GPT con conclusión conciliada)?
metadatos_proveedor()	756	pública	Ficha del proveedor indicado, o del activo si no se indica ninguno.
get_cliente_ia()	779	pública	Devuelve el cliente IA según IA_PROVEEDOR configurado.
_instanciar()	864	interna	Único punto de instanciación de un proveedor (nunca se salta a otro).
cliente_dedicado()	869	pública	Cliente NUEVO (sin caché) de un proveedor concreto, opcionalmente con otro
reset_cliente_ia()	891	pública	Limpia la caché del cliente (para tests o recarga de configuración).
prueba_de_humo()	902	pública	Llamada mínima real al proveedor activo. Confirma cuatro cosas de una vez:
class ClienteIA	63	clase	Interfaz abstracta para clientes IA Métodos: generar_mensaje, validar_conexion
class ClienteAnthropicIA	96	clase	Cliente para Anthropic Claude Métodos: generar_mensaje, validar_conexion
class ClienteAzureOpenAIIA	362	clase	Cliente para Azure OpenAI Métodos: detectar_modelo_deployment, perfil, generar_mensaje, validar_conexion
class ClienteOpenAIIA	628	clase	Cliente para la API pública de OpenAI con una sola API key.

config/modelos.py

Propósito: CATÁLOGO DE MODELOS — parámetros óptimos por modelo, no por proveedor.

Tamaño: 283 líneas

Comunidades del grafo: Config · Perfil del modelo (contexto) · Tests · Catálogo y detección de modelos · Configuración · Parámetros

Lo usan: config/ia.py, modulos/explicador_modelo.py, modulos/revision_codigo.py, modulos/ventana_contexto.py, web/services/presupuesto_service.py

Tests que lo importan: test_azure_mocks.py, test_estructura_config.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
precio_entrada_por_mtok()	232	pública	USD por millón de tokens de entrada del modelo, por coincidencia de prefijo.
perfil_modelo()	260	pública	Devuelve el perfil del modelo por coincidencia de PREFIJO más larga.
class PerfilModelo	66	clase	Parámetros óptimos para un modelo concreto. Métodos: contexto_chars, max_ctx, chunk_size, max_chunks, presupuesto_doc

config/parametros.py

Propósito: ÚNICA FUENTE DE VERDAD DE TODOS LOS PARÁMETROS AJUSTABLES DEL PROYECTO.

Tamaño: 802 líneas

Comunidades del grafo: Configuración · Parámetros · Tests · Regresión del núcleo

Lo usan: agente.py, config/ia.py, config/red.py, config/virf_plus.py, modulos/abstencion.py, modulos/automatizaciones.py, modulos/chunker.py, modulos/chunker_estructural.py, modulos/document_cache.py, modulos/entailment.py, modulos/excel_a_notebook.py, modulos/feedback_mejorado.py …

Tests que lo importan: test_azure_mocks.py, test_beta_calibracion.py, test_estructura_config.py, test_regresion.py, test_solo_informe_ia.py, test_virf_plus.py

Nombre	Línea	Tipo	Qué hace
_num_env()	54	interna	Lee `nombre` del entorno con `conv`; ante valor inválido usa `defecto`.
_entero()	79	interna	
_decimal()	83	interna	
_booleano()	88	interna	
identidad_validador_entorno()	661	pública	Lee la identidad del validador del entorno EN EL MOMENTO de la llamada.
tiers_doble_firma()	720	pública	Tiers que exigen doble firma, leídos del entorno EN EL MOMENTO de la llamada.
identidad_segundo_validador_entorno()	739	pública	Segundo firmante desde el entorno. Mismo criterio que el primero.

config/red.py

Propósito: Capa de RED: proxy, inspección TLS y reintento ante cuota agotada.

Tamaño: 222 líneas

Comunidades del grafo: Configuración · Reintentos de red · Configuración · Clientes IA y proveedor · Tipos · typing (compartido)

Lo usan: config/ia.py

Tests que lo importan: test_azure_mocks.py

Nombre	Línea	Tipo	Qué hace
cliente_http()	48	pública	Cliente `httpx` configurado para la red corporativa, o None si no hace falta.
diagnostico_https()	108	pública	Explica un fallo de TLS/proxy en términos accionables.
es_transitorio()	169	pública	
espera_sugerida()	174	pública	Segundos a esperar. Prefiere el `Retry-After` del servidor.
con_reintento()	202	pública	Ejecuta `fn`, reintentando ante errores transitorios de cuota.

config/rutas.py

Propósito: Rutas del proyecto. Única fuente de verdad de dónde vive cada cosa.

Tamaño: 113 líneas

Comunidades del grafo: Seguridad · Nombres de archivo seguros · Tipos · typing (compartido)

Lo usan: agente.py, config/ia.py, config/virf_plus.py, modulos/explicador_modelo.py, modulos/huella_prompts.py, modulos/motor_ia_v2.py, modulos/registro_emisiones.py, motor/Mx_00_CargaRutas.py, motor/Mx_03_LlamarIA.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
nombre_seguro()	35	pública	Convierte un texto suministrado por el usuario en un nombre de fichero seguro.

config/virf_plus.py

Propósito: VIRF PLUS — el agente principal (GPT) suma a Claude como segunda opinión.

Tamaño: 318 líneas

Comunidades del grafo: VIRF PLUS · Consenso entre dos modelos · Automatizaciones · Cuadro de observaciones · Tipos · typing (compartido) · Configuración · Clientes IA y proveedor

Lo usan: config/ia.py, modulos/automatizaciones.py, web/routers/config.py

Tests que lo importan: test_virf_plus.py

Nombre	Línea	Tipo	Qué hace
_json_laxo()	87	interna	Primer objeto/lista JSON del texto (tolera fences y prosa). None si no hay.
_campos_decision()	114	interna	Aplana los campos de decisión de un JSON (incluidos los anidados).
_normalizar()	130	interna	
comparar_respuestas()	134	pública	Compara dos respuestas. Devuelve:
_nombre_modelo()	161	interna	
class ClienteConsensoIA	168	clase	Cliente IA que consulta a Claude y a GPT y concilia sus respuestas. Métodos: modelo, descripcion, generar_mensaje, resumen, validar_conexion

Carpeta motor/

Archivo	Líneas	Propósito
motor/Mx_00_CargaRutas.py	489	Configuración central del Motor VIRF. Define rutas, parámetros de la API, constantes del dominio y el rol del agente como Auditor Senior ind
motor/Mx_01_LecturaEvidencias.py	1376	
motor/Mx_02_EstructuraPrompt.py	304	Construcción de todos los prompts que se envían al modelo de IA activo. Cada función devuelve el string listo para usar como mensaje "user".
motor/Mx_03_LlamarIA.py	375	Capa de comunicación con la API del proveedor de IA activo.
motor/Mx_04_MotorControlesData.py	451	Motor de controles de calidad automatizados sobre datos.

motor/Mx_00_CargaRutas.py

Propósito: Configuración central del Motor VIRF. Define rutas, parámetros de la API, constantes del dominio y el rol del agente como Auditor Senior independiente.

Tamaño: 489 líneas

Comunidades del grafo: Estándares · Flag --estandares por CLI · Multimodelo · Raíz de evidencia · Tipos · typing (compartido)

Lo usan: agente.py, modulos/cuestionamientos_ext.py, modulos/explicador_modelo.py, modulos/generador_word.py, modulos/huella_prompts.py, modulos/motor_ia.py, modulos/motor_ia_v2.py, modulos/prevuelo.py, modulos/revision_codigo.py, motor/Mx_01_LecturaEvidencias.py, motor/Mx_02_EstructuraPrompt.py, motor/Mx_03_LlamarIA.py …

Tests que lo importan: test_estandares_sesion.py, test_prompts_editables.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
redirigir_estandar()	80	pública	Reapunta `RUTAS["estandares_virf"]` a otra ruta explícita.
redirigir_evidencia()	108	pública	Reapunta las carpetas de evidencia a otra raíz (flag `--raiz-evidencia`).

motor/Mx_01_LecturaEvidencias.py

Propósito: (sin docstring de módulo)

Tamaño: 1376 líneas

Comunidades del grafo: Motor · Lectura de evidencias (Mx_01) · Estándares · Lectura del pipeline · Lectura · Caché de documentos · Código muerto · Evaluación paralela

Lo usan: agente.py, modulos/explicador_modelo.py, modulos/motor_ia.py, modulos/motor_ia_v2.py, modulos/prevuelo.py, modulos/revision_codigo.py, src/rag/indexador_lexico.py, web/routers/chat.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_leer_pdf()	68	interna	Lee texto de un PDF **digital**. Dos vías: pdfminer y, si falla, PyPDF2.
_leer_docx()	163	interna	
_leer_pptx()	189	interna	
_leer_excel()	206	interna	[AUDIT-FIX] `df.head(max_filas)` recortaba EN SILENCIO. Medido con un XLSX
_leer_python()	239	interna	Lee un fichero .py. Con `max_chars=None` (el default) devuelve el código
_leer_ipynb_completo()	276	interna	Lee un notebook Jupyter íntegro: celdas de código, markdown y un extracto
_extraer_outputs_celda()	335	interna	Texto legible de los outputs de una celda, acotado y sin binarios.
_leer_json()	366	interna	
leer_archivo()	443	pública	Lee un archivo y retorna dict con el texto extraído SIN chunking.
_es_evidencia()	650	interna	True si el archivo debe tratarse como evidencia documental del modelo.
leer_carpeta()	659	pública	Lee todos los archivos soportados de una carpeta (recursivo).
leer_repositorio_informes()	681	pública	Consolida el texto de todos los informes de referencia para aprendizaje de estilo.
cargar_todos_inputs()	692	pública	Carga todos los inputs definidos en RUTAS['inputs'].
inputs_para_dimension()	703	pública	Filtra y une los documentos relevantes para una dimensión específica.
leer_estandares_virf()	724	pública	Lee Estandares_VIRF.xlsx y retorna actividades filtradas por Tier.
_recortar_declarando()	1041	interna	Aplica el tope del pipeline por control DEJANDO CONSTANCIA del recorte.
leer_txt_csv_log()	1076	pública	Lee .txt, .csv, .log y retorna el contenido como string.

Nombre	Línea	Tipo	Qué hace
leer_sql()	1089	pública	Lector de .sql para el pipeline por CONTROL, acotado a MAX_CHARS_CODIGO.
leer_r()	1109	pública	Lector de .R para el pipeline por CONTROL, acotado a MAX_CHARS_CODIGO.
leer_ipynb()	1122	pública	Lector de notebook para el pipeline por CONTROL, acotado a MAX_CHARS_CODIGO.
leer_archivo_generico()	1148	pública	Dispatcher: selecciona lector según extensión.
compilar_textos_de_carpeta()	1181	pública	Recorre recursivo una carpeta, concatena contenido de archivos soportados.
obtener_evidencia_por_control()	1222	pública	Lee evidencia dentro de EVIDENCIA_FOLDER/ctrl_id/ y concatena.
obtener_conocimiento_por_modelo()	1238	pública	Compila contenido en Conocimiento/<NombreModelo>/* para contexto.
_score_manual()	1254	interna	Calcula score para detectar si es "manual" (documentación).
_score_script()	1280	interna	Calcula score para detectar si es "script" (código de proceso).
detectar_manual_y_script()	1307	pública	Encuentra el "mejor" manual y script por scoring de nombres.
guardar_parametros_ejecucion()	1339	pública	Crea un CSV _params_<modelo>_<fecha>.csv con parámetros de ejecución.

motor/Mx_02_EstructuraPrompt.py

Propósito: Construcción de todos los prompts que se envían al modelo de IA activo. Cada función devuelve el string listo para usar como mensaje "user".

Tamaño: 304 líneas

Comunidades del grafo: Motor IA v2 · Evaluación con datos

Lo usan: modulos/motor_ia_v2.py, motor/Mx_03_LlamarIA.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
prompt_aprender_estilo()	56	pública	Prompt para que el modelo aprenda el lenguaje de los informes de referencia.
estilo_por_defecto()	77	pública	Estilo base cuando no hay informes de referencia disponibles.
prompt_generar_replica_virf()	94	pública	Prompt para generar el script de control de calidad de datos de segunda línea.
prompt_sintesis()	145	pública	Prompt para generar la síntesis ejecutiva y el dictamen final.
construir_prompt_final()	217	pública	Arma el prompt final para evaluación de un control individual.

motor/Mx_03_LlamarIA.py

Propósito: Capa de comunicación con la API del proveedor de IA activo.

Tamaño: 375 líneas

Comunidades del grafo: Motor IA v2 · Evaluación con datos · Tipos · typing (compartido) · Motor · Llamada IA con contrato JSON

Lo usan: modulos/motor_ia.py, modulos/motor_ia_v2.py, web/services/analisis_codigo.py

Tests que lo importan: test_azure_mocks.py, test_prompts_editables.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_obtener_cliente()	75	interna	Devuelve el cliente IA. La caché vive ÚNICAMENTE en config.get_cliente_ia().
_llamar_modelo()	89	interna	Llama a la API usando el cliente centralizado desde config.py.
_parsear_json()	166	interna	Extrae y parsea el primer objeto JSON encontrado en el texto.
_crop_to_json()	203	interna	Extrae substring entre la primera "{" y la última "}" (fallback).
_validar_contrato()	219	interna	Valida que el JSON retornado cumple con el contrato esperado.
aprender_estilo()	313	pública	Llama al modelo con el texto de informes de referencia.
generar_replica_virf()	325	pública	Genera el código del script _replica_virf.py como segunda línea.
generar_sintesis()	348	pública	Genera la síntesis ejecutiva, dictamen global y plan de acción.

motor/Mx_04_MotorControlesData.py

Propósito: Motor de controles de calidad automatizados sobre datos.

Tamaño: 451 líneas

Comunidades del grafo: Motor · Controles sobre datos (Mx_04) · Cuestionamientos · Catálogo activo

Lo usan: modulos/motor_ia.py, modulos/motor_ia_v2.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_catalogo_para_deteccion()	26	interna	Fusiona el catálogo por defecto con los personalizados que el operador
detectar_cuestionamientos()	48	pública	Analiza scripts Python en busca de patrones metodológicamente cuestionables.
tiene_replica_virf()	114	pública	Verifica si existe un script _replica_virf.py entre los documentos.
calcular_riesgo_global()	123	pública	Calcula métricas globales de riesgo a partir de todos los resultados.
resumen_cuestionamientos_consola()	160	pública	Imprime en consola un resumen de las alertas detectadas.
alertas_criticas()	174	pública	Retorna lista plana de todas las alertas de todos los archivos.
parsear_lista_campos()	189	pública	Parsea un texto que puede contener lista de campos separados por comas.
parsear_config_control()	212	pública	Parsea una fila del Excel de configuración de controles.
validar_config_unicidad()	276	pública	Valida que la configuración para control de unicidad es suficiente.
ejecutar_control_unicidad()	291	pública	Ejecuta control de unicidad sobre un DataFrame.
ejecutar_motor_data_por_fila()	357	pública	Ejecuta control automático según configuración de una fila Excel.
convertir_resultado_data_a_texto()	410	pública	Convierte resultado del motor de controles automáticos a texto.

Carpeta modulos/

Archivo	Líneas	Propósito
modulos/abstencion.py	103	Compuerta de abstención activa (WP-01, indicador D3).
modulos/aislamiento.py	115	Aislamiento multi-tenant por (tenant, modelo) — WP-02 reducido, indicador D2.
modulos/analisis_estatico.py	707	QUÉ HACE EL CÓDIGO, LEÍDO DEL CÓDIGO — sin ejecutarlo y sin preguntarle al LLM.
modulos/automatizaciones.py	730	Cuadro de observaciones a partir de los Excels de AUTOMATIZACIÓN del producto.
modulos/capturas.py	356	Capturas de evidencia para los informes Word («pantallazos»), sin IA.
modulos/chunker.py	692	Servicio de chunking inteligente para documentos largos.
modulos/chunker_estructural.py	169	Chunking ESTRUCTURAL por secciones (WP · GAP-29, guía de errores/loops RAG).
modulos/comparador.py	378	COMPARACIÓN ENTRE CORRIDAS — qué cambió desde la revisión anterior.
modulos/consola.py	154	Salida de consola a prueba de codificación.
modulos/consolidacion.py	156	Cierre HITL: convierte las decisiones del chat en el PUNTAJE EXACTO final.
modulos/control_evaluacion_data.py	933	ESTRUCTURAS Y UTILIDADES PARA CONTROL DE EVALUACIÓN DE DATOS
modulos/controles_a_notebook.py	794	EL PROBLEMA QUE RESUELVE
modulos/cuestionamientos_ext.py	133	Catálogo extensible de cuestionamientos metodológicos.
modulos/custodia.py	609	CADENA DE CUSTODIA — trazabilidad inmutable de los informes de validación.
modulos/document_cache.py	593	Sistema de memoria de documentos para modo pre-producción (MODO_PREPRDC).
modulos/entailment.py	224	Verificación SEMÁNTICA de citas (entailment / NLI): ¿el fragmento citado IMPLICA la afirmación que se hace sobre él?
modulos/esquemas.py	183	Contrato de salida TIPADO del validador (WP-09 · GAP-43/GAP-14/GAP-03).
modulos/excel_a_notebook.py	1231	INGENIERÍA INVERSA EXCEL → NOTEBOOK PYTHON

Archivo	Líneas	Propósito
modulos/explicador_modelo.py	721	Genera `explicacion_del_modelo_<MODELO>.md`: un documento que permite a un lector —humano o a otro modelo de IA— entender un modelo cuantita
modulos/exportador_excel.py	289	Exporta la tabla de validación a Excel (.xlsx) con las 8 columnas acordadas:
modulos/feedback.py	643	Gestión de feedback del validador sobre las evaluaciones del agente.
modulos/feedback_mejorado.py	373	Sistema de feedback mejorado que REAPRENDE del usuario: • Detecta patrones de error (sobrestima vs subestima) • Ajusta prompts dinámicamente
modulos/fusibles.py	168	Cortacircuitos para bucles de razonamiento y ejecución (WP-05, indicador D6).
modulos/generador_estadistica_feedback.py	252	Genera un informe Excel de precisión del agente basado en el historial de feedback. Se invoca con: python agente.py --estadisticafeedback
modulos/generador_markdown.py	728	Informe de validación en Markdown: el BORRADOR DE TRABAJO del validador.
modulos/generador_word.py	2159	Genera VIRF_Informe_*.docx — Informe de validación en formato PLANO y legible
modulos/huella_informe.py	92	PRIMITIVAS DE HUELLA DE UN INFORME — hash canónico y normalización de nombres.
modulos/huella_prompts.py	117	TRAZABILIDAD DEL PROMPT — qué se le preguntó exactamente al modelo.
modulos/instrucciones_sesion.py	104	Lectura de instrucciones extra (skills + cuestionamientos NL) que la web deja en la raíz de la sesión (v2 · cableado al motor).
modulos/inyeccion.py	117	Defensa contra INYECCIÓN DE INSTRUCCIONES en la evidencia del modelo validado.
modulos/motor_ia.py	2203	Orquestador principal del agente VIRF. Financial Controller / Model Risk Validator — LaR, VaR, Conductuales.
modulos/motor_ia_v2.py	945	Orquestador principal del pipeline VIRF por CONTROL.
modulos/motores_datos.py	609	QUÉ ES El "motor de data": el conjunto de pruebas automáticas que VIRF sabe ejecutar sobre un dataset para validar su calidad.
modulos/motores_entidad.py	489	Lectura de los resultados de los motores de calidad de datos DE LA ENTIDAD.
modulos/notebook_controles.py	512	Genera un .ipynb ENTREGABLE de controles de calidad de datos cuyo output es la tabla de 8 columnas que pide el validador:

Archivo	Líneas	Propósito
modulos/override_humano.py	271	REPORTE DE HUMAN OVERRIDE — dónde el validador corrigió al agente.
modulos/perfilador_datasets.py	342	PERFILADO DE DATASETS — el esquema como evidencia, no los datos.
modulos/pii.py	251	Enmascarado DETERMINISTA de datos personales (PII) en la evidencia, ANTES de que un extracto salga hacia el proveedor de generación.
modulos/plantilla_bcp.py	958	Plantilla común de los informes Word de VIRF (estética «Informe de Resultados»).
modulos/prevuelo.py	289	PRE-VUELO DE LA EVIDENCIA — qué se va a poder leer, antes de gastar.
modulos/registro_emisiones.py	409	REGISTRO APPEND-ONLY DE EMISIONES — control PREVENTIVO de la cadena de custodia.
modulos/reglas_informe.py	239	Reglas puras de calificación de los informes VIRF: conclusión global y estado por control. Las usan los TRES informes Word (Solo Informe, «G
modulos/reporte_docx.py	193	Reporte `.docx` estructurado desde un ValidationReport (WP-08 estructura · GAP-44/45/46/47/48).
modulos/repositorio_controles.py	143	REPOSITORIO de controles de datos ESCRITOS COMO FUNCIONES PYTHON, organizados por tipo de modelo. Cuando el modelo activo es de dominio VaR,
modulos/revision_codigo.py	1053	REVISIÓN DEL CÓDIGO CONTRA LA DOCUMENTACIÓN — una llamada por módulo.
modulos/skills/base.py	310	Infraestructura común de las skills de validación.
modulos/skills/catalogo.py	616	Las ocho skills de validación.
modulos/validador_citas.py	155	Validador DETERMINISTA de citas con id de linaje (WP-03/WP-04 · indicador D4).
modulos/validador_humano.py	456	FIRMA Y RESPONSABILIDAD DEL VALIDADOR HUMANO — segregación de funciones.
modulos/ventana_contexto.py	107	Resolución de la ventana de contexto real del modelo (WP-10 · GAP-40).
modulos/versionado.py	97	Versionado temporal de chunks/estándares (WP-10 · GAP-23).
modulos/word_estilo.py	543	Infraestructura de presentación del informe Word. Reutilizable: aquí vive el CÓMO se ve el documento, no QUÉ dice. `generador_word.py` decid

modulos/abstencion.py

Propósito: Compuerta de abstención activa (WP-01, indicador D3).

Tamaño: 103 líneas

Comunidades del grafo: Abstención y calibración beta

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_abstencion.py, test_beta_calibracion.py

Nombre	Línea	Tipo	Qué hace
umbral_beta()	50	pública	β configurado en el proyecto (import perezoso para no acoplar).
_scores()	59	interna	
evaluar_abstencion()	69	pública	Decide si abstenerse a partir de los scores léxicos recuperados.

modulos/aislamiento.py

Propósito: Aislamiento multi-tenant por (tenant, modelo) — WP-02 reducido, indicador D2.

Tamaño: 115 líneas

Comunidades del grafo: Aislamiento entre clientes (tenant)

Lo usan: src/rag/buscador_lexico.py

Tests que lo importan: test_aislamiento.py

Nombre	Línea	Tipo	Qué hace
etiquetar_chunk()	58	pública	Devuelve una copia del fragmento con su dueño anotado (no muta el original).
pertenece()	67	pública	¿El fragmento pertenece al (tenant, modelo) activo?
filtrar_por_tenant()	83	pública	Conserva solo los fragmentos del (tenant, modelo) activo.
porcentaje_contaminacion()	89	pública	Porcentaje de fragmentos AJENOS al (tenant, modelo) activo. 0.0 = limpio.
assert_sin_contaminacion()	100	pública	Verifica que NINGÚN fragmento del conjunto es ajeno. Lanza si lo hay.
class ContaminacionModelo	44	clase	Se lanza cuando un conjunto recuperado contiene fragmentos ajenos.

modulos/analisis_estatico.py

Propósito: QUÉ HACE EL CÓDIGO, LEÍDO DEL CÓDIGO — sin ejecutarlo y sin preguntarle al LLM.

Tamaño: 707 líneas

Comunidades del grafo: Análisis estático del código · Análisis estático · Contraste doc vs código

Lo usan: agente.py

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
_norm()	127	interna	Minúsculas sin acentos, espacios colapsados.
_numero_en_texto()	134	interna	¿Este número aparece en la documentación?
_identificadores_en_manual()	163	interna	Nombres que el manual invoca como función: `backtesting_kupiec()`.
_codigo_de()	177	interna	Fuente analizable de un `.py` o `.ipynb`.
_hechos_notebook()	215	interna	Hechos de EJECUCIÓN que el notebook guarda de sí mismo.
_contrastar_notebook()	274	interna	Convierte los hechos de ejecución en hallazgos.
_hechos()	320	interna	Extrae del AST todo lo que se puede afirmar sin ejecutar.
_contrastar()	388	interna	Convierte hechos en hallazgos comparándolos con el texto de los manuales.
_funciones_documentadas_ausentes()	471	interna	Funciones que el manual invoca y que NINGÚN script define.
analizar_modulo()	506	pública	Analiza UN fichero. Nunca lanza: un fallo se declara en el resultado.
analizar_arbol()	543	pública	Analiza todos los módulos y consolida. Coste 0: no llama a ninguna API.
informe_markdown()	596	pública	Log legible: qué declara cada script y qué no cuadra con la documentación.

modulos/automatizaciones.py

Propósito: Cuadro de observaciones a partir de los Excels de AUTOMATIZACIÓN del producto.

Tamaño: 730 líneas

Comunidades del grafo: Automatizaciones · Cuadro de observaciones

Lo usan: agente.py, web/routers/automatizaciones.py

Tests que lo importan: test_automatizaciones.py

Nombre	Línea	Tipo	Qué hace
_plano()	105	interna	minúsculas, sin tildes, espacios colapsados.
normalizar_ambito()	112	pública	
normalizar_riesgo()	134	pública	Texto o número de impacto → nivel del cuadro. None si no se reconoce.
localizar_excels()	212	pública	{'00': Path, ...} por prefijo de etapa. Ignora temporales de Excel (~$).
_buscar_columna()	227	interna	
_es_fallo()	235	interna	
_celda()	248	interna	
_leer_hoja()	256	interna	
leer_libro()	306	pública	
leer_carpeta()	326	pública	
tabla_impactos()	332	pública	{ref: {nivel, valor, descripcion}} con ref = «Hoja!fila». Solo filas cuyo
_resolver_ref()	364	interna	Acepta «Hoja!12», «03_Impactos/Hoja!12» o «hoja ! 12».
asignar_responsable()	384	pública	
_fmt_registro()	400	interna	
resumen_libros()	405	pública	
construir_prompt()	481	pública	
_json_laxo()	497	interna	
_cuadro_determinista()	502	interna	Una observación por hoja con resultados no conformes. Coste cero.

Nombre	Línea	Tipo	Qué hace
_hojas_conocidas()	546	interna	
_evidencia_verificada()	552	interna	¿Cada cita apunta a un libro/hoja que realmente se leyó?
_normalizar()	567	interna	
generar_cuadro()	596	pública	Lee los Excels de `carpeta` y devuelve el cuadro:
exportar_excel()	669	pública	
guardar()	720	pública	Escribe JSON + XLSX con sello de tiempo. Devuelve las rutas.
class HojaLeida	187	clase	
class LibroLeido	200	clase	Métodos: n_hallazgos

modulos/capturas.py

Propósito: Capturas de evidencia para los informes Word («pantallazos»), sin IA.

Tamaño: 356 líneas

Comunidades del grafo: Evidencia · Capturas (pantallazos) · Informe web · Generador con capturas

Lo usan: web/services/informe_web.py

Tests que lo importan: test_capturas.py

Nombre	Línea	Tipo	Qué hace
_norm()	47	interna	
_tokens()	53	interna	
_clave()	57	interna	
localizar()	61	pública	Primer fichero con ese nombre (sin distinguir mayúsculas) bajo las raíces.
citas_en_evidencia()	88	pública	«Archivo.pdf → "cita"» → [(archivo, cita)], en el orden del texto.
_fuente()	94	interna	
_textos_pdf()	113	interna	
_pagina_de()	121	interna	(índice de página, rectángulos de la cita, exacta). Primero busca la cita
_render_pdf()	159	interna	
captura_portada()	185	pública	Primera página del PDF. None si no es un PDF legible.
_leer()	202	interna	
_lineas_de()	220	interna	Índices de las líneas que contienen la cita (exacta o por solape).
_render_lineas()	241	interna	
captura_codigo()	277	pública	Código en estilo editor. Con `archivo` existente, se busca el fragmento y se
captura_cita()	308	pública	Captura del documento donde aparece la cita. Devuelve (png, descripción)

modulos/chunker.py

Propósito: Servicio de chunking inteligente para documentos largos.

Tamaño: 692 líneas

Comunidades del grafo: Chunker · Troceado de documentos · Chunker · Caché de resúmenes · Tipos · typing (compartido)

Lo usan: agente.py, src/rag/indexador_lexico.py

Tests que lo importan: test_azure_mocks.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
class ChunkerService	112	clase	Servicio de chunking inteligente con inyección de cliente IA. Métodos: limpiar_cache_resumenes, procesar_doc, procesar_lote, resumen_sesion

modulos/chunker_estructural.py

Propósito: Chunking ESTRUCTURAL por secciones (WP · GAP-29, guía de errores/loops RAG).

Tamaño: 169 líneas

Comunidades del grafo: Chunker estructural

Lo usan: src/rag/indexador_lexico.py

Tests que lo importan: test_chunker_estructural.py

Nombre	Línea	Tipo	Qué hace
_es_tabla()	40	interna	
_es_cerca()	44	interna	
_unidades()	48	interna	Descompone el texto en unidades tipadas, respetando bloques atómicos.
segmentar_estructural()	108	pública	Divide `texto` en segmentos estructurales de a lo sumo `size_max` caracteres.

modulos/comparador.py

Propósito: COMPARACIÓN ENTRE CORRIDAS — qué cambió desde la revisión anterior.

Tamaño: 378 líneas

Comunidades del grafo: Calidad · Comparador de corridas · Calidad · Comparación entre corridas

Lo usan: agente.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_cargar()	139	interna	
_indice_actividades()	143	interna	Actividades indexadas por (ámbito, texto de la actividad).
comparar()	160	pública	Compara dos corridas del mismo modelo. Por defecto, las dos últimas de la
_flecha()	267	interna	
_num()	277	interna	
formatear()	281	pública	
class CambioActividad	58	clase	Métodos: delta
class Comparacion	73	clase	Métodos: delta_score, vara_cambiada, a_dict

modulos/consola.py

Propósito: Salida de consola a prueba de codificación.

Tamaño: 154 líneas

Comunidades del grafo: Utilidades · Consola Unicode y scripts sueltos

Lo usan: agente.py, scripts/auditar_cobertura_rag.py, scripts/explicar_rag.py, scripts/generar_corpus_rag.py, scripts/rebuild_rag_databricks.py, scripts/reconstruir_estandar_desde_informes.py, scripts/verificar_corpus_rag.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_forzar_codepage_utf8_windows()	52	interna	Pone la consola de Windows en UTF-8 (codepage 65001) para que los emoji se
_reconfigurar()	75	interna	Deja `sys.<flujo_nombre>` escribiendo UTF-8 y, sobre todo, con
configurar_consola()	119	pública	Prepara stdout/stderr para imprimir emoji sin riesgo de UnicodeEncodeError.
consola_soporta_unicode()	141	pública	¿Puede la salida actual representar emoji sin sustituirlos?

modulos/consolidacion.py

Propósito: Cierre HITL: convierte las decisiones del chat en el PUNTAJE EXACTO final.

Tamaño: 156 líneas

Comunidades del grafo: Consolidación de decisiones

Lo usan: modulos/exportador_excel.py, web/routers/chat.py, web/routers/resultados.py, web/services/informe_web.py

Tests que lo importan: test_consolidacion.py

Nombre	Línea	Tipo	Qué hace
valor_cumple()	37	pública	Puntaje 1–5 que representa un ítem subsanado (APROBADO) en el recomputo.
_norm()	47	interna	
_indexar_decisiones()	51	interna	(ámbito, actividad normalizada) → observación cerrada del chat.
consolidar_decisiones()	61	pública	Aplica las decisiones cerradas del chat y devuelve el estado FINAL validado,
tabla_final()	130	pública	Toma la tabla intermedia (`datos['tabla_validacion']`) y le aplica las

modulos/control_evaluacion_data.py

Propósito: ESTRUCTURAS Y UTILIDADES PARA CONTROL DE EVALUACIÓN DE DATOS

Tamaño: 933 líneas

Comunidades del grafo: Motor IA v2 · Evaluación con datos · Motor IA v2 · Cálculo de score · Agente · Punto de entrada y cliente IA

Lo usan: agente.py, modulos/motor_ia_v2.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
create_resultado_control_v13()	45	pública	Crea estructura de resultado para un control (v13 con fixes).
_enunciado_control()	746	interna	Extrae QUÉ valida un control a partir del prompt que se le pasó al modelo.
evaluaciones_desde_controles()	766	pública	Traduce los controles automáticos de calidad de datos a evaluaciones VIRF.
class EstadoEvaluacion	37	clase	Estados válidos para evaluación de control
class ValidadorInputs	72	clase	Pre-flight checks antes de evaluar controles Métodos: validar_inputs_basicos, validar_conectividad_api
class DocumentoChunker	173	clase	Divide documentos largos en chunks procesables con overlap Métodos: chunking, calcular_cobertura
class BuscadorEvidencias	272	clase	Localiza evidencias en inputs/ (no en carpeta Evidencias/ fantasma). Métodos: buscar_evidencias
class CalculadorScore	488	clase	Calcula score de cumplimiento (inverso a hallazgos) Métodos: calcular_score_cumplimiento, interpretar_score
class ReportadorEvaluacion	614	clase	Genera reportes detallados de la evaluación Métodos: generar_resumen_pipeline, generar_tabla_resultados

modulos/controles_a_notebook.py

Propósito: EL PROBLEMA QUE RESUELVE

Tamaño: 794 líneas

Comunidades del grafo: Controles · Notebook desde documento · Tipos · typing (compartido) · Controles · Construcción del notebook

Lo usan: agente.py, modulos/notebook_controles.py, web/services/solo_informe_ia.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
leer_documento()	56	pública	Extrae texto plano de .docx, .pdf, .md o .txt.
extraer_controles_del_documento()	179	pública	Usa el LLM para convertir la prosa del documento en controles estructurados.
leer_controles_excel()	230	pública	Lee la hoja 'Controles' y normaliza cada fila.
_firma()	284	interna	Conjunto de tokens significativos para comparar controles heurísticamente.
_similitud()	291	interna	Jaccard entre dos firmas.
_norm()	298	interna	Normaliza un nombre de columna para comparar (minúsculas, sin espacios).
_score_par()	303	interna	Puntúa el parecido entre un control del documento y uno del Excel.
analizar_brechas()	350	pública	Cruza los controles del documento con los del Excel.
_md()	418	interna	
_code()	419	interna	
_codigo_control()	422	interna	Genera el código pandas de un control usando el catálogo de motores.
construir_notebook_controles()	500	pública	
generar_notebook_controles()	723	pública	Pipeline completo: documento → controles → notebook + brechas.

modulos/cuestionamientos_ext.py

Propósito: Catálogo extensible de cuestionamientos metodológicos.

Tamaño: 133 líneas

Comunidades del grafo: Tipos · typing (compartido) · Cuestionamientos · Catálogo activo

Lo usan: motor/Mx_04_MotorControlesData.py, skills/base_tipada.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_leer()	44	interna	
_valida_regex()	53	interna	
_traducir_personalizado()	61	interna	Convierte un dict del formato del router web al formato interno
catalogo_activo()	91	pública	Devuelve `(catalogo, avisos)`.

modulos/custodia.py

Propósito: CADENA DE CUSTODIA — trazabilidad inmutable de los informes de validación.

Tamaño: 609 líneas

Comunidades del grafo: Custodia · Cadena de hashes · Tipos · typing (compartido) · Custodia · Huellas de evidencia · Custodia · Sellado del informe

Lo usan: agente.py, modulos/comparador.py, modulos/motor_ia_v2.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_hash_cadena()	64	interna	Eslabón de la cadena. Incluye el timestamp para que alterarlo rompa la
informes_de_modelo()	80	pública	Informes JSON de un modelo, ordenados SIGUIENDO LOS ENLACES de la cadena.
version_evidencia()	152	pública	Congela la versión de cada documento de evidencia usado.
preservar_si_existe()	183	pública	Si en `ruta_destino` ya hay un informe SELLADO, lo archiva antes de que se
sellar()	233	pública	Añade el bloque `custodia` al informe. Devuelve el mismo dict, mutado.
_ruta_lock()	300	interna	Fichero de bloqueo por MODELO.
sellar_y_escribir()	313	pública	Sella y escribe el informe en UNA sola operación protegida por lock.
verificar_cadena()	409	pública	Recorre todos los informes de un modelo y verifica la cadena completa.
verificar_evidencia()	499	pública	Comprueba si los documentos citados por un informe siguen intactos.
imprimir_cadena()	552	pública	
imprimir_evidencia()	589	pública	

modulos/document_cache.py

Propósito: Sistema de memoria de documentos para modo pre-producción (MODO_PREPRDC).

Tamaño: 593 líneas

Comunidades del grafo: Caché de documentos · Tests · Borrado del caché · Tipos · typing (compartido) · Lectura · Caché de documentos

Lo usan: agente.py, modulos/chunker.py, modulos/custodia.py, modulos/feedback.py, modulos/registro_emisiones.py, modulos/validador_humano.py, motor/Mx_01_LecturaEvidencias.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_lock_archivo()	80	interna	Lock cooperativo entre procesos basado en creación atómica de un fichero.
_escribir_json_atomico()	153	interna	[CONCURRENCIA-FIX] Escribe JSON de forma atómica: fichero temporal en el
inicializar_cache()	532	pública	Inicializa (o reinicializa) la instancia global del caché.
get_cache()	547	pública	Retorna la instancia global. Si no fue inicializada, crea una deshabilitada
cerrar_cache()	565	pública	Persiste el caché e imprime el resumen de sesión.
_reset_cierre_para_tests()	590	interna	Permite que los tests ejerciten cerrar_cache() más de una vez.
class DocumentCache	185	clase	Caché persistente de documentos parseados. Métodos: get, invalidar_archivo, purgar_huerfanos, persistir, estadisticas, limpiar_todo, resumen_sesion

modulos/entailment.py

Propósito: Verificación SEMÁNTICA de citas (entailment / NLI): ¿el fragmento citado IMPLICA la afirmación que se hace sobre él?

Tamaño: 224 líneas

Comunidades del grafo: Salvaguardas · Entailment (LLM como juez)

Lo usan: modulos/motor_ia.py

Tests que lo importan: test_entailment.py

Nombre	Línea	Tipo	Qué hace
activo()	69	pública	¿Está habilitada la verificación (VIRF_ENTAILMENT=1)?
_na()	83	interna	Resultado neutro: idéntico en forma al del stub histórico.
_prompt()	97	interna	
verificar()	122	pública	Verifica si `fragmento` implica `afirmacion`.
_parsear()	199	interna	Extrae el JSON del juez. Reutiliza el parser robusto del motor si está a
_subcadena_laxa()	217	interna	¿`aguja` aparece en `pajar`, ignorando espacios y mayúsculas? Los

modulos/esquemas.py

Propósito: Contrato de salida TIPADO del validador (WP-09 · GAP-43/GAP-14/GAP-03).

Tamaño: 183 líneas

Comunidades del grafo: Informe · Contrato ValidationReport · Esquemas · Validación de afirmaciones · Esquemas de datos (contratos)

Lo usan: modulos/reporte_docx.py

Tests que lo importan: test_docx_regresion.py, test_esquemas.py

Nombre	Línea	Tipo	Qué hace
desde_dict()	153	pública	Construye un `ValidationReport` a partir del dict sellado del pipeline.
validar_reporte()	169	pública	Alias explícito: valida y devuelve, o levanta `ValidationError`.
intentar_validar()	174	pública	Modo *warn-first* (plan WP-09): devuelve `(reporte|None, errores:list[str])`
class EstadoRequisito	56	clase	Clasificación tri-state extendida de cada requisito (guía §7 checklist).
class ValidationClaim	65	clase	Un hallazgo atómico de validación: un requisito documental contrastado
class ValidationReport	123	clase	Reporte de validación completo. Es el objeto que el borde del pipeline

modulos/excel_a_notebook.py

Propósito: INGENIERÍA INVERSA EXCEL → NOTEBOOK PYTHON

Tamaño: 1231 líneas

Comunidades del grafo: Controles · Excel a notebook ejecutable

Lo usan: agente.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
analizar_libro()	99	pública	Recorre un libro y produce:
_slug_libro()	233	interna	`parametros.xlsx` → `parametros`. Slug legal como variable Python.
_slug_hoja()	239	interna	
analizar_libro_externo_ligero()	243	pública	Análisis mínimo de un libro externo: solo hojas, encabezados y una muestra
resolver_libros_externos()	280	pública	Para cada ref externa detectada, verifica si el libro está en la carpeta.
extraer_vba()	310	pública	Extrae módulos VBA de un .xlsm con olevba.
construir_grafo_libros()	360	pública	Escanea todos los .xlsx/.xlsm de la carpeta, detecta refs [otro.xlsx],
_consultar_llm_traduccion()	526	interna	Pide al modelo la traducción. Fallback a plan mínimo si falla.
_consultar_llm_vba()	553	interna	Pide al modelo la traducción de los módulos VBA a Python.
verificar_orden_topologico()	587	pública	Comprueba que en el motor_calculo cada paso solo use variables ya
_md()	627	interna	
_code()	628	interna	
construir_notebook()	631	pública	
_valores_esperados_excel()	842	interna	Prefiere valores cacheados por Excel; si no hay, intenta evaluar con
_ejecutar_notebook()	905	interna	Ejecuta las celdas de código del notebook. Retorna (ok, namespace, mensaje).
auto_validar()	944	pública	Ejecuta el notebook, compara cada output declarado con el valor
_serializable()	1002	interna	
inyectar_reporte_validacion()	1009	pública	Añade un anexo D con el reporte de auto-validación y una alerta arriba.
generar_notebook_de_archivo()	1062	pública	

Nombre	Línea	Tipo	Qué hace
_plan_fallback()	1158	interna	
generar_notebook_de_carpeta()	1183	pública	

modulos/explicador_modelo.py

Propósito: Genera `explicacion_del_modelo_<MODELO>.md`: un documento que permite a un lector —humano o a otro modelo de IA— entender un modelo cuantitativo de riesgo SIN leer el código fuente.

Tamaño: 721 líneas

Comunidades del grafo: Explicador del modelo · Explicador · Verificar truncado · Explicador · Inventario del modelo · Tipos · typing (compartido)

Lo usan: agente.py

Tests que lo importan: test_azure_mocks.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_render_supuestos()	201	interna	Vuelca el catálogo al prompt en un formato compacto y legible.
class ExplicadorModelo	362	clase	Genera la explicación consolidada de un modelo a partir de las 4 fuentes. Métodos: inventariar, verificar, generar

modulos/exportador_excel.py

Propósito: Exporta la tabla de validación a Excel (.xlsx) con las 8 columnas acordadas:

Tamaño: 289 líneas

Comunidades del grafo: Exportador Excel · Acta de cierre (Excel)

Lo usan: web/routers/chat.py, web/routers/resultados.py

Tests que lo importan: test_acta_cierre.py, test_exportador_excel.py

Nombre	Línea	Tipo	Qué hace
_aplica()	42	interna	
construir_libro()	46	pública	Devuelve un `openpyxl.Workbook` con la tabla y una cabecera de contexto.
a_bytes()	108	pública	Serializa el libro a bytes (para servirlo por HTTP).
_norm_act()	123	interna	
acta_cierre_bytes()	127	pública	Acta de cierre en .xlsx (3 hojas) generada al finalizar el chat:

modulos/feedback.py

Propósito: Gestión de feedback del validador sobre las evaluaciones del agente.

Tamaño: 643 líneas

Comunidades del grafo: Feedback · Aprendizaje y persistencia · Feedback · Estadísticas · Validador · Correcciones humanas (override) · Feedback · Calidad del proceso y autoría

Lo usan: agente.py, modulos/feedback_mejorado.py, modulos/generador_estadistica_feedback.py, modulos/motor_ia.py, modulos/override_humano.py, web/routers/chat.py

Tests que lo importan: test_aprendizaje_chat.py, test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
autor_de()	54	pública	Autor de una corrección, con los registros antiguos tratados como SIN_AUTOR.
cargo_de()	68	pública	Cargo del autor de una corrección. Mismo criterio que `autor_de`.
identidad_para_feedback()	73	pública	Identidad que firma las correcciones de esta sesión.
_id_registro()	110	interna	Identidad estable de un registro, para poder fusionar sin duplicar.
cargar_historial()	123	pública	Carga el historial completo de feedback desde el JSON.
guardar_historial()	153	pública	Persiste el historial en el JSON.
_mostrar_resumen_evaluaciones()	201	interna	Imprime un índice numerado de todas las actividades evaluadas.
_leer()	227	interna	[CI-FIX] input() seguro para entornos NO interactivos (cron, CI, nohup,
bucle_feedback_consola()	245	pública	Bucle interactivo al final de la ejecución.
_bucle_feedback_interactivo()	280	interna	Implementación interactiva real (ver bucle_feedback_consola).
construir_contexto_feedback()	405	pública	Busca en el historial si hay feedback previo relevante para esta actividad
_norm_act()	446	interna	
_valor_cumple_aprendizaje()	450	interna	Puntaje que representa 'Cumple' al aprender de un APROBADO. Igual regla que
registrar_aprendizaje_desde_chat()	461	pública	Persiste como FEEDBACK las decisiones cerradas del chat de sustentación, para
_registros_utilizables()	537	interna	[AUDIT-FIX] Filtra los registros con los que SÍ se puede calcular.
calcular_estadisticas()	565	pública	Calcula métricas de precisión del agente basadas en el historial de feedback.
class _SinEntradaInteractiva	223	clase	[CI-FIX] stdin cerrado o interrumpido: se aborta el feedback sin fallar.

modulos/feedback_mejorado.py

Propósito: Sistema de feedback mejorado que REAPRENDE del usuario: • Detecta patrones de error (sobrestima vs subestima) • Ajusta prompts dinámicamente • Calibra scores basado en feedback acumulado • Aprende qué criterios buscar en cada ámbito

Tamaño: 373 líneas

Comunidades del grafo: Motor IA · Evaluar actividades y ámbitos

Lo usan: modulos/motor_ia.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
cargar_historial()	20	pública	Carga el historial de feedback delegando en `modulos.feedback`.
analizar_patron_errores()	43	pública	Analiza patrones de error del agente en un ámbito específico.
construir_bloque_aprendizaje()	133	pública	Construye un bloque de instrucción dinámico basado en errores anteriores.
ajustar_puntaje_por_feedback()	187	pública	Ajusta el puntaje basado en feedback histórico similar.
obtener_recomendaciones_evaluacion()	240	pública	Retorna recomendaciones específicas basadas en errores previos.
generar_resumen_aprendizaje()	269	pública	Genera un resumen de qué ha aprendido el sistema del feedback.
registrar_feedback_mejorado()	304	pública	Registra feedback Y lo analiza para extraer lecciones.

modulos/fusibles.py

Propósito: Cortacircuitos para bucles de razonamiento y ejecución (WP-05, indicador D6).

Tamaño: 168 líneas

Comunidades del grafo: Seguridad · Cortacircuitos (fusibles)

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_circuit_breakers.py

Nombre	Línea	Tipo	Qué hace
class CortacircuitosActivado	46	clase	Se lanza cuando un fusible se dispara. Lleva el motivo estructurado.
class Cortacircuitos	55	clase	Controla cap de iteraciones + fusibles de tiempo y tokens. Métodos: desde_config, iniciar, reiniciar, segundos_transcurridos, nueva_iteracion, sumar_tokens, verificar, escalar

modulos/generador_estadistica_feedback.py

Propósito: Genera un informe Excel de precisión del agente basado en el historial de feedback. Se invoca con: python agente.py --estadisticafeedback

Tamaño: 252 líneas

Comunidades del grafo: Feedback · Estadísticas

Lo usan: agente.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_fill()	20	interna	
_font()	21	interna	
_align()	23	interna	
_border()	24	interna	
_color_mae()	38	interna	
_color_pct()	46	interna	
generar_informe_estadistica()	54	pública	

modulos/generador_markdown.py

Propósito: Informe de validación en Markdown: el BORRADOR DE TRABAJO del validador.

Tamaño: 728 líneas

Comunidades del grafo: Informe Markdown

Lo usan: agente.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_pj()	58	interna	Puntaje 1-5, `0` si no se aportó evidencia, `None` si NO se pudo evaluar.
_orden_pj()	90	interna	
_etiqueta_pj()	95	interna	Cómo se rotula el puntaje en el encabezado de una actividad.
_txt()	104	interna	
_firma_agente()	108	interna	Versión y motor, legibles.
_semaforo_txt()	125	interna	Etiqueta del semáforo. También llega como dict `{label, color}`, no como
_bloque_citas()	136	interna	Reindenta la evidencia como cita de Markdown.
_cabecera()	155	interna	
_como_leer()	189	interna	Cómo interpretar este documento. Va antes que cualquier cifra.
_politicas()	250	interna	Aviso de políticas §4.2 y §6 aplicadas al informe.
_resumen()	298	interna	
_inconsistencias()	338	interna	Todas las contradicciones del informe, juntas y ordenadas por severidad.
_metodologia()	394	interna	Cuestionamientos sobre la elección metodológica, agrupados.
_skills()	437	interna	Hallazgos de los análisis transversales.
_pedidos()	517	interna	Todas las preguntas al modelador, juntas y sin repetir.
_detalle()	565	interna	
_limitaciones()	633	interna	Qué NO se pudo evaluar. La sección que hace defendible el informe.
_pie()	687	interna	
generar_informe_md()	701	pública	Escribe el informe Markdown a partir del JSON de resultados.

modulos/generador_word.py

Propósito: Genera VIRF_Informe_*.docx — Informe de validación en formato PLANO y legible

Tamaño: 2159 líneas

Comunidades del grafo: Informe Word · Primitivas del entregable · Informe Word · Generar entregable del pipeline · Informe Word · Estilos y formato · Tests · Informe Word sin puntajes falsos

Lo usan: agente.py

Tests que lo importan: test_pendientes_auditoria.py, test_plantilla_bcp.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_hex_to_rgb()	56	interna	
_score_color_hex()	61	interna	
_label_semaforo()	81	interna	
_puntaje_label()	99	interna	
_puntaje_color_hex()	112	interna	
_status_resultado()	120	interna	Etiqueta de estado para el campo 'Resultado de la actividad'.
_score_o_puntaje()	138	interna	Devuelve el score si trae valor, y si no el puntaje.
_normalizar_score()	154	interna	Convierte score (0-100) o puntaje (1-5) a la escala de política 0-5.
_es_error()	199	interna	Detecta si el texto ES un mensaje de error técnico (no si lo menciona).
_generar_recomendacion_automatica()	243	interna	Genera una recomendación automática según el puntaje y contexto.
_set_cell_bg()	260	interna	
_set_cell_borders()	270	interna	
_cell_text()	284	interna	
_add_heading()	301	interna	
_add_para()	309	interna	
_add_numbered()	328	interna	
_add_bullet()	336	interna	
_hr()	344	interna	

Nombre	Línea	Tipo	Qué hace
_add_ambito_heading()	361	interna	Encabezado de ámbito con línea decorativa inferior.
_seccion_skills()	388	interna	Análisis transversales en el ENTREGABLE.
_sombrear_parrafo()	515	interna	Fondo de un párrafo. Equivalente a `_set_cell_bg` pero sobre `w:pPr`.
_add_hallazgos_codigo()	534	interna	Pinta cada discrepancia código↔documentación con el FRAGMENTO a la vista.
_add_hallazgos_cdd()	665	interna	Hallazgos de los controles automáticos de calidad de datos.
_partir_por_vallas()	749	interna	Separa un texto en tramos `(es_codigo, contenido)` según las vallas ```.
_add_field_block()	781	interna	Etiqueta en línea propia (bold) y contenido en el párrafo siguiente.
_procedencia_nota()	842	interna	Compone la nota al pie de procedencia de UNA actividad.
_add_result_line()	905	interna	Resultado de la actividad con score coloreado y etiqueta de estado.
_tabla_scores()	942	interna	
_indice_controles_ambito()	990	interna	Lista numerada de todos los controles/estándares aplicables en el ámbito.
_ficha_control()	1003	interna	Ficha de una actividad/control con:
_seccion_ambito()	1137	interna	Sección de un ámbito:
_portada_desde_json()	1177	interna	Genera la portada basada en los datos JSON.
_metadata_desde_json()	1217	interna	Inserta metadata en el documento.
_resumen_ejecutivo_desde_json()	1238	interna	Inserta el resumen ejecutivo.
_objetivos_desde_json()	1244	interna	Inserta los objetivos.
_alcance_desde_json()	1256	interna	Inserta el alcance con dimensiones y controles.
_informacion_relevante_desde_json()	1268	interna	Inserta información relevante.
_conclusion_desde_json()	1293	interna	Inserta la conclusión.
_hallazgos_desde_json()	1318	interna	Inserta hallazgos y recomendaciones.
_tabla_contenido_desde_json()	1365	interna	Inserta la tabla de contenido.
_informe_detallado_desde_json()	1374	interna	Inserta el informe detallado con controles.
_hoja_firma()	1476	interna	Hoja de aprobación formal: quién firma, con qué cargo y cuándo.

Nombre	Línea	Tipo	Qué hace
_metadatos_custodia()	1562	interna	Metadatos de trazabilidad: permiten verificar el informe a posteriori.
_anexo_correcciones()	1673	interna	Anexo: Correcciones del Validador — evidencia de que el juicio humano manda.
generar_word_informe_desde_json()	1782	pública	Genera el informe Word completo basado en el formato JSON estructura.
generar_word_informe()	1879	pública	Genera el entregable Word con la plantilla común «Informe de Resultados»

modulos/huella_informe.py

Propósito: PRIMITIVAS DE HUELLA DE UN INFORME — hash canónico y normalización de nombres.

Tamaño: 92 líneas

Comunidades del grafo: Tipos · typing (compartido) · Custodia · Huellas de evidencia

Lo usan: modulos/custodia.py, modulos/registro_emisiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_sin_campos_derivados()	50	interna	Copia del informe sin los campos que derivan del propio hash.
hash_contenido()	63	pública	SHA-256 canónico del contenido de un informe.
hash_archivo()	78	pública	SHA-256 del contenido binario. None si no se puede leer.
slug()	90	pública	Normaliza un nombre de modelo para compararlo entre ficheros.

modulos/huella_prompts.py

Propósito: TRAZABILIDAD DEL PROMPT — qué se le preguntó exactamente al modelo.

Tamaño: 117 líneas

Comunidades del grafo: Trazabilidad · Huella de prompts · Tipos · typing (compartido)

Lo usan: agente.py, modulos/motor_ia.py, modulos/motor_ia_v2.py, modulos/revision_codigo.py, modulos/skills/base.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
sha256_texto()	63	pública	SHA-256 hexadecimal de un texto UTF-8.
huella_plantilla_prompts()	68	pública	Huella de la LÓGICA de construcción de prompts.

modulos/instrucciones_sesion.py

Propósito: Lectura de instrucciones extra (skills + cuestionamientos NL) que la web deja en la raíz de la sesión (v2 · cableado al motor).

Tamaño: 104 líneas

Comunidades del grafo: Agente · Punto de entrada y cliente IA

Lo usan: agente.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_leer_json_seguro()	41	interna	
leer()	51	pública	Devuelve las instrucciones NL de la sesión, en el orden en el que
leer_entendimiento()	76	pública	Mejora 4 · v2.2 — Devuelve el contenido del entendimiento del

modulos/inyeccion.py

Propósito: Defensa contra INYECCIÓN DE INSTRUCCIONES en la evidencia del modelo validado.

Tamaño: 117 líneas

Comunidades del grafo: Salvaguardas · Inyección de prompts

Lo usan: modulos/motor_ia.py, web/services/solo_informe_ia.py

Tests que lo importan: test_inyeccion.py

Nombre	Línea	Tipo	Qué hace
neutralizar()	80	pública	Neutraliza tramos que parecen instrucciones inyectadas y devuelve
contiene_inyeccion()	114	pública	¿Hay al menos un tramo con pinta de instrucción inyectada?

modulos/motor_ia.py

Propósito: Orquestador principal del agente VIRF. Financial Controller / Model Risk Validator — LaR, VaR, Conductuales.

Tamaño: 2203 líneas

Comunidades del grafo: Motor IA · Evaluar actividades y ámbitos · Motor IA · Scores y semáforo · Tests · Contrato y estadísticas · RAG · Enrutado por dominio

Lo usan: agente.py, config/ia.py, modulos/consolidacion.py, modulos/entailment.py, modulos/revision_codigo.py, modulos/skills/base.py, motor/Mx_03_LlamarIA.py, scripts/auditar_cobertura_rag.py, web/routers/preview_prompt.py, web/services/solo_informe_ia.py

Tests que lo importan: test_azure_mocks.py, test_pendientes_auditoria.py, test_regresion.py, test_tabla_validacion.py

Nombre	Línea	Tipo	Qué hace
_enriquecer_con_rag()	169	interna	Recupera contexto adicional del índice RAG del dominio correspondiente
_es_ambito_metodologia()	275	interna	¿Este ámbito es el de Metodología?
_enmarcar_rag()	298	interna	Envuelve el bloque RAG con las reglas de uso. Devuelve "" si no hay bloque.
_extraer_json()	391	interna	[PARSEO-FIX] Extrae el primer objeto JSON completo de la respuesta del modelo.
_normalizar_inconsistencias()	460	interna	Deja la lista de inconsistencias en una forma que el resto pueda consumir.
_verificar_citas()	505	interna	Comprueba que las citas literales existan REALMENTE en el contexto.
_validar_contrato_evaluacion()	544	interna	[CONTRATO-FIX] Valida el contrato de una evaluación de actividad.
_evaluacion_invalida()	591	interna	[CONTRATO-FIX] Construye una evaluación marcada como NO VÁLIDA.
evaluar_actividad()	636	pública	Evalúa una actividad específica del estándar VIRF.
_pedir_evaluacion()	1259	interna	Llama al modelo hasta obtener una respuesta que cumpla el contrato.
_construir_bloque_antisesgo()	1318	interna	[FIX-BIAS] Construye un bloque de instrucción en el prompt que corrige
evaluar_ambito_completo()	1380	pública	Evalúa todas las actividades requeridas de un ámbito.
generar_resumen_ejecutivo()	1495	pública	Genera el resumen ejecutivo narrativo para el informe Word.
calcular_score_ambito()	1621	pública	Promedio de puntajes de un ámbito, en la escala de política 0–5.
calcular_score_con_ic()	1678	pública	[D1/D6-FIX] Calcula score con intervalo de confianza (IC 90%) y

Nombre	Línea	Tipo	Qué hace
validar_masa_critica_feedback()	1755	pública	[D6-FIX] Verifica que hay suficiente historial de feedback para que el
calcular_score_global()	1790	pública	Score global ponderado por ámbito, en la escala de política 0–5.
aplicar_politicas_procedimiento()	1826	pública	Aplica las políticas de `PROCEDIMIENTO_VALIDACION.md` que la entidad
_umbral_env()	1898	interna	Umbral configurable por el operador; cae al defecto si no es parseable.
resultados_para_sustentar()	1906	pública	Estados cuya fila DEBE ir al chat a sustentarse. 'No cumple' es obligatorio;
veredicto_actividad()	1921	pública	Traduce una evaluación de actividad al veredicto formal de 5 estados. NO
_norm_act()	1953	interna	Normaliza el texto de una actividad para cruzar estándar ↔ evaluación.
_fila_tabla()	1958	interna	
construir_tabla_validacion()	1985	pública	Arma la tabla intermedia (una fila por actividad del estándar) que se muestra
get_semaforo()	2037	pública	Retorna label y color hex según el score. Usa SCORE_SEMAFORO de config.
orquestar_pipeline_virf()	2149	pública	Pipeline 1 — Evaluación por CONTROL (ControlesData.xlsx).
class _ContratoInvalido	387	clase	[CONTRATO-FIX] La respuesta del modelo no cumple el esquema esperado.

modulos/motor_ia_v2.py

Propósito: Orquestador principal del pipeline VIRF por CONTROL.

Tamaño: 945 líneas

Comunidades del grafo: Pipeline · Orquestación por control · Motor IA v2 · Evaluación con datos

Lo usan: agente.py, modulos/motor_ia.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
evaluar_control()	123	pública	Evalúa un control individual mediante el pipeline completo (v13 con fixes).
evaluar_todos_controles()	302	pública	Itera todos los controles del Excel de configuración y los evalúa.
generar_informe_final()	481	pública	Genera informe Word consolidando todos los resultados de controles.
orquestar_pipeline_virf()	726	pública	Orquesta el pipeline completo end-to-end.

modulos/motores_datos.py

Propósito: QUÉ ES El "motor de data": el conjunto de pruebas automáticas que VIRF sabe ejecutar sobre un dataset para validar su calidad.

Tamaño: 609 líneas

Comunidades del grafo: Datos · Motores de calidad y notebook

Lo usan: agente.py, modulos/controles_a_notebook.py, modulos/perfilador_datasets.py, scripts/pipeline_calidad_datos.py, web/routers/controles.py

Tests que lo importan: test_notebook_controles.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_cols()	57	interna	'Cuenta, FechaApertura' → ['Cuenta', 'FechaApertura']
_estado_por_tasa()	69	interna	Traduce el resultado de una prueba a estado, RESPETANDO el umbral que el
_base()	113	interna	
_falta()	122	interna	
_motor_unicidad()	133	interna	Un valor de CAMPO_OBJETIVO por cada combinación de GROUP_COLS.
_motor_duplicados()	169	interna	Filas duplicadas según GROUP_COLS (o la fila completa si no se indica).
_motor_missing()	192	interna	Valores nulos en CAMPO_OBJETIVO (o en todo el dataset).
_motor_outliers()	220	interna	Outliers en CAMPO_OBJETIVO por IQR robusto (no Z-score).
_motor_rango()	266	interna	CAMPO_OBJETIVO dentro de [MIN, MAX] declarados en la config.
_motor_integridad_referencial()	304	interna	CAMPO_OBJETIVO de este dataset debe existir en el dataset de referencia.
_motor_tipo_dato()	350	interna	CAMPO_OBJETIVO debe ser convertible al TIPO_ESPERADO (numerico|fecha|texto).
_motor_completitud()	414	interna	El dataset debe tener al menos MIN_FILAS y todas las columnas de COLUMNAS_REQ.
motores_disponibles()	505	pública	Nombres de las pruebas registradas.
describir_motores()	510	pública	Texto legible del catálogo — se inyecta en los prompts del LLM.
ejecutar_motor()	524	pública	Ejecuta la prueba `tipo` sobre `df` con la configuración `cfg`.
cargar_datasets()	559	pública	Carga todos los datasets de una carpeta en un dict {nombre: DataFrame}.

modulos/motores_entidad.py

Propósito: Lectura de los resultados de los motores de calidad de datos DE LA ENTIDAD.

Tamaño: 489 líneas

Comunidades del grafo: Motores de la entidad · Leer resultados · Datos · Resultados de la entidad · Scripts · Preguntas de oro del RAG

Lo usan: agente.py, scripts/pipeline_calidad_datos.py, web/routers/controles.py

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
_norm()	89	interna	minúsculas, sin acentos, sin signos: para comparar encabezados.
_mapear_columnas()	114	interna	Empareja los encabezados reales con los cuatro campos que se esperan.
_a_fraccion()	140	interna	Convierte «12,5 %», «12.5» o 0.125 a fracción 0-1. None si no es un número.
_numero()	162	interna	Un número o None. Tolera «0,05», «5 %» y celdas vacías.
_escala_de_la_columna()	175	interna	¿La columna viene en porcentaje (0-100) o en fracción (0-1)?
_carpeta()	217	interna	
_leer_tabla()	221	interna	Lee un fichero tabular a lista de dicts. `[]` si no se puede.
leer_resultados()	242	pública	Lee todas las tablas de la carpeta. NUNCA lanza y NUNCA ejecuta nada.
puntaje_por_tasa()	328	pública	Traduce una tasa de incumplimiento (0-1) al puntaje 1-5 del estándar.
_veredicto()	357	interna	True = OK, False = falla, None = no se reconoce.
evaluaciones_desde_motores_entidad()	369	pública	Convierte la tabla de la entidad en evaluaciones del ámbito Calidad de datos.

modulos/notebook_controles.py

Propósito: Genera un .ipynb ENTREGABLE de controles de calidad de datos cuyo output es la tabla de 8 columnas que pide el validador:

Tamaño: 512 líneas

Comunidades del grafo: Tests · Notebook de controles · Controles · Repositorio por dominio · Controles · Construcción del notebook

Lo usan: web/routers/controles.py

Tests que lo importan: test_endpoint_notebook_controles.py, test_notebook_controles.py

Nombre	Línea	Tipo	Qué hace
_normalizar_controles()	71	interna	Renumera a C01.. y adjunta Dimensión / Control Esperado / Evidencia /
construir_notebook_8cols()	102	pública	Arma el notebook. `meta` admite: titulo, fuente, resumen, proyecto_dir,
a_ipynb_bytes()	229	pública	
construir_notebook_repositorio()	291	pública	Notebook cuyos controles son FUNCIONES PYTHON del repositorio (una celda por
generar_controles_desde_entendimiento()	410	pública	El agente propone controles cuando no hay documento. Usa el LLM.
generar_ipynb_controles()	444	pública	Orquestador. PRIORIDAD 1: repositorio de controles Python del dominio

modulos/override_humano.py

Propósito: REPORTE DE HUMAN OVERRIDE — dónde el validador corrigió al agente.

Tamaño: 271 líneas

Comunidades del grafo: Validador · Correcciones humanas (override) · Tests · Anexo de correcciones · Feedback · Calidad del proceso y autoría · Agente · Punto de entrada y cliente IA

Lo usan: agente.py, modulos/generador_word.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_mismo_modelo()	39	interna	
overrides_de_modelo()	43	pública	Todas las correcciones registradas para un modelo, en orden de registro.
_sesion()	55	interna	Agrupador de sesión de revisión: la FECHA (día) del registro.
_orden_sesion()	66	interna	Clave de orden cronológico a partir de 'dd/mm/aaaa'.
ultimo_ciclo()	86	pública	Correcciones de la sesión de revisión más reciente de ese modelo.
resumen_override()	100	pública	Datos del anexo y de la métrica de independencia.
linea_independencia()	157	pública	Línea para el resumen ejecutivo del informe (cara al regulador).
calidad_proceso()	172	pública	Señal interna: ¿este validador está revisando a fondo o aprobando en bloque?
guardar_calidad_proceso()	252	pública	Persiste el reporte interno junto a las salidas, en fichero separado.

modulos/perfilador_datasets.py

Propósito: PERFILADO DE DATASETS — el esquema como evidencia, no los datos.

Tamaño: 342 líneas

Comunidades del grafo: Datos · Perfilado de datasets · Datos · Perfilado de carpetas · Tipos · typing (compartido)

Lo usan: agente.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_describir_rango()	144	interna	Mínimo y máximo como texto, o (None, None) si la columna no los admite.
_intentar_fechas()	169	interna	Si una columna `object` contiene fechas, devuelve su rango.
perfilar_dataframe()	201	pública	Perfila un DataFrame ya cargado. No lee disco, no llama al LLM.
perfilar_datasets()	264	pública	Perfila un dict {nombre: DataFrame} y devuelve el bloque Markdown.
perfilar_carpeta()	299	pública	Carga los datasets de una carpeta y devuelve su perfil.
documento_perfil()	314	pública	Devuelve el perfil envuelto como documento de evidencia, o None si no hay
class PerfilColumna	88	clase	Descripción de una columna. Sin datos, solo esquema y estadística. Métodos: a_linea
class PerfilDataset	120	clase	Descripción de un dataset completo. Métodos: a_texto

modulos/pii.py

Propósito: Enmascarado DETERMINISTA de datos personales (PII) en la evidencia, ANTES de que un extracto salga hacia el proveedor de generación.

Tamaño: 251 líneas

Comunidades del grafo: Salvaguardas · Enmascarado de PII

Lo usan: modulos/motor_ia.py, web/services/solo_informe_ia.py

Tests que lo importan: test_pii.py

Nombre	Línea	Tipo	Qué hace
_luhn_valido()	71	interna	Algoritmo de Luhn (tarjetas de pago). Rechaza la inmensa mayoría de las
_iban_valido()	92	interna	Validación mod-97 (ISO 13616): mueve los 4 primeros caracteres al final,
_dni_nie_valido()	110	interna	DNI (8 dígitos + letra) o NIE (X/Y/Z + 7 dígitos + letra) con letra de
_sub_validado()	159	interna	Sustituye las coincidencias de `patron` que pasen `validar` por su
enmascarar()	178	pública	Enmascara PII en `texto` y devuelve (texto_enmascarado, hallazgos).
contiene_pii()	248	pública	¿Hay al menos un dato personal detectable? Útil para tests y avisos.

modulos/plantilla_bcp.py

Propósito: Plantilla común de los informes Word de VIRF (estética «Informe de Resultados»).

Tamaño: 958 líneas

Comunidades del grafo: Informes · Plantilla común BCP · Informes · API de la plantilla · Informes · Reglas de calificación y cumplimiento · Informes · Citas y resaltado en párrafo

Lo usan: modulos/generador_word.py, scripts/generar_guia_agente.py, web/services/informe_web.py, web/services/solo_informe_word.py

Tests que lo importan: test_plantilla_bcp.py

Nombre	Línea	Tipo	Qué hace
_set_cell_bg()	64	interna	
_cell_left_accent()	72	interna	Barra de color en el borde izquierdo de la celda (acento de callout).
_cell_margins()	83	interna	Padding interno de una celda (en twips: 20 = 1 pt aprox.).
_run()	94	interna	
_p()	105	interna	
_etiqueta_valor()	116	interna	Línea «Etiqueta: valor», etiqueta en color, cuerpo justificado.
_caption()	128	interna	Pie de tabla/figura: pequeño, gris, centrado.
_forzar_ancho()	134	interna	Fija el ancho de columnas (cm) en TODAS las filas y desactiva autofit.
_tbl_header_repeat()	150	interna	Repite la fila de encabezado si la tabla se parte entre páginas.
_tabla()	156	interna	Tabla editorial: encabezado sólido y sutil, columnas proporcionales,
_callout()	185	interna	
_bloque_codigo()	204	interna	
_linea_horizontal()	216	interna	
_salto_pagina()	224	interna	
_estilo_base()	228	interna	Fuente Flexo, interlineado, control de viudas y estilos de título.
fecha_larga()	303	pública	
periodo_validacion()	311	pública	«octubre de 2026» si inicio y entrega caen en el mismo mes; si no,
_parrafos()	327	interna	

Nombre	Línea	Tipo	Qué hace
_texto_o_gancho()	335	interna	
_bullet()	342	interna	
no_partir()	352	pública	Filas indivisibles; con `entera`, además cada fila pegada a la siguiente
resaltar()	376	pública	Fondo y barra izquierda a nivel de PÁRRAFO (sin tabla).
campo()	395	pública	Línea «Etiqueta: texto» del barrido (texto plano estructurado).
cita()	409	pública	Cita o aviso destacado en PÁRRAFO sombreado (sin tabla), para el barrido.
titulo()	438	pública	
_fld()	450	interna	
_hipervinculo()	455	interna	
_indice()	467	interna	Campo TOC real (Word lo regenera con números de página al abrir: se activa
_logo()	491	interna	
_encabezado_pie()	496	interna	
_es_periodica()	526	interna	
_portada()	530	interna	
_objetivos()	561	interna	
_alcance()	568	interna	
observaciones_por_control()	583	pública	
control_satisfactorio()	590	pública	Un control no evaluado (fallo técnico) no puede darse por satisfactorio.
cumplimiento()	595	pública	Fila por ámbito: total, satisfactorios y no cumplidos (con %).
_cumplimiento()	612	interna	
_ficha_documento()	638	interna	Cuadro de control del documento al inicio de «Información Relevante».
_info_relevante()	664	interna	

Nombre	Línea	Tipo	Qué hace
_cuadro_resumen()	710	interna	
_glosario()	735	interna	
_imagen()	744	interna	
_sustento()	756	interna	
_barrido()	776	interna	Barrido de TODO el estándar en texto estructurado (sin tablas): por ámbito,
_quitar_salto_final()	855	interna	Quita un salto de página vacío al final de un anexo: la plantilla ya abre
generar()	873	pública	Escribe el informe en `ruta`. `inf` (normalizado por cada informe):
class Marcadores	425	clase	Métodos: envolver

modulos/prevuelo.py

Propósito: PRE-VUELO DE LA EVIDENCIA — qué se va a poder leer, antes de gastar.

Tamaño: 289 líneas

Comunidades del grafo: Prevuelo · Hallazgos e informe · Prevuelo · Chequeo previo de evidencia · Validación · Política sin HuggingFace

Lo usan: agente.py, web/routers/dominio_prevuelo.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
_clasificar_fallo()	112	interna	Distingue un PDF escaneado de otro fallo de lectura.
ejecutar_prevuelo()	122	pública	Recorre las carpetas de evidencia y clasifica cada fichero.
formatear()	217	pública	Informe legible para consola.
class Hallazgo	53	clase	Un fichero que no va a llegar al evaluador, y por qué. Métodos: nombre
class InformePrevuelo	66	clase	Métodos: chars_totales, n_problemas, hay_bloqueantes, a_dict

modulos/registro_emisiones.py

Propósito: REGISTRO APPEND-ONLY DE EMISIONES — control PREVENTIVO de la cadena de custodia.

Tamaño: 409 líneas

Comunidades del grafo: Tipos · typing (compartido) · Custodia · Registro de emisiones · Agente · Punto de entrada y cliente IA

Lo usan: agente.py, modulos/custodia.py

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
_añadir_linea()	101	interna	Añade UNA línea al registro, con lock entre procesos y persistencia física.
_firmante()	144	interna	Extrae validador y cargo del propio informe.
registrar_emision()	162	pública	Anota en el registro que un informe SELLADO se ha emitido.
leer_registro()	204	pública	Devuelve las entradas del registro, en orden de escritura.
_indexar_outputs()	236	interna	Índice `hash de contenido → fichero` de todos los informes en `outputs/`.
verificar_emisiones()	266	pública	Contrasta el registro de emisiones contra los informes presentes.
imprimir_emisiones()	362	pública	

modulos/reglas_informe.py

Propósito: Reglas puras de calificación de los informes VIRF: conclusión global y estado por control. Las usan los TRES informes Word (Solo Informe, «Generar informe» de la web y el entregable del pipeline), para que una misma situación reciba siempre la misma calificación.

Tamaño: 239 líneas

Comunidades del grafo: Informes · Reglas de calificación y cumplimiento

Lo usan: modulos/generador_word.py, modulos/plantilla_bcp.py, web/services/informe_web.py, web/services/solo_informe_reglas.py

Tests que lo importan: test_plantilla_bcp.py

Nombre	Línea	Tipo	Qué hace
_sin_tildes()	59	interna	
clave_ambito()	64	pública	Clave canónica de un ámbito (desplegable o estándar). '' si no casa.
normalizar_riesgo()	73	pública	Devuelve el riesgo del catálogo que corresponde a `texto` (tolerante a tildes).
riesgo_maximo()	86	pública	Riesgo más grave entre las observaciones de un control. None si no hay.
es_om()	94	pública	
control_cumple()	98	pública	
conclusion_control()	117	pública	Frase de conclusión del barrido de controles.
_contar()	127	interna	
concluir()	145	pública	Conclusión global del informe a partir de las observaciones registradas.
riesgo_desde_puntaje()	218	pública	
texto_conclusion()	236	pública	Párrafo de conclusión para la sección IV.

modulos/reporte_docx.py

Propósito: Reporte `.docx` estructurado desde un ValidationReport (WP-08 estructura · GAP-44/45/46/47/48).

Tamaño: 193 líneas

Comunidades del grafo: Informe · Contrato ValidationReport · Tests · DOCX de regresión

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_docx_regresion.py

Nombre	Línea	Tipo	Qué hace
veredicto_de_ambito()	49	pública	Deriva el veredicto de un ámbito a partir de los `status` de sus hallazgos.
_agrupar_por_ambito()	70	interna	
_añadir_pie_con_paginacion()	78	interna	Inserta en el pie de cada sección un campo dinámico "Página X de Y".
_run_texto()	95	interna	
_run_campo()	103	interna	Añade un campo de Word (fldChar begin → instrText → fldChar end).
generar_docx_validacion()	118	pública	Genera el `.docx` de validación con las cuatro secciones obligatorias.

modulos/repositorio_controles.py

Propósito: REPOSITORIO de controles de datos ESCRITOS COMO FUNCIONES PYTHON, organizados por tipo de modelo. Cuando el modelo activo es de dominio VaR, el notebook toma los controles de `inputs/repositorio_controles/var/`; si es LaR, de `.../lar/`; etc. Así el .ipynb entregable es ESPECIALIZADO, no genérico.

Tamaño: 143 líneas

Comunidades del grafo: Controles · Repositorio por dominio · Tipos · typing (compartido)

Lo usan: modulos/notebook_controles.py

Tests que lo importan: test_endpoint_notebook_controles.py

Nombre	Línea	Tipo	Qué hace
carpeta_de_dominio()	73	pública	Normaliza el nombre de dominio al de su carpeta en el repositorio.
_extraer_meta()	81	interna	Ejecuta el .py del control en un espacio aislado para leer META y validar
cargar_controles()	100	pública	Devuelve (controles, avisos) para el `dominio` + `shared`. Cada control:

modulos/revision_codigo.py

Propósito: REVISIÓN DEL CÓDIGO CONTRA LA DOCUMENTACIÓN — una llamada por módulo.

Tamaño: 1053 líneas

Comunidades del grafo: Revisión de código · Hallazgos · Configuración · Parámetros · Tests · SQL y R como código del modelo · Agente · Punto de entrada y cliente IA

Lo usan: agente.py

Tests que lo importan: test_azure_mocks.py, test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_aparece()	124	interna	¿El identificador aparece en el texto, como palabra completa?
recategorizar()	131	pública	Corrige la categoría cuando CONTRADICE los hechos. Devuelve `(cat, motivo)`.
verificar_referencia_manual()	189	pública	¿La referencia al manual existe DE VERDAD en los manuales?
_excluir_dependencias_activo()	324	interna	Lee el flag EN RUNTIME, no al importar.
descubrir_modulos()	339	pública	Módulos de código bajo `raiz`, recursivo.
inventario()	382	pública	Inventario con coste estimado. NO llama al modelo.
formatear_inventario()	440	pública	Inventario legible para consola. Se imprime SIEMPRE antes de gastar.
_numerar_lineas()	476	interna	Prefija cada línea con su número.
_sentencia_cerrada()	516	interna	True si el texto acumulado termina en el terminador de sentencia.
_lenguaje_de()	530	interna	Lenguaje del fichero a partir de su extensión.
_partir_por_definiciones()	546	interna	Parte un módulo grande en bloques que empiezan en una frontera de definición.
informe_markdown()	965	pública	Informe Markdown del resultado consolidado.
class HallazgoCodigo	277	clase	Un hallazgo atribuido a un punto concreto del código. Métodos: a_dict, ubicacion
class RevisorCodigo	687	clase	Revisa el código de un modelo contra su documentación, módulo a módulo. Métodos: revisar_bloque, revisar_modulo, revisar, consolidar

modulos/skills/base.py

Propósito: Infraestructura común de las skills de validación.

Tamaño: 310 líneas

Comunidades del grafo: Skills · Catálogo y clase base · Skills · Citas inventadas (challenger, supuestos) · Skills · Skill de síntesis

Lo usan: modulos/skills/catalogo.py, skills/base_tipada.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class ResultadoSkill	62	clase	Lo que devuelve una skill. Es un dict para que viaje al JSON sin conversión.
class SkillBase	71	clase	Base de todas las skills. Subclasificar e implementar `construir_prompt`. Métodos: construir_prompt, ejecutar

modulos/skills/catalogo.py

Propósito: Las ocho skills de validación.

Tamaño: 616 líneas

Comunidades del grafo: Skills · Catálogo y clase base · Skills · Citas inventadas (challenger, supuestos) · Skills · Skill de síntesis · Skills · Regulatoria con RAG

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
obtener_skill()	601	pública	Clase de la skill por nombre. Lanza ValueError con las opciones válidas.
listar_skills()	609	pública	Catálogo para `--listar-skills`.
class SkillCoherencia	43	clase	El hallazgo más valioso de una validación y el más caro de obtener a mano. Métodos: construir_prompt
class SkillSupuestos	125	clase	Todo método tiene supuestos. Casi ninguna documentación los verifica. Métodos: construir_prompt
class SkillPapers	190	clase	Una cita no es una justificación. Métodos: construir_prompt
class SkillChallenger	247	clase	Qué habría hecho un equipo independiente con los mismos datos. Métodos: construir_prompt
class SkillRegulatorio	317	clase	Qué exige la norma que aquí no está. Métodos: construir_prompt
class SkillGobierno	378	clase	Quién aprueba, quién revisa, quién puede tocarlo. Métodos: construir_prompt
class SkillDatos	436	clase	Un modelo no es mejor que los datos que lo alimentan. Métodos: construir_prompt
class SkillSintesis	494	clase	De todo lo anterior, ¿qué es lo verdaderamente grave? Métodos: construir_prompt

modulos/validador_citas.py

Propósito: Validador DETERMINISTA de citas con id de linaje (WP-03/WP-04 · indicador D4).

Tamaño: 155 líneas

Comunidades del grafo: Citas · Validación determinista

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_citas_deterministas.py

Nombre	Línea	Tipo	Qué hace
extraer_ids()	50	pública	Devuelve los ids de cita `[doc:pagina:chunk]` presentes en el texto.
id_de_chunk()	58	pública	Construye el id de linaje de un fragmento recuperado, si trae los campos.
_ids_disponibles()	75	interna	
validar_afirmacion()	84	pública	Valida UNA afirmación factual contra el contexto recuperado.
verificar_afirmaciones()	123	pública	Valida una lista de afirmaciones. Resumen agregado para el informe/verificador.
verificar_entailment()	143	pública	Verificación SEMÁNTICA (NLI/entailment) — DIFERIDA al día de API.

modulos/validador_humano.py

Propósito: FIRMA Y RESPONSABILIDAD DEL VALIDADOR HUMANO — segregación de funciones.

Tamaño: 456 líneas

Comunidades del grafo: Agente · Punto de entrada y cliente IA · Tipos · typing (compartido) · Firma humana · Doble firma · Validador · Cumplimiento regulatorio

Lo usan: agente.py, modulos/feedback.py, modulos/override_humano.py

Tests que lo importan: test_pendientes_auditoria.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_leer()	76	interna	Lectura de consola. Devuelve None si la entrada se agotó (EOF/Ctrl-C).
solicitar_identidad()	92	pública	Resuelve la identidad del validador responsable del informe.
_cargar_registro()	173	interna	
registrar_validacion()	194	pública	Añade una firma al registro acumulado, con lock y escritura atómica.
historial_por_validador()	234	pública	Agrupa el registro por validador: cuántos modelos y con qué fechas.
firmas_consecutivas()	261	pública	Cuántas firmas consecutivas más recientes del modelo son de este validador.
aviso_segunda_revision()	279	pública	Devuelve el texto del aviso si falta segunda revisión, o None.
_normalizar_nombre()	315	interna	Forma canónica de un nombre, para comparar dos firmantes.
requiere_doble_firma()	333	pública	¿Este tier exige dos firmas? Ver `config.parametros.tiers_doble_firma`.
solicitar_segunda_firma()	338	pública	Resuelve y valida la firma del SEGUNDO validador.
imprimir_historial()	433	pública	
class IdentidadRequerida	58	clase	Se intentó emitir un informe de producción sin validador identificable.
class SegundaFirmaRequerida	62	clase	Se intentó emitir un informe de un tier con doble firma sin segundo

modulos/ventana_contexto.py

Propósito: Resolución de la ventana de contexto real del modelo (WP-10 · GAP-40).

Tamaño: 107 líneas

Comunidades del grafo: RAG · Vigencia y ventana de contexto

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_versionado.py

Nombre	Línea	Tipo	Qué hace
resolver_ventana()	48	pública	Resuelve el presupuesto de contexto (en caracteres) del modelo configurado.
max_ctx_efectivo()	104	pública	Atajo: solo el presupuesto de contexto en caracteres.

modulos/versionado.py

Propósito: Versionado temporal de chunks/estándares (WP-10 · GAP-23).

Tamaño: 97 líneas

Comunidades del grafo: RAG · Vigencia y ventana de contexto · Tipos · typing (compartido)

Lo usan: src/rag/buscador_lexico.py

Tests que lo importan: test_versionado.py

Nombre	Línea	Tipo	Qué hace
_a_fecha()	34	interna	Parsea una fecha ISO (YYYY-MM-DD) de forma tolerante. None si no aplica.
chunk_activo()	48	pública	¿Este chunk está vigente?
filtrar_activos()	75	pública	Pre-filtro duro: devuelve solo los chunks vigentes, preservando el orden.
marcar_version()	81	pública	Estampa metadatos de versión sobre un chunk (aditivo; no toca su texto).

modulos/word_estilo.py

Propósito: Infraestructura de presentación del informe Word. Reutilizable: aquí vive el CÓMO se ve el documento, no QUÉ dice. `generador_word.py` decide el contenido.

Tamaño: 543 líneas

Comunidades del grafo: Informe Word · Estilos y formato · Informe Word · Logo · Informes · Reglas de calificación y cumplimiento

Lo usan: modulos/generador_word.py, modulos/plantilla_bcp.py

Tests que lo importan: test_pendientes_auditoria.py, test_solo_informe.py

Nombre	Línea	Tipo	Qué hace
fuentes_instaladas()	63	pública	Familias tipográficas visibles para el sistema. Vacío si no se puede.
resolver_fuente()	91	pública	Decide con qué familia se escribe el documento, y lo DECLARA.
aplicar_fuente()	121	pública	Fija la familia en las CUATRO ranuras que usa Word.
registrar_respaldo_fuente()	141	pública	Declara en `fontTable.xml` por qué fuente sustituir si `Flexo` no está.
aplicar_estilo_documento()	188	pública	Fija la tipografía y la justificación en el estilo `Normal`.
_nodo()	230	interna	
justificar()	241	pública	Justifica un párrafo de prosa.
_sombrear()	261	interna	
_borde()	270	interna	`lados`: cualquier combinación de t/b/l/r.
add_bloque_codigo()	287	pública	Inserta código con aspecto de celda: monoespaciado, fondo, borde y números.
add_codigo_inline()	338	pública	Un fragmento monoespaciado dentro de una frase, con fondo tenue.
_asegurar_parte_footnotes()	402	interna	Devuelve el elemento `<w:footnotes>`, creándolo la primera vez.
_asegurar_estilos_nota()	418	interna	Crea los estilos que Word espera para las notas, si no existen.
add_nota_pie()	435	pública	Ancla una nota al pie al final de `parrafo` y devuelve su número.
localizar_logo()	501	pública	Primer fichero de imagen en `inputs/logoempresa/`. None si no hay.
logo_para_word()	519	pública	Devuelve el logo en un formato que Word admita.
class _FootnotesPart	384	clase	Parte OPC mínima que envuelve el XML de notas al pie. Métodos: element, blob

Carpeta src/

Archivo	Líneas	Propósito
src/rag/ALM/indexer.py	24	Indexador RAG para el dominio ALM (modelos conductuales: prepago, retiro de depósitos, NMD; IRRBB). Indexa manuales conductuales, EBA guidel
src/rag/ALM/retriever.py	11	Retriever RAG para el dominio ALM (modelos conductuales: prepago, supervivencia/ retiro de depósitos, NMD, IRRBB). Recupera contexto de comp
src/rag/base_indexer.py	124	Clase base para indexadores de todos los dominios RAG.
src/rag/base_retriever.py	117	Clase base para retrievers de todos los dominios RAG.
src/rag/buscador_lexico.py	747	Consulta el índice léxico de un dominio y devuelve los top-K chunks.
src/rag/config_rag.py	190	Parámetros del RAG LÉXICO (BM25 + TF-IDF). Sin embeddings, sin HuggingFace.
src/rag/custom_retriever.py	54	Retriever + indexer para dominios RAG PERSONALIZADOS (Mejora 1 · v2.2).
src/rag/indexador_lexico.py	733	Construye y persiste el índice LÉXICO (BM25 + TF-IDF) de un dominio RAG.
src/rag/LaR/indexer.py	39	Indexador RAG para modelos de Liquidity at Risk (LaR).
src/rag/LaR/retriever.py	23	Retriever RAG para modelos de Liquidity at Risk (LaR).
src/rag/shared/indexer.py	23	Indexador RAG de conocimiento regulatorio compartido (transversal a todos los dominios). Indexa: SR 11-7, EBA/GL/2023/01, BCBS 239, papers r
src/rag/shared/retriever.py	10	Retriever de conocimiento regulatorio compartido. Fallback cuando el dominio del modelo no tiene índice específico. Uso: SharedRetriever().q
src/rag/tesauro.py	228	Tesauro de dominio: los sinónimos que la búsqueda léxica no puede adivinar.
src/rag/VaR/indexer.py	23	Indexador RAG para modelos de Value at Risk (VaR). Indexa manuales metodológicos VaR, papers FRTB/Basilea III y backtesting. Hereda de BaseI
src/rag/VaR/retriever.py	10	Retriever RAG para modelos de Value at Risk (VaR). Recupera contexto de riesgo de mercado (FRTB, Basilea III, Kupiec) para el prompt. Uso: V

src/rag/ALM/indexer.py

Propósito: Indexador RAG para el dominio ALM (modelos conductuales: prepago, retiro de depósitos, NMD; IRRBB). Indexa manuales conductuales, EBA guidelines, documentos de IRRBB y de seguimiento. Hereda de BaseIndexer. Uso: ALMIndexer().build_index()

Tamaño: 24 líneas

Comunidades del grafo: RAG · Fuentes por dominio · Tipos · typing (compartido) · RAG · Indexadores por dominio y reconstrucción

Lo usan: scripts/generar_corpus_rag.py, scripts/rebuild_rag_databricks.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class ALMIndexer	13	clase	Métodos: get_source_paths

src/rag/ALM/retriever.py

Propósito: Retriever RAG para el dominio ALM (modelos conductuales: prepago, supervivencia/ retiro de depósitos, NMD, IRRBB). Recupera contexto de comportamiento de clientes y de riesgo de tasa para el prompt. Uso: ALMRetriever().query_para_prompt(texto, top_k=3)

Tamaño: 11 líneas

Comunidades del grafo: RAG · Recuperador y resolución de dominio

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class ALMRetriever	10	clase	

src/rag/base_indexer.py

Propósito: Clase base para indexadores de todos los dominios RAG.

Tamaño: 124 líneas

Comunidades del grafo: RAG · Indexador base · RAG · Indexadores por dominio y reconstrucción · Tipos · typing (compartido) · Tests · RAG no indexa la evidencia

Lo usan: src/rag/ALM/indexer.py, src/rag/LaR/indexer.py, src/rag/VaR/indexer.py, src/rag/custom_retriever.py, src/rag/shared/indexer.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class BaseIndexer	57	clase	Indexador base para un dominio RAG. Métodos: get_source_paths, build_index, cargar_indice, estado

src/rag/base_retriever.py

Propósito: Clase base para retrievers de todos los dominios RAG.

Tamaño: 117 líneas

Comunidades del grafo: RAG · Recuperador y resolución de dominio

Lo usan: src/rag/ALM/retriever.py, src/rag/LaR/retriever.py, src/rag/VaR/retriever.py, src/rag/custom_retriever.py, src/rag/shared/retriever.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class BaseRetriever	34	clase	Retriever base para un dominio RAG. Métodos: query, query_para_prompt, estado

src/rag/buscador_lexico.py

Propósito: Consulta el índice léxico de un dominio y devuelve los top-K chunks.

Tamaño: 747 líneas

Comunidades del grafo: RAG · Buscador léxico · RAG · Indexador léxico

Lo usan: scripts/auditar_cobertura_rag.py, scripts/explicar_rag.py, scripts/preguntas_oro_formula.py, scripts/preguntas_oro_rag.py, src/rag/base_retriever.py

Tests que lo importan: test_aislamiento.py, test_azure_mocks.py, test_beta_calibracion.py, test_pendientes_auditoria.py, test_rag_lexico.py, test_versionado.py

Nombre	Línea	Tipo	Qué hace
get_buscador()	737	pública	Devuelve el buscador del dominio, reutilizándolo dentro del proceso.
_reset_para_tests()	745	interna	Vacía la caché de proceso. Solo para los tests.
class BuscadorLexico	49	clase	Buscador léxico de un dominio. Carga perezosa y caché en proceso. Métodos: buscar, buscar_para_prompt, diagnostico_consulta, estado

src/rag/config_rag.py

Propósito: Parámetros del RAG LÉXICO (BM25 + TF-IDF). Sin embeddings, sin HuggingFace.

Tamaño: 190 líneas

Comunidades del grafo: Tipos · typing (compartido) · RAG · Carpeta de índices

Lo usan: scripts/explicar_rag.py, scripts/rebuild_rag_databricks.py, src/rag/base_indexer.py, src/rag/buscador_lexico.py, src/rag/indexador_lexico.py, web/routers/rag.py

Tests que lo importan: test_azure_mocks.py, test_rag_lexico.py, test_regresion.py

Nombre	Línea	Tipo	Qué hace
_indice_dir_defecto()	88	interna	Carpeta donde se persisten los índices léxicos, uno por dominio.

src/rag/custom_retriever.py

Propósito: Retriever + indexer para dominios RAG PERSONALIZADOS (Mejora 1 · v2.2).

Tamaño: 54 líneas

Comunidades del grafo: Tipos · typing (compartido) · RAG · Indexadores por dominio y reconstrucción · RAG · Recuperador y resolución de dominio · RAG · Fuentes por dominio

Lo usan: scripts/rebuild_rag_databricks.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class CustomDomainRetriever	26	clase	Retriever paramétrico: `CustomDomainRetriever("capital_economico")`
class CustomDomainIndexer	40	clase	Indexer paramétrico: `CustomDomainIndexer("capital_economico") Métodos: get_source_paths

src/rag/indexador_lexico.py

Propósito: Construye y persiste el índice LÉXICO (BM25 + TF-IDF) de un dominio RAG.

Tamaño: 733 líneas

Comunidades del grafo: RAG · Indexador léxico · RAG · Buscador léxico · Tests · RAG no indexa la evidencia · RAG · Fuentes por dominio

Lo usan: agente.py, scripts/auditar_cobertura_rag.py, scripts/explicar_rag.py, scripts/preguntas_oro_formula.py, scripts/preguntas_oro_rag.py, scripts/rebuild_rag_databricks.py, scripts/verificar_corpus_rag.py, src/rag/base_indexer.py, src/rag/buscador_lexico.py, src/rag/tesauro.py, web/services/solo_informe_ia.py

Tests que lo importan: test_azure_mocks.py, test_rag_lexico.py, test_regresion.py, test_versionado.py

Nombre	Línea	Tipo	Qué hace
quitar_acentos()	170	pública	Normaliza acentos: «ponderación» y «ponderacion» deben ser el mismo token.
_despluralizar()	187	interna	Quita el plural aplicando la regla real del español (que también cubre el
_stem()	216	interna	Stemmer conservador: despluralizar y, como mucho, UN recorte derivativo.
tokenizar()	249	pública	Texto → lista de tokens normalizados. MISMA función para chunks y consultas.
vocabulario_crudo()	279	pública	Palabras del corpus SIN stem, para corregir erratas de la consulta.
texto_normalizado_para_tfidf()	295	pública	Versión del texto lista para `TfidfVectorizer`.
raiz_proyecto()	312	pública	Raíz del proyecto. Un único sitio donde calcularla.
fuentes_rag()	317	pública	Carpeta de LITERATURA DE REFERENCIA del dominio: `inputs/rag/<dominio>/`.
ruta_indice()	346	pública	Fichero donde se persiste el índice léxico de un dominio.
class IndiceLexico	356	clase	Lo que se guarda en disco. Deliberadamente, DATOS y no objetos vivos.
class IndexadorLexico	400	clase	Construye, persiste y recarga el índice léxico de un dominio. Métodos: get_source_paths, construir, cargar, estado

src/rag/LaR/indexer.py

Propósito: Indexador RAG para modelos de Liquidity at Risk (LaR).

Tamaño: 39 líneas

Comunidades del grafo: RAG · Fuentes por dominio · Tipos · typing (compartido) · RAG · Indexadores por dominio y reconstrucción

Lo usan: scripts/generar_corpus_rag.py, scripts/rebuild_rag_databricks.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class LaRIndexer	27	clase	Métodos: get_source_paths

src/rag/LaR/retriever.py

Propósito: Retriever RAG para modelos de Liquidity at Risk (LaR).

Tamaño: 23 líneas

Comunidades del grafo: RAG · Recuperador y resolución de dominio

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class LaRRetriever	22	clase	

src/rag/shared/indexer.py

Propósito: Indexador RAG de conocimiento regulatorio compartido (transversal a todos los dominios). Indexa: SR 11-7, EBA/GL/2023/01, BCBS 239, papers regulatorios generales. Es el fallback cuando no hay un dominio específico. Hereda de BaseIndexer.

Tamaño: 23 líneas

Comunidades del grafo: RAG · Fuentes por dominio · Tipos · typing (compartido) · RAG · Indexadores por dominio y reconstrucción

Lo usan: scripts/generar_corpus_rag.py, scripts/rebuild_rag_databricks.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class SharedIndexer	12	clase	Métodos: get_source_paths

src/rag/shared/retriever.py

Propósito: Retriever de conocimiento regulatorio compartido. Fallback cuando el dominio del modelo no tiene índice específico. Uso: SharedRetriever().query_para_prompt(texto, top_k=3)

Tamaño: 10 líneas

Comunidades del grafo: RAG · Recuperador y resolución de dominio

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SharedRetriever	9	clase	

src/rag/tesauro.py

Propósito: Tesauro de dominio: los sinónimos que la búsqueda léxica no puede adivinar.

Tamaño: 228 líneas

Comunidades del grafo: RAG · Buscador léxico · RAG · Tesauro

Lo usan: src/rag/buscador_lexico.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_normalizar()	146	interna	Un término del tesauro → los tokens que produciría el índice.
cargar_tesauro()	158	pública	Construye el índice de expansión: token → tokens equivalentes.
expandir_token()	226	pública	Token → él mismo más sus equivalentes conocidos.

src/rag/VaR/indexer.py

Propósito: Indexador RAG para modelos de Value at Risk (VaR). Indexa manuales metodológicos VaR, papers FRTB/Basilea III y backtesting. Hereda de BaseIndexer. Uso: VaRIndexer().build_index()

Tamaño: 23 líneas

Comunidades del grafo: RAG · Fuentes por dominio · Tipos · typing (compartido) · RAG · Indexadores por dominio y reconstrucción

Lo usan: scripts/generar_corpus_rag.py, scripts/rebuild_rag_databricks.py

Tests que lo importan: test_regresion.py

Nombre	Línea	Tipo	Qué hace
class VaRIndexer	12	clase	Métodos: get_source_paths

src/rag/VaR/retriever.py

Propósito: Retriever RAG para modelos de Value at Risk (VaR). Recupera contexto de riesgo de mercado (FRTB, Basilea III, Kupiec) para el prompt. Uso: VaRRetriever().query_para_prompt(texto, top_k=3)

Tamaño: 10 líneas

Comunidades del grafo: RAG · Recuperador y resolución de dominio

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_rag_lexico.py

Nombre	Línea	Tipo	Qué hace
class VaRRetriever	9	clase	

Carpeta web/

Archivo	Líneas	Propósito
web/app.py	117	VIRF Web — capa web sobre el agente CLI.
web/config_web.py	18	Rutas y constantes del backend web.
web/routers/auth.py	58	Autenticación por código + nombre (v2 · fase 1).
web/routers/automatizaciones.py	109	Cuadro de observaciones desde los Excels de automatización (00–03).
web/routers/chat.py	280	Chat argumentativo por sesión (v2 · fase 7).
web/routers/config.py	266	Configuración del proveedor IA (v2 · fase 2).
web/routers/controles.py	162	Controles de calidad de datos (v2 · fase 2).
web/routers/cuestionamientos.py	168	Cuestionamientos metodológicos — paso 9/10.
web/routers/cuestionamientos_sesion.py	58	Cuestionamientos POR SESIÓN — endpoints (v2 · fase 6).
web/routers/debug_conexiones.py	23	Endpoint HTTP para el verificador de conexiones (Mejora 5 · v2.2). Misma contraseña que el visor de prompt.
web/routers/dominio_prevuelo.py	51	Pre-vuelo sobre la EVIDENCIA de la sesión activa (v2 · fase 2).
web/routers/entendimiento.py	63	Endpoints del mapa de entendimiento del modelo (Mejora 4 · v2.2).
web/routers/estandares.py	84	Router del paso 02 · Estándares (v2.4).
web/routers/guia.py	81	Guía del procedimiento: en qué paso está el expediente.
web/routers/informe.py	106	Informe consolidado por sesión (v2 · fase 8).
web/routers/modelos.py	92	Modelos del validador logueado + selección del modelo activo (v2 · fase 1).
web/routers/preview_prompt.py	127	Visor del prompt del agente, protegido con contraseña (Mejora 2 · v2.2).
web/routers/prompts.py	93	Endpoints CRUD de prompts editables por sesión (v2.3).
web/routers/rag.py	253	Dominio & RAG: papers al RAG global + reconstrucción + dominios custom.
web/routers/resultados.py	194	Lectura de resultados de validación (v2 · fase 2).
web/routers/skills.py	65	Endpoints de Skills — capacidades del agente por sesión (v2 · fase 5).
web/routers/solo_informe.py	249	Router del apartado «Solo Informe»: genera el Informe de Resultados sin pipeline.
web/routers/upload.py	288	Endpoints de subida de evidencia y visualización del árbol de inputs.

Archivo	Líneas	Propósito
web/routers/validacion.py	117	Endpoints REST + WebSocket del pipeline de validación (v2 · fase 2).
web/services/analisis_codigo.py	389	Análisis estructural del ZIP de implementación (Mejora 4 · v2.2).
web/services/chat_service.py	589	Chat argumentativo por sesión — v2 · fase 7.
web/services/contexto_sesion.py	307	Resolución del contexto de sesión (v2 · fase 2).
web/services/cuestionamientos_sesion.py	154	Cuestionamientos POR SESIÓN (v2 · fase 6).
web/services/dominios_custom_store.py	170	Dominios RAG personalizados (Mejora 1 · v2.2).
web/services/env_service.py	113	Lectura y escritura del `.env` del proyecto SIN romper comentarios ni orden de las variables que no se tocan.
web/services/estandares_service.py	247	Estándares de validación por sesión (v2.4).
web/services/informe_web.py	750	Generador de informe consolidado por sesión («Generar informe» de la web).
web/services/modelos_store.py	217	Modelos por validador — cada uno con su carpeta y su `sesion.json`.
web/services/pipeline_runner.py	314	Ejecuta `agente.py --prdc` como subproceso para UNA sesión (v2 · fase 2).
web/services/presupuesto_service.py	66	Presupuesto de contexto dinámico por modelo detectado (v2.3).
web/services/prompt_service.py	170	Prompts editables por sesión (v2.3).
web/services/sesion_store.py	142	Registro de validadores + login por token (v2 · fase 1).
web/services/skills_store.py	215	Skills por sesión (v2 · fase 5).
web/services/solo_informe_ia.py	459	Asistencia con IA del apartado «Solo Informe» (opcional).
web/services/solo_informe_reglas.py	8	Compatibilidad: las reglas de calificación viven en `modulos.reglas_informe` porque también las usa el entregable del pipeline, que no puede
web/services/solo_informe_service.py	344	Apartado «Solo Informe»: compone el informe de resultados SIN ejecutar el pipeline.
web/services/solo_informe_word.py	59	Word del apartado «Solo Informe» (Informe de Resultados).
web/verificar_conexiones.py	290	Verificador end-to-end de conexiones del VIRF Agent Web (Mejora 5 · v2.2).

web/app.py

Propósito: VIRF Web — capa web sobre el agente CLI.

Tamaño: 117 líneas

Comunidades del grafo: Web · Modelos de petición de los routers

Lo usan: scripts/generar_guia_agente.py

Tests que lo importan: test_acta_cierre.py, test_endpoint_notebook_controles.py, test_solo_informe.py, test_solo_informe_ia.py, test_upload_zip.py

Nombre	Línea	Tipo	Qué hace
_version_assets()	88	interna	Token de caché derivado del mtime de los estáticos. Cambia en cuanto se
index()	106	pública	
salud()	116	pública	

web/config_web.py

Propósito: Rutas y constantes del backend web.

Tamaño: 18 líneas

Comunidades del grafo: Web · Modelos de petición de los routers

Lo usan: web/routers/controles.py, web/routers/cuestionamientos.py, web/routers/rag.py, web/routers/upload.py, web/services/contexto_sesion.py, web/services/dominios_custom_store.py, web/services/env_service.py, web/services/informe_web.py, web/services/modelos_store.py, web/services/pipeline_runner.py, web/services/prompt_service.py, web/services/sesion_store.py …

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

web/routers/auth.py

Propósito: Autenticación por código + nombre (v2 · fase 1).

Tamaño: 58 líneas

Comunidades del grafo: Web · Modelos de petición de los routers · Web · Login y sesiones de validadores

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
hacer_login()	27	pública	
hacer_logout()	40	pública	
validador_actual()	48	pública	Devuelve el validador logueado o 401 si no hay sesión.
class Login	21	clase	

web/routers/automatizaciones.py

Propósito: Cuadro de observaciones desde los Excels de automatización (00–03).

Tamaño: 109 líneas

Comunidades del grafo: Automatizaciones · Cuadro de observaciones · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
carpeta_entrada()	31	pública	
carpeta_salida()	35	pública	
ultimo_cuadro()	39	pública	El cuadro más reciente de la sesión, o None.
_contexto_modelo()	54	interna	Entendimiento del modelo (si ya se generó) para explicar bien el producto.
estado()	66	pública	
generar()	78	pública	
descargar()	95	pública	
class PedirCuadro	26	clase	

web/routers/chat.py

Propósito: Chat argumentativo por sesión (v2 · fase 7).

Tamaño: 280 líneas

Comunidades del grafo: Web · Rutas del chat · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
iniciar_chat()	29	pública	
estado_chat()	51	pública	
observaciones()	57	pública	Resumen enfocado para dashboards, sin arrastrar todos los mensajes.
adjuntar_chat()	79	pública	El validador sube un archivo con evidencia nueva para la observación en curso.
consolidar_chat()	122	pública	Cierre HITL: recomputa el PUNTAJE EXACTO final con las decisiones del chat y
descargar_acta_cierre()	163	pública	Acta de cierre en Excel (3 hojas: Resumen · Validación · Decisiones) con el
_reconstruye_ctx()	207	interna	Rehidrata un ctx a partir de la clave `codigo/slug` sin las cookies —
ws_chat()	231	pública	Requiere cookies `virf_token` (login) y `virf_modelo` (modelo activo).
class IniciarChat	24	clase	

web/routers/config.py

Propósito: Configuración del proveedor IA (v2 · fase 2).

Tamaño: 266 líneas

Comunidades del grafo: Web · Configuración y archivo .env

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_snapshot_env()	81	interna	
_estado_virf_plus()	98	interna	Estado del panel VIRF PLUS, sin secretos.
_gpt_principal()	119	interna	El GPT de VIRF PLUS es el del agente principal; Azure si el principal no es GPT.
_validador_o_401()	125	interna	
get_config()	133	pública	Estado del .env + validador logueado (sin secretos).
set_modelo()	144	pública	Guarda título (etiqueta visible) y tier del MODELO ACTIVO. El modelo
set_proveedor()	168	pública	
set_virf_plus()	189	pública	Botón VIRF PLUS: suma (o quita) a Claude como segunda opinión del agente
verificar_conexion()	245	pública	
class ConfigModelo	55	clase	
class ConfigVirfPlus	60	clase	
class ConfigProveedor	68	clase	

web/routers/controles.py

Propósito: Controles de calidad de datos (v2 · fase 2).

Tamaño: 162 líneas

Comunidades del grafo: Datos · Motores de calidad y notebook · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_doc_controles_de_sesion()	23	interna	Primer manual de extracción/controles subido a la sesión.
_entendimiento_del_modelo()	37	interna	Resumen textual del modelo para el modo SIN documento: metadatos +
descargar_notebook_controles()	64	pública	Genera y devuelve el .ipynb ENTREGABLE de controles de datos (8 columnas).
catalogo()	109	pública	Las pruebas genéricas de `modulos.motores_datos.REGISTRO_MOTORES`.
resultados_de_la_entidad()	126	pública	Lee las tablas de resultados que la entidad deja bajo

web/routers/cuestionamientos.py

Propósito: Cuestionamientos metodológicos — paso 9/10.

Tamaño: 168 líneas

Comunidades del grafo: Web · Rutas de cuestionamientos · Web · Cuestionamientos (modelo)

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_leer_json_seguro()	42	interna	
_cuestionamientos_default()	51	interna	
listar_default()	106	pública	Los cuestionamientos del catálogo + su flag `activo` (default True).
toggle_default()	124	pública	
listar_personalizados()	136	pública	
agregar_personalizado()	144	pública	
borrar_personalizado()	160	pública	
class Cuestionamiento	58	clase	Estructura pública. `tipo` mapea al concepto del prompt del usuario.
class ToggleActivo	99	clase	

web/routers/cuestionamientos_sesion.py

Propósito: Cuestionamientos POR SESIÓN — endpoints (v2 · fase 6).

Tamaño: 58 líneas

Comunidades del grafo: Web · Modelos de petición de los routers · Web · Verificar conexiones y cuestionamientos

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
get_estado()	30	pública	
toggle()	35	pública	
crear_custom()	43	pública	
borrar_custom()	53	pública	
class Toggle	20	clase	
class NuevoCustom	25	clase	

web/routers/debug_conexiones.py

Propósito: Endpoint HTTP para el verificador de conexiones (Mejora 5 · v2.2). Misma contraseña que el visor de prompt.

Tamaño: 23 líneas

Comunidades del grafo: Web · Modelos de petición de los routers · RAG · Enrutado por dominio

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
conexiones()	17	pública	

web/routers/dominio_prevuelo.py

Propósito: Pre-vuelo sobre la EVIDENCIA de la sesión activa (v2 · fase 2).

Tamaño: 51 líneas

Comunidades del grafo: Web · Router de prevuelo · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_rutas_de_sesion()	20	interna	Mapeo `nombre_carpeta_agente → ruta_fisica` para el pre-vuelo.
prevuelo()	39	pública	Coste 0. Recorre `web/data/sesiones/{val}/{slug}/inputs/` y devuelve

web/routers/entendimiento.py

Propósito: Endpoints del mapa de entendimiento del modelo (Mejora 4 · v2.2).

Tamaño: 63 líneas

Comunidades del grafo: Web · Entendimiento del modelo · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
analizar()	23	pública	Analiza el ZIP descomprimido y genera `entendimiento_modelo_{slug}.md`.
obtener()	42	pública	
feedback()	56	pública	
class Feedback	17	clase	

web/routers/estandares.py

Propósito: Router del paso 02 · Estándares (v2.4).

Tamaño: 84 líneas

Comunidades del grafo: Web · Servicio de estándares (Excel)

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
get_estado()	24	pública	
get_detalle()	29	pública	
upload()	41	pública	
confirmar()	63	pública	Confirma el estándar YA parseado (upload lo dejó en el .xlsx de la sesión).
borrar()	82	pública	

web/routers/guia.py

Propósito: Guía del procedimiento: en qué paso está el expediente.

Tamaño: 81 líneas

Comunidades del grafo: Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_ficheros()	27	interna	
_paso()	33	interna	
progreso()	38	pública	

web/routers/informe.py

Propósito: Informe consolidado por sesión (v2 · fase 8).

Tamaño: 106 líneas

Comunidades del grafo: Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
estado()	29	pública	Panorama para el panel «Documentar Todo».
versiones()	58	pública	
consolidar()	63	pública	
descargar()	90	pública	
class Consolidar	24	clase	

web/routers/modelos.py

Propósito: Modelos del validador logueado + selección del modelo activo (v2 · fase 1).

Tamaño: 92 líneas

Comunidades del grafo: Web · Gestión de modelos (store y rutas)

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_validador_logueado_o_401()	26	interna	
listar_modelos()	34	pública	
crear_modelo()	40	pública	
activar_modelo()	52	pública	Marca el modelo como activo (cookie); el resto del backend lo usa.
modelo_activo()	67	pública	Modelo actualmente seleccionado (o `None`).
desactivar()	78	pública	
borrar_modelo()	86	pública	
class NuevoModelo	20	clase	

web/routers/preview_prompt.py

Propósito: Visor del prompt del agente, protegido con contraseña (Mejora 2 · v2.2).

Tamaño: 127 líneas

Comunidades del grafo: RAG · Enrutado por dominio · Web · Modelos de petición de los routers

Lo usan: web/app.py, web/routers/debug_conexiones.py, web/routers/prompts.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_muestra_rag()	32	interna	Top 3 chunks del retriever del dominio para dar visibilidad al RAG.
_leer_parametros()	50	interna	Snapshot de parámetros que la UI del visor muestra al final.
preview_prompt()	63	pública	

web/routers/prompts.py

Propósito: Endpoints CRUD de prompts editables por sesión (v2.3).

Tamaño: 93 líneas

Comunidades del grafo: Web · Edición de prompts · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_password_o_403()	33	interna	
get_base()	39	pública	Los prompts por defecto (para el editor cuando aún no hay sesión).
get_efectivos()	45	pública	Los prompts efectivos de la sesión (custom si existen, base si no).
editar()	54	pública	
restaurar()	72	pública	
presupuesto()	84	pública	Presupuesto de contexto para la sesión. Usa el modelo detectado por
class BodyEditar	24	clase	
class BodyRestaurar	29	clase	

web/routers/rag.py

Propósito: Dominio & RAG: papers al RAG global + reconstrucción + dominios custom.

Tamaño: 253 líneas

Comunidades del grafo: Web · Rutas RAG y dominios personalizados · Web · Reconstruir índice RAG

Lo usan: web/app.py

Tests que lo importan: test_rag_desactualizado.py

Nombre	Línea	Tipo	Qué hace
_login_o_401()	33	interna	
_leer_indice()	40	interna	Estado del índice sin ejecutar el script — carga el .pkl directamente.
_esta_desactualizado()	61	interna	True si el índice está OBSOLETO respecto al corpus: falta el `.pkl` habiendo
_papers_en_disco()	87	interna	
_modelos_que_usan()	98	interna	Lista de "codigo/slug" de modelos que declaran este dominio.
listar_dominios()	118	pública	Fijos primero, custom después. Cada uno con estado del índice y papers.
crear_dominio()	143	pública	
borrar_dominio()	159	pública	
borrar_paper()	179	pública	Elimina un libro/manual CONCRETO del corpus `inputs/rag/{dom_id}/` y
_ejecutar_rebuild()	215	interna	Lanza el script de reindexado como subproceso. Coste 0 (RAG léxico).
reconstruir()	241	pública	Reconstruye el índice del dominio dado. Acepta cualquier dominio
class NuevoDominio	136	clase	
class Reconstruir	210	clase	

web/routers/resultados.py

Propósito: Lectura de resultados de validación (v2 · fase 2).

Tamaño: 194 líneas

Comunidades del grafo: Web · Descarga de resultados

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_outputs()	25	interna	
_resolver()	29	interna	
_leer_json()	45	interna	
listar_resultados()	53	pública	Todos los `resultados_*.json` DE LA SESIÓN activa.
resultado_de_la_ultima_corrida()	82	pública	
_resumen_tabla()	94	interna	Conteo por Resultado + cuántas filas van al chat a sustentarse.
_payload_tabla()	107	interna	
tabla_de_la_ultima_corrida()	119	pública	Output intermedio: la tabla de validación por actividad (Ámbito | Control |
tabla_por_ruta()	138	pública	
descargar_excel()	145	pública	Descarga la tabla de validación en Excel.
resultado_por_ruta()	191	pública	

web/routers/skills.py

Propósito: Endpoints de Skills — capacidades del agente por sesión (v2 · fase 5).

Tamaño: 65 líneas

Comunidades del grafo: Web · Catálogo de skills · Web · Modelos de petición de los routers

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
listar_defaults()	28	pública	Catálogo por defecto SIN estado (para inspección/documentación).
get_skills()	34	pública	Skills DE LA SESIÓN activa: defaults (con flag `activo` según el
toggle()	43	pública	
crear_custom()	51	pública	
borrar_custom()	60	pública	
class ToggleDefault	18	clase	
class NuevoCustom	23	clase	

web/routers/solo_informe.py

Propósito: Router del apartado «Solo Informe»: genera el Informe de Resultados sin pipeline.

Tamaño: 249 líneas

Comunidades del grafo: Solo Informe · Router y endpoints · Solo Informe · Estándar, borrador e imágenes · Tests · Modelo simulado de Solo Informe · Solo Informe · Reglas de conclusión

Lo usan: web/app.py

Tests que lo importan: test_solo_informe.py, test_solo_informe_ia.py

Nombre	Línea	Tipo	Qué hace
base_solo_informe()	43	pública	
estado()	82	pública	
subir_estandar()	92	pública	
borrar_estandar()	107	pública	
guardar_borrador()	114	pública	
subir_imagen()	122	pública	
ver_imagen()	132	pública	
generar()	140	pública	
descargar()	165	pública	
_ia()	190	interna	Traduce los errores de la IA a HTTP: 503 sin credenciales, 502 si falla.
ia_estado()	203	pública	
ia_pulir()	208	pública	
ia_grafico()	213	pública	
ia_manuales()	218	pública	
manuales()	231	pública	
subir_manual()	236	pública	
borrar_manual()	246	pública	
class Sustento	50	clase	

Nombre	Línea	Tipo	Qué hace
class Observacion	56	clase	
class DatosInforme	68	clase	
class PedidoTexto	178	clase	
class PedidoManuales	183	clase	

web/routers/upload.py

Propósito: Endpoints de subida de evidencia y visualización del árbol de inputs.

Tamaño: 288 líneas

Comunidades del grafo: Web · Subida de archivos y ZIP

Lo usan: web/app.py

Tests que lo importan: test_upload_zip.py

Nombre	Línea	Tipo	Qué hace
_nombre_seguro()	48	interna	
_clasificar_estado()	63	interna	
_serializar_dir()	80	interna	
listar_destinos()	109	pública	Slugs válidos con etiqueta y alcance (sesion|global).
arbol_inputs()	115	pública	Árbol de `inputs/` DE LA SESIÓN activa. El RAG global no se muestra
listar_carpeta()	125	pública	Ficheros ya subidos a `slug` — para pintar cada card del upload guiado.
borrar_archivo()	132	pública	Borra un fichero SUBIDO desde este panel. Solo permite eliminar
subir_archivos()	156	pública	
subir_zip()	192	pública	Descomprime un ZIP en `destino`. Antes: el flujo especial para el

web/routers/validacion.py

Propósito: Endpoints REST + WebSocket del pipeline de validación (v2 · fase 2).

Tamaño: 117 líneas

Comunidades del grafo: Web · Modelos de petición de los routers · Web · Descarga de resultados

Lo usan: web/app.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
estado_actual()	29	pública	
lanzar_validacion()	48	pública	
cancelar_validacion()	66	pública	
ws_validar()	75	pública	Emite el stream de eventos del runner identificado por `run_id`.
class LanzarValidacion	21	clase	

web/services/analisis_codigo.py

Propósito: Análisis estructural del ZIP de implementación (Mejora 4 · v2.2).

Tamaño: 389 líneas

Comunidades del grafo: Web · Estructura del código del modelo · Web · Entendimiento del modelo · Tipos · typing (compartido) · Informe web · Generador con capturas

Lo usan: web/routers/entendimiento.py, web/routers/preview_prompt.py, web/services/informe_web.py, web/verificar_conexiones.py

Tests que lo importan: test_prompts_editables.py

Nombre	Línea	Tipo	Qué hace
_analizar_py()	35	interna	Extrae funciones, clases e imports de un .py con AST.
_analizar_ipynb()	70	interna	Cuenta celdas y extrae imports+defs del código Python.
_analizar_generico()	109	interna	
estructura()	117	pública	Recorre `base_impl` y devuelve el mapa estructural entero.
_armar_prompt()	211	interna	
_mock_entendimiento()	265	interna	Salida canned SIN API. Aprovecha la estructura real para que el
generar_entendimiento()	310	pública	Genera el documento .md de entendimiento. Devuelve el texto.
ruta_md()	340	pública	
ruta_feedback()	344	pública	
guardar_md()	348	pública	
leer_md()	355	pública	
leer_feedbacks()	365	pública	
agregar_feedback()	376	pública	

web/services/chat_service.py

Propósito: Chat argumentativo por sesión — v2 · fase 7.

Tamaño: 589 líneas

Comunidades del grafo: Web · Chat de validación · Chat · Adjuntar evidencia · Tests · Plantilla común de informes

Lo usan: web/routers/chat.py, web/routers/informe.py, web/routers/preview_prompt.py, web/routers/resultados.py, web/services/informe_web.py, web/verificar_conexiones.py

Tests que lo importan: test_acta_cierre.py, test_chat_adjuntar.py, test_plantilla_bcp.py, test_tabla_validacion.py

Nombre	Línea	Tipo	Qué hace
_fichero()	112	interna	
_ahora()	116	interna	
_guardar()	120	interna	
_cargar_de_disco()	139	interna	
sesion_activa()	159	pública	Devuelve la sesión de chat de este `ctx` (memoria o disco).
_estados_para_sustentar()	180	interna	Qué veredictos van al chat. 'No cumple' SIEMPRE; 'Parcial' y 'No evidenciado'
_extraer_observaciones()	194	interna	De los resultados de validación, saca las observaciones que van al chat a
_renderiza_prompt()	314	interna	
_respuesta_mock()	349	interna	Mock rotativo para desarrollo sin API. Segundo intento cierra
_parse_json_laxo()	371	interna	Parsea el JSON del veredicto tolerando fences markdown y prosa alrededor
_invocar_modelo()	396	interna	
iniciar()	416	pública	Crea (o reinicia) la sesión de chat del `ctx` a partir del JSON
_emitir_pregunta()	443	interna	
_agregar()	474	interna	
responder()	486	pública	Procesa la respuesta del validador y avanza el chat.
adjuntar_evidencia()	535	pública	El validador adjunta un archivo con evidencia NUEVA para la observación en

Nombre	Línea	Tipo	Qué hace
a_dict()	567	pública	
clear_todo()	586	pública	Solo para tests.
class Observacion	40	clase	
class Mensaje	66	clase	
class SesionChat	75	clase	Métodos: actual, resumen

web/services/contexto_sesion.py

Propósito: Resolución del contexto de sesión (v2 · fase 2).

Tamaño: 307 líneas

Comunidades del grafo: Web · Subida de archivos y ZIP · Web · Modelos de petición de los routers

Lo usan: web/routers/automatizaciones.py, web/routers/chat.py, web/routers/controles.py, web/routers/cuestionamientos_sesion.py, web/routers/debug_conexiones.py, web/routers/dominio_prevuelo.py, web/routers/entendimiento.py, web/routers/estandares.py, web/routers/guia.py, web/routers/informe.py, web/routers/preview_prompt.py, web/routers/prompts.py …

Tests que lo importan: test_acta_cierre.py, test_endpoint_notebook_controles.py, test_upload_zip.py

Nombre	Línea	Tipo	Qué hace
contexto_sesion()	39	pública	
destinos_disponibles()	233	pública	Lista los slugs con metadatos ricos para que el frontend pinte las
resolver_destino()	261	pública	Traduce un slug de destino a la carpeta física. Los slugs `rag_*`
inicializar_carpetas_sesion()	273	pública	Materializa las subcarpetas de destino en la primera petición.
es_solo_zip()	285	pública	Slugs que rechazan ficheros sueltos (implementacion/).
listar_carpeta_sesion()	291	pública	Ficheros en la carpeta `slug` de la sesión (o global si es rag_*).

web/services/cuestionamientos_sesion.py

Propósito: Cuestionamientos POR SESIÓN (v2 · fase 6).

Tamaño: 154 líneas

Comunidades del grafo: Web · Verificar conexiones y cuestionamientos

Lo usan: web/routers/cuestionamientos_sesion.py, web/routers/preview_prompt.py, web/verificar_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_fichero()	45	interna	
_ahora()	49	interna	
_leer()	53	interna	
_escribir()	66	interna	
get_estado()	72	pública	Snapshot: los 15 defaults con `activo` según lo guardado en la sesión,
toggle_default()	97	pública	
add_custom()	112	pública	
remove_custom()	130	pública	
get_for_prompt()	141	pública	Instrucciones NL a inyectar en el prompt del agente. Los defaults

web/services/dominios_custom_store.py

Propósito: Dominios RAG personalizados (Mejora 1 · v2.2).

Tamaño: 170 líneas

Comunidades del grafo: Motores de la entidad · Leer resultados · Web · Rutas RAG y dominios personalizados

Lo usan: web/routers/rag.py, web/services/modelos_store.py, web/verificar_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
slugify()	40	pública	«Capital Económico» → «capital_economico»
_leer()	55	interna	
_escribir()	65	interna	
listar()	71	pública	Devuelve la lista completa de dominios (fijos + custom).
crear()	98	pública	Crea un dominio custom. Devuelve la entrada creada.
existe()	141	pública	
borrar()	145	pública	Retira un dominio custom. Mueve la carpeta a

web/services/env_service.py

Propósito: Lectura y escritura del `.env` del proyecto SIN romper comentarios ni orden de las variables que no se tocan.

Tamaño: 113 líneas

Comunidades del grafo: Web · Configuración y archivo .env

Lo usan: web/routers/config.py, web/routers/guia.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_parsear_valor()	38	interna	Quita comillas envolventes y desescapa el interior si la envoltura era
_serializar_valor()	52	interna	Escribe siempre entre comillas dobles.
leer_env()	64	pública	Devuelve las variables actualmente definidas en `.env`.
actualizar_env()	79	pública	Aplica `cambios` sobre `.env` preservando lo demás.

web/services/estandares_service.py

Propósito: Estándares de validación por sesión (v2.4).

Tamaño: 247 líneas

Comunidades del grafo: Web · Servicio de estándares (Excel) · Solo Informe · Estándar, borrador e imágenes

Lo usan: web/routers/estandares.py, web/routers/guia.py, web/routers/solo_informe.py, web/services/pipeline_runner.py, web/services/solo_informe_service.py

Tests que lo importan: test_estandares_sesion.py, test_solo_informe.py

Nombre	Línea	Tipo	Qué hace
carpeta()	38	pública	
ruta_xlsx()	42	pública	
ruta_json()	46	pública	
_normalizar_marca()	56	interna	Cualquier valor no vacío cuenta como 'aplica'; vacío o None cuenta como no.
_cabeceras_ok()	64	interna	Mapea cabeceras del Excel a índices de columna. Tolera espacios extra
parsear()	79	pública	Lee el Excel y devuelve el árbol {ambito: [{control, actividades}]}.
guardar_xlsx()	186	pública	
confirmar()	193	pública	Marca el estándar como confirmado y lo guarda en JSON. Este JSON es
estado()	211	pública	Devuelve el estado del paso 02 para la sesión:
limpiar()	240	pública	Borra xlsx + json de la sesión (usado en 'Cambiar archivo').
class EstandarInvalido	52	clase	El .xlsx no tiene el formato esperado.

web/services/informe_web.py

Propósito: Generador de informe consolidado por sesión («Generar informe» de la web).

Tamaño: 750 líneas

Comunidades del grafo: Informe web · Generador con capturas · Informes · Plantilla común BCP · Informes · API de la plantilla

Lo usan: web/routers/guia.py, web/routers/informe.py

Tests que lo importan: test_automatizaciones.py, test_capturas.py, test_informe_tabla_final.py, test_plantilla_bcp.py

Nombre	Línea	Tipo	Qué hace
salida_dir_de()	55	pública	
_nombre_canonico()	64	interna	(nombre, versión). La versión es 1 + max(V*) existente en la carpeta.
listar_versiones()	80	pública	Enumera los informes generados en esta sesión, más nuevos primero.
_titulo()	123	interna	Título con estilo REAL de Word (alimenta el índice). Nivel 1 centrado.
_first()	136	interna	
_sev()	144	interna	
_terminos_presentes()	173	interna	Términos del catálogo que REALMENTE aparecen en el informe, alfabéticos.
_ultimo_cuadro()	190	interna	Último cuadro de observaciones de las automatizaciones (Excels 00–03).
_cuadro_observaciones()	207	interna	Anexo: Ámbito · Riesgo · Observación · Recomendación · Responsable/Fecha,
_seccion_tabla_final()	248	interna	Tabla de 8 columnas tras el chat. Con `titulo=None` no pinta título
_seccion_entendimiento()	321	interna	Anexo fijo (siempre presente). Placeholder si no hay entendimiento.
_seccion_trazabilidad()	359	interna	
_puntaje()	402	interna	
_chat()	407	interna	(ámbito, actividad) → observación del chat; id de observación → mensajes.
_figura()	448	interna	
_captura_cita()	454	interna	

Nombre	Línea	Tipo	Qué hace
_portadas()	470	interna	Controles de Documentación: portada de cada documento PDF que revisan.
_detalle_control()	495	interna	Lo que el agente y el chat aportan sobre el control, en texto estructurado
a_plantilla()	603	pública	Resultados de la corrida + chat + automatizaciones → informe normalizado.
_hay_entendimiento()	737	interna	
generar()	745	pública	Genera el .docx versionado con la plantilla común. Devuelve `(ruta, version)`.
class _Capturas	427	clase	Estado de las capturas de un informe: dónde se guardan, dónde se buscan los Métodos: doc

web/services/modelos_store.py

Propósito: Modelos por validador — cada uno con su carpeta y su `sesion.json`.

Tamaño: 217 líneas

Comunidades del grafo: Web · Gestión de modelos (store y rutas)

Lo usan: web/routers/chat.py, web/routers/config.py, web/routers/informe.py, web/routers/modelos.py, web/routers/rag.py, web/routers/validacion.py, web/services/contexto_sesion.py, web/verificar_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_ahora()	37	interna	
slug_de()	41	pública	Convierte «VAR BCP 2027» → «VAR_BCP_2027`. Rechaza cadenas vacías o
_normaliza_tier()	54	interna	
_valida_dominio()	63	interna	Acepta los 3 fijos del selector + shared + cualquier custom
_base_validador()	78	interna	Carpeta raíz de todos los modelos de un validador.
_base_modelo()	83	interna	
_leer_sesion()	87	interna	
_escribir_sesion()	94	interna	
_snapshot()	100	interna	Vista pública que consume la UI.
crear()	114	pública	Crea la sesión del modelo. Falla si ya existe uno con el mismo slug:
listar()	144	pública	Modelos del validador, más recientes primero.
cargar()	161	pública	
actualizar_actividad()	169	pública	Marca que hubo movimiento en la sesión (validación, chat, informe…).
actualizar_meta()	184	pública	Edita metadatos de una ficha existente EN SITIO. No mueve la carpeta:
borrar()	209	pública	Elimina la sesión entera del disco. Irreversible; usar con cuidado.

web/services/pipeline_runner.py

Propósito: Ejecuta `agente.py --prdc` como subproceso para UNA sesión (v2 · fase 2).

Tamaño: 314 líneas

Comunidades del grafo: Web · Descarga de resultados · Web · Presupuesto de prompts · Web · Modelos de petición de los routers

Lo usan: web/routers/informe.py, web/routers/resultados.py, web/routers/validacion.py

Tests que lo importan: test_estandares_sesion.py, test_invariante_asyncio.py

Nombre	Línea	Tipo	Qué hace
_clasificar_fase()	43	interna	
_construir_env_hijo()	70	interna	Copia os.environ y le añade dos capas propias de la sesión web:
run_de_sesion()	127	pública	
run_por_id()	131	pública	
_snapshot_outputs_hoy()	135	interna	
_detectar_json_nuevo()	143	interna	
construir_args()	160	pública	
_bombear_stdout()	189	interna	
lanzar()	204	pública	Inicia una corrida en el contexto de sesión `ctx`. Devuelve `run_id`.
cancelar()	304	pública	
class Run	53	clase	

web/services/presupuesto_service.py

Propósito: Presupuesto de contexto dinámico por modelo detectado (v2.3).

Tamaño: 66 líneas

Comunidades del grafo: Web · Presupuesto de prompts · Tipos · typing (compartido)

Lo usan: web/routers/config.py, web/routers/prompts.py, web/services/pipeline_runner.py

Tests que lo importan: test_prompts_editables.py

Nombre	Línea	Tipo	Qué hace
calcular()	26	pública	Devuelve el presupuesto de contexto que corresponde al modelo detectado.
aplicar_env()	54	pública	Copia el env y sobreescribe VIRF_MAX_CTX y VIRF_PRESUPUESTO_DOC con los

web/services/prompt_service.py

Propósito: Prompts editables por sesión (v2.3).

Tamaño: 170 líneas

Comunidades del grafo: Web · Servicio de prompts · Web · Prompts editables

Lo usan: web/routers/prompts.py, web/services/analisis_codigo.py, web/services/chat_service.py, web/services/pipeline_runner.py

Tests que lo importan: test_prompts_editables.py

Nombre	Línea	Tipo	Qué hace
_cargar_base()	32	interna	
prompts_base()	39	pública	Todos los prompts base con metadatos (para el GET del router).
_ruta_custom()	44	interna	
_leer_custom()	48	interna	
_escribir_custom()	58	interna	
_ahora()	64	interna	
prompts_efectivos()	68	pública	Base + overrides de la sesión, con marca de cuál está personalizado.
get_prompt()	88	pública	Devuelve el contenido efectivo (custom si existe, si no el base).
_placeholders_en()	104	interna	
editar_prompt()	108	pública	Guarda un override para la sesión. Devuelve la entrada resultante.
restaurar_prompt()	153	pública	Elimina el override. Idempotente.
clear_cache_para_tests()	167	pública	Los tests recargan prompts_base.json cuando lo modifican.

web/services/sesion_store.py

Propósito: Registro de validadores + login por token (v2 · fase 1).

Tamaño: 142 líneas

Comunidades del grafo: Web · Login y sesiones de validadores · Web · Modelos de petición de los routers

Lo usan: web/routers/auth.py, web/routers/chat.py, web/routers/config.py, web/routers/modelos.py, web/routers/prompts.py, web/routers/rag.py, web/routers/solo_informe.py, web/routers/validacion.py, web/services/contexto_sesion.py, web/services/pipeline_runner.py, web/verificar_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_ahora()	37	interna	
_leer_json()	41	interna	
leer_validadores()	50	pública	Snapshot del registro. NUNCA muta el estado en disco.
_escribir_validadores()	56	interna	
normalizar_codigo()	62	pública	
formato_valido()	66	pública	
alta_validador()	70	pública	Registra un nuevo validador si el código está bien formado.
login()	95	pública	Autentica (si existe) o registra al vuelo y autentica. Devuelve
sesion_activa()	115	pública	
set_atributo_sesion()	121	pública	Guarda un atributo volátil en la sesión de login (in-memory).
logout()	138	pública	

web/services/skills_store.py

Propósito: Skills por sesión (v2 · fase 5).

Tamaño: 215 líneas

Comunidades del grafo: Web · Catálogo de skills

Lo usan: web/routers/preview_prompt.py, web/routers/skills.py, web/verificar_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_fichero()	88	interna	
_leer()	92	interna	
_escribir()	105	interna	
_ahora()	111	interna	
get_skills()	115	pública	Snapshot para la UI: defaults con `activo` + custom.
toggle_default()	136	pública	Marca/desmarca un skill del catálogo por defecto. `activo=True` lo saca
add_custom()	166	pública	Añade un skill personalizado en lenguaje natural.
remove_custom()	191	pública	
get_active_skills_for_prompt()	202	pública	Instrucciones a inyectar en el prompt del agente (cuando llegue la

web/services/solo_informe_ia.py

Propósito: Asistencia con IA del apartado «Solo Informe» (opcional).

Tamaño: 459 líneas

Comunidades del grafo: Solo Informe · Asistencia con IA

Lo usan: web/routers/solo_informe.py

Tests que lo importan: test_solo_informe_ia.py

Nombre	Línea	Tipo	Qué hace
proveedor()	69	pública	
estado()	74	pública	Qué proveedor se usará y si tiene credenciales. No llama a la API.
_llamar()	94	interna	
_limpiar_entrada()	122	interna	PII enmascarada e instrucciones inyectadas neutralizadas antes de salir.
_texto_plano()	130	interna	Quita el Markdown que algunos modelos añaden aunque se pida texto
_json()	143	interna	Primer objeto JSON de la respuesta (tolera fences y prosa alrededor).
pulir()	175	pública	
_numeros()	195	interna	Cifras del texto en forma canónica («99,0» y «99» → «99»; «1,5» → «1.5»).
_validar_spec()	201	interna	
_dibujar()	227	interna	
grafico()	268	pública	Propone y dibuja un gráfico del texto. Devuelve {imagen_id, spec}.
dir_manuales()	291	pública	
_indice_path()	297	interna	
_nombre_seguro()	301	interna	
_fragmentar()	307	interna	Fragmentos de ~TAM_FRAGMENTO caracteres por párrafo, con número de página
_reindexar()	326	interna	
guardar_manual()	339	pública	
listar_manuales()	355	pública	

Nombre	Línea	Tipo	Qué hace
borrar_manual()	365	pública	
_fragmentos()	374	interna	
buscar()	384	pública	Fragmentos de los manuales más relevantes para la consulta (BM25, local).
desde_manuales()	419	pública	objetivo="contexto" → redacta «Contexto del modelo y antecedentes».
class IANoConfigurada	47	clase	No hay credenciales del proveedor: la web muestra dónde ponerlas.
class IAFallo	51	clase	El proveedor respondió con error o con algo inutilizable.

web/services/solo_informe_reglas.py

Propósito: Compatibilidad: las reglas de calificación viven en `modulos.reglas_informe` porque también las usa el entregable del pipeline, que no puede importar `web/`.

Tamaño: 8 líneas

Comunidades del grafo: Informes · Reglas de calificación y cumplimiento

Lo usan: web/routers/solo_informe.py, web/services/solo_informe_service.py

Tests que lo importan: test_solo_informe.py

web/services/solo_informe_service.py

Propósito: Apartado «Solo Informe»: compone el informe de resultados SIN ejecutar el pipeline.

Tamaño: 344 líneas

Comunidades del grafo: Solo Informe · Estándar, borrador e imágenes · Solo Informe · Reglas de conclusión

Lo usan: web/routers/solo_informe.py, web/services/solo_informe_ia.py, web/services/solo_informe_word.py

Tests que lo importan: test_solo_informe.py

Nombre	Línea	Tipo	Qué hace
base_de()	38	pública	
ruta_xlsx()	44	pública	
ruta_json()	48	pública	
ruta_borrador()	52	pública	
dir_imagenes()	56	pública	
dir_salida()	62	pública	
_norm()	91	interna	
_mapear_cabecera()	95	interna	
parsear_estandar()	116	pública	Devuelve {ambitos: [nombre…], controles: [{id, ambito, clave_ambito, control,
_extraer()	140	interna	
filtrar_por_tier()	189	pública	Estándar reducido a las actividades que aplican al tier (y sus ámbitos).
limpiar_om()	198	pública	Una Oportunidad de Mejora no lleva responsable ni fecha límite.
guardar_estandar()	210	pública	
cargar_estandar()	223	pública	
guardar_borrador()	244	pública	
cargar_borrador()	249	pública	
guardar_imagen()	264	pública	
ruta_imagen()	278	pública	Ruta de una imagen subida. None si el id no es válido o no existe.

Nombre	Línea	Tipo	Qué hace
observaciones_por_control()	288	pública	
cumplimiento()	295	pública	Fila por ámbito del estándar: total, satisfactorios y no cumplidos (con %).
validar_datos()	316	pública	Errores de formulario que impiden generar el informe (lista vacía = OK).

web/services/solo_informe_word.py

Propósito: Word del apartado «Solo Informe» (Informe de Resultados).

Tamaño: 59 líneas

Comunidades del grafo: Tests · Solo Informe (Word y endpoints) · Tests · Capturas y Solo Informe Word

Lo usan: web/routers/solo_informe.py

Tests que lo importan: test_solo_informe.py, test_solo_informe_ia.py

Nombre	Línea	Tipo	Qué hace
nombre_fichero()	25	pública	
a_plantilla()	32	pública	Formulario + estándar → informe normalizado de `plantilla_bcp.generar`.
generar()	55	pública	Genera el .docx en `base/salida/` y devuelve su ruta.

web/verificar_conexiones.py

Propósito: Verificador end-to-end de conexiones del VIRF Agent Web (Mejora 5 · v2.2).

Tamaño: 290 líneas

Comunidades del grafo: Web · Verificador de conexiones · RAG · Enrutado por dominio · Web · Modelos de petición de los routers · Scripts · Reconstruir estándar y verificar corpus

Lo usan: web/routers/debug_conexiones.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_construir_ctx()	111	interna	
verificar()	129	pública	
main()	274	pública	
class _Reporte	39	clase	Acumula OK/FAIL/N/A y agrupa por bloque para el print final. Métodos: bloque, check, resumen, a_dict, imprimir

Carpeta skills/

Archivo	Líneas	Propósito
skills/base_tipada.py	200	Cada skill de la guía es una HERRAMIENTA declarada para function calling: · entrada/salida tipada con Pydantic, · un método `declaracion_her
skills/calidad_datos.py	220	Única de las 6 skills puramente DETERMINISTA: no llama al modelo. Ingiere un dataset y devuelve JSON estructurado con outliers (IQR), nulos
skills/cola_pendiente.py	111	Antipatrón «Exceso de Agencia» (guía §5.1): el agente NUNCA ejecuta directamente una acción de impacto externo (enviar un correo de resultad
skills/documentacion.py	38	DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Parsea versión/estructura del documento y detecta mod
skills/gobierno.py	37	DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Deriva el Tier real del modelo por complejidad, lo cr
skills/implementacion.py	38	DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Analiza el código estáticamente (AST), revisa lineami
skills/metodologia.py	43	DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Busca la justificación teórica de fórmulas y parámetr
skills/uso.py	42	DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Ingiere canales operativos (correos .msg/.eml, .pptx,

skills/base_tipada.py

Propósito: Cada skill de la guía es una HERRAMIENTA declarada para function calling: · entrada/salida tipada con Pydantic, · un método `declaracion_herramienta()` que emite el JSON Schema de parámetros, · un ALCANCE DE REVISIÓN DECLARADO — qué rubro le pertenece (por ID, resuelto desde la config VERSIONADA, no hardcodeado) y qué queda FUERA de su frontera, · comportamiento fuera de alcance: se abstiene o deriva (nunca opina sobre el dominio de otra skill).

Tamaño: 200 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno · Tests · Skills tipadas · Skills · Catálogo y clase base

Lo usan: skills/calidad_datos.py, skills/documentacion.py, skills/gobierno.py, skills/implementacion.py, skills/metodologia.py, skills/uso.py

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class DeclaracionHerramienta	37	clase	Declaración de una herramienta para function calling (formato OpenAI/Azure).
class AlcanceRubro	57	clase	Alcance de revisión resuelto de una skill: qué rubro le pertenece.
class ResultadoAlcance	72	clase	Decisión determinista de si una consulta cae dentro de la frontera de la skill.
class SkillTipada	92	clase	Base de las 6 skills tipadas. Métodos: declaracion_herramienta, resolver_rubro, evaluar_alcance, a_resultado_skill, ejecutar

skills/calidad_datos.py

Propósito: Única de las 6 skills puramente DETERMINISTA: no llama al modelo. Ingiere un dataset y devuelve JSON estructurado con outliers (IQR), nulos en variables clave, duplicados exactos y solapamiento train/test (data leakage).

Tamaño: 220 líneas

Comunidades del grafo: Skills · Calidad de datos · Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_outliers_iqr()	67	interna	Nº de valores fuera de [q1 - 1.5·IQR, q3 + 1.5·IQR]. Determinista.
run_data_quality_check()	82	pública	Analiza `csv_path` y devuelve un `DataQualityResult`. Cálculo 100% determinista.
_evaluar_leakage()	124	interna	
_ejecutar_en_sandbox()	184	interna	
_main()	205	interna	
class DataLeakageResult	36	clase	
class DataQualityResult	45	clase	Reporte determinista de calidad de datos de un dataset.
class SkillCalidadDatos	149	clase	Skill 2 de la guía: Data Quality Guard (determinista, en sandbox). Métodos: ejecutar

skills/cola_pendiente.py

Propósito: Antipatrón «Exceso de Agencia» (guía §5.1): el agente NUNCA ejecuta directamente una acción de impacto externo (enviar un correo de resultados, marcar un modelo como aprobado para producción). Toda acción de impacto se ENCOLA en estado `PENDIENTE` y espera la confirmación de un validador senior.

Tamaño: 111 líneas

Comunidades del grafo: Skills · Cola de acciones pendientes · Tipos · typing (compartido)

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
ruta_cola_por_defecto()	36	pública	`web/data/pendientes.json` bajo la raíz del proyecto.
_leer()	62	interna	
_escribir()	72	interna	
encolar_accion()	77	pública	Registra una acción de impacto en la cola `PENDIENTE`. NO la ejecuta.
listar_pendientes()	103	pública	Lee la cola. Por defecto solo las que siguen en estado `PENDIENTE`.
class AccionPendiente	41	clase	Una acción de impacto externo, encolada a la espera de confirmación humana.

skills/documentacion.py

Propósito: DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Parsea versión/estructura del documento y detecta modificaciones no declaradas u omisión de apéndices. Su detalle de criterios vive en el ámbito «Documentación» del estándar versionado (resuelto por el coordinador).

Tamaño: 38 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SkillDocumentacion	17	clase	

skills/gobierno.py

Propósito: DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Deriva el Tier real del modelo por complejidad, lo cruza contra el Tier autodeclarado y levanta bandera si discrepan; audita actas/correos institucionales. Toda acción de impacto (aprobar, enviar) pasa por la cola PENDIENTE (skills.cola_pendiente). El detalle de criterios vive en el ámbito «Gobierno» del estándar versionado.

Tamaño: 37 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SkillGobierno	18	clase	

skills/implementacion.py

Propósito: DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Analiza el código estáticamente (AST), revisa lineamientos de seguridad (credenciales en entorno, logs, backups) y valida réplicas exactas de la lógica matemática. El analizador AST ya existe (`modulos/analisis_estatico.py`); aquí se declara como herramienta tipada. La ejecución de tests unitarios corre en sandbox.

Tamaño: 38 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SkillImplementacion	18	clase	

skills/metodologia.py

Propósito: DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Busca la justificación teórica de fórmulas y parámetros calibrados en los manuales metodológicos aprobados, con filtro duro por `model_id` (multi-tenancy). El RAG léxico ya existe (`src/rag/`); aquí se declara como herramienta tipada.

Tamaño: 43 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SkillMetodologia	17	clase	

skills/uso.py

Propósito: DECLARADA para function calling; NO operativa hasta el Coordinador ReAct (día de API). Ingiere canales operativos (correos .msg/.eml, .pptx, hojas de cálculo) para rastrear aprobaciones y comparar métricas mostradas a dirección contra la base de producción.

Tamaño: 42 líneas

Comunidades del grafo: Skills tipadas · Documentación y gobierno

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
class SkillUso	22	clase	

Carpeta scripts/

Archivo	Líneas	Propósito
scripts/auditar_cobertura_rag.py	312	¿Contra qué actividades del estándar VIRF está CIEGO el corpus de literatura?
scripts/explicar_rag.py	273	Abre el RAG léxico en canal: enseña, paso a paso y con NÚMEROS REALES del índice que tengas construido, qué le pasa a una consulta desde que
scripts/generar_corpus_rag.py	863	
scripts/generar_guia_agente.py	868	Genera `docs/agente_corporativo/GUIA_VIRF_AGENTE_CORPORATIVO.docx` (y `.pdf` con `--pdf`) a partir del PROPIO repositorio, para que la guía
scripts/pipeline_calidad_datos.py	350	Pipeline de calidad de datos DEL VALIDADOR. Genera la tabla que el agente lee.
scripts/preguntas_oro_formula.py	284	Golden set DISCRIMINANTE de fórmula/tabla + medición IR léxica (API-free).
scripts/preguntas_oro_rag.py	713	Set de preguntas-oro del RAG léxico: ¿recupera el corpus el documento correcto?
scripts/rebuild_rag_databricks.py	236	Script de mantenimiento para reconstruir los índices RAG en Databricks.
scripts/reconstruir_estandar_desde_informes.py	293	
scripts/verificar_corpus_rag.py	337	VERIFICADOR DEL CORPUS RAG — ¿el índice contiene lo que dice contener?

scripts/auditar_cobertura_rag.py

Propósito: ¿Contra qué actividades del estándar VIRF está CIEGO el corpus de literatura?

Tamaño: 312 líneas

Comunidades del grafo: RAG · Buscador léxico

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_clasificar()	89	interna	
_anclas()	100	interna	Términos con IDF > 0 que la consulta y el fragmento comparten.
auditar()	121	pública	
main()	291	pública	

scripts/explicar_rag.py

Propósito: Abre el RAG léxico en canal: enseña, paso a paso y con NÚMEROS REALES del índice que tengas construido, qué le pasa a una consulta desde que se escribe hasta que sale el top-K.

Tamaño: 273 líneas

Comunidades del grafo: RAG · Buscador léxico

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_regla()	57	interna	
explicar()	61	pública	
main()	253	pública	

scripts/generar_corpus_rag.py

Propósito: (sin docstring de módulo)

Tamaño: 863 líneas

Comunidades del grafo: RAG · Indexadores por dominio y reconstrucción

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_construir_pdf()	746	interna	Genera un PDF. Devuelve el número de párrafos escritos.
generar()	815	pública	
listar()	842	pública	

scripts/generar_guia_agente.py

Propósito: Genera `docs/agente_corporativo/GUIA_VIRF_AGENTE_CORPORATIVO.docx` (y `.pdf` con `--pdf`) a partir del PROPIO repositorio, para que la guía no se quede atrás cuando cambia el código:

Tamaño: 868 líneas

Comunidades del grafo: Guía · Generador del PDF del agente corporativo

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
archivos_codigo()	171	pública	
_rel()	183	interna	
_dotted()	187	interna	
proposito()	191	pública	Primer párrafo con contenido real (salta líneas que solo repiten el nombre del archivo).
_primera_linea()	208	interna	
_imports()	215	interna	Módulos importados (dotted), resolviendo imports relativos.
analizar_modulos()	233	pública	
comunidades_por_archivo()	277	pública	
comunidades_del_grafo()	290	pública	
nodos_centrales()	316	pública	
endpoints()	337	pública	Rutas reales de la app FastAPI (importarla no arranca el servidor).
catalogo_tests()	367	pública	
variables_entorno()	387	pública	
_inline()	422	interna	
_toc()	550	interna	
_portada()	566	interna	
_seccion_interfaz()	587	interna	

Nombre	Línea	Tipo	Qué hace
_seccion_informes()	607	interna	
_ficha_modulo()	647	interna	
_seccion_modulos()	671	interna	
_seccion_endpoints()	691	interna	
_seccion_tests()	706	interna	
_seccion_variables()	724	interna	
_seccion_grafo()	735	interna	
_seccion_matriz_fallos()	757	interna	Tablas «Errores frecuentes» de todas las skills, juntas.
construir()	784	pública	
a_pdf()	838	pública	Convierte con Microsoft Word (COM vía PowerShell) actualizando índice y campos.
main()	855	pública	
class Escritor	437	clase	Envoltorio del documento: títulos numerados con marcador, md, imágenes. Métodos: titulo, p, salto, imagen, tabla, markdown

scripts/pipeline_calidad_datos.py

Propósito: Pipeline de calidad de datos DEL VALIDADOR. Genera la tabla que el agente lee.

Tamaño: 350 líneas

Comunidades del grafo: Datos · Pipeline de calidad de datos

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
_control_fechas_coherentes()	127	interna	FechaVencimiento debe ser posterior a FechaApertura.
_veredicto()	166	interna	Aplica TU umbral. No inventa criterio propio.
ejecutar_plan()	182	pública	Corre el plan y devuelve las filas de la tabla. NUNCA lanza por control.
escribir_tabla()	289	pública	Escribe la tabla donde el agente la busca.
main()	302	pública	

scripts/preguntas_oro_formula.py

Propósito: Golden set DISCRIMINANTE de fórmula/tabla + medición IR léxica (API-free).

Tamaño: 284 líneas

Comunidades del grafo: Scripts · Preguntas de oro de fórmulas · RAG · Indexador léxico

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
norm()	44	pública	Normaliza el espacio en blanco (la extracción PDF mete espacios dobles).
_buscadores()	179	interna	
_ndcg()	193	interna	
evaluar_chunker()	200	pública	
verificar_ground_truth()	234	pública	Falla si alguna frase_clave NO existe en su documento (no inventar GT).

scripts/preguntas_oro_rag.py

Propósito: Set de preguntas-oro del RAG léxico: ¿recupera el corpus el documento correcto?

Tamaño: 713 líneas

Comunidades del grafo: Scripts · Preguntas de oro del RAG

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: test_beta_calibracion.py, test_pendientes_auditoria.py

Nombre	Línea	Tipo	Qué hace
_elegible_r3()	378	interna	
_chunks_del_dominio()	426	interna	Cuántos fragmentos aporta cada documento al índice de ese dominio.
_fuentes_del_dominio()	451	interna	Ficheros realmente presentes en el corpus de ese dominio.
evaluar()	461	pública	Lanza el set y devuelve el detalle. No lanza: un dominio sin índice se
evaluar_adversarias()	520	pública	Pregunta cada consulta al dominio EQUIVOCADO y comprueba que el sistema no
_metricas()	550	interna	recall@1 sobre todas las aplicables; recall@3 sobre las ELEGIBLES.
informe()	580	pública	Imprime el resultado. Devuelve el código de salida sugerido.
main()	693	pública	

scripts/rebuild_rag_databricks.py

Propósito: Script de mantenimiento para reconstruir los índices RAG en Databricks.

Tamaño: 236 líneas

Comunidades del grafo: RAG · Indexadores por dominio y reconstrucción

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_dominios_custom()	69	interna	Lee los dominios personalizados que la web haya registrado.
_dominios_conocidos()	86	interna	
_get_indexer()	90	interna	Importa y retorna el indexer del dominio especificado.
rebuild_dominio()	114	pública	Reconstruye el índice RAG de un dominio.
mostrar_status()	138	pública	Muestra el estado de cada índice RAG léxico.
main()	173	pública	

scripts/reconstruir_estandar_desde_informes.py

Propósito: (sin docstring de módulo)

Tamaño: 293 líneas

Comunidades del grafo: Scripts · Reconstruir estándar y verificar corpus

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_norm()	90	interna	Normaliza para detectar variantes del mismo texto (erratas incluidas).
extraer()	96	pública	Recolecta (ámbito → control → actividad) de todos los informes emitidos.
filas()	134	pública	Aplana a filas: UNA por (ámbito, actividad).
escribir()	173	pública	Genera el Excel con el formato que espera Mx_01.leer_estandares_virf().
main()	248	pública	

scripts/verificar_corpus_rag.py

Propósito: VERIFICADOR DEL CORPUS RAG — ¿el índice contiene lo que dice contener?

Tamaño: 337 líneas

Comunidades del grafo: Scripts · Verificación del corpus RAG

Lo usan: nadie lo importa directamente (punto de entrada, script o carga dinámica)

Tests que lo importan: ninguno lo importa directamente (puede cubrirse vía otro módulo)

Nombre	Línea	Tipo	Qué hace
_norm()	94	interna	
_ficheros_esperados()	106	interna	Nombres de los ficheros fuente que ESTE dominio debe contener.
verificar_dominio()	116	pública	Comprobaciones 1-4 sobre un dominio.
verificar_informe()	230	pública	Comprobación 5: ¿el RAG llegó realmente al prompt en una corrida real?
main()	265	pública	

Parte 8 · Catálogo de endpoints de la web

Leído de la aplicación FastAPI real (`web.app`). La pantalla llama a estas rutas desde `web/static/js/app.js`; para saber qué archivo tocar, busca la ruta aquí.

web/app.py

Método	Ruta	Función	Descripción
GET	/	index	
GET	/api/salud	salud	

web/routers/auth.py

Método	Ruta	Función	Descripción
GET	/api/auth/actual	validador_actual	Devuelve el validador logueado o 401 si no hay sesión.
POST	/api/auth/login	hacer_login	
POST	/api/auth/logout	hacer_logout	

web/routers/automatizaciones.py

Método	Ruta	Función	Descripción
POST	/api/automatizaciones/cuadro	generar	
GET	/api/automatizaciones/descargar/{fichero}	descargar	
GET	/api/automatizaciones/estado	estado	

web/routers/chat.py

Método	Ruta	Función	Descripción
GET	/api/chat/acta.xlsx	descargar_acta_cierre	Acta de cierre en Excel (3 hojas: Resumen · Validación · Decisiones) con el
POST	/api/chat/adjuntar	adjuntar_chat	El validador sube un archivo con evidencia nueva para la observación en curso.
POST	/api/chat/consolidar	consolidar_chat	Cierre HITL: recomputa el PUNTAJE EXACTO final con las decisiones del chat y

Método	Ruta	Función	Descripción
GET	/api/chat/estado	estado_chat	
POST	/api/chat/iniciar	iniciar_chat	
GET	/api/chat/observaciones	observaciones	Resumen enfocado para dashboards, sin arrastrar todos los mensajes.
WS	/ws/chat	ws_chat	Requiere cookies `virf_token` (login) y `virf_modelo` (modelo activo).

web/routers/config.py

Método	Ruta	Función	Descripción
GET	/api/config	get_config	Estado del .env + validador logueado (sin secretos).
POST	/api/config/modelo	set_modelo	Guarda título (etiqueta visible) y tier del MODELO ACTIVO. El modelo
POST	/api/config/proveedor	set_proveedor	
POST	/api/config/verificar	verificar_conexion	
POST	/api/config/virf_plus	set_virf_plus	Botón VIRF PLUS: suma (o quita) a Claude como segunda opinión del agente

web/routers/controles.py

Método	Ruta	Función	Descripción
GET	/api/motores/catalogo	catalogo	Las pruebas genéricas de `modulos.motores_datos.REGISTRO_MOTORES`.
GET	/api/motores/entidad	resultados_de_la_entidad	Lee las tablas de resultados que la entidad deja bajo
GET	/api/motores/notebook	descargar_notebook_controles	Genera y devuelve el .ipynb ENTREGABLE de controles de datos (8 columnas).

web/routers/cuestionamientos.py

Método	Ruta	Función	Descripción
GET	/api/cuestionamientos/default	listar_default	Los cuestionamientos del catálogo + su flag `activo` (default True).

Método	Ruta	Función	Descripción
POST	/api/cuestionamientos/default/{cid}/toggle	toggle_default	
GET	/api/cuestionamientos/personalizados	listar_personalizados	
POST	/api/cuestionamientos/personalizados	agregar_personalizado	
DELETE	/api/cuestionamientos/personalizados/{cid}	borrar_personalizado	

web/routers/cuestionamientos_sesion.py

Método	Ruta	Función	Descripción
GET	/api/cuestionamientos-sesion	get_estado	
POST	/api/cuestionamientos-sesion/custom	crear_custom	
DELETE	/api/cuestionamientos-sesion/custom/{cid}	borrar_custom	
POST	/api/cuestionamientos-sesion/toggle	toggle	

web/routers/debug_conexiones.py

Método	Ruta	Función	Descripción
GET	/api/debug/conexiones	conexiones	

web/routers/dominio_prevuelo.py

Método	Ruta	Función	Descripción
POST	/api/prevuelo	prevuelo	Coste 0. Recorre `web/data/sesiones/{val}/{slug}/inputs/` y devuelve

web/routers/entendimiento.py

Método	Ruta	Función	Descripción
GET	/api/entendimiento	obtener	
POST	/api/entendimiento/analizar	analizar	Analiza el ZIP descomprimido y genera `entendimiento_modelo_{slug}.md`.

Método	Ruta	Función	Descripción
POST	/api/entendimiento/feedback	feedback	

web/routers/estandares.py

Método	Ruta	Función	Descripción
DELETE	/api/estandares	borrar	
POST	/api/estandares/confirmar	confirmar	Confirma el estándar YA parseado (upload lo dejó en el .xlsx de la sesión).
GET	/api/estandares/detalle	get_detalle	
GET	/api/estandares/estado	get_estado	
POST	/api/estandares/upload	upload	

web/routers/guia.py

Método	Ruta	Función	Descripción
GET	/api/guia/progreso	progreso	

web/routers/informe.py

Método	Ruta	Función	Descripción
POST	/api/informe/consolidado	consolidar	
GET	/api/informe/descargar/{fichero}	descargar	
GET	/api/informe/estado	estado	Panorama para el panel «Documentar Todo».
GET	/api/informe/versiones	versiones	

web/routers/modelos.py

Método	Ruta	Función	Descripción
GET	/api/modelos	listar_modelos	
POST	/api/modelos	crear_modelo	

Método	Ruta	Función	Descripción
GET	/api/modelos/activo	modelo_activo	Modelo actualmente seleccionado (o `None`).
POST	/api/modelos/desactivar	desactivar	
DELETE	/api/modelos/{slug}	borrar_modelo	
POST	/api/modelos/{slug}/activar	activar_modelo	Marca el modelo como activo (cookie); el resto del backend lo usa.

web/routers/preview_prompt.py

Método	Ruta	Función	Descripción
POST	/api/validacion/preview-prompt	preview_prompt	

web/routers/prompts.py

Método	Ruta	Función	Descripción
GET	/api/prompts	get_efectivos	Los prompts efectivos de la sesión (custom si existen, base si no).
GET	/api/prompts/base	get_base	Los prompts por defecto (para el editor cuando aún no hay sesión).
POST	/api/prompts/editar	editar	
GET	/api/prompts/presupuesto	presupuesto	Presupuesto de contexto para la sesión. Usa el modelo detectado por
POST	/api/prompts/restaurar	restaurar	

web/routers/rag.py

Método	Ruta	Función	Descripción
GET	/api/rag/dominios	listar_dominios	Fijos primero, custom después. Cada uno con estado del índice y papers.
POST	/api/rag/dominios/crear	crear_dominio	
DELETE	/api/rag/dominios/{dom_id}	borrar_dominio	
DELETE	/api/rag/dominios/{dom_id}/paper/{nombre}	borrar_paper	Elimina un libro/manual CONCRETO del corpus `inputs/rag/{dom_id}/` y

Método	Ruta	Función	Descripción
POST	/api/rag/reconstruir	reconstruir	Reconstruye el índice del dominio dado. Acepta cualquier dominio

web/routers/resultados.py

Método	Ruta	Función	Descripción
GET	/api/resultados/actual	resultado_de_la_ultima_corrida	
GET	/api/resultados/excel	descargar_excel	Descarga la tabla de validación en Excel.
GET	/api/resultados/lista	listar_resultados	Todos los `resultados_*.json` DE LA SESIÓN activa.
GET	/api/resultados/tabla	tabla_de_la_ultima_corrida	Output intermedio: la tabla de validación por actividad (Ámbito | Control |
GET	/api/resultados/tabla/{fecha}/{fichero}	tabla_por_ruta	
GET	/api/resultados/{fecha}/{fichero}	resultado_por_ruta	

web/routers/skills.py

Método	Ruta	Función	Descripción
GET	/api/skills	get_skills	Skills DE LA SESIÓN activa: defaults (con flag `activo` según el
POST	/api/skills/custom	crear_custom	
DELETE	/api/skills/custom/{skill_id}	borrar_custom	
GET	/api/skills/defaults	listar_defaults	Catálogo por defecto SIN estado (para inspección/documentación).
POST	/api/skills/toggle	toggle	

web/routers/solo_informe.py

Método	Ruta	Función	Descripción
PUT	/api/solo-informe/borrador	guardar_borrador	
GET	/api/solo-informe/descargar/{nombre}	descargar	

Método	Ruta	Función	Descripción
GET	/api/solo-informe/estado	estado	
POST	/api/solo-informe/estandar	subir_estandar	
DELETE	/api/solo-informe/estandar	borrar_estandar	
POST	/api/solo-informe/generar	generar	
GET	/api/solo-informe/ia/estado	ia_estado	
POST	/api/solo-informe/ia/grafico	ia_grafico	
POST	/api/solo-informe/ia/manuales	ia_manuales	
POST	/api/solo-informe/ia/pulir	ia_pulir	
POST	/api/solo-informe/imagen	subir_imagen	
GET	/api/solo-informe/imagen/{ident}	ver_imagen	
GET	/api/solo-informe/manuales	manuales	
POST	/api/solo-informe/manuales	subir_manual	
DELETE	/api/solo-informe/manuales/{nombre}	borrar_manual	

web/routers/upload.py

Método	Ruta	Función	Descripción
GET	/api/inputs/arbol	arbol_inputs	Árbol de `inputs/` DE LA SESIÓN activa. El RAG global no se muestra
DELETE	/api/inputs/archivo/{slug}/{fichero}	borrar_archivo	Borra un fichero SUBIDO desde este panel. Solo permite eliminar
GET	/api/inputs/carpeta/{slug}	listar_carpeta	Ficheros ya subidos a `slug` — para pintar cada card del upload guiado.
GET	/api/inputs/destinos	listar_destinos	Slugs válidos con etiqueta y alcance (sesion|global).
POST	/api/inputs/upload	subir_archivos	
POST	/api/inputs/upload-zip	subir_zip	Descomprime un ZIP en `destino`. Antes: el flujo especial para el

web/routers/validacion.py

Método	Ruta	Función	Descripción
POST	/api/validar/cancelar/{run_id}	cancelar_validacion	
GET	/api/validar/estado	estado_actual	
POST	/api/validar/lanzar	lanzar_validacion	
WS	/ws/validar/{run_id}	ws_validar	Emite el stream de eventos del runner identificado por `run_id`.

Parte 9 · Catálogo de tests

Todos los tests corren sin coste (IA simulada) con `.venv\Scripts\python.exe -m pytest -q`. Tras un cambio, corre al menos el archivo del área.

Archivo	Tests	Qué protege
test_abstencion.py	6	Verifica la compuerta de abstención activa (modulos/abstencion.py) de forma DETERMINISTA y SIN API: la decisión se toma sobre los scores léxicos ya re
test_acta_cierre.py	5	Pruebas del ACTA DE CIERRE en Excel (3 hojas) generada al finalizar el chat. Coste 0: no llama al LLM; usa datos sintéticos y consolidación determinis
test_aislamiento.py	7	Verifica el aislamiento multi-tenant (modulos/aislamiento.py) de forma DETERMINISTA y SIN API. Simula una recuperación con fragmentos de dos tenants y
test_aprendizaje_chat.py	4	(contexto + ajuste automático en futuras corridas). Coste 0; historial aislado.
test_arbol_entornos.py	3	Los recorridos del árbol deben reconocer un entorno virtual por `pyvenv.cfg`, no por su nombre. Con el filtro por nombre, un entorno llamado `env/` o
test_automatizaciones.py	12	Cuadro de observaciones desde los Excels 00–03 de automatización. Coste cero.
test_azure_mocks.py	70	Verificación del flujo completo con Azure OpenAI SIMULADO.
test_beta_calibracion.py	5	β se fijó (config.ABSTENCION_BETA) desde la distribución de `cobertura` léxica sobre las preguntas-oro: respondibles ∈ [0.379, 1.0], evidencia ausente
test_capturas.py	6	Capturas de evidencia para los informes («pantallazos»). Coste cero: sin IA; los PDF se generan al vuelo con Pillow y todo vive en `tmp_path`.
test_chat_adjuntar.py	2	Coste 0: usa el mock del chat (VIRF_MOCK_CHAT=1), no llama al modelo.
test_chunker_estructural.py	7	Verifica que la segmentación estructural (modulos/chunker_estructural.py) respeta encabezados y NO parte tablas ni fórmulas por la mitad. Determinista
test_circuit_breakers.py	8	Verifica el cortacircuitos (modulos/fusibles.py) de forma DETERMINISTA y SIN API: el reloj se inyecta, así que no se duerme ni se llama al modelo. Cub
test_citas_deterministas.py	7	Verifica el validador de citas con id de linaje (modulos/validador_citas.py) de forma DETERMINISTA y SIN API: una afirmación factual solo se acepta si
test_consolidacion.py	6	Cierre HITL: el chat produce el puntaje EXACTO final (WP-A).
test_docx_regresion.py	6	ValidationReport mock. Sin API (no usa el pipeline real).
test_endpoint_notebook_controles.py	2	Prueba del endpoint GET /api/motores/notebook (coste 0: LLM interceptado).
test_entailment.py	7	Verificación semántica de citas (modulos/entailment.py) — SIN API.
test_esquemas.py	10	Verifica que un dict correcto produce un ValidationReport válido y que uno malformado (status inválido, cita ausente, confianza fuera de rango) FALLA

Archivo	Tests	Qué protege
test_estandares_sesion.py	10	Tests del paso 02 · Estándares de validación por sesión (WEB-V2.4).
test_estructura_config.py	22	Estructura de config/ y preparación para Azure OpenAI.
test_exportador_excel.py	2	
test_guia_agente.py	7	Guía del agente corporativo (scripts/generar_guia_agente.py + docs/agente_corporativo/). Coste cero: sin IA y sin Word (el PDF no se genera en los tes
test_informe_tabla_final.py	2	
test_invariante_asyncio.py	2	Invariante: ninguna `asyncio.create_task(...)` puede quedar como sentencia suelta.
test_inyeccion.py	7	Defensa anti-inyección de instrucciones (modulos/inyeccion.py): alta precisión, 100% local. Neutraliza órdenes dirigidas al evaluador SIN mutilar pros
test_notebook_controles.py	10	Pruebas API-free del generador de notebook de controles de datos (8 columnas).
test_pendientes_auditoria.py	231	Tests de los pendientes de auditoría externa (SBS / Big4).
test_pii.py	9	Enmascarado PII (modulos/pii.py): alta precisión, 100% local, sin red.
test_plantilla_bcp.py	4	Plantilla común de los tres informes Word (Solo Informe, «Generar informe» de la web y entregable del pipeline). Coste cero: sin IA, todo en `tmp_path
test_prompts_editables.py	17	Tests de la capa de prompts editables por sesión y presupuesto dinámico (WEB-V2.3).
test_rag_desactualizado.py	4	tras añadir manuales (cierra el footgun de «subí papers pero no reconstruí»).
test_rag_lexico.py	39	Tests del RAG LÉXICO (BM25 + TF-IDF) que sustituyó al RAG vectorial el 2026-07-31.
test_regresion.py	463	Origen: creada durante la auditoría de regresión del 2026-07-26, tras aplicar limpieza de código muerto, corrección del flujo secuencial, chunking ada
test_requirements_web_fijados.py	1	web/requirements_web.txt debe fijar versiones exactas (==), como requirements.txt.
test_solo_informe.py	32	Tests del apartado «Solo Informe» (informe sin pipeline). Coste cero: no hay IA en ningún punto del flujo, y todo escribe en `tmp_path`.
test_solo_informe_ia.py	19	Tests de la asistencia con IA de «Solo Informe». Coste cero: el modelo se sustituye por `_llm_de_prueba`, que registra el prompt y devuelve una respue
test_tabla_validacion.py	18	Veredicto formal por actividad + tabla intermedia + puente al chat.
test_upload_zip.py	5	POST /api/inputs/upload-zip — extracción segura de ZIP (coste 0, carpeta temporal).

Archivo	Tests	Qué protege
test_validador_entorno_hf.py	2	Guardia de la política «cero HuggingFace / cero ChromaDB» (validate_project.py).
test_versionado.py	13	Dos piezas, ambas deterministas y sin API: · Pre-filtro duro is_active en la recuperación léxica (no validar contra metodologías derogadas). · Detecci
test_virf_plus.py	12	VIRF PLUS (GPT principal + Claude como segunda opinión) — coste cero, con dobles.

test_abstencion.py

Verifica la compuerta de abstención activa (modulos/abstencion.py) de forma DETERMINISTA y SIN API: la decisión se toma sobre los scores léxicos ya recuperados. Fixture de "retrieval de score bajo" → devuelve la abstención estándar "EVIDENCIA INSUFICIENTE".

sin_fragmentos_se_abstiene · retrieval_debil_por_debajo_de_beta_se_abstiene · evidencia_fuerte_no_se_abstiene · beta_por_defecto_solo_abstiene_ante_ausencia_total · scores_no_numericos_se_ignoran · la_etiqueta_es_estable

test_acta_cierre.py

Pruebas del ACTA DE CIERRE en Excel (3 hojas) generada al finalizar el chat. Coste 0: no llama al LLM; usa datos sintéticos y consolidación determinista.

acta_tiene_tres_hojas_y_columnas · acta_resumen_puntaje_sube_con_aprobado · acta_hoja_decisiones_lista_cada_veredicto · acta_marca_decision_en_filas_cambiadas · endpoint_acta_descarga_xlsx

test_aislamiento.py

Verifica el aislamiento multi-tenant (modulos/aislamiento.py) de forma DETERMINISTA y SIN API. Simula una recuperación con fragmentos de dos tenants y comprueba que, filtrando por el Tenant A, el 0% de lo devuelto es ajeno (D2 = 0%), y que la aserción de defensa en profundidad grita ante un spillover.

no_spillover · conjunto_mixto_dispara_la_asercion · fragmento_sin_dueno_se_considera_ajeno · filtrar_solo_por_tenant_ignora_modelo · etiquetar_no_muta_el_original · conjunto_vacio_es_limpio · buscar_acepta_filtro_tenant_sin_indice

test_aprendizaje_chat.py

(contexto + ajuste automático en futuras corridas). Coste 0; historial aislado.

aprobado_se_guarda_como_correccion · idempotente_por_sesion · pendiente_no_se_aprende · la_correccion_entra_como_contexto_futuro

test_arbol_entornos.py

Los recorridos del árbol deben reconocer un entorno virtual por `pyvenv.cfg`, no por su nombre. Con el filtro por nombre, un entorno llamado `env/` o un respaldo `.venv_respaldo/` dentro del proyecto daba 7 tests en rojo falsos.

entorno_con_nombre_arbitrario_se_reconoce · site_packages_se_reconoce · codigo_del_proyecto_no_se_confunde_con_entorno

test_automatizaciones.py

Cuadro de observaciones desde los Excels 00–03 de automatización. Coste cero.

localiza_por_prefijo_e_ignora_temporales · lectura_detecta_encabezado_y_no_conformes · impactos_prefiere_nivel_cualitativo · normalizar_riesgo · responsable_y_fecha_por_origen_y_riesgo · cuadro_con_ia_riesgo_desde_impactos · sin_impactos_el_riesgo_es_estimado · borrador_determinista_y_orden · ia_sin_json_falla_con_mensaje · carpeta_vacia · exporta_excel_con_columnas_del_cuadro · informe_docx_usa_el_cuadro_en_resumen_y_anexo

test_azure_mocks.py

Verificación del flujo completo con Azure OpenAI SIMULADO.

el_sistema_va_como_primer_mensaje_no_como_parametro · usa_el_deployment_no_el_nombre_del_modelo · devuelve_str_no_el_objeto_del_sdk · formato_json_activa_response_format · sin_formato_json_no_se_envia_response_format · memoriza_el_modelo_real_de_la_respuesta · modelo_clasico_acepta_max_tokens · modelo_de_razonamiento_recibe_max_completion_tokens · el_parametro_negociado_se_memoriza · si_ambos_parametros_fallan_se_reporta_con_claridad · choices_vacio_por_filtro_de_contenido · content_none_se_reporta_con_su_finish_reason · un_error_de_red_no_se_traga · el_fallo_de_azure_NO_cae_a_anthropic · el_parser_extrae_el_contrato · el_texto_posterior_con_llaves_no_confunde_al_parser · dos_objetos_seguidos_no_se_fusionan · json_invalido_no_devuelve_un_dict_inventado · evaluar_actividad_pipeline_2 · chunking_resume_con_azure · el_score_y_el_semaforo_se_calculan · no_se_instancia_ningun_cliente_anthropic · pipeline_1_evalua_un_control_con_azure · explicador_genera_las_9_secciones_con_azure · explicador_inventaria_sin_llamar_al_modelo · revision_de_codigo_con_azure · la_revision_de_codigo_no_cae_a_anthropic · credencial_invalida_lanza_y_no_devuelve_texto · endpoint_incorrecto_se_reporta_con_el_detalle · deployment_inexistente_da_404_accionable · el_mensaje_del_preflight_nombra_al_proveedor_activo · no_existe_fallback_automatico_en_el_codigo · el_perfil_se_adapta_al_modelo_que_sirva_el_deployment · un_modelo_desconocido_no_rompe_y_es_conservador · el_cliente_memoriza_el_modelo_real · un_429_se_reintenta_y_la_corrida_continua · un_401_NO_gasta_reintentos · al_agotarse_propaga_el_error_real · respeta_el_retry_after_del_servidor · sin_retry_after_usa_backoff_exponencial_acotado · clasifica_bien_lo_transitorio · el_parametro_esta_conectado_a_config · el_cliente_azure_reintenta_de_verdad · sin_configuracion_no_se_construye_cliente_http · un_ca_bundle_inexistente_se_rechaza_con_mensaje_util · un_ca_bundle_corrupto_se_explica · no_se_usa_la_forma_deprecada_de_verify · el_diagnostico_nombra_la_causa_real · produccion_rechaza_tls_desactivado · los_parametros_viven_en_config · config_embeddings_ya_no_existe · los_parametros_de_embeddings_no_se_declaran · requirements_no_declara_ningun_paquete_de_huggingface · requirements_declara_el_motor_lexico · el_rag_no_importa_chromadb_ni_pide_embeddings · los_defaults_de_config_rag_no_divergen_de_config · indexar_y_buscar_no_toca_la_red · ningun_modulo_de_negocio_importa_modelos_externos · el_parametro_del_modelo_local_no_se_declara · el_sdk_toma_posesion_del_http_client · el_cliente_de_chat_tambien_se_reconstruye · el_diagnostico_de_content_filter_nombra_la_categoria · el_filtro_en_la_salida_tambien_se_diagnostica · un_deployment_tipo_o1_valida_correctamente · el_parametro_negociado_queda_memorizado · un_modelo_clasico_acierta_a_la_primera · una_credencial_mala_sigue_reportandose · no_se_reintenta_indefinidamente · el_ping_no_pide_100_tokens · el_codigo_no_vuelve_a_hardcodear_el_parametro

test_beta_calibracion.py

β se fijó (config.ABSTENCION_BETA) desde la distribución de `cobertura` léxica sobre las preguntas-oro: respondibles ∈ [0.379, 1.0], evidencia ausente = 0.0. β = 0.19 cae en el hueco de separación. Sin API (léxico determinista).

beta_esta_en_el_hueco_de_separacion · una_respondible_no_se_abstiene · todas_las_respondibles_superan_beta · evidencia_ausente_se_abstiene · beta_es_configurable_por_el_operador

test_capturas.py

Capturas de evidencia para los informes («pantallazos»). Coste cero: sin IA; los PDF se generan al vuelo con Pillow y todo vive en `tmp_path`.

localizar_y_citas_en_evidencia · pdf_pagina_exacta_resaltada · pdf_cita_inexistente_no_inventa_pagina · portada_y_cache · md_y_codigo_con_contexto · informe_web_inserta_capturas

test_chat_adjuntar.py

Coste 0: usa el mock del chat (VIRF_MOCK_CHAT=1), no llama al modelo.

adjuntar_enriquece_evidencia_y_reevalua · adjuntar_sin_texto_falla

test_chunker_estructural.py

Verifica que la segmentación estructural (modulos/chunker_estructural.py) respeta encabezados y NO parte tablas ni fórmulas por la mitad. Determinista, sin API.

no_parte_la_tabla_a_la_mitad · no_parte_la_formula_a_la_mitad · segmenta_por_encabezados · preserva_erratas · ningun_segmento_corta_a_mitad_de_linea · texto_vacio_devuelve_lista_vacia · bloque_atomico_mayor_que_el_tope_se_emite_entero

test_circuit_breakers.py

Verifica el cortacircuitos (modulos/fusibles.py) de forma DETERMINISTA y SIN API: el reloj se inyecta, así que no se duerme ni se llama al modelo. Cubre el cap de iteraciones (<=3), el fusible de tiempo (~10 s), el de tokens (~20.000) y el escalado a HITL con hallazgos intermedios.

cap_de_iteraciones_corta_al_cuarto_intento · cap_por_defecto_es_tres · fusible_de_tiempo_corta_pasados_los_diez_segundos · verificar_tiempo_explicito · fusible_de_tokens_corta_pasados_los_veinte_mil · tokens_negativos_se_rechazan · escalar_preserva_los_hallazgos_intermedios · max_iter_invalido

test_citas_deterministas.py

Verifica el validador de citas con id de linaje (modulos/validador_citas.py) de forma DETERMINISTA y SIN API: una afirmación factual solo se acepta si trae un id [doc:pagina:chunk] bien formado cuyo chunk EXISTE en el contexto recuperado. La verificación semántica (NLI) queda diferida y se comprueba que lo declara.

cita_determinista_bien_formada_y_localizable_se_acepta · cita_determinista_sin_cita_se_rechaza · cita_determinista_chunk_inexistente_se_rechaza · id_de_chunk_se_construye_desde_las_claves_del_proyecto · extraer_ids_multiples · agregado_alimenta_tasa_para_d4 · entailment_esta_diferido_a_api

test_consolidacion.py

Cierre HITL: el chat produce el puntaje EXACTO final (WP-A).

aprobado_sube_a_cumple_5 · negado_mantiene_puntaje · no_muta_el_eval_original · valor_cumple_configurable · sin_decisiones_score_final_igual_al_inicial · aprobado_marca_cumple_en_la_fila_final

test_docx_regresion.py

ValidationReport mock. Sin API (no usa el pipeline real).

veredicto_de_ambito_reglas · genera_docx_con_veredictos_por_ambito · seccion_sin_observaciones · seccion_referencias_consolidada · pie_con_paginacion_tiene_campo_page · abstencion_se_declara_en_el_docx

test_endpoint_notebook_controles.py

Prueba del endpoint GET /api/motores/notebook (coste 0: LLM interceptado).

endpoint_repositorio_dominio · endpoint_fallback_llm_sin_repositorio

test_entailment.py

Verificación semántica de citas (modulos/entailment.py) — SIN API.

apagado_por_defecto_devuelve_na · implica_con_cita_real · no_implica · veredicto_sin_cita_real_se_degrada · insuficiente · entradas_cortas_no_llaman_al_juez · juez_que_revienta_no_tumba_la_evaluacion

test_esquemas.py

Verifica que un dict correcto produce un ValidationReport válido y que uno malformado (status inválido, cita ausente, confianza fuera de rango) FALLA la validación. Determinista, sin API.

dict_correcto_produce_reporte_valido · status_invalido_falla · cita_ausente_falla · desviado_sin_codigo_falla · omitido_admite_codigo_vacio · confianza_fuera_de_rango_falla · model_id_en_blanco_falla · sinonimo_hallazgos_se_acepta · intentar_validar_no_levanta · reporte_abstenido_sin_hallazgos_es_valido

test_estandares_sesion.py

Tests del paso 02 · Estándares de validación por sesión (WEB-V2.4).

parseo_basico_con_merged_simulado · rechaza_sin_cabeceras · rechaza_sin_actividades · orden_ambitos_es_de_aparicion · roundtrip_upload_confirmar_estado · limpiar_borra_ambos_ficheros · redirigir_estandar_actualiza_rutas · redirigir_estandar_lanza_si_no_existe · pipeline_runner_pasa_estandares_si_confirmado · pipeline_runner_omite_estandares_si_no_override

test_estructura_config.py

Estructura de config/ y preparación para Azure OpenAI.

nadie_importa_del_directorio_archivo · no_reaparece_config_py_en_la_raiz · config_resuelve_al_paquete · la_api_publica_de_config_esta_completa · ningun_modulo_declara_su_propia_ruta_de_config · el_default_vive_en_el_codigo_no_en_el_env · la_seleccion_de_proveedor_vive_solo_en_config · no_hay_fallback_automatico_entre_proveedores · ambos_clientes_cumplen_la_misma_interfaz · el_negocio_no_toca_el_sdk · no_quedan_endpoints_de_proveedor_fuera_de_config · resuelve_por_prefijo_con_fecha_de_version · la_coincidencia_mas_larga_gana · reconoce_el_modelo_dentro_de_un_deployment_con_prefijo · un_modelo_desconocido_cae_en_perfil_conservador · los_modelos_de_razonamiento_exigen_max_completion_tokens · gpt4_clasico_reduce_el_presupuesto · el_chunk_size_se_acota_por_calidad · max_chunks_nunca_baja_del_umbral_de_coste · el_comando_existe · no_exige_api_key_en_el_preflight · reporta_sin_lanzar_cuando_no_hay_credencial

test_exportador_excel.py

xlsx_valido_con_8_columnas_y_aplica · final_incluye_puntaje_validado

test_guia_agente.py

Guía del agente corporativo (scripts/generar_guia_agente.py + docs/agente_corporativo/). Coste cero: sin IA y sin Word (el PDF no se genera en los tests).

rutas_citadas_existen · skill_tiene_frontmatter_y_secciones · hay_doce_skills_y_capturas · proposito_salta_la_linea_que_repite_el_nombre · endpoints_incluye_los_routers_anidados · catalogo_de_tests_y_variables · genera_el_docx_con_todas_las_partes

test_informe_tabla_final.py

seccion_tabla_final_agrega_tabla_y_puntaje · seccion_vacia_no_revienta

test_invariante_asyncio.py

Invariante: ninguna `asyncio.create_task(...)` puede quedar como sentencia suelta.

ninguna_create_task_queda_sin_referencia · pipeline_runner_retiene_sus_tareas

test_inyeccion.py

Defensa anti-inyección de instrucciones (modulos/inyeccion.py): alta precisión, 100% local. Neutraliza órdenes dirigidas al evaluador SIN mutilar prosa legítima de un documento de riesgos.

ignorar_instrucciones_se_neutraliza · forzar_puntaje_se_neutraliza · reasignar_rol_se_neutraliza · anular_hallazgos_se_neutraliza · marca_de_sistema_se_neutraliza · prosa_de_riesgos_legitima_no_se_toca · nunca_lanza_y_es_idempotente

test_notebook_controles.py

Pruebas API-free del generador de notebook de controles de datos (8 columnas).

normaliza_renumera_y_mapea_dimension · umbral_se_formatea_segun_semantica_del_motor · todos_los_motores_tienen_dimension_y_decision · rutas_van_al_inicio_y_es_ipynb_valido · ejecuta_y_entrega_tabla_8_columnas · decisiones_y_porcentajes_correctos · csv_persistido · orquestador_modo_documento · orquestador_modo_entendimiento · orquestador_sin_controles_inserta_plantilla

test_pendientes_auditoria.py

Tests de los pendientes de auditoría externa (SBS / Big4).

informe_borrado_se_detecta_como_ausente · borrar_outputs_entero_no_borra_la_prueba · el_registro_solo_crece · registro_nunca_abre_en_modo_escritura · informe_archivado_sigue_contando_como_presente · informe_editado_se_detecta_como_alterado · registro_corrupto_no_se_descarta_en_silencio · la_emision_registra_quien_firma · ambas_ramas_de_sellar_y_escribir_registran · flag_verificar_emisiones_disponible · el_registro_se_versiona · la_suite_no_escribe_en_el_registro_real · registros_antiguos_no_se_descartan · validador_nulo_no_rompe_la_lectura · el_anexo_dice_quien_hizo_cada_correccion · calidad_proceso_ya_no_aproxima_por_modelo · la_senal_declara_su_limite_con_historial_antiguo · ambos_modulos_de_feedback_firman · los_dos_modulos_usan_el_mismo_resolutor_de_identidad · la_identidad_del_feedback_es_la_del_informe · el_feedback_no_bloquea_sin_identidad · identidad_explicita_gana · mismo_nombre_dos_veces_es_rechazado · la_comparacion_normaliza · segundo_validador_distinto_es_aceptado · sin_segunda_firma_el_tier1_no_emite · tiers_no_declarados_no_exigen_doble_firma · desactivado_por_defecto · varios_tiers · el_tier_se_compara_como_texto · hay_via_no_interactiva · flags_disponibles · ambas_firmas_llegan_al_word · la_segunda_firma_viaja_al_json · la_doble_firma_se_resuelve_antes_de_gastar · el_segundo_firmante_queda_en_el_registro · la_decision_de_bloquear_esta_justificada_en_el_codigo · no_enruta_por_trozos_de_palabra · los_titulos_legitimos_siguen_enrutando · el_dominio_explicito_manda · el_dominio_explicito_admite_mayusculas · dominio_inexistente_falla_ruidoso · la_procedencia_distingue_declarado_de_adivinado · el_dominio_se_resuelve_una_sola_vez · el_informe_registra_dominio_y_procedencia · el_dominio_explicito_llega_al_retriever · ningun_documento_desaparece_entero · la_cobertura_declarada_es_la_real · sin_recorte_declara_100 · el_recorte_se_declara_dentro_del_contexto · los_documentos_pequenos_entran_completos · las_skills_tienen_presupuesto_propio · la_cobertura_viaja_a_los_resultados · agente_usa_el_reparto_y_no_un_corte_plano · agente_pasa_bloque_rag_a_las_skills · la_skill_regulatoria_usa_el_bloque · sin_bloque_la_skill_declara_el_limite · cada_documento_llega_rotulado_con_su_rol · agente_rotula_al_construir_el_contexto · el_prompt_manda_usar_el_rotulo · sin_implementacion_no_inventa_discrepancias · el_recorte_no_se_confunde_con_discrepancia · ambos_lados_sobreviven_al_reparto · el_bloque_llega_encuadrado · sin_fragmentos_no_se_encuadra_nada · prohibe_citarlo_como_evidencia · la_ausencia_en_el_corpus_no_es_prueba · no_sustituye_al_estandar · avisa_si_el_fragmento_no_es_texto_oficial · la_regla_de_citas_desambigua_los_dos_bloques · el_pipeline_encuadra_el_bloque · las_skills_reciben_el_bloque_encuadrado · el_cuestionamiento_metodologico_solo_va_en_metodologia · el_filtro_no_depende_de_como_se_teclee_el_ambito · el_bloque_se_inyecta_condicionalmente_en_el_prompt · ya_no_queda_la_nota_de_autofiltro · el_recuento_de_fragmentos_no_cambia_al_encuadrar · referencia_inventada_se_marca_sin_anclar

· referencia_real_se_verifica · la_verificacion_tolera_espacios_del_extractor · una_referencia_demasiado_corta_no_acredita · las_categorias_de_ausencia_no_exigen_cita · no_documentado_sobre_algo_que_el_manual_si_declara · documentado_ausente_sobre_algo_que_si_esta_en_el_codigo · no_se_corrige_cuando_la_evidencia_es_ambigua · no_se_corrige_por_nombres_demasiado_cortos · la_correccion_deja_rastro · la_correccion_ocurre_antes_de_verificar_la_cita · cita_con_elision_se_verifica_por_tramos · trocear_no_relaja_la_prueba · toda_referencia_aportada_se_verifica · el_hallazgo_viaja_con_su_estado_de_anclaje · el_informe_cuenta_anclados_y_sin_anclar · el_revisor_recibe_metodologico_y_seguimiento · el_dry_run_mide_los_mismos_manuales · el_informe_transcribe_el_codigo · el_informe_dice_por_que_esta_mal · el_informe_muestra_contra_que_se_contrasta · el_informe_marca_lo_no_anclado · sin_hallazgos_no_se_pinta_seccion · la_fusion_conserva_los_hallazgos_integros · el_word_pinta_los_hallazgos · el_prompt_exige_y_declara_la_verificacion · los_hallazgos_de_las_skills_llegan_al_word · una_skill_que_fallo_consta_en_el_word · las_citas_no_verificadas_se_declaran_en_el_word · sin_skills_no_se_pinta_el_anexo · agente_pasa_las_skills_al_word · el_word_declara_que_el_corpus_no_es_oficial · el_word_dice_si_el_dominio_se_declaro_o_se_dedujo · sin_rag_no_se_advierte_de_un_corpus_que_no_se_uso · un_json_truncado_se_reintenta_y_se_recupera · si_acierta_a_la_primera_no_gasta_de_mas · si_falla_siempre_se_declara_el_fallo_sin_inventar_puntaje · un_contrato_incompleto_tambien_reintenta · el_numero_de_intentos_es_configurable_y_central · los_errores_de_red_no_se_reintentan_aqui · las_skills_no_revientan_con_actividades_sin_puntaje · una_actividad_no_evaluada_no_se_presenta_como_cero · las_puntuadas_van_primero_y_de_peor_a_mejor …

test_pii.py

Enmascarado PII (modulos/pii.py): alta precisión, 100% local, sin red.

correo_se_enmascara · tarjeta_valida_luhn_se_enmascara · numero_largo_no_valido_no_se_toca · iban_valido_se_enmascara_y_uno_falso_no · dni_valido_se_enmascara_y_cifra_sola_no · importe_financiero_no_es_pii · telefono_solo_con_contexto · texto_limpio_pasa_igual · nunca_lanza_con_entradas_raras

test_plantilla_bcp.py

Plantilla común de los tres informes Word (Solo Informe, «Generar informe» de la web y entregable del pipeline). Coste cero: sin IA, todo en `tmp_path`.

riesgo_desde_puntaje · plantilla_minima_y_anexos · informe_web_con_plantilla · entregable_pipeline_con_plantilla

test_prompts_editables.py

Tests de la capa de prompts editables por sesión y presupuesto dinámico (WEB-V2.3).

get_prompt_fallback_a_base · editar_texto_plano_persiste · editar_template_valida_placeholders · editar_template_acepta_todos_los_placeholders · evaluacion_ambito_no_editable · restaurar_es_idempotente · editar_con_mismo_contenido_que_base_restaura · prompt_id_desconocido_lanza · gpt_4o_tiene_ventana_grande · gpt_4_clasico_tiene_ventana_pequena · desconocido_usa_conservador · none_usa_conservador · aplicar_env_no_muta_el_real · chat_service_usa_prompt_custom · analisis_codigo_usa_prompt_custom · mx03_lee_env_override · mx03_fallback_cuando_no_hay_override

test_rag_desactualizado.py

tras añadir manuales (cierra el footgun de «subí papers pero no reconstruí»).

indice_ausente_con_papers_es_desactualizado · indice_mas_nuevo_esta_al_dia · source_mas_nuevo_marca_desactualizado · readme_y_dotfiles_no_cuentan

test_rag_lexico.py

Tests del RAG LÉXICO (BM25 + TF-IDF) que sustituyó al RAG vectorial el 2026-07-31.

singular_y_plural_caen_en_el_mismo_token · los_acentos_no_parten_el_vocabulario · los_acronimos_del_dominio_sobreviven · las_stopwords_se_filtran · el_stemming_no_destruye_los_tecnicismos · una_consulta_vacia_no_revienta · el_indice_se_recarga_sin_reconstruir · construir_sin_force_reutiliza_lo_persistido · un_documento_nuevo_invalida_el_indice · un_indice_corrupto_se_descarta_en_vez_de_reventar · un_indice_de_formato_anterior_se_descarta · un_dominio_sin_literatura_produce_un_indice_vacio · no_se_persisten_objetos_de_sklearn · recupera_el_documento_correcto · una_consulta_ajena_al_corpus_devuelve_vacio · respeta_top_k · el_orden_es_determinista · combina_las_dos_señales · filtro_por_fuente · un_corpus_minusculo_no_deja_la_busqueda_muda · la_cobertura_es_comparable_entre_consultas · un_termino_ausente_del_corpus_no_penaliza_la_cobertura · el_peso_informativo_delata_una_consulta_pobre · una_consulta_pobre_declara_su_incertidumbre_en_el_prompt · el_prompt_reporta_la_cobertura_no_el_score · la_cobertura_minima_descarta_el_solape_marginal · el_tesauro_une_sinonimos_del_dominio · se_corrige_una_errata_del_estandar · la_correccion_no_inventa_parecidos · solo_se_expande_a_terminos_que_existen_en_el_corpus · el_vocabulario_crudo_se_persiste · desactivar_el_tesauro_lo_desactiva · el_peso_informativo_no_cuenta_dos_veces_la_misma_idea · el_retriever_de_dominio_expone_la_api_que_motor_ia_consume · la_inferencia_de_dominio_sigue_funcionando · el_bloque_para_prompt_no_lleva_cabecera_vacia · el_bloque_para_prompt_cita_la_fuente · el_fallo_del_rag_no_tumba_una_validacion · la_consulta_es_de_milisegundos

test_regresion.py

Origen: creada durante la auditoría de regresión del 2026-07-26, tras aplicar limpieza de código muerto, corrección del flujo secuencial, chunking adaptativo y reubicación de outputs. Antes de esto el proyecto NO tenía suite automatizada.

pesos_suman_uno · ambitos_coinciden_en_los_cuatro_sistemas · cliente_ia_se_cachea_por_proceso · env_example_sin_claves_reales · gitignore_protege_env · estandares_filtran_por_tier · el_estandar_reconstruido_declara_su_limitacion · tier_inexistente_no_revienta · datasets_no_son_evidencia_documental · config_del_validador_no_es_evidencia · cobertura_total_sin_truncar · documento_corto_no_se_chunkea · documento_vacio_no_revienta · api_caida_degrada_sin_excepcion · cobertura_de_una_pasada_cubre_300_paginas · un_manual_de_300_paginas_se_cubre_entero · mx01_no_trunca_antes_del_chunker · cobertura_se_mide_contra_el_archivo_real · extraer_json_tolera_formatos · contrato_valido_pasa · contrato_rechaza_invalidos · evaluacion_fallida_no_inventa_puntaje · cobertura_se_propaga_hasta_el_prompt · promedio_simple · excluye_sin_evidencia · excluye_evaluaciones_invalidas · ambito_totalmente_vacio_es_falta_critica · score_global_excluye_del_denominador_el_fallo_tecnico · semaforo_cubre_toda_la_escala · patron_con_historial_vacio · ajuste_exige_muestras_minimas · estadisticas_con_historial_vacio · bucle_sin_tty_no_revienta · antisesgo_exige_masa_critica · escritura_atomica_sin_residuos · purgar_huerfanos · escritores_concurrentes_no_pierden_datos · singleton · catalogo_completo · deteccion_exacta_de_defectos · umbral_se_respeta · motor_inexistente_no_revienta · dataset_ausente_no_revienta · score_none_no_revienta · genera_word_con_ambito_sin_evidencia · lee_el_excel_de_controles · nan_del_excel_no_revienta · buscador_encuentra_evidencia_markdown · buscador_excluye_configuracion · raiz_sin_artefactos_generados · outputs_organizados_por_fecha · notebook_de_controles_escribe_en_outputs · todos_los_modulos_importan · codigo_muerto_fuera_del_arbol_activo · patrones_metodologicos_completos · rag_degrada_sin_chromadb · resumenes_caben_en_el_presupuesto_de_contexto · el_excel_real_cumple_los_invariantes_estructurales · forward_fill_de_ambito_es_intencional · tolera_columnas_reordenadas · falla_explicito_si_falta_columna_obligatoria · descarta_filas_espurias · avisa_si_falta_la_columna_tier · columna_a_vacia_no_se_confunde_con_ambito · sin_codigo_paralelo_huerfano · all_no_exporta_funciones_inexistentes · extraccion_integra_sin_truncar · ninguna_seccion_se_pierde_en_la_extraccion · las_tablas_sobreviven_completas · chunking_cubre_el_100_por_ciento · resumen_cabe_en_el_presupuesto · no_hay_segmentos_vacios_ni_anomalos · segunda_corrida_no_llama_al_llm · documento_modificado_invalida_el_cache · cambiar_la_configuracion_invalida_el_cache · sin_cache_dir_funciona_igual · escritura_del_cache_es_atomica · bajar_el_limite_dispara_mas_llamadas_no_menos · el_acantilado_avisa_con_umbral_accionable · el_modo_adaptativo_no_avisa · la_cobertura_sigue_al_100_en_ambos_lados · el_default_esta_por_encima_del_acantilado · reproduce_el_fallo_sin_configurar · configurar_consola_evita_el_crash · stderr_tambien_queda_protegido · flujo_sin_reconfigure_degrada_sin_romper · es_idempotente · nunca_propaga_excepciones · los_entrypoints_configuran_la_consola · config_py_no_coexiste_con_el_paquete · base_dir_apunta_a_la_raiz_no_a_config · ningun_modulo_redeclara_parametros_de_negocio · un_cambio_de_entorno_llega_a_todos_los_modulos · la_api_publica_se_mantiene · los_submodulos_estan_separados_por_responsabilidad · no_descarta_evidencia_que_no_lleva_el_nombre_del_modelo · solo_coincidentes_restringe_de_forma_explicita · excluir_omite_ficheros_indicados · una_fuente_vacia_se_declara_no_se_disimula · sin_ninguna_evidencia_falla_en_vez_de_inventar · dry_run_no_llama_al_modelo_ni_escribe · verificar_detecta_secciones_faltantes ·

verificar_detecta_truncamiento_a_media_frase · las_nueve_secciones_estan_en_los_bloques_del_prompt · un_fallo_de_api_conserva_lo_ya_generado · los_fixtures_de_las_tres_familias_existen · el_andamiaje_no_contiene_jerga_de_LaR · cada_familia_se_inventaria_completa · el_prompt_de_cada_familia_lleva_su_propia_evidencia · el_catalogo_cubre_las_familias_de_riesgo_habituales · cada_familia_del_catalogo_tiene_disparador_y_puntos · los_supuestos_clave_por_familia_estan_vigilados · el_catalogo_es_extensible_sin_tocar_la_logica · el_catalogo_llega_al_prompt_con_su_regla_de_seleccion · una_evaluacion_real_nunca_puede_dar_cero · ambito_sin_evidencia_puntua_cero · ambito_con_fallo_tecnico_no_puntua · la_actividad_sin_evidencia_arrastra_el_promedio_del_ambito · la_actividad_con_fallo_tecnico_no_arrastra_el_promedio · el_fallo_tecnico_gana_a_la_marca_de_sin_evidencia …

test_requirements_web_fijados.py

web/requirements_web.txt debe fijar versiones exactas (==), como requirements.txt.

dependencias_web_con_version_exacta

test_solo_informe.py

Tests del apartado «Solo Informe» (informe sin pipeline). Coste cero: no hay IA en ningún punto del flujo, y todo escribe en `tmp_path`.

cumplimiento_por_ambito · validar_datos · cargar_estandar_antiguo_se_rehace_desde_el_xlsx · imagen_rechaza_extension_y_id_invalido · word_estructura · word_om_no_va_en_rojo_ni_lleva_responsable · word_filtra_por_tier · periodo_validacion · ficha_del_documento · word_primera_vez_sin_logo · flujo_endpoints · generar_exige_tier_si_el_estandar_lo_tiene · generar_sin_estandar_o_con_errores · descargar_rechaza_rutas · casos_basicos · excepcion_documentacion_gobierno_dos_moderadas_es_aceptable · excepcion_tres_moderadas_es_regular_por_prudencia · excepcion_cuatro_moderadas_es_regular · dos_moderadas_sin_gobierno_no_activan_la_excepcion · ambito_del_estandar_tambien_casa_la_excepcion · conteos_y_supuestos · riesgo_tolerante_a_tildes_y_desconocido_falla · sin_observaciones_es_adecuado · frase_toma_el_riesgo_mas_grave · om_no_incumple · limpiar_om_quita_responsable_y_fecha · clave_ambito · formato_usuario_con_forward_fill · columnas_tier_y_filtro · estandar_real_del_repo · sin_columnas_falla · guardar_estandar_invalido_no_deja_residuos

test_solo_informe_ia.py

Tests de la asistencia con IA de «Solo Informe». Coste cero: el modelo se sustituye por `_llm_de_prueba`, que registra el prompt y devuelve una respuesta fija. Ninguno llama a Azure ni a Anthropic.

azure_por_defecto_y_lo_que_falta · override_de_proveedor_y_modelo · azure_usa_deployment_propio_del_apartado · clave_rechazada_da_mensaje_claro · pulir_enmascara_pii_y_marca_el_texto · respuesta_sin_markdown · pulir_vacio_falla · grafico_flujo · grafico_barras_con_cifras_del_texto · grafico_rechaza_cifras_inventadas · grafico_texto_corto_o_json_roto · manual_fragmentos_con_pagina_y_busqueda · busqueda_no_repite_bloques_de_plantilla · manual_formato_no_admitido · desde_manuales_cita_fuentes · desde_manuales_sin_manuales · endpoints_ia · endpoint_sin_credenciales_responde_503 · word_incluye_el_grafico

test_tabla_validacion.py

Veredicto formal por actividad + tabla intermedia + puente al chat.

cumple_desde_4 · parcial_es_frontera_de_3 · no_cumple_por_debajo_de_3 · no_evidenciado_por_sin_evidencia · fallo_tecnico_es_no_evidenciado_no_no_cumple · umbral_configurable · columnas_y_mapeo · no_aplicable_para_actividad_no_requerida · requerida_sin_evaluacion_es_no_evidenciado · no_cumple_va_al_chat · anota_resultado_en_la_evaluacion · anota_todas_las_evals_incluso_las_no_cruzadas · cumple_no_va_al_chat · solo_no_cumple_va_al_chat · fallo_tecnico_no_va_al_chat · placeholder_sin_evidencia_no_va_al_chat · dedup_actividad_repetida_en_el_estandar · fallback_json_sin_resultado

test_upload_zip.py

POST /api/inputs/upload-zip — extracción segura de ZIP (coste 0, carpeta temporal).

zip_normal_se_extrae · zip_slip_rechazado · demasiadas_entradas_413 · tamano_declarado_excesivo_413 · zip_bomba_upload_pequeno_expansion_grande

test_validador_entorno_hf.py

Guardia de la política «cero HuggingFace / cero ChromaDB» (validate_project.py).

paquete_instalado_pero_roto_se_reporta · paquete_ausente_no_se_reporta

test_versionado.py

Dos piezas, ambas deterministas y sin API: · Pre-filtro duro is_active en la recuperación léxica (no validar contra metodologías derogadas). · Detección config-driven de la ventana de contexto real del modelo, en vez del fallback fijo 15000.

chunk_inactivo_no_se_recupera · chunk_sin_metadatos_esta_activo · is_active_false_es_inactivo · aun_no_vigente_por_effective_date · derogado_por_expiration_date · fecha_ilegible_no_revienta · filtrar_activos_preserva_orden · marcar_version_es_aditivo · gpt4o_resuelve_su_ventana_real_no_15000 · gpt4_clasico_tiene_ventana_menor · override_explicito_del_operador_gana · deployment_desconocido_cae_a_conservador_declarado · max_ctx_efectivo_atajo

test_virf_plus.py

VIRF PLUS (GPT principal + Claude como segunda opinión) — coste cero, con dobles.

comparar_detecta_discrepancia_de_puntaje · comparar_acuerdo_ignora_redaccion · arbitro_recibe_ambas_opiniones_y_discrepancias · modo_discrepancia_ahorra_arbitro_y_vale_gpt · arbitro_claude_opcional · por_defecto_gpt_es_principal_y_arbitro · defaults_de_parametros_ponen_a_gpt_al_frente · sin_conclusion_con_un_solo_evaluador · bitacora_registra_cada_conciliacion · get_cliente_ia_solo_con_flag_explicito · openai_exige_api_key · openai_reutiliza_negociacion_de_azure

Parte 10 · Variables de entorno

Variables leídas por el código (en `.env`, que NO se sube al agente; la plantilla con comentarios está en `.env.example`). La descripción sale del comentario de `.env.example` cuando lo hay.

Variable	Se lee en	Descripción
ANTHROPIC_API_KEY	ia.py, Mx_00_CargaRutas.py	── Anthropic (SECUNDARIO — solo si IA_PROVEEDOR=anthropic) ──────────────────────── Obtén tu clave en: https://console.anthropic.com/
ANTHROPIC_MODELO	parametros.py, solo_informe_ia.py	
ANTHROPIC_WORKSPACE_ID	ia.py	slug "default"); sácalo del API de la org: GET /api/console/organizations/<org_uuid>/workspaces/default
AZURE_OPENAI_API_KEY	ia.py	No hay que tocar código después. El sistema detecta solo qué modelo sirve el deployment (GPT-4, GPT-4-turbo, GPT-5…) y ajusta el chunking a su contexto.
AZURE_OPENAI_API_VERSION	parametros.py	
AZURE_OPENAI_DEPLOYMENT	ia.py, solo_informe_ia.py	
AZURE_OPENAI_ENDPOINT	ia.py	
CACHE_DIR	rutas.py	En Databricks: /dbfs/FileStore/virf/cache
HTTPS_PROXY	red.py	navegador confía y Python no. El proxy en sí lo detecta solo — basta con las variables estándar:
HTTP_PROXY	red.py	
IA_PROVEEDOR	parametros.py, Mx_03_LlamarIA.py, config.py, preview_prompt.py	anthropic | azure_openai
MODO	parametros.py	debug | produccion
NO_PROXY	red.py	
OPENAI_API_KEY	ia.py	── OpenAI directo (GPT con una API key; alternativa a Azure OpenAI) ──────────── Obtén tu clave en: https://platform.openai.com/api-keys
OPENAI_MODELO	parametros.py	
RAG_INDICE_DIR	parametros.py	En Databricks: /dbfs/FileStore/virf/rag_lexico

Variable	Se lee en	Descripción
SOLO_INFORME_DEPLOYMENT	solo_informe_ia.py	deployment de Azure más barato (si existe)
SOLO_INFORME_IA_PROVEEDOR	solo_informe_ia.py	azure_openai | anthropic | openai
SOLO_INFORME_MODELO	solo_informe_ia.py	modelo para anthropic / openai
VIRF_AGENTE_VERSION	parametros.py	viaja al informe: debe reflejar el evaluador real
VIRF_CA_BUNDLE	parametros.py	La CA hay que declararla. Pídele a IT «el certificado raíz de la CA que usa el proxy para la inspección TLS, en formato .pem»:
VIRF_CHAT_INCLUYE_NO_EVIDENCIADO	motor_ia.py, chat_service.py	
VIRF_CHAT_INCLUYE_PARCIAL	motor_ia.py, chat_service.py	
VIRF_CHUNK_SIZE	preview_prompt.py	tamaño INICIAL; el modo adaptativo lo sube
VIRF_DIR_LOGO	parametros.py	imagen que encuentre (.png .jpg .webp ...). Si no hay ninguno, la portada sale sin logo.
VIRF_DOBLE_FIRMA_TIER	parametros.py	El segundo firmante debe ser una persona DISTINTA de la primera: la comparación normaliza mayúsculas, espacios y acentos.
VIRF_FUENTE	parametros.py	Flexo»—, decláralo aquí: debe coincidir con el nombre de familia que muestra Word en su desplegable de fuentes.
VIRF_FUENTE_MONO	parametros.py	Monoespaciada de los bloques de código. Debe ser de ancho fijo.
VIRF_FUENTE_RESPALDO	parametros.py	Sustituta declarada cuando la corporativa no está instalada.
VIRF_IA_TEMPERATURE	preview_prompt.py	reproducibilidad que exige un informe firmado. Súbela solo para explorar, nunca para emitir.
VIRF_MAX_CHUNKS	preview_prompt.py	coste medido: 40→$1,14 vs 60→$1,45 (misma
VIRF_MAX_CTX	preview_prompt.py	sube VIRF_PRESUPUESTO_DOC por separado y deja MAX_CTX más bajo: ganas fidelidad en ese manual sin agrandar todos los prompts.
VIRF_MOCK_CHAT	analisis_codigo.py, chat_service.py	
VIRF_PLUS	agente.py, ia.py	ANTHROPIC_API_KEY a lo que ya usa el agente GPT. ~3 llamadas por evaluación. Si uno de los dos falla, la evaluación falla: no hay «consenso» con un solo evaluad

Variable	Se lee en	Descripción
VIRF_PLUS_ARBITRAJE	parametros.py	siempre | discrepancia (ahorra el árbitro si coinciden)
VIRF_PLUS_ARBITRO	parametros.py	gpt | anthropic — quién redacta la conclusión
VIRF_PLUS_GPT	parametros.py	azure_openai | openai (por defecto, el de IA_PROVEEDOR)
VIRF_PLUS_LOG	virf_plus.py	
VIRF_PUNTAJE_CUMPLE	consolidacion.py, feedback.py	
VIRF_RESP_MODELAMIENTO	parametros.py	── Cuadro de observaciones (Excels 00–03 de automatización) ──────────── Responsable según dónde se levanta la observación, y plazo (días) por riesgo.
VIRF_RESP_SEGUIMIENTO	parametros.py	
VIRF_ROL_AGENTE_OVERRIDE	Mx_03_LlamarIA.py	
VIRF_VALIDADOR	parametros.py	Alternativas: los flags --validador/--cargo, o la pregunta interactiva. Definirlas aquí equivale a firmar: hazlo solo en tu entorno personal.
VIRF_VALIDADOR_2	parametros.py	los flags --validador-2/--cargo-2, o la pregunta interactiva. Definirlas equivale a firmar: hazlo solo en tu entorno personal.
VIRF_VALIDADOR_2_CARGO	parametros.py	
VIRF_VALIDADOR_CARGO	parametros.py	

Parte 11 · El grafo del código

Cómo usarlo está en la skill S11 (Parte 6). Aquí, la referencia generada del grafo actual: los nodos más conectados y todas las comunidades con sus archivos principales.

Nodos centrales (cambiarlos afecta a mucho)

Nodo	Archivo	Conexiones
app.js	web/static/js/app.js	179
main()	agente.py	87
pedirJSON()	web/static/js/app.js	63
escapeHtml()	web/static/js/app.js	58
ChunkerService	modulos/chunker.py	44
IndexadorLexico	src/rag/indexador_lexico.py	43
BuscadorLexico	src/rag/buscador_lexico.py	39
generar_word_informe()	modulos/generador_word.py	33
_indexador()	tests/test_rag_lexico.py	33
DocumentCache	modulos/document_cache.py	30
evaluar_actividad()	modulos/motor_ia.py	29
TestJuezCodigoAnclado	tests/test_pendientes_auditoria.py	29
cliente_azure()	tests/test_azure_mocks.py	27
columns()	tests/test_regresion.py	27
RevisorCodigo	modulos/revision_codigo.py	26
get_cliente_ia()	config/ia.py	26
generar()	modulos/plantilla_bcp.py	25
_buscador()	tests/test_rag_lexico.py	25
contexto_sesion()	web/services/contexto_sesion.py	25
tokenizar()	src/rag/indexador_lexico.py	24
a_plantilla()	web/services/informe_web.py	24
SkillTipada	skills/base_tipada.py	24

Nodo	Archivo	Conexiones
enmascarar()	modulos/pii.py	23
sellar_y_escribir()	modulos/custodia.py	22
ExplicadorModelo	modulos/explicador_modelo.py	22

Comunidades del grafo

Cada comunidad agrupa funciones y clases muy conectadas entre sí. Las marcadas como archivadas corresponden a código retirado (`archivo/`, `codigo_muerto_*`).

#	Comunidad	Nodos	Archivos principales
5	Archivado v28 · Memoria y feedback (archivado)	153	
184	Tipos · typing (compartido)	145	modulos/registro_emisiones.py, modulos/validador_humano.py, modulos/huella_informe.py
2	Web · Modelos de petición de los routers	95	web/routers/chat.py, web/routers/informe.py, web/routers/validacion.py
10	RAG · Buscador léxico	66	src/rag/buscador_lexico.py, src/rag/indexador_lexico.py, tests/test_rag_lexico.py
78	Solo Informe · Asistencia con IA	64	web/services/solo_informe_ia.py, tests/test_solo_informe_ia.py, web/routers/solo_informe.py
6	Guía · Generador del PDF del agente corporativo	62	scripts/generar_guia_agente.py, tests/test_guia_agente.py
34	Automatizaciones · Cuadro de observaciones	61	modulos/automatizaciones.py, web/routers/automatizaciones.py, config/virf_plus.py
20	Informe Word · Primitivas del entregable	60	modulos/generador_word.py, tests/test_regresion.py
7	Tests · RAG léxico (recuperación)	58	tests/test_rag_lexico.py
23	RAG · Indexador léxico	57	src/rag/indexador_lexico.py, tests/test_azure_mocks.py, tests/test_rag_lexico.py
8	Motor IA v2 · Evaluación con datos	57	modulos/control_evaluacion_data.py, motor/Mx_02_EstructuraPrompt.py, motor/Mx_03_LlamarIA.py
4	Archivado v28 · Orquestador de flujos (archivado)	55	
99	Datos · Motores de calidad y notebook	55	modulos/motores_datos.py, web/routers/controles.py, tests/test_regresion.py
135	RAG · Enrutado por dominio	53	tests/test_pendientes_auditoria.py, src/rag/__init__.py, tests/test_regresion.py
28	Motor · Lectura de evidencias (Mx_01)	49	motor/Mx_01_LecturaEvidencias.py
18	Informes · Plantilla común BCP	48	modulos/plantilla_bcp.py, web/services/informe_web.py
106	Informe web · Generador con capturas	47	web/services/informe_web.py, modulos/capturas.py, modulos/plantilla_bcp.py
12	Motor IA · Evaluar actividades y ámbitos	47	modulos/motor_ia.py, modulos/feedback_mejorado.py, tests/test_regresion.py

#	Comunidad	Nodos	Archivos principales
163	Tests · Entorno y configuración central	47	tests/test_regresion.py, tests/test_arbol_entornos.py, tests/_arbol.py
22	Revisión de código · Juez anclado y hallazgos	47	tests/test_pendientes_auditoria.py, modulos/revision_codigo.py
13	Web · Servicio de estándares (Excel)	46	web/services/estandares_service.py, tests/test_estandares_sesion.py, web/routers/estandares.py
19	Controles · Excel a notebook ejecutable	46	modulos/excel_a_notebook.py, tests/test_regresion.py
11	Agente · Punto de entrada y cliente IA	46	agente.py, modulos/validador_humano.py, modulos/instrucciones_sesion.py
26	Web · Gestión de modelos (store y rutas)	44	web/services/modelos_store.py, web/routers/modelos.py
17	Informe Word · Estilos y formato	43	modulos/word_estilo.py, modulos/generador_word.py
24	Archivado · Generador Word antiguo (Mx_05) (archivado)	43	
27	Archivado v28 · Herramientas SQL y pandas (archivado)	43	
44	Configuración · Clientes IA y proveedor	42	config/ia.py, config/__init__.py, config/red.py
41	Tests · Simulación de Azure (pipeline)	41	tests/test_azure_mocks.py
213	Archivado v28 · Evaluadores (archivado)	40	
29	Seguridad · Cortacircuitos (fusibles)	40	modulos/fusibles.py, tests/test_circuit_breakers.py
52	Solo Informe · Estándar, borrador e imágenes	40	web/services/solo_informe_service.py, tests/test_solo_informe.py, web/services/estandares_service.py
68	Frontend · Arranque, login y navegación	40	web/static/js/app.js
56	Skills · Catálogo y clase base	39	modulos/skills/base.py, modulos/skills/catalogo.py, modulos/skills/__init__.py
16	Archivado v28 · Vector store Chroma (archivado)	38	
114	Estándares · Lectura del pipeline	37	tests/test_regresion.py, motor/Mx_01_LecturaEvidencias.py, tests/test_pendientes_auditoria.py
121	Web · Descarga de resultados	36	web/routers/resultados.py, web/services/pipeline_runner.py, web/routers/validacion.py
31	Web · Rutas RAG y dominios personalizados	36	web/routers/rag.py, tests/test_rag_desactualizado.py, web/services/dominios_custom_store.py
40	RAG · Vigencia y ventana de contexto	36	tests/test_versionado.py, modulos/versionado.py, modulos/ventana_contexto.py

#	Comunidad	Nodos	Archivos principales
71	Informe Markdown	36	modulos/generador_markdown.py
3	Datos · Pipeline de calidad de datos	35	tests/test_pendientes_auditoria.py, scripts/pipeline_calidad_datos.py
45	VIRF PLUS · Consenso entre dos modelos	35	tests/test_virf_plus.py, config/virf_plus.py
47	Web · Configuración y archivo .env	35	web/routers/config.py, web/services/env_service.py
57	Firma humana · Doble firma	35	tests/test_pendientes_auditoria.py, modulos/validador_humano.py
43	Archivado v28 · Agente VIRF antiguo (archivado)	34	
55	Skills · Calidad de datos	34	skills/calidad_datos.py, tests/skills/test_calidad_datos.py
49	Web · Subida de archivos y ZIP	34	web/routers/upload.py, web/services/contexto_sesion.py
74	Skills tipadas · Documentación y gobierno	33	skills/base_tipada.py, skills/documentacion.py, skills/gobierno.py
32	Informes · Reglas de calificación y cumplimiento	33	modulos/reglas_informe.py, modulos/plantilla_bcp.py, modulos/word_estilo.py
83	Motores de la entidad · Leer resultados	33	modulos/motores_entidad.py, web/services/dominios_custom_store.py, src/rag/__init__.py
105	Datos · Resultados de la entidad	32	tests/test_pendientes_auditoria.py, modulos/motores_entidad.py
38	Caché de documentos	32	modulos/document_cache.py, tests/test_regresion.py
53	Revisión de código · Hallazgos	32	modulos/revision_codigo.py, tests/test_pendientes_auditoria.py, tests/test_regresion.py
15	RAG · Recuperador y resolución de dominio	31	src/rag/base_retriever.py, src/rag/ALM/retriever.py, src/rag/LaR/retriever.py
51	Web · Chat de validación	31	web/services/chat_service.py
82	Chunker · Troceado de documentos	31	tests/test_regresion.py, modulos/chunker.py, src/rag/indexador_lexico.py
93	Tabla de validación y veredictos	31	tests/test_tabla_validacion.py, modulos/motor_ia.py
286	Tests · Regresión del núcleo	30	tests/test_regresion.py, config/parametros.py
36	Web · Catálogo de skills	30	web/services/skills_store.py, web/routers/skills.py
42	Archivado v28 · Compresión de contexto (archivado)	30	
61	Archivado · Config monolítica antigua (archivado)	30	

#	Comunidad	Nodos	Archivos principales
62	Salvaguardas · Entailment (LLM como juez)	30	tests/test_entailment.py, modulos/entailment.py, tests/test_regresion.py
63	Tests · RAG como pericia (no evidencia)	30	tests/test_pendientes_auditoria.py, modulos/motor_ia.py
64	Verificador VIRF · Paquete de verificación	30	verificador_virf.py
65	Tests · Estructura de la configuración	30	tests/test_estructura_config.py
67	Salvaguardas · Enmascarado de PII	30	modulos/pii.py, tests/test_pii.py
0	Archivado v28 · Skills con historial (archivado)	29	
37	Utilidades · Consola Unicode y scripts sueltos	29	tests/test_regresion.py, modulos/consola.py
110	Fixtures · IFRS9 y VaR de demostración	29	modelos/VaR_FixtureDrift/inputs/implementacion/calcula_var.py, tests/fixtures/modelos_sinteticos/IFRS9_Provisiones/inputs/implementacion/implementacion_IFRS9_Provisiones.py, tests/fixtures/var_injection/modelo_var_demo.py
21	Archivado · Parches v13 (código muerto) (archivado)	28	
33	Solo Informe · Reglas de conclusión	28	tests/test_solo_informe.py, web/services/solo_informe_service.py, web/routers/solo_informe.py
50	Custodia · Registro de emisiones	28	tests/test_pendientes_auditoria.py, modulos/registro_emisiones.py
70	Aislamiento entre clientes (tenant)	28	modulos/aislamiento.py, tests/test_aislamiento.py
72	Motor IA · Scores y semáforo	28	tests/test_regresion.py, modulos/motor_ia.py
9	RAG · Indexadores por dominio y reconstrucción	28	scripts/rebuild_rag_databricks.py, scripts/generar_corpus_rag.py, src/rag/base_indexer.py
77	Tests · Notebook de controles	27	tests/test_notebook_controles.py, modulos/notebook_controles.py
73	Archivado · Generador Excel antiguo (archivado)	27	
80	Web · Pantalla Solo Informe (frontend)	27	web/static/js/app.js
69	Solo Informe · Router y endpoints	26	web/routers/solo_informe.py
75	Feedback · Calidad del proceso y autoría	26	tests/test_pendientes_auditoria.py, modulos/feedback.py, modulos/override_humano.py
81	Motor · Controles sobre datos (Mx_04)	26	motor/Mx_04_MotorControlesData.py
96	Frontend · Dominios RAG y documentación	26	web/static/js/app.js

#	Comunidad	Nodos	Archivos principales
148	Tests · Skills tipadas	25	tests/skills/test_skills_tipadas.py, skills/base_tipada.py, skills/__init__.py
58	Web · Login y sesiones de validadores	25	web/services/sesion_store.py, web/routers/auth.py, web/verificar_conexiones.py
84	Skills · Coherencia manual vs implementación	25	tests/test_pendientes_auditoria.py, modulos/skills/catalogo.py
89	Archivado v28 · Detector de fórmulas y metodología (archivado)	25	
92	Feedback · Aprendizaje y persistencia	25	modulos/feedback.py, modulos/document_cache.py, tests/test_regresion.py
101	Tests · Catálogo y detección de modelos	24	config/ia.py, tests/test_estructura_config.py, config/modelos.py
14	Archivado v28 · Traza de decisiones (archivado)	24	
88	Tests · Controles de calidad de datos	24	tests/test_pendientes_auditoria.py
107	Evidencia · Capturas (pantallazos)	24	modulos/capturas.py
102	Análisis estático del código	24	modulos/analisis_estatico.py
85	Abstención y calibración beta	24	tests/test_abstencion.py, tests/test_beta_calibracion.py, modulos/abstencion.py
87	Citas · Validación determinista	24	modulos/validador_citas.py, tests/test_citas_deterministas.py
134	Tests · Conexión Azure y parámetros	23	tests/test_azure_mocks.py
151	Tests · Auditoría integral 2026-08-07	23	tests/test_pendientes_auditoria.py
35	Prevuelo · Chequeo previo de evidencia	23	tests/test_regresion.py, modulos/prevuelo.py
95	Tests · Análisis estático	23	tests/test_pendientes_auditoria.py
97	Frontend · Guía interactiva	23	web/static/js/guia.js
104	Tests · SQL y R como código del modelo	22	tests/test_regresion.py, modulos/revision_codigo.py
112	Tests · Revisión de código	22	tests/test_regresion.py, modulos/revision_codigo.py
118	Tests · Informe Word sin puntajes falsos	22	tests/test_regresion.py, modulos/generador_word.py
170	Tests · Contrato y estadísticas	22	tests/test_regresion.py, modulos/motor_ia.py
191	Configuración · Parámetros	22	config/parametros.py, modulos/revision_codigo.py, config/modelos.py
111	Motor IA v2 · Cálculo de score	21	modulos/control_evaluacion_data.py

#	Comunidad	Nodos	Archivos principales
117	Custodia · Huellas de evidencia	21	tests/test_regresion.py, modulos/huella_informe.py, modulos/custodia.py
125	Skills · Citas inventadas (challenger, supuestos)	21	modulos/skills/catalogo.py, tests/test_pendientes_auditoria.py, modulos/skills/base.py
144	Informe Word · Generar entregable del pipeline	21	modulos/generador_word.py, tests/test_regresion.py, tests/test_pendientes_auditoria.py
39	Calidad · Comparación entre corridas	21	tests/test_regresion.py, modulos/comparador.py
79	Tests · Presentación del Word	21	tests/test_pendientes_auditoria.py, modulos/generador_word.py
119	Tests · Capturas y Solo Informe Word	21	tests/test_capturas.py, web/services/solo_informe_word.py, modulos/plantilla_bcp.py
76	Esquemas de datos (contratos)	21	tests/test_esquemas.py, modulos/esquemas.py
109	Archivado · Lector de inputs antiguo (archivado)	21	
115	Tests · Resultados por celda del notebook	20	tests/test_pendientes_auditoria.py
120	Informe · Contrato ValidationReport	20	modulos/reporte_docx.py, modulos/esquemas.py
133	Informes · API de la plantilla	20	modulos/plantilla_bcp.py, web/services/informe_web.py
138	Tests · Automatizaciones	20	tests/test_automatizaciones.py
94	Skills · Cola de acciones pendientes	20	skills/cola_pendiente.py, tests/skills/test_cola_pendiente.py
98	Archivado v28 · Challenger con RAG (archivado)	20	
48	Scripts · Preguntas de oro del RAG	20	scripts/preguntas_oro_rag.py, modulos/motores_entidad.py, tests/test_pendientes_auditoria.py
139	Tests · Solo Informe (Word y endpoints)	20	tests/test_solo_informe.py, web/services/solo_informe_word.py
155	Frontend · Navegación y vistas	20	web/static/js/app.js
159	Tests · Borrado del caché	19	tests/test_regresion.py, modulos/document_cache.py
160	Custodia · Cadena de hashes	19	modulos/custodia.py, tests/test_regresion.py
30	Web · Prompts editables	19	tests/test_prompts_editables.py, web/services/prompt_service.py, web/services/analisis_codigo.py
141	Feedback · Estadísticas	19	modulos/generador_estadistica_feedback.py, modulos/feedback.py, agente.py
25	Web · Verificar conexiones y cuestionamientos	19	web/services/cuestionamientos_sesion.py, web/routers/cuestionamientos_sesion.py

#	Comunidad	Nodos	Archivos principales
199	Frontend · Cuestionamientos y skills	19	web/static/js/app.js
127	Tests · PDF real y chunking	18	tests/test_regresion.py
128	Tests · Parser de estándares	18	tests/test_regresion.py
129	Tests · Neutralidad de proveedor IA	18	tests/test_regresion.py
165	Trazabilidad · Huella de prompts	18	tests/test_regresion.py, modulos/huella_prompts.py
168	Tests · Umbral recall@3	18	tests/test_pendientes_auditoria.py
123	Archivado · Comparativa doc vs código (archivado)	18	
1	Controles · Notebook desde documento	18	modulos/controles_a_notebook.py
124	Chunker estructural	18	modulos/chunker_estructural.py, tests/test_chunker_estructural.py
59	Frontend · Pantalla de resultados	18	web/static/js/app.js
91	Validación del proyecto (validate_project)	18	validate_project.py
130	Agente · Acotar contexto de skills	17	tests/test_pendientes_auditoria.py, agente.py
132	Firma humana · Identidad del validador	17	tests/test_regresion.py, modulos/validador_humano.py
136	Tests · Explicador del modelo	17	tests/test_regresion.py
154	Tests · Cuestionamientos extensibles	17	tests/test_regresion.py
86	Explicador del modelo	17	modulos/explicador_modelo.py, tests/test_regresion.py
131	Salvaguardas · Inyección de prompts	17	tests/test_inyeccion.py, modulos/inyeccion.py
175	RAG · Fuentes por dominio	17	src/rag/ALM/indexer.py, src/rag/indexador_lexico.py, src/rag/LaR/indexer.py
116	Web · Rutas de cuestionamientos	16	web/routers/cuestionamientos.py
122	Archivado v28 · Challenger sin RAG (archivado)	16	
126	Tests · Tope por tolerancia cero	16	tests/test_pendientes_auditoria.py
140	Seguridad · Nombres de archivo seguros	16	tests/test_regresion.py, config/rutas.py
179	Custodia · Sellado del informe	16	tests/test_regresion.py, modulos/custodia.py, tests/test_pendientes_auditoria.py
182	Web · Entendimiento del modelo	16	web/services/analisis_codigo.py, web/routers/entendimiento.py
183	Web · Servicio de prompts	16	web/services/prompt_service.py

#	Comunidad	Nodos	Archivos principales
142	Tests · Skills en el entregable	15	tests/test_pendientes_auditoria.py
145	Datos · Perfilado de datasets	15	modulos/perfilador_datasets.py, tests/test_regresion.py
147	Multimodelo · Raíz de evidencia	15	tests/test_regresion.py, motor/Mx_00_CargaRutas.py
149	Tests · Reintento ante cuota (429)	15	tests/test_azure_mocks.py
150	Tests · Red corporativa y TLS	15	tests/test_azure_mocks.py
152	Tests · Caché de resúmenes	15	tests/test_regresion.py
153	Tests · Explicador agnóstico al modelo	15	tests/test_regresion.py
194	Calidad · Comparador de corridas	15	modulos/comparador.py
209	Tests · RAG no indexa la evidencia	15	tests/test_regresion.py, src/rag/indexador_lexico.py, src/rag/base_indexer.py
243	Pipeline · Orquestación por control	15	tests/test_regresion.py, modulos/motor_ia_v2.py
193	Código muerto · Evaluación paralela	15	motor/Mx_01_LecturaEvidencias.py, modulos/motor_ia.py
146	Tests · DOCX de regresión	15	tests/test_docx_regresion.py, modulos/reporte_docx.py
143	Exportador Excel	15	modulos/exportador_excel.py, tests/test_exportador_excel.py
203	Validador · Correcciones humanas (override)	15	modulos/override_humano.py, modulos/feedback.py
162	Tests · Preguntas de oro RAG	14	tests/test_pendientes_auditoria.py
164	Tests · Score de control ponderado	14	tests/test_regresion.py
214	Tests · Anexo de correcciones	14	tests/test_regresion.py, modulos/override_humano.py, modulos/feedback.py
158	Controles por dominio · Funciones de control	14	inputs/repositorio_controles/var/christoffersen.py, inputs/repositorio_controles/alm/delta_eve.py, inputs/repositorio_controles/alm/sot_eve_tier1.py
137	Web · Rutas del chat	14	web/routers/chat.py
169	Controles · Repositorio por dominio	14	modulos/repositorio_controles.py, modulos/notebook_controles.py
113	Skills · Síntesis y contrato	13	tests/test_pendientes_auditoria.py
166	Archivado v28 · Lector de código (archivado)	13	
167	Tests · Filtro de contenido Azure	13	tests/test_azure_mocks.py, config/ia.py
171	Tests · Acantilado de coste	13	tests/test_regresion.py

#	Comunidad	Nodos	Archivos principales
172	Tests · Umbral antisesgo	13	tests/test_regresion.py
173	Tests · Lectura íntegra de código e ipynb	13	tests/test_regresion.py
181	Web · Presupuesto de prompts	13	tests/test_prompts_editables.py, web/services/presupuesto_service.py, web/services/pipeline_runner.py
192	Config · Perfil del modelo (contexto)	13	config/modelos.py
218	Archivado v28 · Embeddings (archivado)	13	
220	Archivado v28 · Motor de razonamiento (archivado)	13	
230	Código muerto · Buscador de evidencias (archivado)	13	
60	Archivado v28 · Troceo de texto (archivado)	13	
66	Web · Edición de prompts	13	web/routers/prompts.py
157	Tests · Subida ZIP (límites)	13	tests/test_upload_zip.py, web/routers/__init__.py
100	Scripts · Reconstruir estándar y verificar corpus	13	scripts/reconstruir_estandar_desde_informes.py, web/verificar_conexiones.py
174	Frontend · Chat	13	web/static/js/app.js
180	Tests · Políticas del procedimiento 2026	12	tests/test_regresion.py, modulos/motor_ia.py
185	Tests · Reintento de contrato	12	tests/test_pendientes_auditoria.py
186	Tests · Sin evidencia no es fallo	12	tests/test_regresion.py
187	Tests · Caché de resúmenes con bloqueo	12	tests/test_regresion.py
188	Tests · Lectura fallida declarada	12	tests/test_regresion.py
189	Tests · Custodia bajo concurrencia	12	tests/test_regresion.py
190	Tests · Ninguna llamada externa	12	tests/test_regresion.py
200	Tests · Respuestas de Azure OpenAI	12	tests/test_azure_mocks.py, modulos/motor_ia.py
234	Archivado v28 · Gestor de flujos (archivado)	12	
236	Prevuelo · Hallazgos e informe	12	modulos/prevuelo.py
237	Archivado v28 · Skill de reconstrucción (archivado)	12	
246	Datos · Perfilado de carpetas	12	modulos/perfilador_datasets.py, tests/test_regresion.py

#	Comunidad	Nodos	Archivos principales
241	Tests · Endpoint del notebook	12	tests/test_endpoint_notebook_controles.py
247	RAG · Indexador base	12	src/rag/base_indexer.py
239	Web · Estructura del código del modelo	12	web/services/analisis_codigo.py
177	Archivado v28 · Demo challenger (archivado)	12	
178	Consolidación de decisiones	12	modulos/consolidacion.py
196	Tests · Historial de feedback	11	tests/test_regresion.py
197	Tests · Índice RAG sin contaminación	11	tests/test_regresion.py
198	Tests · Contexto se llena sin abandonar	11	tests/test_regresion.py
250	Archivado v28 · Cadena de razonamiento (archivado)	11	
255	Archivado v28 · Plantilla de skills (archivado)	11	
259	Tests · Plantilla común de informes	11	tests/test_plantilla_bcp.py, web/services/chat_service.py
161	Tests · Consolidación	10	tests/test_consolidacion.py
201	Archivado v28 · Skill consolidador (archivado)	10	
202	Archivado v28 · Skill análisis de fallos (archivado)	10	
207	Tests · Fallo de credencial Azure	10	tests/test_azure_mocks.py
210	Tests · OCR retirado	10	tests/test_regresion.py
260	Archivado v28 · Skill de auditoría documental (archivado)	10	
262	Archivado v28 · Skill de papers (archivado)	10	
287	Configuración · Cliente OpenAI	10	config/ia.py, tests/test_virf_plus.py
308	Tests · Feedback	10	tests/test_regresion.py
309	Web · Verificador de conexiones	10	web/verificar_conexiones.py
206	Tests · Aprendizaje desde el chat	10	tests/test_aprendizaje_chat.py
306	Validación · Política sin HuggingFace	10	tests/test_validador_entorno_hf.py, validate_project.py, modulos/prevuelo.py
231	Scripts · Verificación del corpus RAG	10	scripts/verificar_corpus_rag.py

#	Comunidad	Nodos	Archivos principales
290	Scripts · Preguntas de oro de fórmulas	10	scripts/preguntas_oro_formula.py
291	Controles · Construcción del notebook	10	modulos/controles_a_notebook.py, modulos/notebook_controles.py
204	Cuestionamientos · Catálogo activo	10	motor/Mx_04_MotorControlesData.py, modulos/cuestionamientos_ext.py
205	Acta de cierre (Excel)	10	tests/test_acta_cierre.py, modulos/exportador_excel.py
208	Chat · Adjuntar evidencia	10	tests/test_chat_adjuntar.py, web/services/chat_service.py, validate_project.py
217	Motor · Llamada IA con contrato JSON	10	motor/Mx_03_LlamarIA.py
307	Ejemplo · Implementación PD Retail	10	tests/fixtures/modelos_sinteticos/PD_Retail/inputs/implementacion/implementacion_PD_Retail.py
195	Skills · Regulatoria con RAG	9	tests/test_pendientes_auditoria.py, modulos/skills/catalogo.py
216	Tests · Un solo parser JSON	9	tests/test_regresion.py
232	Estándares · Flag --estandares por CLI	9	tests/test_estandares_sesion.py, motor/Mx_00_CargaRutas.py
311	Configuración · Normalización de tier	9	tests/test_regresion.py, config/dominio.py
313	Validador · Cumplimiento regulatorio	9	modulos/validador_humano.py, modulos/motor_ia.py, validate_project.py
314	Tests · Fallos de red	9	tests/test_regresion.py
90	Esquemas · Validación de afirmaciones	9	modulos/esquemas.py
215	Fixtures · Modelo sintético Fraude/AML	9	tests/fixtures/modelos_sinteticos/FraudeAML/inputs/implementacion/implementacion_FraudeAML.py
219	Frontend · Estándares	9	web/static/js/app.js
222	Tests · Sin documentos no hay puntaje	8	tests/test_regresion.py, modulos/generador_word.py
225	Tests · Cobertura documental ponderada	8	tests/test_regresion.py
226	Tests · Respuesta vacía del modelo	8	tests/test_regresion.py
227	Tests · Columna de control opcional	8	tests/test_regresion.py
228	Tests · Filtrado de controles	8	tests/test_regresion.py
229	Tests · El resumen final cuadra	8	tests/test_regresion.py
315	Archivado v28 · Skill de metodología (archivado)	8	
316	Web · Cuestionamientos (modelo)	8	web/routers/cuestionamientos.py

#	Comunidad	Nodos	Archivos principales
224	Tests · Textos largos sin truncar	8	tests/test_regresion.py
288	Tests · Modelo simulado de Solo Informe	8	tests/test_solo_informe_ia.py, tests/test_solo_informe.py, web/routers/solo_informe.py
103	Archivado v28 · Priorización de contexto (archivado)	8	
233	Frontend · Lanzar validación	8	web/static/js/app.js
211	Archivado v28 · Skill detección de fórmulas (archivado)	7	
212	Archivado v28 · Skill metodología (archivado)	7	
223	Tests · Pipeline completo con Azure	7	tests/test_azure_mocks.py
235	Tests · Cierre del caché	7	tests/test_regresion.py, modulos/document_cache.py
240	Skills · Skill de síntesis	7	modulos/skills/catalogo.py, modulos/skills/base.py
242	Tests · Nombres de ámbito	7	tests/test_regresion.py
244	Tests · Pipeline de controles	7	tests/test_regresion.py
245	Tests · Ningún recorte silencioso	7	tests/test_regresion.py
317	Herramienta · Excel de estándar de prueba	7	create_estandares_excel.py
221	Análisis estático · Contraste doc vs código	7	modulos/analisis_estatico.py
249	Web · Router de prevuelo	7	web/routers/dominio_prevuelo.py
318	Explicador · Inventario del modelo	7	tests/test_azure_mocks.py, modulos/explicador_modelo.py, tests/test_regresion.py
252	Tests · Dos métricas de cobertura	6	tests/test_regresion.py
253	Tests · Ruido de pdfminer	6	tests/test_regresion.py
320	Tests · Rol del agente editable	6	tests/test_prompts_editables.py
46	Tests · Corpus RAG como resumen secundario	6	tests/test_regresion.py
156	Informe Word · Logo	6	modulos/word_estilo.py
319	Lectura · Caché de documentos	6	motor/Mx_01_LecturaEvidencias.py, modulos/document_cache.py, tests/test_regresion.py
248	Chunker · Caché de resúmenes	6	modulos/chunker.py
251	Tests · Invariante de asyncio	6	tests/test_invariante_asyncio.py

#	Comunidad	Nodos	Archivos principales
54	Configuración · Reintentos de red	6	config/red.py
256	Tests · Un solo acceso al historial	5	tests/test_regresion.py
238	Tests · Fixtures compartidas (conftest)	5	tests/conftest.py
176	Agente · Punto de entrada antiguo	5	modulos/motor_ia.py
254	Explicador · Verificar truncado	5	tests/test_regresion.py, modulos/explicador_modelo.py
323	Tests · Tabla final	4	tests/test_consolidacion.py
326	Tests · Hash por ambas ramas	4	tests/test_regresion.py
325	Tests · Contexto de sesión de prueba	4	tests/test_prompts_editables.py
257	Control VaR · Kupiec	4	inputs/repositorio_controles/var/kupiec.py
258	Control VaR · Semáforo de Basilea	4	inputs/repositorio_controles/var/semaforo_basilea.py
261	Tests · Fallo de API conserva lo hecho	4	tests/test_regresion.py
305	Código muerto · Prompt de dimensión (archivado)	4	
321	Informes · Citas y resaltado en párrafo	4	modulos/plantilla_bcp.py
322	RAG · Tesauro	4	src/rag/tesauro.py
324	Tests · Tabla final en el Word	4	tests/test_informe_tabla_final.py
328	Tests · Caché sin heurística	3	tests/test_regresion.py
327	RAG · Carpeta de índices	3	src/rag/config_rag.py
108	Código muerto · Versión del estándar (archivado)	3	
263	Control ALM · Backtesting de prepago	3	inputs/repositorio_controles/alm/backtesting_prepago.py
264	Control ALM · Beta de depósitos	3	inputs/repositorio_controles/alm/beta_deposito.py
265	Control ALM · Brecha de duración	3	inputs/repositorio_controles/alm/brecha_duracion.py
266	Control ALM · CPR de prepago	3	inputs/repositorio_controles/alm/cpr_prepago.py
267	Control ALM · Decaimiento NMD	3	inputs/repositorio_controles/alm/decaimiento_nmd.py
268	Control ALM · Sensibilidad NII	3	inputs/repositorio_controles/alm/delta_nii.py
269	Control ALM · Escenarios IRRBB	3	inputs/repositorio_controles/alm/escenarios_irrbb_completos.py
270	Control ALM · Gap de reprecios	3	inputs/repositorio_controles/alm/gap_reprecios.py

#	Comunidad	Nodos	Archivos principales
271	Control ALM · Vida media NMD	3	inputs/repositorio_controles/alm/vida_media_nmd.py
272	Control LaR · Brecha de liquidez	3	inputs/repositorio_controles/lar/brecha_liquidez_acumulada.py
273	Control LaR · Concentración de fondeo	3	inputs/repositorio_controles/lar/concentracion_fondeo.py
274	Control LaR · Horizonte de supervivencia	3	inputs/repositorio_controles/lar/horizonte_supervivencia.py
275	Control LaR · Composición HQLA	3	inputs/repositorio_controles/lar/hqla_composicion.py
276	Control LaR · LCR	3	inputs/repositorio_controles/lar/lcr.py
277	Control LaR · NSFR	3	inputs/repositorio_controles/lar/nsfr.py
278	Control transversal · Columnas constantes	3	inputs/repositorio_controles/shared/columnas_constantes.py
279	Control transversal · Completitud	3	inputs/repositorio_controles/shared/completitud_datos.py
280	Control transversal · Duplicados	3	inputs/repositorio_controles/shared/duplicados_registros.py
281	Control transversal · Frescura	3	inputs/repositorio_controles/shared/frescura_datos.py
282	Control transversal · Volumen mínimo	3	inputs/repositorio_controles/shared/volumen_minimo.py
283	Control VaR · Excedencias	3	inputs/repositorio_controles/var/backtesting_excedencias.py
284	Control VaR · Severidad de excepciones	3	inputs/repositorio_controles/var/severidad_excepciones.py
285	Control VaR · VaR negativos	3	inputs/repositorio_controles/var/var_negativos.py
310	Tests · Dependencias web fijadas	3	tests/test_requirements_web_fijados.py
289	Tests · Notebook sin ejecutar	2	tests/test_pendientes_auditoria.py
329	Informe Word · Sombreado de párrafo	2	modulos/generador_word.py
292	Paquete · agent (archivado) (archivado)	1	
293	Paquete · prompts (archivado) (archivado)	1	
294	Paquete · reasoning (archivado) (archivado)	1	
295	Paquete · tools (archivado) (archivado)	1	
296	Paquete · motor	1	motor/__init__.py
297	Paquete · src	1	src/__init__.py
298	Paquete · RAG ALM	1	src/rag/ALM/__init__.py
299	Paquete · RAG LaR	1	src/rag/LaR/__init__.py

#	Comunidad	Nodos	Archivos principales
300	Paquete · RAG compartido	1	src/rag/shared/__init__.py
301	Paquete · RAG VaR	1	src/rag/VaR/__init__.py
302	Paquete · web	1	web/__init__.py
303	Nodo suelto · post (archivado)	1	
304	Web · Reconstruir índice RAG	1	web/routers/rag.py
312	Tipos · Clase abstracta (ABC) (archivado)	1	
330	Tipos · Document (python-docx) (archivado)	1	
331	Tests · parametrize (archivado)	1	
332	Tipos · get (archivado)	1	
333	Tipos · RGBColor (archivado)	1	
334	Tipos · Any (archivado)	1	

Parte 12 · Matriz de fallos frecuentes

Reúne las tablas «Errores frecuentes» de todas las skills: síntoma, causa y arreglo.

Skill	Síntoma / error	Causa	Arreglo
S01	sesión de login expirada	El servidor se reinició (las sesiones viven en memoria)	Volver a entrar; no es un fallo
S01	no hay modelo activo	Se llamó a un endpoint de sesión sin elegir modelo	S04
S01	rechazó la clave (401)	Clave mal copiada en .env (p. ej. sin la «s» inicial)	S05
S01	faltan columnas obligatorias	Cabecera del Excel distinta	S06
S01	Word sin imágenes de evidencia	El documento citado no está en la sesión o la cita no es literal	S09
S01	ModuleNotFoundError (torch, chromadb…)	Se usó el Python del sistema en vez de .venv	Usar .venv
S02	Word: «contenido ilegible»	Elemento XML fuera de orden (p. ej. w:shd tras w:jc)	Usar resaltar; no append directo en pPr
S02	Hoja en blanco entre anexos	La función del anexo termina con salto de página	La plantilla ya los quita (_quitar_salto_final); no añadir otro
S02	Índice sin números	Campos sin actualizar	Abrir en Word (se actualiza solo: updateFields)
S02	Tabla cortada entre páginas	Falta no_partir(tabla)	Añadirlo tras crear la tabla
S02	Imagen de sustento ausente	Ruta inexistente	Ver S09
S03	Sale Regular con 3 moderadas Doc+Gob	Supuesto A2 (hueco entre «más de 3» y «menos de 3»)	Confirmar con la gerencia y cambiar la condición
S03	Una OM deja el control en rojo	OM_INCUMPLE = True	Debe ser False (decisión del usuario)
S04	Cambio de CSS/JS no se ve	Caché del navegador	index() añade ?v=<mtime>; recargar
S04	Campo no se oculta con hidden	La clase fija display	Regla [hidden]{display:none!important} en el panel
S04	401 tras reiniciar el servidor	Sesiones en memoria	Volver a entrar
S04	400 «no hay modelo activo»	Endpoint de sesión sin modelo	Entrar a un modelo o usar dependencia de solo login
S05	401 «invalid x-api-key»	Clave incompleta o con comillas/espacios	Corregir en .env y reiniciar la web

Skill	Síntoma / error	Causa	Arreglo
S05	400 «workspace» con Anthropic	Clave «identity-linked»	Añadir ANTHROPIC_WORKSPACE_ID
S05	Azure «deployment not found»	Nombre de deployment ≠ nombre del modelo	Usar el nombre del deployment del portal
S05	403 content_filter	Política de contenido de Azure	Ver categoría en el mensaje; ajustar prompt
S05	La web sigue usando la clave vieja	.env se lee al arrancar	Reiniciar uvicorn
S06	Paso 02 rechaza el Excel real	Cabecera en fila 2 (fila 1 vacía)	Usar el lector tolerante o mover la cabecera
S06	Actividades con ámbito equivocado	Celda combinada no propagada	Revisar forward-fill de la columna
S06	Tier IV muestra todas las actividades	Sin columnas TIER	El estándar no distingue tiers: es lo esperado
S07	Sin literatura en el informe	Índice no construido / dominio inferido mal	Reconstruir; declarar --dominio-rag
S07	Resultados repetidos	Corpus con párrafos de plantilla	Deduplicar en el buscador
S07	BM25 = 0 con corpus pequeño	IDF nulo en Okapi con pocos documentos	Usar BM25Plus o más documentos